基于评分端点重建与缺陷约束的啤酒风味预测及风格迁移

摘　要　针对啤酒化合物到感官画像的预测、关键因素识别与风格迁移三个问题，本文先重建评分端点，再分别建立预测、解释与决策模型。

附件 4 的感官值是逐人中心化后的连续值，不是 0–6 原始分。本文据公开的 2197 条 0–6 评分（16 名品评员，每款酒 3–14 人）重建了 50 个目标的面板均分与七级分布；对 8.xlsx 的 70 个参考字段逐项核查：37 个符合对数规则、1 个恒等对应，另 32 个字段含 80 处差异。附件中显式缺失仅 73 格，另有 18.7% 的输入单元格为负值，其含义需按字段变换规则判断，二者必须分开处理。

第一问按酒三折五次验证。极端随机树与梯度提升树的等权融合在核心 30 项上的面板均分平均绝对误差为 0.50222 分，明显优于训练均值（0.60450 分），却只比风格中位数基线（0.50969 分）低 0.00747 分，差值的 95% 重采样区间 $[-0.01914, 0.00418]$ 含 0：固定预测比较不足以确认融合模型优于已知风格基线；补充的同参数 ET 对照中，风格加化学相对仅风格的核心 MAE 低 0.01426 分，但相对风格中位数的区间仍跨 0。七级分布模型的累计分布平方距离为 0.02178（总体基线 0.02820），面板众数一致率 80.52%，但非零众数一致率仅 24.33%，总体一致率主要由 78.03% 的零等级贡献。保形区间逐属性平均覆盖 90.06%、宽 1.7342 分，按品评人数分组后 3–4 人组降至 74.00%。缺失机制实验显示，系统性低值缺失使核心误差增加 0.0273 分，约为随机缺失 20%（0.0139 分）的两倍，不同缺失机制和缺失比例需要共同解释。

第二问以 $|r| \geq 0.7$ 构建 231 节点、109 条边的共表达图。图正则模型在单折平均口径下略差于无图（0.537035 对 0.536911），在聚合预测口径下略好（0.524865 对 0.526148），最终模型取 $\lambda = 0$；网络主要用于描述共变结构；150 条 Top-3 候选中 36 条满足“频率 ≥ 0.5、至少 5 折入选、置换贡献为正”的条件，将 80 个异常单元格置缺后 Top-3 保留 97.64%，仅用一致字段与有单位登记字段的 32 项交集时降至 36.18%。

第三问对 Tripel、Kriek、Lambic 各生成 40 条谱。受约束局部凸组合把 Tripel 的平均目标风格概率提高到 0.3358（重采样 0.2992），判为目标风格的比例为 0.695（重采样 0.565），但分布距离反而更大（0.1158 对 0.1034），说明“更像目标风格”与“更接近真实分布”不是一回事。迁移在 55 条记录上搜索可行 51 条、独立代理支持 11 条；严格浓度协议下最大 Tripel 份额全为 0，且 55 条记录均找到正余量不可行证书，阻塞主要来自 DMS 与反式-2-壬烯醛；改用阈值超额协议后，参考阈值下 30 条记录允许正份额，但没有候选通过联合代理复核；2 倍阈值敏感性下 34 条允许正份额，其中 9 条重复划分记录获得代理支持，并未经过实际制备与盲评。

结果显示：风格基线与复杂模型的误差接近；关键因素应按稳定支持与探索性候选分层解释；阈值约束与代理判断显著影响迁移候选，实际效果需要制备和盲评。

关键词　啤酒风味；感官评分重建；有序分布；化合物共表达网络；风格迁移；缺陷阈值约束

目录

1 问题重述与分析 **1**
1.1 三项任务要求的证据强度不同 . . . 1
1.2 技术路线 . . . 1
1.3 本文的交付物 . . . 1

2 模型假设与符号说明 **1**
2.1 基本假设 . . . 1
2.2 符号说明 . . . 1

3 数据核查与评分端点 **1**
3.1 数据全景与样本对应 . . . 1
3.2 化学尺度与异常 . . . 2
3.3 缺失、低值和变换 . . . 3
3.4 评分端点重建 . . . 3
3.5 维度映射与提交口径 . . . 5

4 第一问：感官画像的预测 **6**
4.1 三类输出与评价指标 . . . 6
4.2 面板均分预测 . . . 7
4.3 已知风格下的化学输入对照 . . . 7
4.4 七级分布与等级识别 . . . 8
4.5 预测区间与缺失条件下的鲁棒性 . . . 9
4.6 讨论：风格信息与化学输入 . . . 10

5 第二问：化合物关联结构与关键因素 **10**
5.1 共表达图的构建 . . . 10
5.2 图约束模型与可解性 . . . 11
5.3 双口径结果 . . . 12
5.4 Top-3 的稳定性 . . . 13
5.5 化学解释及其边界 . . . 13

6 第三问：风格生成与迁移可行域 **14**
6.1 风格条件生成与评价体系 . . . 14
6.2 生成结果 . . . 15
6.3 迁移搜索与冻结候选 . . . 16
6.4 严格浓度约束与不可行证书 . . . 17
6.5 阈值风险协议与联合复核 . . . 17

7 模型评价与推广 **18**
7.1 方法上的可取之处 . . . 18
7.2 局限 . . . 18
7.3 可推广的部分 . . . 19

8 结论 **19**

A 全部 50 项的 Top-3 候选及证据分层 **21**

B 敏感性分析所用 32 字段清单 **23**

C 源程序清单 **23**

1 问题重述与分析

啤酒的风味由挥发性化合物、理化条件与酿造工艺共同决定：酯类与高级醇带来果香与醇厚感，酒花萜烯提供草本与柑橘调，硫化物、双乙酰、反式-2-壬烯醛等则构成缺陷。附件给出了 250 款商业啤酒的化合物测量与感官评分，要求完成三项任务：由化合物预测 0–6 分的感官画像；识别各感官维度的关键化合物并论证其稳定性与化学合理性；在给定风格下生成化合物谱，并把 Pils/Lager 迁移为 Tripel 而缺陷不恶化。

1.1 三项任务要求的证据强度不同

三项任务共享同一份数据与同一套验证框架，但它们要求的证据强度依次递增。第一问是**预测**：只要在留出酒样上误差可控、区间可信，结论即可成立。第二问是**解释**：模型选出的重要化合物必须同时满足统计稳定与化学可解释，而相关系数与回归系数都不构成因果。第三问是**决策**：生成谱与迁移候选要同时满足风格、感官与缺陷三类约束，模型内可行与真实可酿造之间还隔着制备与品评。

因此本文不为三问套用同一种评价：第一问报告按酒划分的留出误差与区间覆盖，第二问报告重采样频率与置换贡献，第三问报告冻结候选的三层复核与线性规划可行性证书。任何一问的好成绩都不自动支持另一问。

1.2 技术路线

图 1 给出全文结构。数据层完成四件事：附件结构核查、化学尺度与缺失诊断、评分端点重建、评分卡到目标列的维度映射。任务层按三问展开，但共用同一批按酒分层的划分与同一套预处理规则。最底层是统一的验证约定：插补、截尾、标准化与调参只在训练折内估计，生成与迁移的候选先冻结再复核，失败与反直觉结果全部保留。

1.3 本文的交付物

对应三问，本文给出四类可复核的结果：一是 250 款酒在 50 个感官目标上的面板均分与七级评分分布，以及据此得到的众数与中位数等级；二是 50 个目标各 3 个候选化合物、共 150 条因素记录，附重采样入选频率、置换贡献与化学依据；三是 Tripel、Kriek、Lambic 三种风格的生成谱及其感官画像；四是 Pils/Lager 到 Tripel 的迁移可行域，包括严格协议下的不可行证书与阈值协议下的可行记录数。

上述结果全部由可重跑的脚本产生，输入文件哈希、随机种子与运行配置随结果保存。第 3 节交代数据口径，第 4 至 6 节分别给出三问的模型与结果，第 7 节讨论适用范围与推广。

2 模型假设与符号说明

2.1 基本假设

A1 附件 1 的字段可能采用对数、恒等或其他处理；对经过核查的对数字段，负值可表示原始浓度小于 1。第 3.2 节逐字段比较 70 个参考字段，不对所有列统一反变换。

A2 1.csv 中 73 个空单元格的真实缺失机制未知，建模时使用训练折中位数填补；10797 个负值保留为观测值，其原始尺度含义取决于字段变换。第 3.3 节以模拟缺失检验预测敏感性。

A3 附件 4 的连续值是同一批原始评分经逐人中心化、标准化后的结果，可由公开的 2197 条 0–6 原始评分重建出面板均分与七级分布；同一品评员的重复评分可先平均，品评员之间等权。

A4 风格标签 `tasting_category_fine` 在样本中可用且定义一致；使用仅依赖训练折的风格中位数作为对照，评价化学输入的预测价值。

A5 缺陷阈值取自文献与 8.xlsx 的阈值中位数，可近似迁移到本数据；但阈值依赖啤酒基质与测量方法，故在第三问中把阈值写成可缩放参数，并在 0.5、1、2 倍三档上报告结论。

A6 面板均分可代表该款酒的感官强度，众数可代表最常见的等级；两者对应不同的决策损失，不能互相替代。

2.2 符号说明

表 1 汇总全文记号。矩阵用大写粗体，向量用小写粗体，标量用斜体；下标 b 表示酒款、j 表示感官目标、t 表示品评员、r 表示同一品评员的重复次数、d 表示缺陷化合物。

3 数据核查与评分端点

3.1 数据全景与样本对应

八个附件表可按“化学输入—感官目标—评分卡—旁证”四层归并，样本主键在 1.csv、4.csv、7.csv 与 8.xlsx 之间一致。表 2 汇总了各表的规模、内容与用途：1.csv 是化学主表，4.csv 是感官监督表，2.csv、5.csv、6.csv 给出相关结构，3.xlsx 定义评分维度，7.csv 与 8.xlsx 分别提供评论旁证和外部校准。附件与公开逐人评分的数据来源见 [1, 2]。

样本规模为 250 款酒、22 种风格。1.csv 的 231 个数值字段由 226 个非综合化学与理化字段和 5 个综合量（`kcalperc`、`esters_sum`、`aroma_hops_sum`、`acids_sum`、`sulfur_sum`）组成；

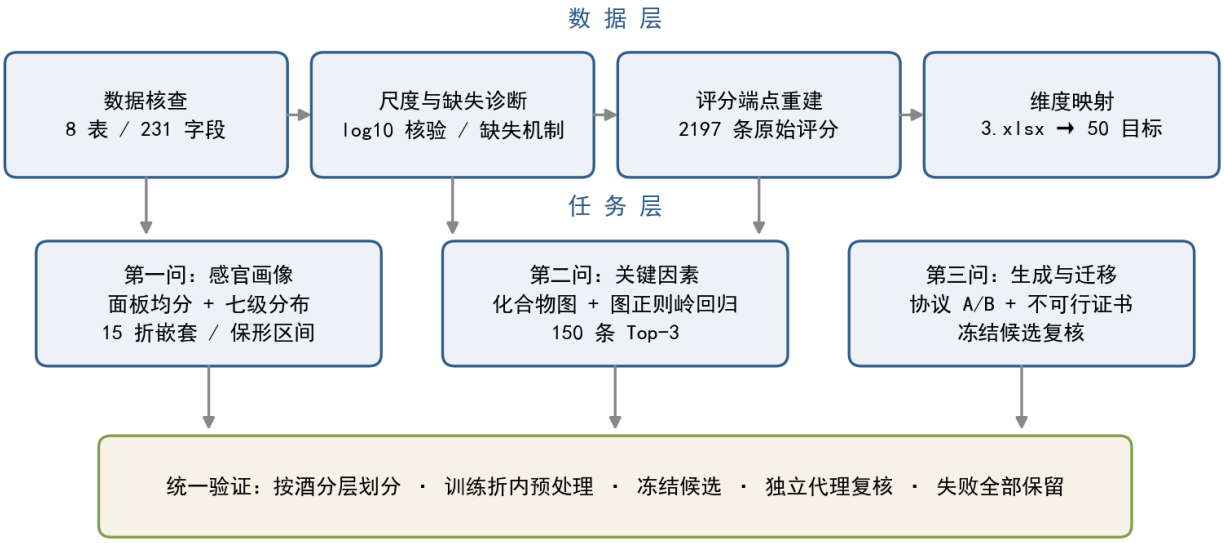


图 1　全文技术路线。数据层的四项核查共同决定任务层的输入口径；三问共用按酒分层划分与训练折内预处理，避免同一款酒同时出现在训练与验证中。

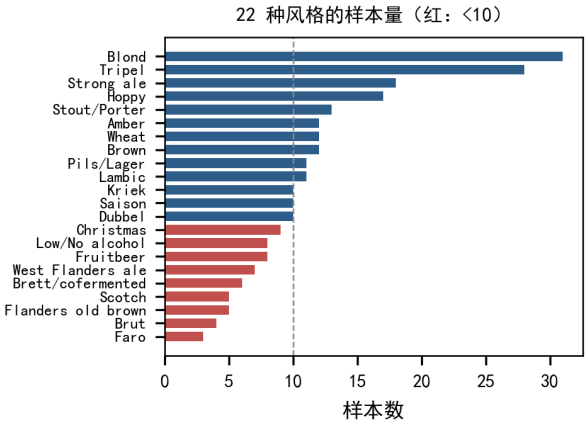


图 2　22 种风格的样本量。红色为不足 10 款的 9 种风格，最少 3 款；分层划分与风格基线都受这一分布约束。

4.csv 的 50 个目标由 15 个 Aroma、15 个 Flavour 和 20 个其他维度组成。1.csv 与 4.csv 均以 beer 为唯一键、beer_id 为条码辅助键，250 行的风格标签在两表中一致，样本对应可按编号与行序双重核对，风格分层因此可直接复用，不需要另行匹配。风格样本量从 Faro 的 3 款到 Blond 的 31 款，9 种风格不足 10 款，后续按风格分层划分，避免稀有风格整体落入某一折。

5.csv 与 6.csv 分别刻画感官目标之间、化学字段与感官目标之间的秩相关结构；两表用于检查目标分块和因素筛选的方向一致性，不进入预测模型，也不进入第一问特征矩阵，避免把目标间相关当作输入信息造成泄漏。

题面与附件存在三处口径差异，本文以附件为准。题面称 226 款酒，附件为 250 款；题面称约 40 维感官目标，附件为 50 维，其中 Aroma/Flavour 各 15 项、合计 30 项；题面的约 40 维来自两栏各约 20 项的口径，附件把三组汇总和共用维度拆开后为 50 项。题面称评分卡 10 个 sheet，附件为 12 个 beer_* 评分卡，另加 1 个 summary。四个表共用 beer 编号，使按酒划分训练折时不会出现样本错位。提交口径相应定为：50 项全部建模并输出，30 个核心项单列误差与分布指标，20 个其他项用于缺陷约束和总体评价，不与核心项合并平均。

附件 2 是秩相关矩阵，不是题面所称的皮尔逊相关矩阵。在 212 个可匹配字段上，2.csv 与本地重算 Spearman 的平均绝对差为 1.6×10^{-16}，与 Pearson 的平均绝对差为 0.0883；前者处于机器精度，后者高出一个量级。因此第二问的化学网络按秩相关构建，2.csv 直接作为相关结构使用，题面的“皮尔逊”表述予以纠正。231 个字段中有 19 个因命名或汇总口径无法与 8.xlsx 的化学名匹配，核对只在 212 个可匹配字段上进行；这一限制不影响秩相关判断。8.xlsx 的 7 个 sheet 分别承载校准化学值、感官阈值、SHAP、MDI 与价格；阈值表 261 行中只有 43 行有数值，第三问的缺陷阈值只取有数值的行。

3.2　化学尺度与异常

1.csv 包含经不同规则处理的化学与理化字段，不能把所有字段统一按 10^x 还原。以附件 8 校准表与附件 1 中 70 个同名数值字段进行逐字段比较：对每个字段分别测试恒等、非正值替换及替换后取 $\log_{10}$ 的候选规则。38 个字段在全部可比记录上满足数值容差，其中 37 个符合“非正值替换为该字段最小正值的一半，再取 $\log_{10}$”的规则，另一个 `diethyl.succinate` 满足恒等对应；其余 32 个字

表 1　主要符号

符号	含义
n, p, q	酒款数（250）、化合物字段数（231）、感官目标数（50）
$X \in \mathbb{R}^{n\times p}$	化合物与理化特征矩阵
$Y \in \mathbb{R}^{n\times q}$	感官目标矩阵（面板均分）
s_{btrj}	品评员 t 对酒 b、目标 j 的第 r 次原始评分
$\bar{s}_{bj}$	酒 b 在目标 j 上的面板均分
q_{bjk}	酒 b 在目标 j 上取等级 k 的面板比例
$\hat{g}_{\mathrm{mode}}, \hat{g}_{\mathrm{med}}$	预测众数等级、预测中位数等级
D	累计分布平方距离
RPS	有序概率评分
Z	训练折内标准化后的特征矩阵
W, L	化合物相关图的带符号权重矩阵、图拉普拉斯矩阵
B	多目标回归系数矩阵
α, λ	岭惩罚系数、图正则系数
c_d, t_d, R_d	缺陷 d 的浓度、参考阈值、阈值超额风险
$\boldsymbol{w}$	混配中锚点酒的份额向量
$\boldsymbol{M}$	混配约束矩阵（各锚点与来源在缺陷上的浓度差）

段共保留 80 个不一致单元格。两组的 38 和 32 互不重叠，覆盖全部 70 个字段。

对 `diethyl.succinate`，247 个共同可比记录中，两表原值的最大绝对差为 5.0×10^{-10}。该字段属于恒等对应：若把附件 1 的值误作对数并计算 10^{x}/cal，比值中位数为 13.95，但各记录范围为 6.26–738.07，说明这是套用不匹配变换产生的差异，不能据此声称该字段有固定倍数的单位偏差。表间一致也不能替代独立仪器标定。

图 3(a) 展示 37 个对数规则字段的 8452 个正参考值配对；绘图仅对每个字段使用其原报告单位比较，零值替换记录不在该图中。(b) 按字段统计 80 处例外，排名前八的字段分别列出，其余 24 字段合并。两幅子图使用不同的数量口径，完整字段分类、变换规则和异常数见附录 B 与支撑材料。

第二问的另一组 32 个字段记为 $V = R \cap U \cap X_{226}$：$R$ 是上述 38 个全部可比记录一致字段，U 是单位注册表中的 61 个字段，X_{226} 是剔除 5 个综合量后的非综合输入。V 包括 31 个对数规则字段和 1 个恒等对应字段，与 80 处例外所涉及的 32 个字段交集为空。该交集仅说明数值对应和单位记录具备，不构成完整化学标定；具体字段与单位列在附录 B。异常处理敏感性保留原始字段并将 80 处单元格置缺，填补参数只由训练折估计；另一项对照则更换为 V，两者回答不同问题。

3.3　缺失、低值和变换

显式缺失为 73 格（0.126%）；另有 18.7% 的数值单元格为负值，二者分别统计。73 个缺失分布在 9 列、63 款酒上，最多的一列是 `amyl.acetate` 的 27 格；9 列中 4 列的缺失不超过 2 格，`amyl.acetate` 一列接近全部缺失的 37%。缺失列中既有建模需要的香气化合物，也有 1 个综合量 `aroma_hops_sum`；整列删除会改变输入维度，综合量又不能按单列均值填补，否则会破坏它与子项的关系，故按单元格保留。缺失呈点状分布，没有整行删除；本文不删除缺失行，树模型与线性模型均使用训练折估计的中位数进行填补，避免测试折信息泄漏。

数值小于 0 的单元格有 10797 个，占 250×231 输入矩阵的 18.7%，分布于 175 个字段。对于已核实采用对数变换的字段，负值表示原始数值小于该字段报告单位下的 1；其他字段须依各自的处理规则解释，负值本身不能判定为低于检出限。本文保留负值；树模型先以训练折中位数填补缺失，线性模型另在训练折内估计标准化参数。

7.csv 的 750 个缺失集中在 3 列，且 186 个字段来自 Ratebeer 评论词，不是品评小组的 0–6 评分。750 个缺失对应 3 个整列缺失，说明这些感官词在评论语料中没有覆盖，而不是随机漏填；评论词的出现还受文本长度和用户习惯影响，缺失机制不是随机的。因此 7.csv 只作旁证，不能作训练标签；它的 186 个评论词与 4.csv 的 50 个目标并非同一套词表，不能按名称直接合并，若用评论词训练，会把文本噪声当成感官信号。

图 4(a) 显示显式缺失的列分布，(b) 显示输入矩阵中缺失与负值的位置。显式缺失由训练折填补，负值保留；4.csv 没有 NaN，但每款酒只有 3–14 名品评员，因此后文同时报告面板聚合结果及按人数分组的区间覆盖。

3.4　评分端点重建

4.csv 是标准化后的感官分值，不能直接当作 0–6 分标签；0–6 端点需由公开逐人评分重建。4.csv 为 250×53，50 个目标列的均值为 -0.063～0.063、标准差为 0.37～0.86，符合中心化/标准化后的尺度，与 0–6 原始分不符。各列均值接近 0、标准差小于 1，说明 4.csv 是对原始 0–6 分做中心化/标准化后的结果；列间标准差不齐，说明标准化未统一到单位方差。该表用于样本对应、目标定义和标准化核查，不作为回归标签；它仍保留样本间的相对差异，可用于检查目标分布和异常酒样，但不能还原 0–6 分，也不能替代逐人评分。面板均分的取值范围仍是 0–6，而 4.csv 的标准化值可正可负，两者不能在同一损失函数下混用；4.csv 的列均值和标准差只能确认标准化，不能反推出原始 0–6 分的线性变换参数。

表 2　八个附件表的规模、内容与本文用途

文件	规模	内容	本文用途
1.csv	250 × 234	3 列样本信息（beer、beer_id、风格）+ 231 个数值字段：226 个非综合化学与理化字段、5 个综合量	化学输入；风格标签与样本对齐
2.csv	231 × 231	231 个化学字段的秩相关矩阵（含行名列共 232 列）	核对相关结构；构建化学网络
3.xlsx	12 卡 + 汇总	12 张 beer_* 评分卡 + 1 个 summary；单卡 100 × 33，summary 13 × 52	评分维度定义与目标映射
4.csv	250 × 53	3 列样本信息 + 50 个标准化感官目标（15 A + 15 F + 20 其他）	目标定义与样本对应；不作 0–6 标签
5.csv	50 × 51	50 个感官目标间的 Spearman 矩阵	目标相关性核查与分组
6.csv	44 × 51	44 个化学字段 × 50 个感官目标的 Spearman 矩阵	关键因素筛选的旁证；不作因果解释
7.csv	250 × 189	3 列样本信息 + 186 个 Ratebeer 评论感官词	旁证；750 个缺失，不作训练标签
8.xlsx	7 个 sheet	70 个参考化学与理化字段（250 × 73）；阈值 261 行（43 个有数值）；SHAP 500 行；MDI 232 行；价格表	尺度反查、第三问阈值、重要性交叉核对

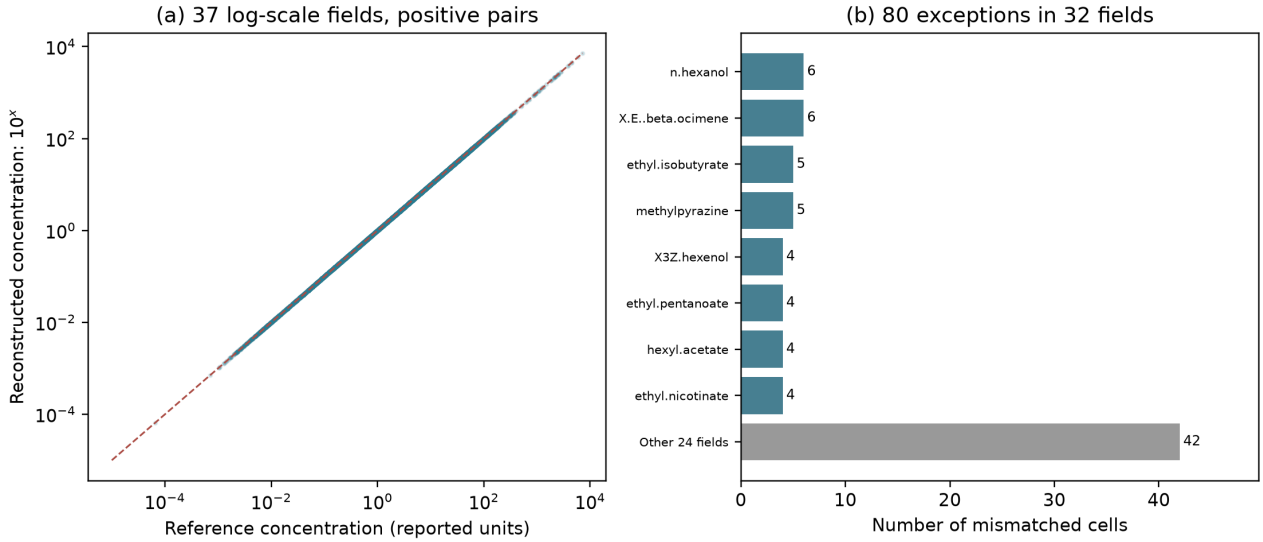


图 3　化学尺度核查。(a) 37 个对数规则字段的正值配对，各字段按自身报告单位比较；(b) 32 个不一致字段中异常单元格的数量分布，前八字段单列，其余 24 字段合计。

表 3　化学尺度核查的集合定义

集合	字段数	核查结果
逐点一致 R	38	37 对数规则、1 恒等规则
有例外字段 E	32	共 80 个异常单元格
敏感性子集 V	32	$R \cap U \cap X_{226}$；$V \cap E = \varnothing$

8.xlsx 的字段说明指向作者公开的逐人评分数据。据此重建的面板包含 16 名品评员，每款酒 3–14 人、中位 8 人，共 2197 条 0–6 评分。16 名品评员以匿名编号出现，每款酒的品评员集合不同；按 beer 编号与附件对齐后，30 个 Aroma/Flavour 属性分成两组核对，20 个共用项单独核对。面板人数不等意味着端点精度不等，直接比较不同酒的均分会把精度差异混入误差。2197 条评分是重建端点的全部原始信息；任何汇总都会损失一部分分布信息，故本文保留逐条评分用于分布端点。

记第 b 款酒、第 j 个属性、第 t 名品评员的第 r 次评分为 s_{btrj}，T_b 为评过该酒的品评员集合，n_{bt} 为同一人的重复次数。面板均分为

$$\bar{s}_{bj} = \frac{1}{|T_b|} \sum_{t \in T_b} \frac{1}{n_{bt}} \sum_{r=1}^{n_{bt}} s_{btrj}, \tag{1}$$

七级分布为

$$q_{bjk} = \frac{1}{|T_b|} \sum_{t \in T_b} \frac{1}{n_{bt}} \sum_{r=1}^{n_{bt}} \mathbf{1}\{s_{btrj} = k\}, \tag{2}$$

其中 $k = 0, \ldots, 6$。

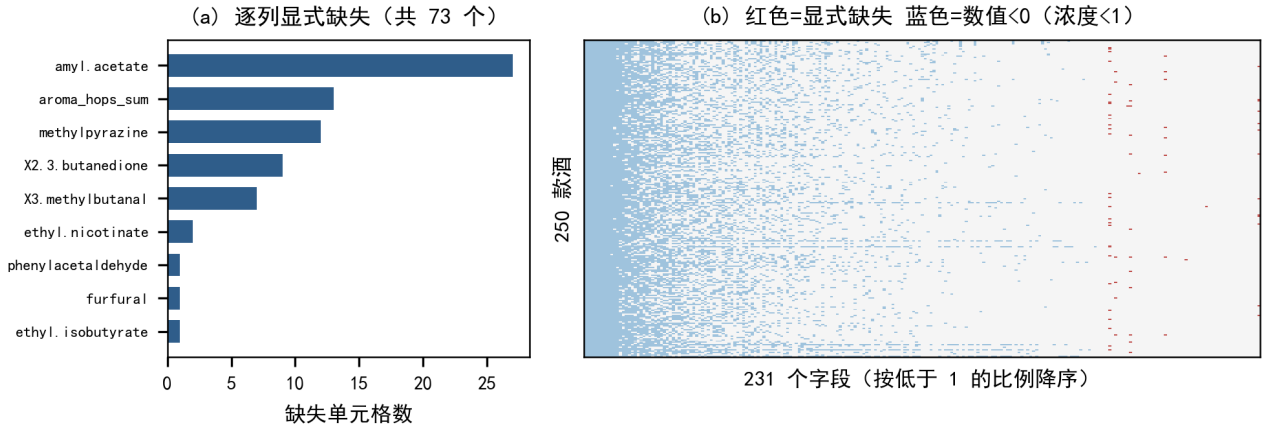


图 4　缺失与低值结构。(a) 231 个数值字段的显式缺失量，9 列有缺失，`amyl.acetate` 最多 27 格；(b) 250 × 231 矩阵中缺失与负值的空间分布，缺失呈点状，负值呈面状。

分母 $|T_b|$ 是实际评过该酒的品评员数，不是全体 16 人；每款酒的端点只使用自己的面板，未评人员不按 0 分计入。式 (1) 与式 (2) 都先在人内对重复评分取平均，再在人之间等权平均。同一人对同一杯酒的重复评分高度相关，若按原始条目数加权，评分次数多的人会主导端点；等权处理则让每名品评员对一款酒只投一票。同一人可能对同一杯酒给出多次评分，n_{bt} 可能大于 1；先平均再等权等价于把每人的重复评分压缩成一条观测。面板均分与七级分布共享同一套人员权重，因此二者的差异只来自汇总方式，不来自样本集合。

平方损失下均值最优，绝对损失下中位数最优，累计分布损失下完整分布最优，0–1 损失下众数最优。三种端点对应三种决策损失：面板均分对应平方损失，七级分布对应累计分布或 RPS 损失，众数 $\hat{g}_{bj} = \arg\max_k q_{bjk}$ 对应 0–1 精确等级损失。本文不把三者压成一个标签，而是在第一问分别报告均分误差、分布距离和众数一致率。3–4 人面板的端点不确定性明显大于 8 人以上面板，第一问按人数分组报告覆盖率和误差，避免把面板人数差异误判为模型差异；众数对人数少的面板更敏感，3–4 人时众数可能只反映一两个品评员的选择。

3.5　维度映射与提交口径

评分卡与 4.csv 的目标可按字典一一映射，但字段名和模板公式不能直接照搬。3.xlsx 含 12 个 beer_* 评分卡和 1 个 summary，12 张评分卡对应 12 款示例酒，summary 汇总这些酒的目标列；评分卡只用于维度定义，不直接提供 250 款酒的得分。每张卡左栏 Aroma、右栏 Flavour，两栏各含 Malt、Hops、Esters 三组，每组 4 个描述词加 1 个组汇总；乙醛 1 项，口感 4 项，缺陷 6–7 项，另有香料与 Lambic 专项。映射结果见表 4：A/F 前缀各 15 项，共 30 项核心；其余 20 项为口感、缺陷、香料、酸味与总体。这 50 个目标列同时定义第一问的输出、第二问的响应变量和第三问的感官画像维度。

4.csv 中 A_malt_*、A_hops_*、A_esters_* 各 5 项，F 侧同构；组汇总以 _all 结尾，共 6 项，三组汇总分别为 A/F_malt_all、A/F_hops_all、A/F_esters_all，组汇总与 4 个描述词共同构成 5 项，不能只取子项而漏掉汇总。Aroma 栏可映射 15 项（3 组 × 5），Flavour 栏同理；口感 4 项、缺陷 7 项、香料与总体 9 项构成其余 20 项。乙醛 1 项在 4.csv 中没有独立目标列，故不进入 50 项目标。字段名差异来自评分卡模板与监督表的两次命名：A/F_ester_fruity 在 4.csv 中写作 A/F_esters_fruity，4vg 写作 X4vg；若按字符串直接匹配，这几个字段会丢失。本文按显式字典映射，字典以 4.csv 的 50 个列名为准，逐项记录评分卡组别、A/F 归属和是否汇总；映射表写入交付包的字段清单，预测时按列名而非列序取数，不依赖评分卡模板公式。bitternes 等拼写按原表保留，避免合并时产生重复列。

提交口径为 50 项全部建模并输出，核心 30 项单列；20 个其他项用于缺陷约束、风格画像和总体评价。核心 30 项是训练面板的主评分维度，20 个其他项多为缺陷或风格专项，两类项目的量纲和稀疏性不同，分开报告更便于按决策场景取用。评分卡的组别划分与 0–6 强度标度符合训练面板的感官评价流程 [1]；但乙醛位置没有独立目标列，组汇总也不能按模板公式直接重算，直接套公式会把无监督项算进目标，因此本文以字段名和组别字典为准。预测输出按 50 列对齐 4.csv 的列序，核心 30 项与其余 20 项用同一套字段清单标记，便于下游按组取用；映射表同时作为训练、预测和评价三处的字段白名单，避免不同脚本各自解释列名。

表 4　评分卡分组与 4.csv 目标的字典映射。A/F 前缀各 15 项，其余 20 项为共用目标；乙醛项没有独立目标列。

评分卡分组（项数）	4.csv 字段	归属	目标数
Malt（4 子项 + 1 汇总）	A_malt_all, A_malt_grain, A_malt_bread, A_malt_cara, A_malt_burn；F 侧同构	A_/F_	10
Hops（4 子项 + 1 汇总）	A_hops_all, A_hops_citrus, A_hops_tropical, A_hops_noble, A_hops_woody；F 侧同构	A_/F_	10
Esters（4 子项 + 1 汇总）	A_esters_all, A_esters_ethac, A_esters_isoaa, A_esters_flower, A_esters_fruity；F 侧同构	A_/F_	10
Acetaldehyde（1）	—	—	0
口感（4）	body, bitternes, co2, aftertaste	共用	4
缺陷（6–7）	dms, metallic, X4vg, diacetyl, t2n, stale_hops, barnyard	共用	7
香料、酸味与总体（9）	acidity, sweetness, orange, coriander, clove, lactic, acetic, alcohol, overall	共用	9
合计	A 15 项、F 15 项、共用 20 项		50

4　第一问：感官画像的预测

第一问要求仅凭 231 个数值输入（226 个化合物与 5 个综合量）预测 50 项感官画像。本文把画像拆成均值、分布和等级三层输出，三层各有对应的损失与评价口径；均分与分布的模型比较在同一套按酒划分的 15 个留出折上完成，区间与缺失实验另设划分，后文分别注明。

4.1　三类输出与评价指标

第一问的交付物是三类可分别评价的输出，而不是一个等级。第 3 节已把第 b 款酒、第 j 个属性的面板均分记为 $\bar{s}_{bj}$，把七级评分分布记为 q_{bjk}；本节沿用这两组记号。

交付物包括连续的面板均分 $\bar{s}_{bj} \in [0,6]$、七维概率 $(q_{bj0}, \ldots, q_{bj6})$（非负且和为 1），以及由分布导出的众数等级与中位数等级；前两者描述平均强度与评分分歧，后者给出整数决策。众数取概率最大的等级，中位数取累计概率首次达到 0.5 的等级；在零膨胀的偏态分布下两者经常不同，不能互相替代。

三类输出对应三种决策。若只需要一个总体强度，取面板均分；若要报告不确定性，取七级分布；若必须给出一个整数等级，才在众数和中位数之间选择。把三者压成一个数字，会丢失评分分歧，也会让评价口径随下游任务漂移。

面板均分用平均绝对误差评价：

$$\mathrm{MAE} = \frac{1}{nm} \sum_{b=1}^{n} \sum_{j=1}^{m} \left| \bar{s}_{bj} - \hat{s}_{bj} \right|. \tag{3}$$

其中 n 为酒数，m 为核心 30 项或全 50 项，$\hat{s}_{bj}$ 为留出预测。MAE 的单位是分，可直接与 0—6 的评分尺度对照。

分布质量用累计分布平方距离评价。记某款酒某属性的预测概率为 $\hat{p}_h$、观测分布为 q_h，$P_k = \sum_{h=0}^{k} \hat{p}_h$ 与 $Q_k = \sum_{h=0}^{k} q_h$ 分别为预测分布与观测面板分布在等级边界 k 的累计概率，则

$$D = \frac{1}{6} \sum_{k=0}^{5} (P_k - Q_k)^2 . \tag{4}$$

式 (4) 对六个内部边界取平均，使 D 与七级概率处于同一尺度；逐条记录计算后取平均，得到表 7 中的 D。D 越小，预测分布越接近该酒实际的面板经验分布。

若把预测分布发给一位随机抽出的品评员，其实际等级为 Y，则单次评分的秩序概率评分为

$$\mathrm{RPS}(P, Y) = \frac{1}{6} \sum_{k=0}^{5} (P_k - \mathbf{1}\{Y \le k\})^2 . \tag{5}$$

式 (5) 是严格适当的评分规则，鼓励模型报告真实概率而不是把质量压向某一等级 [10]。对 $Y \sim q$ 取期望，并记面板分歧项 $\Delta = \frac{1}{6} \sum_{k=0}^{5} Q_k(1 - Q_k)$，则

$$\mathbb{E}_{Y \sim q} \left[\mathrm{RPS}(P, Y) \right] = D + \Delta. \tag{6}$$

式 (6) 说明个人期望 RPS 由两部分组成：第一部分正是 D，第二部分 Δ 只由观测分布的分歧程度决定。核心 30 项上 $D = 0.02178$、期望 RPS $= 0.11044$，两者相差 0.08866，恰好等于面板分歧项的平均值。把 0.02178 称为 RPS，或把 0.11044 当作分布距离，都会把“分布拟合”和“个人评分预测”混为一谈；本文对两者分别报告、分别解释。

从数值看，分歧项 0.08866 远大于 D 的 0.02178，其中分歧项来自面板经验评分分布的离

散度；它反映人员及重复评分差异，不能单独识别其成因。这决定了后文的改进重点：继续压低 D 的空间有限，提高个人等级命中率更困难。

4.2 面板均分预测

风格中位数是较强的已知风格基线；融合模型与其误差差距较小，需要结合不确定性解释。

面板均分预测以 15 个留出折评价：按风格分层、以酒为单位做三折，重复五次，每款酒在五次中各被留出一次；先把五次留出预测取平均，再按式 (3) 计算 MAE。这样同一款酒只贡献一个误差，避免重复留出被当作独立样本。

比较的方法包括训练均值、总体中位数、风格中位数、岭回归、极端随机树、梯度提升树，以及极端随机树与梯度提升树的固定等权融合 [3, 4, 5, 6]。岭回归的正则系数在训练折内选择，其余方法使用预先固定的参数。

三折而非五折，是因为 22 种风格中有 9 种少于 10 款、最少只有 3 款；三折能保证每个训练折仍覆盖全部风格，五次重复则降低单次划分的偶然性。融合权重固定为等权而不是在留出折内学习，是为了避免把模型选择误差带进比较；风格中位数对每个目标只估计一组风格常数，参数个数等于风格数，因而是一个很强的朴素基线。

结果见表 6。融合在核心 30 项上的 MAE 为 0.50222 分，在全部 50 项上为 0.43560 分；相对训练均值的降幅分别为 0.10228 分和 0.11319 分。真正需要解释的是这 0.10228 分从哪里来。风格中位数只使用风格标签、不使用任何化合物，核心 MAE 已达 0.50969 分；使用 231 个化学输入的融合模型只把误差压到 0.50222 分，融合低 0.00747 分。

以酒为单位、在固定预测条件下重采样，融合减风格中位数的差值 95% 区间为 $[-0.01914, 0.00418]$，含 0。在本数据与固定预测的重采样口径下，没有足够证据确认融合模型优于风格中位数；这不等于证明化合物信息的条件增量为零。风格中位数到总体均值的误差差，占总体均值到融合模型误差差的 92.7%，该比例仅为误差对比，不是信息贡献分解。本文报告的区间均为固定预测条件下的重采样区间，未纳入重新训练和模型选择的不确定性。

4.3 已知风格下的化学输入对照

为进一步区分风格信息与化学信息，新增相同外层划分和相同极端随机树参数的探索性比较。一个模型只输入风格的独热编码，另一个模型输入同样的风格编码及 231 个化学字段；两者的预处理均在训练折拟合，并对每款酒的五次留出预测取平均。仍保留按训练折构造的风格中位数作为面向 MAE 的非树基线。

表 5　已知风格条件下的面板均分预测对照

模型	核心 30 项 MAE	全 50 项 MAE
风格中位数	0.50969	0.44712
仅风格 ET	0.51701	0.45762
风格与化学 ET	0.50275	0.44019

核心 30 项中，风格与化学 ET 相对于仅风格 ET 的 MAE 差为 -0.01426 分，以酒为单位、按风格分层对固定留出预测重采样 5000 次，95% 区间为 $[-0.02427, -0.00395]$；相对于风格中位数的差值为 -0.00694 分，区间为 $[-0.01778, 0.00398]$。因此，在该固定树模型和已知风格条件下，加入化学字段改善了对仅风格 ET 的预测，但相对风格中位数的优势尚不确定。两种 ET 的输入维数不同，`max_features` 按各自维数取比例，树的候选分裂集合并不完全相同；本对照也不包含重训与模型选择不确定性。

融合相对极端随机树仍有可辨的改进：核心 MAE 低 0.00367 分，融合减 ET 的差值区间 $[-0.00668, -0.00056]$ 不含 0。融合只是把两个已训练好的树模型等权平均，没有引入新信息，模型平均可以降低部分不一致误差；该实验没有将误差变化分解为方差与偏差，故不对来源作唯一归因。

若以平方误差衡量且两模型误差方差相等，记误差相关系数为 ρ，等权平均的误差方差为单模型的 $(1+\rho)/2$；ρ 越低，融合收益越大，但平均不会消除系统性偏差。上述推导依赖等方差假设，本实验未验证该假设，不能由 MAE 差值直接推断方差贡献。

逐目标看，模型差异并不均匀。图 6 给出 50 项目标在 5 种模型下的 MAE 热图：高误差集中在低方差目标与口感、整体印象类维度，多数目标的模型间色差很小，只有少数目标对模型选择敏感。继续更换回归器最多只能在这些目标上取得局部改进，不能改变整体误差格局。

预测值本身是否可信，还要看校准。把 12500 条留出预测按预测值分成十等份，比较每箱的预测均值与实测均值：十箱中有九箱的偏差不超过 0.04 分，期望校准误差 ECE 为 0.029（图 5）。唯一系统偏差在最高一箱，预测 3.19 分、实测 3.31 分，低估 0.12 分。模型在低分段略高估、在高分段压缩，这与树模型对极端值取平均的行为一致。校准良好不等于排序正确：ECE 小说明整体水平没有漂移，非零众数一致率低说明单个等级仍难判对，两者衡量的是不同层面。

逐目标看，最容易的是 `coriander`（0.108 分）与 `acetic`（0.166 分），最难的是 `A_esters_isoaa`（0.711 分）、`A_esters_ethac`（0.631 分）与 `A_esters_all`（0.620 分）。难的目标集中在酯

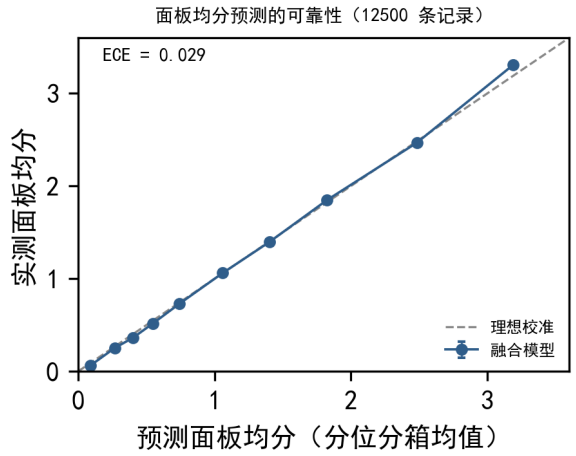


图 5　面板均分预测的可靠性曲线。十等分箱的实测均值基本落在对角线上，ECE 为 0.029；最高一箱（预测 3.19 分）低估 0.12 分，说明模型压缩了高分端。误差棒为一倍标准误。

表 6　七种方法的面板均分留出误差（单位：分）

方法	核心 30 项/分	全 50 项/分
训练均值	0.60450	0.54879
总体中位数	0.58785	0.52275
风格中位数	0.50969	0.44712
岭回归	0.52270	0.45708
极端随机树	0.50589	0.44336
梯度提升树	0.50819	0.43780
固定等权融合	0.50222	0.43560

注：留出预测为五次留出结果的平均，MAE 以酒为单位计算。

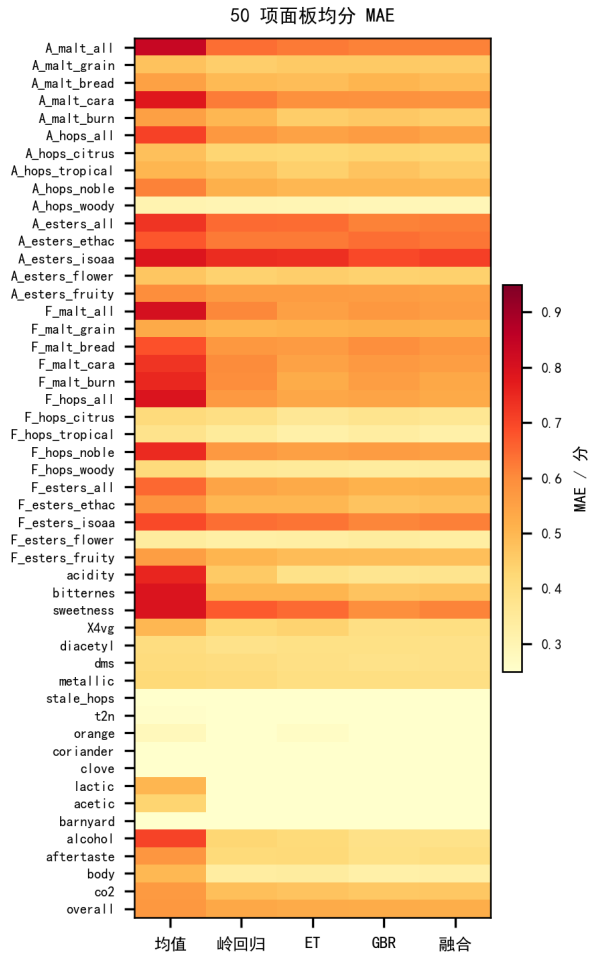


图 6　50 项目标在 5 种模型下的留出 MAE。误差集中在低方差目标与口感类维度，模型差异只出现在少数目标。

类总量与乙酸酯，这类维度在风格间差异大、面板分歧也大，误差不是模型选错造成的。

两个汇总口径也会改变模型排名：核心 30 项上 ET 略好于 GBR（0.50589 对 0.50819），全 50 项上 GBR 反而更好（0.43780 对 0.44336）。单一口径的排名并不稳固，这也是本文同时报告两个口径、并用融合降低单模型波动的原因。

4.4　七级分布与等级识别

七级分布模型在分布指标上优于两条基线，非零等级的识别仍是短板。

以第 3 节的七级分布 q_{bjk} 为监督目标，用多输出极端随机树直接估计七维概率 $\hat{p}_{bjk}$，再取众数和中位数作为等级输出。表 7 比较三条路线：训练集的总体经验分布、训练折内的风格经验分布，以及频率极端随机树。三者使用同一套留出折与同一批 7500 条核心记录。

频率 ET 的 D 为 0.02178，低于总体基线的 0.02820 和风格基线的 0.02313；期望 RPS 为 0.11044，低于总体基线的 0.11685 和风格基线的 0.11179。两项分布指标均优于这两条基线；由于模型形式与输入不同，该比较不能单独分离化学信息的贡献。

但等级指标给出相反的画面：总体众数一致率 80.52%，仅比总体基线的 79.31% 高 1.21 个百分点，还低于风格基线的 80.72%；非零众数一致率 24.33%，低于风格基线的 26.58%。这里出现了指标间的冲突：分布距离和期望 RPS 都支持频率 ET，等级一致率却支持风格基线。冲突的根源是评价对象不同——前两者衡量整条分布，后者只衡量概率最大的那一级；当零等级占近八成时，众数一致率对少数非零等级的错分不敏感。

分布指标改善而等级指标不改善，原因在零膨胀。核心 30 项的 7500 条记录中，观测众数为 0 的占 78.03%，频率 ET 预测为 0 的占 81.29%，多出 3.26 个百分点。总体一致率主要由“预测 0 且观测也是 0”撑起；只要把零等级判对，总体一致率就接近 80%。

零膨胀下的类别不平衡会使总体一致率失真，这是分类评价中的常见现象 [7]。进一步对七个观测等级等权计算宏平均 F1 与平衡准确率，并报告

表 7　核心 30 项七级分布模型的比较

方法	D	期望 RPS	众数一致率	非零一致率
总体分布基线	0.02820	0.11685	79.31%	21.30%
风格分布基线	0.02313	0.11179	80.72%	26.58%
频率 ET	0.02178	0.11044	80.52%	24.33%

注：D 与 RPS 无量纲；非零一致率在观测众数非零的 1648 条记录上计算。

有序等级平均绝对误差。本文保留总体一致率是为了与常规报告口径可比，但同时报告非零一致率，避免用一个被零等级主导的数字下结论。

7500 条核心记录中，频率 ET 判对 6039 条，其中 5638 条观测与预测均为零；总体基线判对 5948 条，其中零等级判对 5597 条。前者多判对的 91 条包括零等级净增 41 条和非零等级净增 50 条。对 1648 条非零观测，401 条判对、459 条判为零、788 条误判为其他非零等级。`A_hops_noble` 的 49 条非零记录全部被预测为 0。图 7 展示各等级之间的完整混淆。

针对零膨胀，本文尝试过先判零、再估非零等级的两阶段模型：在与频率模型同为 32 棵树的条件下，非零众数一致率由 24.76% 升到 25.49%，提高 0.73 个百分点，但提高幅度的固定条件 95% 区间 $[-0.91, 2.30]$ 个百分点含 0，D 也由 0.02178 升到 0.02200。表中 200 棵树的主模型非零一致率为 24.33%，与 32 棵树对照的口径不同。证据不足以支持替换，主模型仍取频率 ET；这与 4.2 节的判断一致，化合物输入能修正分布的整体形状，却不足以改变具体等级的排序。

表 8 的平衡准确率和宏 F1 显示，频率 ET 低于已知风格基线，而等级 MAE 较低。三种指标关注的分类与等级距离不同；各方法使用相同外层留出酒和训练折构造基线，风格基线额外依赖风格标签，ET 使用化学字段。结果支持保留多种端点及强基线，不支持仅用总体一致率概括非零等级能力。

4.5　预测区间与缺失条件下的鲁棒性

保形区间的逐属性平均覆盖接近名义水平，人数分组覆盖存在差异；模拟缺失实验比较不同缺失模式的预测影响。

区间用五次分割的保形预测构造 [9]：每次 150 款酒训练、50 款酒校准、50 款酒测试，按属性在校准集上取绝对残差的分位数，得到该属性的固定半宽。五次分割的逐属性平均覆盖率为 90.06%，平均宽度为 1.7342 分。该汇总值接近 90% 的名义水平，不能保证每个属性或人数分组均达到该水平。

保形预测只要求校准集与测试集可交换，不要求残差服从正态分布；按属性分别校准，也避免了

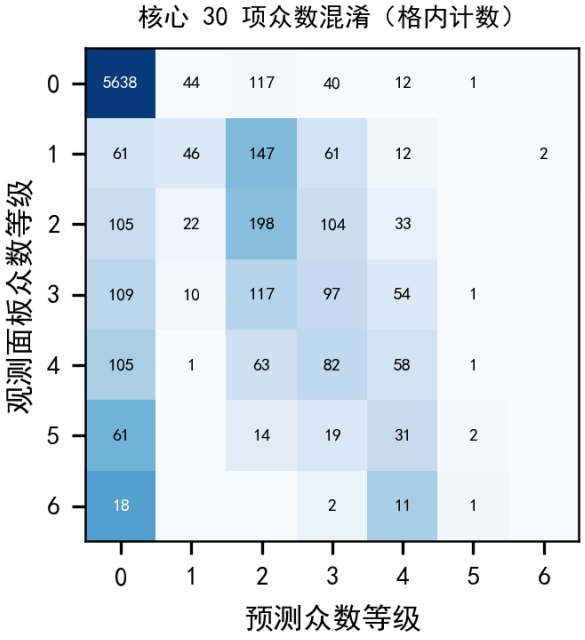


图 7　核心 30 项留出预测的面板众数等级混淆矩阵。观测非零的 1648 条中，459 条判为零，788 条误判为其他非零等级。

跨属性的尺度差异。这两点是它适合面板均分的原因，但也意味着覆盖率是边际的，不是对每款酒、每个人数分组都成立。

边际覆盖不等于条件覆盖。按历史面板人数把测试记录分组，3—4 人组（12 款酒）覆盖率只有 74.00%，5—7 人组为 86.89%，8 人及以上组为 92.94%，随人数单调上升。人数较少时面板均分的抽样变异可能较大，而本实验的半宽按属性统一给出；这是一种解释，现有数据不能将组间差异归因于人数。五次分割重复使用酒样，3—4 人组仅涉及 12 款不同酒，分组结果作为描述性诊断。图 8 同时展示分组覆盖与面板人数分布。

后续可在训练与校准数据中预先制定按人数分组的区间方法，并在独立留出数据上评价；不能依据本次测试分组覆盖直接调整区间宽度。

缺失实验分两块。一块在测试阶段叠加完全随机缺失（MCAR），用训练阶段拟合的插补参数填补；15 折聚合下，核心 MAE 由 0.51086 升到 0.52662（20% 缺失），众数一致率由 80.51% 降到 80.33%。这一块只回答随机丢数的代价，不覆盖系统性缺失。

另一块固定 3 折、重复 2 次，使用 300 棵树的 ET，基准 MAE 为 0.50666，分别施加 MCAR（10%、20%）、MAR（20%）、整块通道缺失（20%）和低值非随机缺失（MNAR，20%）五个条件，结果见表 9 与图 9。

同为 20% 缺失比例，MNAR 低值条件的 MAE 增量为 +0.02727 分，约为 MCAR 20%（+0.01393）

表 8　核心 30 项的留出众数等级补充评价

方法	宏平均 F1	平衡准确率	等级 MAE	总体一致率	非零一致率
总体分布基线	0.2117	0.2490	0.4168	79.31%	21.30%
风格分布基线	0.3352	0.3184	0.4020	80.72%	26.58%
频率 ET	0.2855	0.2835	0.3912	80.52%	24.33%

注：7500 个酒—属性记录先合并再按七个实际出现的等级计算宏 F1 与召回均值；同一酒的 30 项并非独立观测。风格基线使用已知风格，各折只用训练酒的同风格评分分布。等级 MAE 按众数等级差计算。

表 9　六种缺失条件下的核心 30 项 MAE 增量（单位：分）

缺失条件	MAE 增量/分
无缺失	0
MCAR 10%	+0.00656
MCAR 20%	+0.01393
MAR 20%	+0.01148
整块通道 20%	+0.00695
MNAR 低值 20%	+0.02727

注：基准为 ET-300 的 0.50666 分；增量为相对无缺失条件的差值。

的两倍，也高于 MAR 20%（+0.01148）；整块通道缺失的代价反而最小（+0.00695），约为 MCAR 20% 的一半。这些是本实验设定下的模拟结果：低值依赖的掩蔽会使训练折中位数填补替代部分低值，影响模型区分样品；模拟掩蔽不等同于仪器未检出。同一条件下等级精确率由 0.5399 降到 0.5088，Spearman 相关由 0.4472 降到 0.3985，退化不限于均分。

四种掩蔽规则对输入信息的破坏方式不同：MCAR 随机移除数值，MAR 的掩蔽依赖已观测变量，整块缺失移除选定字段，MNAR 低值规则按待掩蔽的数值选择。各条件均采用训练折中位数填补；表中排序适用于所设掩蔽方式和模型。

因此，题面所说的“数据缺失”不能只按缺失比例做压力测试：对仪器数据，低值缺失和整通道未测是两种不同的机制，本实验中前者造成的误差增量更大。两套缺失实验的基准不同：15 折 MCAR 压力的基准是 0.51086，新增机制实验的基准是 ET-300 的 0.50666，折数与模型设置都不同，两个基准不能直接相减。本文只在各自实验内部比较增量，避免把口径差异误读为机制效应。

4.6　讨论：风格信息与化学输入

风格中位数基线在核心 30 项上达到 0.50969 分，融合模型为 0.50222 分，两者的固定留出预测重采样区间包含零。两者使用的信息不同：风格基线需要已知风格，融合模型使用化学字段；它们之间的误差差异不能分解为各类信息的独立贡献。

在同样获得风格标签的 ET 对照中，加入化学字段使核心 MAE 从 0.51701 降至 0.50275，相对仅风格 ET 的固定预测重采样区间未跨零。这一结果表明化学字段在该模型和划分下提供了可用的预测信息；然而与风格中位数比较的区间跨零，不能概括成对所有风格基线都有稳定提升。新对照属于审查后的探索性实验，需用新酒样或预先确定的后续划分检验。

等级预测仍有明显短板。核心属性观测众数为 0 的比例为 78.03%，频率 ET 的总体一致率为 80.52%，非零众数一致率为 24.33%。宏平均 F1、平衡准确率和等级 MAE 展示不同的模型排序，因而部署时应根据错判非零等级与等级距离的代价选择决策指标。逐属性和逐风格结果用于确定哪些目标需要补充数据或专门建模。

第二问的 Top-3 基于标准化岭回归系数筛选，并结合重采样与置换贡献分层；这些条件关联和相关网络不能直接解释为因果作用。第三问的生成画像沿用预测端点，其候选还需经过缺陷约束、独立训练代理与实际制备验证。

5　第二问：化合物关联结构与关键因素

5.1　共表达图的构建

高相关只集中在少数化合物族，多数字段与其他字段近似独立，图正则能够影响的系数范围因此在建模之前就已受限。在 250 款酒的全部 231 个数值字段上计算 Spearman 秩相关，保留 $|r| \geq 0.7$ 的字段对作为边，边权取带符号的相关系数，得到 231 个节点、109 条边。浓度字段跨越多个数量级且含截尾值，秩相关对单调变换不敏感，适合这种分布。有边的字段只有 81 个，其余 150 个字段没有任何邻居，约占全部字段的三分之二。

81 个非孤立字段被划为 19 个模块，最大模块含 15 个字段，109 条边的平均 $|r|$ 为 0.79，模块成员与化合物族基本对应。最大的两块以脂肪酸乙酯、乙酸酯和发酵综合量为主，包括乙酸乙酯、辛酸乙酯、癸酸乙酯、乙酸苯乙酯、乙醇、甘油与酯类总量；随后是酒花倍半萜烯氧化物（石竹烯氧化物、蛇麻烯环氧化物 II、α-木罗醇等）、C_6—C_9 高级醇（3-甲基-2-己醇、1-辛醇、4-甲基-1-戊醇等）与单萜醇（linalool、α-松油醇、香茅醇）。酯类主要来自酵母的酯合成，酒花萜烯随酒花添加进入，高

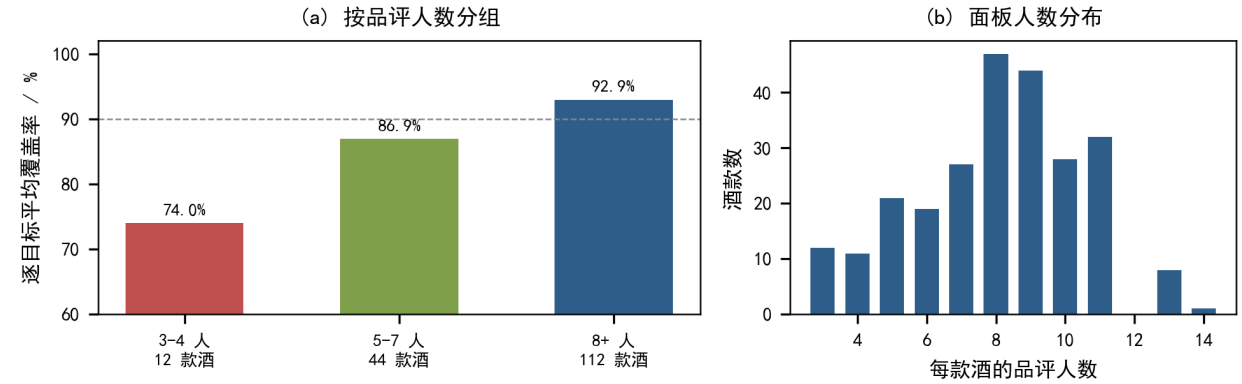


图 8　按品评人数分组的保形区间覆盖率与面板人数分布。覆盖率随人数增加而上升，3—4 人组仅 74.00%，边际覆盖不能保证小面板目标的条件覆盖。

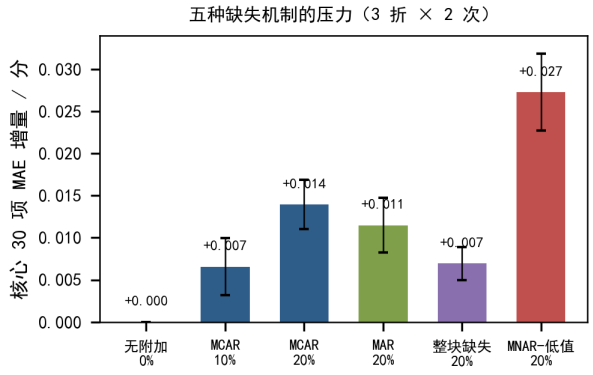


图 9　五种缺失条件下的核心 30 项 MAE 增量（基准 ET-300，0.50666）。系统性低值缺失的代价约为随机缺失的两倍；同为 20% 时，整块通道缺失的代价最小。

级醇是发酵副产物，模块因此沿原料与代谢途径分层；同类化合物共享前体与合成途径，这是高相关只出现在少数族内的化学原因。

109 条边全部为正相关，最小 r 为 0.703，本数据中没有出现 $|r| \geq 0.7$ 的负相关字段对。有符号权重保留了方法的一般性，但“负相关边鼓励系数反向”的机制未被触发，后文的图惩罚实际只做同向平滑。

图 10(a) 把邻接矩阵按模块排序，非零元集中在少数对角块内，块外接近空白；图 10(b) 给出模块规模，除前几个模块外，其余模块只含 2—3 个字段。同类化合物由同一原料或同一代谢途径产生，因而同向共变，这与以化合物谱预测复杂风味的既有结果一致 [1]。孤立字段并非无关紧要，它们只是缺少 $|r| \geq 0.7$ 的强共变伙伴，其系数只能由自身与目标的关系决定；孤立字段的拉普拉斯行恒为零，图惩罚只能在 81 个字段内部传递信息。

以上网络在全部数据上估计，用于描述。

阈值决定图的疏密。把 $|r|$ 的门槛由 0.6 提到 0.9，边数从 266 条降到 9 条，非孤立字段从 130 个降到 15 个（表 10）。0.7 是折中：再低会把经由第

表 10　相关阈值对网络结构的影响（231 个字段）

阈值 $\lvert r\rvert$	边数	连通分量	孤立字段
0.6	266	114	101
0.7	109	169	150
0.8	39	207	194
0.9	9	222	216

三变量产生的间接关联连进来，模块结构被稀释；再高只剩 9 条边，94% 的字段成为孤立点，图惩罚几乎无处可施。0.7 用于全数据网络的描述；模型验证时在每个训练折重算相关图，并在内层候选阈值 0.6、0.7、0.8 中选参。

建模时相关图在每个训练折内单独重算，测试折不参与，图结构与评价样本保持分离。

5.2　图约束模型与可解性

图约束模型在 $\alpha > 0$、$\lambda \geq 0$ 时严格凸、解唯一，三种图设定之间的误差差异因此来自惩罚项中的图结构，而不是求解过程。

记标准化后的化合物矩阵为 $Z \in \mathbb{R}^{250\times 231}$，中心化后的感官均分矩阵为 $Y_c \in \mathbb{R}^{250\times 50}$，待估系数矩阵为 $B \in \mathbb{R}^{231\times 50}$。图用带符号权重矩阵 W 表示，拉普拉斯矩阵取 $L = \mathrm{diag}(\sum_j |W_{ij}|) - W$，模型为

$$\hat{B} = \arg\min_{B} \Big\{ \|Y_c - ZB\|_F^2 + \alpha \|B\|_F^2 + \lambda\, \mathrm{tr}(B^T L B) \Big\}. \tag{7}$$

式 (7) 的第一项是拟合误差，第二项是岭惩罚，第三项让正相关邻边的系数接近、负相关邻边的系数方向相反；$\lambda = 0$ 时模型退化为普通岭回归。λ 增大时，系数更受带符号邻边的约束；若负边构成符号不平衡的环，图惩罚的零空间不必保留每个分量的一个自由度。α 控制整体收缩的强度，λ 控制

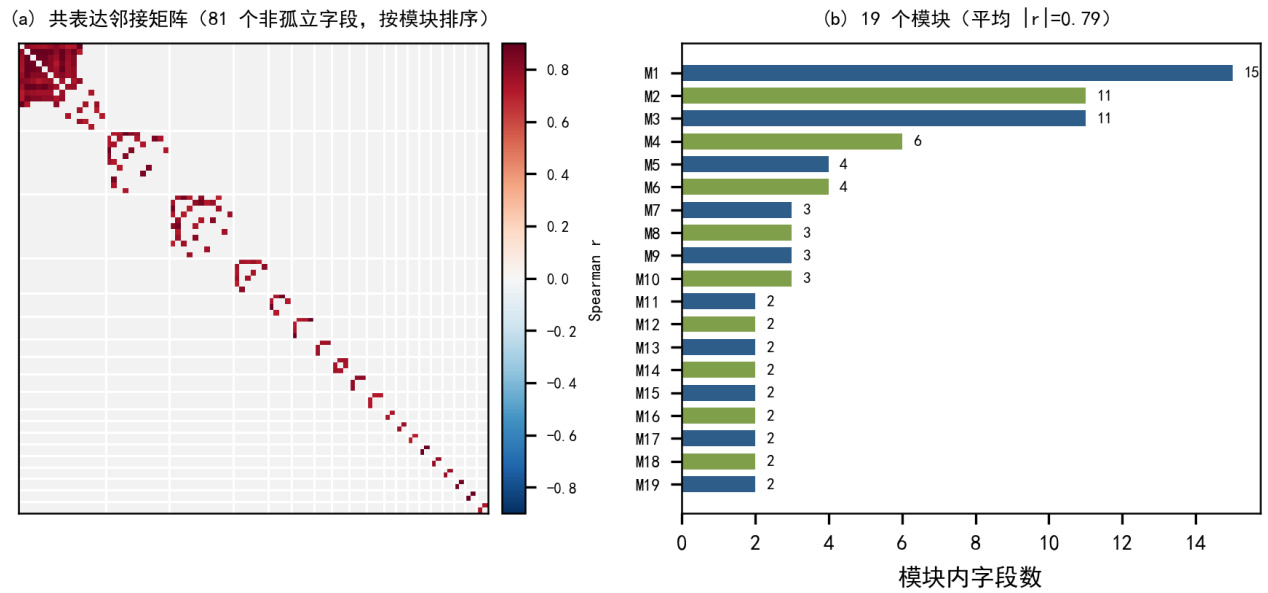


图 10　共表达网络的结构。(a) 81 个非孤立字段的邻接矩阵按模块排序，非零元集中于少数对角块，块外基本空白；(b) 19 个模块的规模，除前几个模块外均只含 2—3 个字段。

沿边差异的强度，两者共同决定模型的有效自由度，这也是二者需要在折内同时选择的原因。

对任一系数向量 b，有

$$b^{T}Lb=\sum_{i<j}|W_{ij}|\,(b_i-\operatorname{sign}(W_{ij})b_j)^2\geq 0. \tag{8}$$

式 (8) 可由 $b^{T}Lb=\sum_i\sum_j|W_{ij}|b_i^2-2\sum_{i<j}W_{ij}b_ib_j$ 逐项配方得到。右边每个平方项非负，故 L 半正定；等号成立当且仅当在 W 的每个连通分量内 $b_i=\operatorname{sign}(W_{ij})b_j$，即系数沿边的符号保持一致。

由式 (8)，λ 项是 B 的凸函数；$\alpha>0$ 使 $\alpha\|B\|_F^2$ 的 Hessian 为 $2\alpha I$，目标函数强凸，式 (7) 因而有唯一解。等价地，正规方程 $(Z^{T}Z+\alpha I+\lambda L)B=Z^{T}Y_c$ 的系数矩阵正定。若 $\alpha=0$，带符号 L 的零空间由各边的符号约束决定，唯一性取决于 Z 与 L 的共同零空间；本文字段存在强共线，正岭惩罚因此不可省去。50 个目标共享同一个 L，惩罚不跨目标耦合，每个目标等价于一个单独的正则回归；图只改变同一目标内部不同字段的系数分配，不改变目标之间的相对难度。

参数与外层评价严格分离：α、λ 与相关阈值都在每个训练折内部由内层留出误差选择，外层测试折不参与；同一折内另取 10 个节点标签随机置换的图，按相同流程重建并重新选参，作为结构无关的零模型。零模型也经过调参，误差中包含选择波动，比较时按同一协议解读。图结构回归的惩罚形式取自图结构变量的回归与选择 [8]。

5.3　双口径结果

真实图的差值随评价口径变号：单折口径比无图差 0.000124 分，五次预测聚合后好 0.001283 分。结合效应量和最终全数据选参，现有证据不支持图正则具有稳定且实际意义明确的预测优势；网络主要用于描述化学共变结构。

两种口径评价不同对象。单折口径把 15 个外层折的留出 MAE 再平均，每次拟合只产生一次预测，对应“训练一次即上线”的误差；聚合口径先对同一款酒的五次留出预测取平均，再在 250 款酒上计算 MAE，对应“重复建模、预测取均值”的误差。前者的样本单位是折，后者是酒；前者保留单次拟合的方差，后者通过平均削去一部分方差。同一批预测在两种口径下给出相反符号，说明图的效应与汇总方式有关，不能脱离口径单独陈述；两种口径对应不同的部署假设，并列报告才能覆盖两种使用场景。

表 11 给出核心 30 项的结果。单折平均为无图 0.536911、真实图 0.537035、打乱图 0.537794；聚合预测为无图 0.526148、真实图 0.524865，差值 -0.001283，固定预测条件下以酒为单位重采样 2000 次的 95% 区间为 $[-0.002341,-0.000229]$。该区间以固定留出预测为条件，不包含重训与选参的不确定性；区间未跨 0，但误差差值仅 0.001283 分，实际意义需要结合任务精度判断。

打乱图作为结构对照，在单折口径下平均误差最高；该结果本身不能分离随机图平滑和选参造成的差异。真实图与无图的差距很小。效应小有结构性原因：150 个字段是孤立节点，其系数行不受图惩罚；15 个外层折中只有 7 折选到正的 λ，其余 8

表 11　核心 30 项在图模型三种设定下的 MAE

评价口径	无图	真实图	打乱图	真实图减无图
单折平均（15 个外层折）	0.536911	0.537035	0.537794	+0.000124
聚合预测（按酒平均）	0.526148	0.524865	—	−0.001283

折的 $\lambda = 0$，模型退化为岭回归。图只在不到一半的折里对 81 个字段中的一部分起作用，最优强度在多数折内被内层准则判为零。

聚合口径把每款酒的五次预测平均。重复划分的训练集相互重叠，五次预测存在相关性，平均后的方差不能假定缩减至五分之一；本实验没有分解图惩罚对偏差与方差的影响。

全 50 项的单折结果为无图 0.470061、真实图 0.470220、打乱图 0.471037，排序与核心 30 项一致，差异同样在万分位。最终描述模型由全部数据上的三折内层验证按核心 30 项 MAE 选择参数，重算网格中 $\alpha = 1000, \lambda = 0$ 的内层误差为 0.539083；最佳正图惩罚组合为 $\alpha = 1000, \lambda = 1, \tau = 0.8$，误差为 0.539086。选参依据是内层误差和既定网格排序，不是使用外层聚合误差或重采样区间直接选取 λ。最终用全部数据拟合的描述模型选得 $\lambda = 0$，Top-3 由岭回归系数给出。图在本文中承担解释功能，不是预测模型的组成部分；把它写成“提升了预测精度”会与最终选择矛盾。

5.4　Top-3 的稳定性

150 条候选中只有 36 条同时满足频率、折数与置换贡献三项条件；Top-3 对单元格级异常稳健，对变量集合敏感，因此不能读作变量选择结论。

对 50 个目标各取标准化岭回归系数绝对值最大的 3 个字段，共 150 条记录。以酒为单位做 200 次 bootstrap，同一款酒的各目标一起进出，目标间的相关性不被打破；每次重建预处理与相关图，并固定全数据选定的 α、λ 与阈值，记录字段是否仍在该目标的前三位，得到入选频率 [12]；同时统计 15 个外层折中的入选次数，以及留出数据上普通置换与风格内置换的误差增量。支持条件为频率不低于 0.5、至少在 5 个外层折入选、两种置换的平均贡献均为正，三项分别对应重采样稳定性、跨折稳定性与预测贡献。

单看频率有 46 条不低于 0.5，且都至少在 5 折入选；再要求两种置换贡献为正，剩 36 条，被剔除的 10 条都是置换贡献不稳定的记录。36 条候选分布在 29 个目标上：22 个目标各 1 条、7 个目标各 2 条，其余 21 个目标没有任何候选通过。图 11(a) 给出 150 条候选的入选频率分布，0.5 的门槛把候选分成通过与否两组，0.2—0.4 区间仍聚集了相当一部分记录，这些字段只是偶尔进入前三；图 11(b) 按目标给出通过比例，空白目标即没有候选通过。频率以固定参数为条件；若在每次重采样中重新选参，评估的是完整选择流程的稳定性。0.5 是本文采用的描述性门槛，不是显著性水平，门槛附近的频率差别不宜过度解读。

化学异常敏感性见表 12。把 80 个异常单元格置为缺失、仅用训练折估计填补后，核心 30 项 MAE 由 0.50641 变为 0.50691，全 50 项由 0.44478 变为 0.44515，预测值平均绝对变化 0.00500 分，Top-3 平均保留 97.64%。仅使用第 3.2 节所定义的 V（38 个逐点一致字段、61 个单位登记字段及 226 个非综合字段的交集）时，固定 $\alpha = 1000$ 的 15 折岭回归 Top-3 平均保留 36.18%。该 V 与含 80 处异常的 32 个字段完全不同，交集为空。

两类扰动都改变输入，性质不同。前者只抹掉 80 个单元格，受影响字段仍由其余酒样支撑，系数排序几乎不动；后者把 226 个非综合字段压到交集 V 的 32 个字段，候选池本身换掉，Top-3 只剩约三分之一。稳定的 Top-3 是“在当前字段集合与当前模型下稳定”，不是变量选择结论；换一套输入，排名会重排。

5.5　化学解释及其边界

三个例子分属三种证据强度：同名端点上的近确定性关联、机制清楚但经由未观测过程的关联、文献支持而统计支持不足的关联。系数、相关与入选频率都是关联量，单独任何一项都不构成因果证据。

X4VG 与 X4vg 的关系最强。X4VG 在该端点上的入选频率为 1.00，15 个外层折全部入选，系数为正。4VG 是阿魏酸经酵母脱羧的产物，赋予丁香与酚类香气，储藏中会下降并转化为香草醛等物质 [13]。这一条的证据链最完整，但端点本身以该化合物命名，关联接近定义对应，属于模型复现命名关系，而不是独立发现。

iron 与 metallic 的关系次之。iron 在 metallic 上的入选频率为 0.94，15 个外层折全部入选，系数为正。铁不是香气化合物，金属感来自口腔中脂质氧化与铁离子的催化过程 [14]；模型看到的是铁含量高的酒更容易被评出金属感这一共变，催化链本身没有进入模型。铁还与水质、设备与风格相关，当前设计无法把铁从这些过程变量中分离。iron 同时进入酒花香气、苦味与甜味等性质不同的端点，这种跨端点重复更像风格或工艺的代理变量，而不是某一感官属性的专属因素。

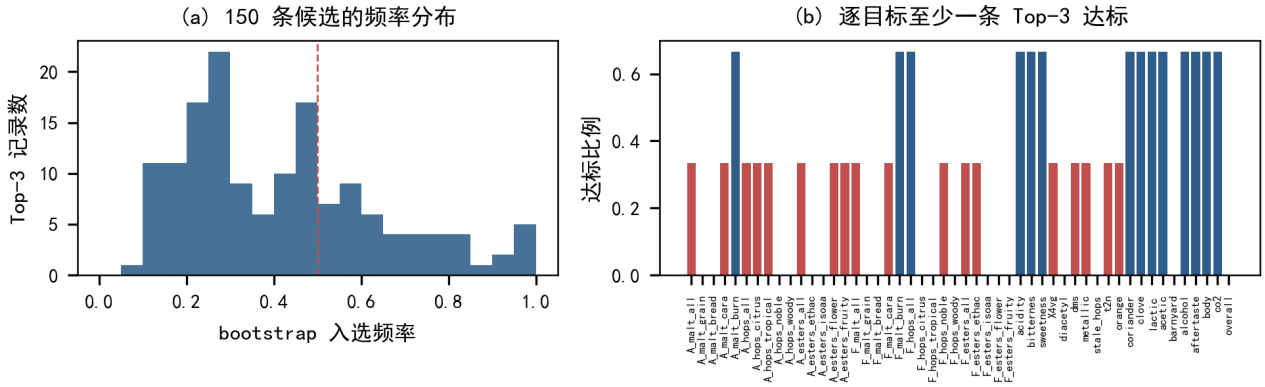


图 11　Top-3 的稳定性。(a) 150 条候选的 bootstrap 入选频率分布，虚线为 0.5 的门槛；(b) 逐目标通过支持条件的候选比例，空白目标表示没有候选通过。

表 12　Top-3 的支持条件与化学异常敏感性

检验项目	数值	备注
通过支持条件的记录	36 / 150	频率 ≥ 0.5、≥ 5 折入选、置换贡献为正
核心 30 项 MAE（异常置缺失前 → 后）	0.50641 → 0.50691	分
全 50 项 MAE（异常置缺失前 → 后）	0.44478 → 0.44515	分
预测值平均绝对变化	0.00500	分
Top-3 平均保留（异常置缺失）	97.64%	—
Top-3 平均保留（仅交集 V 的 32 字段）	36.18%	—

linalool 与 coriander 的关系最弱。linalool 以 0.46 的频率列第三，低于 0.5 的门槛，只在 8 个外层折入选。芫荽籽是小麦啤酒的传统香料，其添加阶段与用量会改变啤酒中的 linalool 含量 [15]，文献给出的来源关系与正系数方向相容，但统计支持没有过线。文献支持与重采样稳定性是两个独立维度，前者不能替代后者。置换贡献衡量字段取值对留出误差的作用，贡献为正只说明该字段携带预测信息，不说明它通过化学途径影响感官。

共线化合物会在系数上互相替代。同一模块内的字段通过 $|r| \geq 0.7$ 的边直接或间接相连，相关变量可能在系数排序中相互替代；模块级联合置换结果提供补充证据，但不能预设所有模块的系数和都稳定。可信的解释应落在模块层面，例如酒花倍半萜烯氧化物或脂肪酸乙酯这样的族，单个化合物名称只作为该族的代表。按模块报告时，“该族整体为正”与“族内某个化合物排名靠前”需要分开陈述，前者对共线替代不敏感。附件 8 的 GBM SHAP 与 MDI 来自原始研究的模型，本文只将其作为外部参照核对化学方向，不计入本队结果 [18]。可检验的解释顺序是：先看模块是否与原料或工艺对应，再看代表化合物的方向与文献是否一致，最后用独立批次验证。

6　第三问：风格生成与迁移可行域

第三问要做两件性质不同的事：在给定风格下生成新的化合物—感官谱，以及判断一款现有酒能否改造成目标风格。本文把二者拆成生成层与判别层：生成层回答哪种采样最像真实酒，判别层回答哪条改造路径在化学上站得住。总的判断是，生成质量无法用单一指标排序；迁移可行域主要由 DMS 与反式-2-壬烯醛卡住，且对阈值高度敏感。

6.1　风格条件生成与评价体系

生成质量在不同评价维度上会给出相反的排序，因此本文的生成体系从一开始就把分布接近与分类器认可分开记录。三个角色的划分互不重叠，是为了让生成器用过的酒不出现在评价里：生成器只用 G，评价器只用 J，真实参考只用 T。同一批 250 款酒在五次划分中被重复使用，因此后文的比较都是划分内比较，不构成外部验证。

生成对象取 Tripel（28 款）、Kriek（10 款）与 Lambic（11 款）。前者的样本量足以支撑 40 条谱的分布比较，后两者稀少，正好用来观察少样本风格上的退化；三个风格的样本量跨度覆盖了够用与不够用两种情形。每次划分把 250 款酒分成互不重叠的 G（生成器训练，100 款）、J（评价器训练，100 款）与 T（留出检验，50 款）；四种方法在每种风格上各生成 40 条谱，五次划分合计 2400 条。生成器只看到 G 中该风格酒样的 226 个非聚合化

学字段。Kriek 与 Lambic 每划分进入 G 的只有 4 至 5 款、进入 T 的真实参考只有 2 款，Tripel 则为 11 款与 6 款，这一不对称决定了后文的读数方式。

四种方法对应四种关于配方空间的假设。重采样有放回地抽取 G 中的同风格酒样，不学习任何结构，是必须被击败的基线 [12]；它不产生新组合，却是唯一保证生成点落在真实数据流形上的方法，任何生成器若在分布距离上赢不过它，就说不上学到了结构。局部凸组合假定新配方落在同风格近邻的连线上：

$$x_{\text{new}} = u\,x_a + (1-u)\,x_b, \qquad 0 \le u \le 1, \qquad (9)$$

其中 x_a, x_b 为 G 中同风格的两个酒样。凸组合对应配比微调这一工艺动作，生成点不会超出数据支撑。受约束局部凸组合在 (9) 之上要求生成点到最近同风格训练样本的距离不超过留一支撑半径，并在对数尺度上混合 DMS 与 4VG、裁剪到上下界。条件 PCA 高斯在 G 的风格内做主成分分解后于主成分空间拟合对角高斯并采样，允许生成点离开凸包，更容易产生训练集中没有的组合。

评价从生成器外部取四个角度。分布距离用固定训练带宽的 RBF 核最大均值差异 MMD^2[11]，真实参考为 T 中该风格酒样的非缺失测量；KS 统计量逐字段计算后取均值，刻画边缘分布的平移；精确复制比例统计与 G 中某训练点距离小于 10^{-8} 的生成谱占比；独立评价器是只在 J 上训练的风格分类器，报告它赋予目标风格的平均概率与判为目标风格的比例。四项指标共用同一套预处理与核带宽，尺度差异不会被误读成分布差异。KS 只看单字段的边缘分布，对字段之间的相关性不敏感；MMD^2 在多维表示上一次比较整个联合分布，两者一起看才能区分单字段偏移与整体结构偏移。

分类器识别率高与分布更接近是两件不同的事。分类器只关心生成点落在决策边界哪一侧，对风格内部的协方差、尾部厚度与稀有组合不敏感；MMD^2 关心整个分布在再生核空间中均值嵌入的位置，对生成物挤向风格中心这类退化格外敏感。一种方法完全可以把样本堆到最典型的 Tripel 附近换来高识别率，同时因丢掉离散度而让 MMD^2 变大。本文因此不设综合评分，只并列报告四个维度。

6.2 生成结果

四种方法在三个风格上互有胜负，没有一种全面占优；重采样在分布距离上仍是难以击败的基线，但它的精确复制比例恒为 1。

表 13 汇总五次划分的平均结果。Tripel 上受约束局部凸组合的平均目标风格概率 0.3358 高于重采样的 0.2992，判为 Tripel 的比例 0.695 也高于 0.565，但它的 MMD^2 为 0.1158、KS 均值 0.4463，两项都劣于重采样的 0.1034 与 0.4245。同一风格上出现了方向相反的评价：按判别边界看受约束凸组合更像目标风格，按分布看重采样离真实酒样更近。重采样在两个距离指标上同时最好，说明它逼近真实分布靠的是直接复制，而不是学到了结构。约束没有换来指标收益，无约束版本的 MMD^2 为 0.1135、平均目标风格概率 0.3432，两项都略好；约束的作用是保证生成谱不越界。

反向的例子同样清楚。条件 PCA 高斯在 Tripel 上的 MMD^2 为 0.1088，仅次于重采样，但独立评价器给它目标风格的概率只有 0.1499、判为目标风格的比例 0.285，两项都最低。这可能与对角高斯忽略主成分之间的相关性、生成点向风格中心收缩有关，结果是谱形趋于平均、反而越过分类边界。距离近而识别率低与识别率高而距离远，指向同一件事：两个指标度量的不是同一个东西。

Kriek 与 Lambic 上的距离整体大了一倍，落在 0.21 至 0.30 之间，四种方法都没有把这两个稀薄风格学到可用程度，而每划分只有 2 款留出酒，抽样误差不可忽略。这两个风格的化学谱离散度大、样本又少，40 条生成谱难以覆盖其变异范围。KS 在 Tripel 上集中在 0.42 至 0.46，在 Kriek 与 Lambic 上集中在 0.62 至 0.66；逐字段取平均会稀释单字段的尾部偏离，因此 KS 更适合判断整体平移，不适合判断尾部风险。Kriek 上条件 PCA 高斯最低（0.2832），重采样 0.2871；Lambic 上条件 PCA 高斯 0.2095、受约束凸组合 0.2181、重采样 0.2226，三者相差不足 0.014，识别率却相反，条件 PCA 高斯只有 0.2091。这两个风格上的读数只能支持重采样没有被稳定击败，不能支持任何方法的优越性。

覆盖度从另一侧描述生成质量。对每个真实酒样，计算它到最近生成点的标准化欧氏距离，再除以它到最近真实酒样的距离，得到相对覆盖比；比值小于 1 表示该真实点到最近生成点的距离小于其到最近真实点的距离。重采样最小（0.60–0.70），两种凸组合居中（0.61–0.73），条件 PCA 高斯最大（0.87–0.93），见表 14。条件 PCA 高斯在 Kriek 与 Lambic 上的 MMD^2 较低，但相对覆盖比更大；两种指标反映生成分布与点对点距离的不同方面，不能仅凭这些结果推断差异的成因。重采样复制率为 1；凸组合产生新组合；条件 PCA 高斯的相对覆盖比最大。

复制率给出了最干净的分界：重采样恒为 1，两种凸组合与条件 PCA 高斯恒为 0。复制率为 1 意味着每条生成谱都是训练集中已有的酒，作为生成新配方没有提供新组合，这正是重采样距离更近却不适合交付的原因。凸组合的复制率为 0，说明它们给出的是新的配比组合，代价是可能落在真实酒样稀疏的区域。

表 13　三种风格下四种生成方法的分布距离、边缘偏移、复制比例与独立评价器判断（五次划分平均；MMD^2 与 KS 越小越好，复制率为与训练点完全重合的生成谱比例，后两列由只在 J 上训练的分类器给出）

风格	方法	MMD^2	KS 均值	复制率	评价器概率	判为目标风格
Tripel	重采样	0.1034	0.4245	1.00	0.2992	0.565
	局部凸组合	0.1135	0.4448	0	0.3432	0.755
	受约束局部凸组合	0.1158	0.4463	0	0.3358	0.695
	条件 PCA 高斯	0.1088	0.4605	0	0.1499	0.285
Kriek	重采样	0.2871	0.6208	1.00	0.5275	0.800
	局部凸组合	0.3005	0.6511	0	0.5779	0.955
	受约束局部凸组合	0.2988	0.6422	0	0.5703	0.950
	条件 PCA 高斯	0.2832	0.6307	0	0.3101	0.595
Lambic	重采样	0.2226	0.6257	1.00	0.4924	0.885
	局部凸组合	0.2206	0.6602	0	0.5543	0.975
	受约束局部凸组合	0.2181	0.6453	0	0.5598	0.995
	条件 PCA 高斯	0.2095	0.6324	0	0.2091	0.385

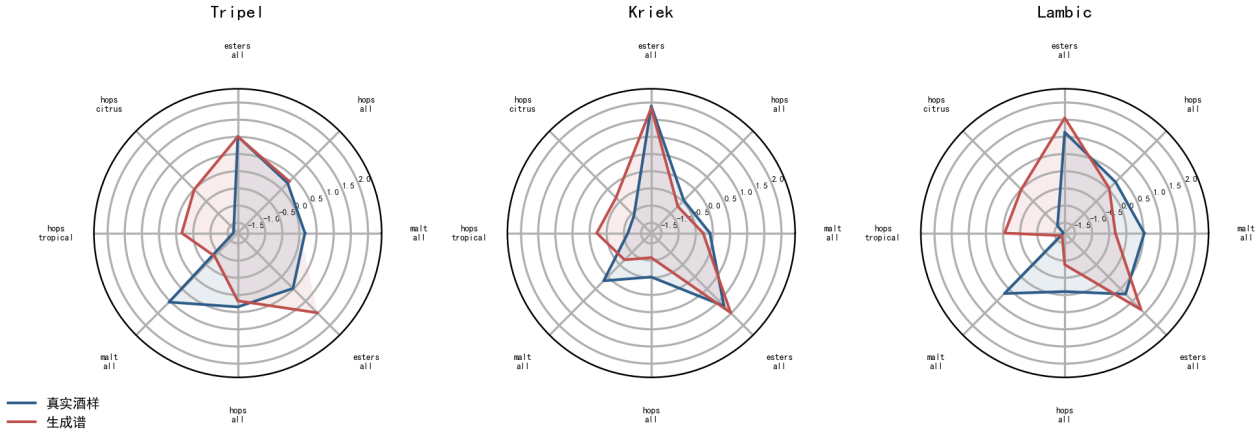


图 12　三种风格的真实酒样与生成谱在 8 个标准化感官维度上的相对强度。纵轴为按 8 维标准化后的相对强度，可正可负，因此比较的是曲线形状而不是绝对得分；生成谱的感官画像由第一问的预测模型给出，是预测值而非新酿酒的实测评价。

表 14　生成集的相对覆盖比（真实酒样到最近生成点距离 / 到最近真实酒样距离）

方法	Tripel	Kriek	Lambic
重采样	0.698	0.653	0.600
局部凸组合	0.729	0.648	0.606
受约束局部凸组合	0.711	0.655	0.634
条件 PCA 高斯	0.919	0.874	0.933

图 12 的曲线在形状上大体重合，没有整体平移或维度错位，差异集中在酒花与麦芽两族维度，说明凸组合把邻近样本的谱形带了进来。

6.3　迁移搜索与冻结候选

迁移的可行性在三个证据层级上依次衰减，搜索可行只是最弱的一层。

迁移任务给定一款 Pils/Lager 源酒，问能否通过调整配比把它推到 Tripel。候选由源酒与 G 内每个 Tripel 锚点之间的线性插值生成，每个锚点取 30 档权重，逐点裁剪回给定上下界；每条记录由此产生 30 档乘锚点数的候选，绝大多数会在下一步被淘汰。筛选条件有五条：标准化空间的均方根改动不超过半径 1；到最近 Tripel 训练点的距离不超过其留一支撑半径；目标风格概率不低于 0.5，且比源酒与仅裁剪对照高出 0.01；四项缺陷不增；两个 G 感官模型给出的 stale hops 代理不增。

半径按标准化特征的均方根改动计量，半径 1 约等于各字段平均改动一个标准差，属于配方微调的量级。半径 0.25、0.5、1、2 都跑过；半径变大时，候选更多，但改动也更大，本文只报告半径 1，其余半径用于检查结论是否随半径翻转。通过的候选再按改动量、目标距离与已知阈值超额做 Pareto，取改动最小者，以免为了风格标签牺牲过多化学余量。半径之外的候选不进入排序，因此本文报告的是在可接受改动内能做到什么，而不是最优改动有多大。

搜索模型使用 G 训练，候选冻结后用 J 训练的模型复核；T 是从同一历史数据集中划出的内部

表 15　半径 1 下迁移候选的三级通过情况（55 条来源与划分记录；后三列依次为 G 搜索可行、J 独立代理支持、以及在原始 0–6 尺度上 stale hops 代理同样不增）

记录集	条数	G 可行	J 支持	原始尺度后
全部来源	55	51	11	6
其中 T 留出（7 款酒）	10	10	2	1

留出酒。全部来源结果包含 G、J 和 T 来源，其中 J 来源酒参与了 J 模型训练；仅 T 来源在两类模型训练之外，仍不是新的外部样本。半径 1 下 55 条来源与划分的组合记录（11 款 Pils/Lager 源酒乘 5 次划分）见表 15。

G 上 55 条中有 51 条找到可行候选，换到 J 的独立模型后只剩 11 条，补上原始 0–6 尺度的 stale hops 不增后只剩 6 条。T 留出的 10 条记录（7 款酒）在同样三级口径下依次为 10、2、1。第一段衰减最大：G 的锚点与源酒共享训练信息，分类器判断偏乐观，换成 J 就掉了 40 条。第二段来自尺度：J 在标准化空间里的代理与原始尺度上的预测并不等价，补上一条后又有 5 条落选。T 的 10 条记录均在 G 上找到候选，其中 1 条通过全部代理条件；该组只有 7 款不同来源酒，不能据此确认训练来源与留出来源具有相同性能。

结论是，搜索可行不等于代理支持，代理支持也不等于真实品评。三层证据的强度依次递增，只能分层报告；把 51 读成 51 款酒改造成功，等于把跨模型的一致性问题当成实验结果。找不到候选时保留源酒并记录失败，不把源酒当作可行解。stale hops 是第一问模型对老化酒花的预测分，它的不增只表示两个模型在这一点上不矛盾，不构成感官实测。

6.4　严格浓度约束与不可行证书

在逐项浓度不增的严格协议（下称协议 A）下，55 条记录的最大 Tripel 份额全部为 0，且每一条都附带可验证的不可行证书；这把无解从数值结果变成了可定位到具体化合物的工艺诊断。

混配只使用 61 个有单位、有报告值的浓度字段，其余字段只有对数化的相对值，无法进入按质量守恒写出的混配式。以某款 Pils/Lager 为基液、G 中的 Tripel 酒样为锚点，混配后的浓度为

$$c(w) = c_0 + \sum_{j=1}^{m} w_j \,(c_j - c_0), \tag{10}$$

(10) 中的 c_0 为源酒浓度向量，c_j 为第 j 个锚点的浓度向量，$w_j \ge 0$ 为 Tripel 的添加份额，剩余 $1 - \mathbf{1}^{\mathsf{T}} w$ 为源酒。四项缺陷逐项不增的要求给出线性规划

$$\begin{aligned} &\max_{w} \ \mathbf{1}^{\mathsf{T}} w \\ &\text{s.t.} \ \ M w \le 0, \quad w \ge 0, \quad \mathbf{1}^{\mathsf{T}} w \le 0.75, \end{aligned} \tag{11}$$

M 的第 (d, j) 元为 $c_{j,d} - c_{0,d}$，份额上界 0.75 对应以现有酒为主体、只做局部调整；该上界是本文设定的搜索约束，0.75 本身不保证成品保留来源酒的工艺或身份。最优值同时是可行性判据：为正，就存在一条提高 Tripel 份额的配比；为 0，说明这个方向上没有任何余量。55 条记录在 (11) 下的最优值全为 0：对于各记录选定的 G 锚点，在所设线性混配及四项逐项不增约束下，任何正的 Tripel 锚点份额都会使至少一项缺陷高于源酒。

无解本身不算结论，能指出被谁挡住才算。对每条记录求解一个对偶证书：存在 $v \ge 0$ 使

$$\left(M_{\mathrm{norm}}^{\mathsf{T}} v\right)_j > 0, \qquad j = 1, \dots, m, \tag{12}$$

其中 M_{norm} 按行最大值归一化，用于消除四项缺陷的量纲差异；按行归一化只改变每项缺陷的尺度，不改变不等号方向，因此证书的结论与原尺度上的线性规划一致。证书的本质是给出一组非负权重，使按该权重加权的四项缺陷偏离在任何正份额下都为正。若 (12) 成立，则任意 $w \ge 0$、$w \ne 0$ 有 $v^{\mathsf{T}} M_{\mathrm{norm}} w = \sum_j (M_{\mathrm{norm}}^{\mathsf{T}} v)_j w_j > 0$，而 $M_{\mathrm{norm}} w \le 0$ 在 $v \ge 0$ 下又要求该内积非正，矛盾，故正份额不可行。55 条记录全部找到这样的证书，归一化余量 $\min_j (M_{\mathrm{norm}}^{\mathsf{T}} v)_j$ 介于 0.0002 与 0.2323 之间，中位数 0.1094。余量越小，约束越贴身。

消融把结论落到化合物上。逐个删除四项约束后，允许正份额的记录数分别为 30、4、0、4 条，对应删除 DMS、双乙酰、4VG 与反式-2-壬烯醛。删 4VG 一条也没有放开，说明它从不单独构成阻塞。按阻塞结构归类，DMS 单独阻塞 14 条、壬烯醛单独阻塞 15 条、两者联合阻塞 6 条、双乙酰 4 条，另有 16 条必须同时放松多项约束才能放开。配方调不动由此变成一句可执行的工艺语言：先把 DMS 与壬烯醛通路降下来，再谈风格迁移。

6.5　阈值风险协议与联合复核

可行域对缺陷阈值的取值极其敏感，把阈值写成可调参数而不是常数，才是可检验的做法。

题面要求的是超阈值项不恶化，而 (11) 的协议 A 要求逐项浓度不增，比题意强得多。协议 A 先于协议 B 被采用，是因为它不需要阈值，只依赖测量本身。协议 A 要求全部四项浓度不超过来源；协议 B 在来源低于阈值时允许浓度上升到阈值，在来源高于阈值时要求浓度不超过来源。为此定义风险函数

$$R_d(c) = \max\left(\frac{c}{t_d} - 1,\ 0\right), \tag{13}$$

表 16　四项化学风险的参考阈值来源

字段	参考阈值	来源及基质
DMS	19 μg/L	附件 8 阈值中位数；基质混合
双乙酰	0.09 mg/L	附件 8 阈值中位数；基质混合
4VG	0.20 mg/L	附件 8 阈值中位数；基质混合
反式-2-壬烯醛	0.1 μg/L	啤酒文献参考 [16, 17]

按 (13) 的定义，协议 B 要求 $R_d(c') \le R_d(c)$，等价于 $c' \le \max(c, t_d)$：低于阈值时以阈值为界，高于阈值时以原值为界。参考阈值取 DMS 19 μg/L、双乙酰 0.09 mg/L、4VG 0.2 mg/L（均取 8.xlsx 对应字段阈值的中位数）与反式-2-壬烯醛 0.1 μg/L（8.xlsx 无该字段阈值，取文献值 [16, 17]）。中位数混合了不同基质与前处理方法，只能当作数量级参考。

这四项是浓度代理，stale hops 另由历史面板均分预测；附件中三项中位数的原始测量基质不统一，不能视作待制备批次的标定阈值。文献的 0.1 μg/L 适用于所述啤酒品评与老化分析条件，个体和基质差异仍需要测量确认。

将四个阈值统一乘以 0.5、1、2 后重解 (11) 并施加协议 B，允许正份额的记录数为 15、30、34。阈值收紧一半会把可行记录压到不足三分之一，放宽一倍却只多放开 4 条。图 13 把化学可行与代理复核两层放在一起。以 1 倍参考阈值为主要分析口径，0.5 倍和 2 倍仅用于考察阈值设定的敏感性。

在化学可行的记录上叠加风格与 stale hops 约束，三层漏斗在 0.5 倍、1 倍、2 倍阈值下依次为：G 找到 0、1、31 条候选，J 复核后剩 0、0、9 条。阈值放宽到 2 倍时化学可行 34 条，其中 31 条能在 G 上找到风格候选，最终只剩 9 条通过 J，落差主要发生在代理层而不是化学层。联合复核把化学可行、风格可达与老化代理放在同一批记录上考察，任何一层不通过都不计为候选。2 倍阈值下的 9 条是代理模型支持的重复划分记录，不是 9 款完成迁移的啤酒；其中 3 条来自 T 内部留出、对应 3 款来源酒。即便如此，最松的设置下 55 条记录也只产出 9 条通过两层代理的候选，不足六分之一，且都未经过真实品评。

阈值本身并不是常数。8.xlsx 的 261 行阈值只有 43 个给出数值，口径与基质（水、麦汁、成品酒）并不统一，文献值也依赖具体的品评方法。本文因此把阈值写成参数 t_d 参与敏感性分析，而不是取固定数宣布可行。在这个设定下，第三问的答案是一个条件命题：逐项不增的严格协议下可行域为空且可被证书化；超额不增协议下部分记录的化学约束允许正份额；参考阈值下当前有限搜索没有候选通过联合代理复核；放宽阈值只能把这条窄缝拓宽，改变不了只有极少数配比能同时满足风格与缺陷的结构。

7　模型评价与推广

7.1　方法上的可取之处

第一，评分端点不是” 拍一个阈值” 得到的。本文先从公开的 2197 条逐人评分重建面板均分与七级分布，再把连续与有序两类目标分开建模，使” 预测均值” 和” 预测等级” 各按自己的损失函数评价。第二，所有模型共享同一批按酒分层的划分，插补、截尾与标准化只在训练折内估计，因此不同方法之间的差值可比。第三，失败被完整保留：风格中位数基线几乎追平融合模型、图正则的收益随汇总口径变号、非零等级识别率只有约四分之一、严格浓度协议下 55 条记录的目标混配份额均为零——这些结果没有一项被删去或改写。

第四，本文把” 无解” 做成了可解释的结论。通过对偶可行性给出正余量证书，再逐项删除约束做消融，可以把笼统的” 迁移不可行” 定位到 DMS 与反式-2-壬烯醛这两个瓶颈上；同一套框架在参考阈值协议下得到 30 条四项化学约束允许正份额的记录；联合风格与 stale hops 代理检查后无候选通过。这比再多试几个生成模型更有信息量。

7.2　局限

L1 没有真实的感官验证。全部结论建立在历史品评数据之上，生成谱与迁移候选的感官画像都是模型预测值。要证明一款候选酒真的好喝，需要新批次的化学测量与配对盲评，本文只给出了可执行的试验方案。

L2 品评人员在训练与测试之间重叠。按酒划分能回答” 新酒款”，不能回答” 新面板”；16 名品评员的构成决定了结论的适用范围。人员留出设计尚未实现。

L3 缺失机制只做到压力测试。MCAR、MAR、MNAR 与整块缺失四种机制给出了误差增量的量级，但没有建立与仪器检出限对应的缺失模型，因此 MNAR 的结果只能作为风险提示，不能外推为具体批次的偏差。

L4 化学侧未完成标定。70 个可对照字段中 32 个存在 80 处不一致，检出限与回收率未知，5 个综合量的聚合公式未复原。第三问的浓度域分析因此限定在 61 个有单位记录的字段上。

L5 小样本生成难以全面胜出。每种风格只有 3–31 款酒，约束凸组合提高了目标风格的识别率，却离真实分布更远；没有一种方法在保真度、覆盖度与复制率三个维度同时优于重采样。

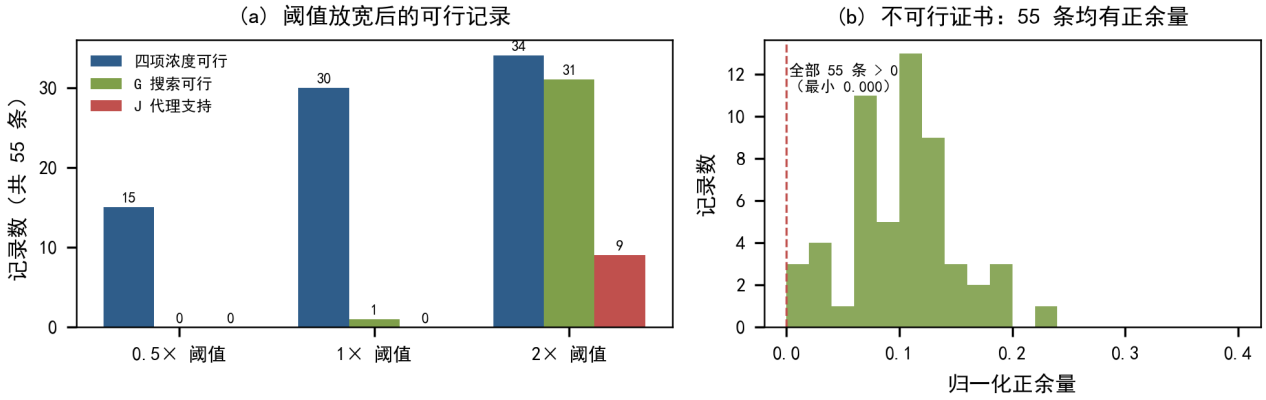


图 13　(a) 缺陷阈值取 0.5 倍、1 倍、2 倍时，55 条记录中四项浓度可行、G 上搜索可行与 J 代理支持的条数；(b) 55 条不可行证书的归一化余量分布，全部大于 0。左图显示可行条数随阈值放宽而非线性增长，右图显示每条不可行记录都带正余量，即所选锚点和线性约束下正份额不可行。

7.3　可推广的部分

代码层面，本文提供了两个可直接复用的入口：一是训练与无标签预测分离的流水线，输入只需与附件 1 一致的 231 列化合物字段，允许缺失、允许列序不同，输出面板均分、众数等级、中位数等级与七级概率四张表，并附带字段顺序、单位与输入哈希；二是”冻结候选—独立复核”的评测协议，可套用到任何”生成候选 + 模型打分”的任务上。

方法层面，本文的三点做法具有一般性。其一，当标签经过中心化/标准化而原始量表不可见时，先追查上游原始评分再重建端点，比在标准化刻度上强行反变换更可靠。其二，把”约束是否可行”表述为线性规划并给出不可行证书，可以在小样本条件下把失败转成工艺诊断。其三，对同一批留出预测同时报告单次拟合与重复聚合两种误差，可以避免”图模型有效/无效”这类依赖汇总口径的误判。

应用层面，该框架可迁移到其他发酵饮品（葡萄酒、清酒、酱油）的风味建模：只要具备”化合物谱 + 感官面板 + 风格标签”三要素，第一问的端点重建、第二问的模块级解释与第三问的可行域分析需要结合新任务的标定、评分和验证条件重新设计。

8　结论

本文围绕啤酒风味的三项任务，先解决”评分端点从哪来”，再分别给出预测、解释与决策三类答案。

第一问：以 2197 条逐人 0–6 评分为依据重建了 50 个感官目标的面板均分与七级分布。按酒三折五次验证下，固定等权融合在核心 30 项上的均分误差为 0.50222 分，低于训练均值 0.60450 分，但只比风格中位数基线（0.50969 分）低 0.00747 分，差值的 95% 重采样区间为 $[-0.01914, 0.00418]$，含 0。该比较未提供融合模型稳定优于已知风格中位数基线的充分证据。新增同参数 ET 对照中，已知风格加化学比仅风格 ET 的核心 MAE 低 0.01426 分，表明在该模型设置下化学输入仍有预测信息；相对风格中位数的差值区间跨 0。七级分布模型的累计分布平方距离为 0.02178（总体基线 0.02820），面板众数一致率 80.52%，但非零众数一致率只有 24.33%：总体一致率主要由 78.03% 的零等级贡献，精细等级识别仍是短板。保形区间逐属性平均覆盖 90.06%、平均宽 1.7342 分，但按品评人数分组后，3–4 人组只有 74.00%。缺失机制实验进一步显示，系统性低值缺失（MNAR）使核心误差增加 0.0273 分，约为随机缺失 20%（0.0139 分）的两倍。

第二问：231 个字段在 $|r| \geq 0.7$ 下构成 109 条边的共表达图，但 150 个字段是孤立节点。图正则模型在单折口径下略差于无图（0.537035 对 0.536911）、在聚合口径下略好（0.524865 对 0.526148，区间 $[-0.002341, -0.000229]$），最终全数据模型选得 $\lambda = 0$，即 Top-3 实际来自普通岭回归。150 条候选中有 36 条通过”频率 ≥ 0.5、至少 5 折入选、置换贡献为正”的描述性条件；把 80 个化学异常单元格置为缺失后 Top-3 保留 97.64%，仅使用 38 个逐点一致字段、61 个单位登记字段和 226 个非综合字段的 32 项交集时降至 36.18%，该交集与 32 个异常字段不重合。36 条为满足描述性支持条件的候选，另外 114 条为探索性候选；网络用于描述共变结构，候选不代表因果效应。

第三问：三种风格各生成 40 条谱。受约束局部凸组合在 Tripel 上的平均目标风格概率为 0.3358、目标预测比例 0.695，均高于重采样的 0.2992 与 0.565，但分布距离 MMD^2 反而更大（0.1158 对 0.1034），说明”更像目标风格”与”更接近真实分布”不是一回事。迁移任务中，55 条”来源 × 划分”记录上搜索可行 51 条、独立代理支持 11 条、

再加原始尺度 stale hops 约束后 6 条；真实留出的 10 条记录（7 款酒）依次为 10、2、1 条。严格协议（四项浓度逐项不增）下最大 Tripel 份额全为 0，且 55 条记录都找到了正余量不可行证书，阻塞主要来自 DMS 与反式-2-壬烯醛；参考阈值下 30 条记录仅满足化学正份额条件，联合风格与 stale hops 检查后没有候选通过 J 代理复核；2 倍阈值敏感性下有 9 条重复划分记录获得 J 代理支持，其中 3 条为 T 内部留出来源，均未经过新批次制备和盲评。

总体判断：在现有数据上，风格基线具有较强预测能力；化合物候选需要结合共变模块和单项稳定性解释；缺陷阈值、代理模型分歧与真实制备共同决定迁移结果。后续工作应按此顺序推进：先补人员留出与真实品评，再谈更复杂的模型。

参考文献

[1] Schreurs M, Piampongsant S, Roncoroni M, et al. Predicting and improving complex beer flavor through machine learning[J]. Nature Communications, 2024, 15: 2368. DOI: 10.1038/s41467-024-46346-0.

[2] Schreurs M, et al. Data from: Predicting and improving complex beer flavor through machine learning (Version v4)[DB/OL]. Zenodo, 2026. DOI: 10.5281/zenodo.23624915. 访问日期: 2026-10-25.

[3] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees[J]. Machine Learning, 2006, 63(1): 3–42.

[4] Friedman J H. Greedy function approximation: a gradient boosting machine[J]. The Annals of Statistics, 2001, 29(5): 1189–1232.

[5] Breiman L. Stacked regressions[J]. Machine Learning, 1996, 24(1): 49–64.

[6] Wolpert D H. Stacked generalization[J]. Neural Networks, 1992, 5(2): 241–259.

[7] Hastie T, Tibshirani R, Friedman J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction[M]. 2nd ed. New York: Springer, 2009: 219–243.

[8] Li C, Li H. Variable selection and regression analysis for graph-structured covariates with an application to genomics[J]. The Annals of Applied Statistics, 2010, 4(3): 1498–1516.

[9] Lei J, G'Sell M, Rinaldo A, et al. Distribution-free predictive inference for regression[J]. Journal of the American Statistical Association, 2018, 113(523): 1094–1111.

[10] Gneiting T, Raftery A E. Strictly proper scoring rules, prediction, and estimation[J]. Journal of the American Statistical Association, 2007, 102(477): 359–378.

[11] Gretton A, Borgwardt K M, Rasch M J, et al. A kernel two-sample test[J]. Journal of Machine Learning Research, 2012, 13: 723–773.

[12] Efron B, Tibshirani R J. An Introduction to the Bootstrap[M]. New York: Chapman & Hall, 1993: 45–82.

[13] Vanbeneden N, Saison D, Delvaux F, et al. Decrease of 4-vinylguaiacol during beer aging and formation of apocynol and vanillin in beer[J]. Journal of Agricultural and Food Chemistry, 2008, 56(24): 11983–11988.

[14] Omur-Ozbek P, Dietrich A M, Duncan S E, et al. Role of lipid oxidation, chelating agents, and antioxidants in metallic flavor development in the oral cavity[J]. Journal of Agricultural and Food Chemistry, 2012, 60(9): 2274–2280.

[15] Pater A, Satora P, Januszek M. Effect of coriander seed addition at different stages of brewing on selected parameters of low-alcohol wheat beers[J]. Molecules, 2024, 29(4): 844.

[16] Meilgaard M C. Individual differences in sensory threshold for aroma chemicals added to beer[J]. Food Quality and Preference, 1993, 4(3): 153–167.

[17] Santos J R, Carneiro J R, Guido L F, et al. Determination of E-2-nonenal by high-performance liquid chromatography with UV detection: assay for the evaluation of beer ageing[J]. Journal of Chromatography A, 2003, 985(1–2): 395–402.

[18] Lundberg S M, Lee S I. A unified approach to interpreting model predictions[C]//Advances in Neural Information Processing Systems 30. 2017: 4765–4774.

AI 工具使用声明　本文的数据核查、代码实现、图表绘制与文字整理使用了 OpenAI Codex（GPT-6 系列）辅助，使用日期为 2026 年 10 月 24 日至 29 日；具体的提示词、采纳内容与人工修改情况见支撑材料中的《AI 工具使用详情说明》。使用记录与队员独立复核情况以实际留存材料为准。

A 全部 50 项的 Top-3 候选及证据分层

下表为全数据描述模型给出的最终 Top-3，括号内为固定全数据参数条件下 200 次 bootstrap 入选频率；标记 * 的 36 项同时满足频率、跨折入选和两类置换贡献条件，未标记的 114 项为探索性候选。标记根据未四舍五入的原始指标计算。最终模型选得 $\lambda = 0$，即这些候选由普通岭回归系数给出；频率不是因果概率，也不是独立检验的 p 值。

感官目标	Top-3 字段（入选频率）
A_esters_all	acetaldehyde*（0.78）；X.E..beta.ionone（0.23）；n.hexanol（0.23）
A_esters_ethac	acetaldehyde（0.40）；ethyl_acetate（0.21）；cis.linalool.oxide（0.17）
A_esters_flower	beta.phellandrene*（0.56）；X.E..alpha.ionone（0.40）；terpinolene（0.14）
A_esters_fruity	beta.phellandrene*（0.86）；X1.10.di.epi.cubenol（0.18）；benzyl.alcohol（0.30）
A_esters_isoaa	n.hexanol（0.40）；ambroxide（0.46）；ethyl.hexadecanoate（0.20）
A_hops_all	iron*（0.56）；DMS（0.34）；cis.dihydro.beta.terpineol（0.20）
A_hops_citrus	X6.methyl.5.hepten.2.one*（0.53）；terpinolene（0.19）；geraniol（0.26）
A_hops_noble	iron（0.36）；ethyl_octanoate（0.23）；ethyl_decanoate（0.16）
A_hops_tropical	DMS*（0.55）；beta.phellandrene（0.26）；citronellol（0.29）
A_hops_woody	iron（0.27）；ethyl_2.methyl_butyrate（0.15）；X2Z.6Z.farnesol（0.23）
A_malt_all	X2.acetylfuran*（0.77）；ethyl.furoate（0.42）；phenethyl_alcohol（0.36）
A_malt_bread	furfural（0.36）；ethyl.furoate（0.28）；ethyl.tiglate（0.13）
A_malt_burn	methylpyrazine*（0.75）；X2.acetylfuran*（0.60）；X4.methyl.1.pentanol（0.11）
A_malt_cara	X2.acetylfuran*（0.60）；phenethyl_alcohol（0.45）；octyl_acetate（0.31）
A_malt_grain	gamma.nonalactone（0.30）；car.3.en.2.one（0.36）；X5.methylfurfural（0.20）
F_esters_all	acetaldehyde*（0.91）；ethyl_isovalerate（0.34）；n.hexanol（0.27）
F_esters_ethac	acetaldehyde*（0.51）；ethyl_acetate（0.47）；n.tetradecanol（0.28）
F_esters_flower	beta.phellandrene（0.30）；eucalyptol（0.29）；terpinolene（0.13）
F_esters_fruity	beta.phellandrene（0.49）；X.E..beta.ionone（0.38）；MeSH（0.17）
F_esters_isoaa	n.hexanol（0.28）；ethyl_decanoate（0.14）；acetaldehyde（0.12）
F_hops_all	cis.dihydro.beta.terpineol（0.70）；iron*（0.70）；ethyl_isovalerate（0.40）

感官目标	Top-3 字段（入选频率）
F_hops_citrus	geraniol（0.47）；citronellol（0.43）；cis.dihydro.beta.terpineol（0.41）
F_hops_noble	iron*（0.68）；citronellol（0.23）；X14.hydroxy.alpha.humulene（0.30）
F_hops_tropical	X6.methyl.5.hepten.2.one（0.46）；DMS（0.24）；citronellol（0.27）
F_hops_woody	manool（0.43）；cis.dihydro.beta.terpineol（0.28）；n.octanol（0.15）
F_malt_all	acetaldehyde*（0.57）；X2.acetylfuran（0.47）；X.E..alpha.ionone（0.44）
F_malt_bread	acetaldehyde（0.26）；dehydroxy.cis.linalool.oxide（0.23）；caryophylla.4.12..8.13..dien.5.alpha.ol（0.15）
F_malt_burn	X2.acetylfuran*（0.51）；methylpyrazine（0.57）；MeSH（0.25）
F_malt_cara	acetaldehyde（0.62）；ethyl.nicotinate（0.27）；isobutyl.hexanoate（0.23）
F_malt_grain	X2.3.butanedione（0.49）；gamma.nonalactone（0.14）；X3.methyl.2.butenal（0.18）
X4vg	X4VG*（1.00）；delta.terpinyl.acetate（0.26）；ethyl_isovalerate（0.22）
acetic	lactic_acid.mg.L.（1.00）；acetic_acid.g.L.（0.62）；DES（0.23）
acidity	lactic_acid.mg.L.*（1.00）；acetic_acid.g.L.*（0.72）；X4.ethyl.guaiacol（0.12）
aftertaste	glycerol*（0.68）；ethyl_isovalerate*（0.62）；cis.dihydro.beta.terpineol（0.34）
alcohol	acetaldehyde*（0.83）；ethanol..v.v.*（0.82）；X3.methyl.2.hexanol（0.47）
barnyard	X2.butyloctanoate（0.32）；gamma.eudesmol（0.27）；iron（0.23）
bitternes	iron*（0.98）；cis.dihydro.beta.terpineol（0.63）；ethyl_octanoate（0.48）
body	glycerol*（0.80）；acetaldehyde（0.68）；X3.methyl.2.hexanol（0.47）
clove	linalool*（0.81）；limonene*（0.67）；eucalyptol（0.48）
co2	ethyl_octanoate*（0.62）；X3.methyl.2.hexanol（0.55）；ethyl_hexanoate（0.34）
coriander	x.methyl.1.octanol*（0.57）；geraniol*（0.55）；linalool（0.46）
diacetyl	CS2（0.21）；X.E..beta.ionone（0.09）；octyl_acetate（0.29）
dms	SO2.mg.L.*（0.56）；caryophylla.4.12..8.13..dien.5.alpha.ol（0.18）；neryl.formate（0.17）
lactic	lactic_acid.mg.L.（1.00）；acetic_acid.g.L.（0.61）；ethyl.phenylacetate（0.28）
metallic	iron*（0.94）；ethyl_isovalerate（0.46）；neryl.formate（0.20）
orange	X1.alpha.terpineol*（0.73）；limonene（0.49）；linalool（0.38）

感官目标	Top-3 字段（入选频率）
overall	ethyl_acetate（0.49）；X2.butyloctanoate（0.34）；ethyl_hexanoate（0.14）
stale_hops	X2.butyloctanoate（0.41）；iron（0.27）；X.E..ortho.methoxy.cinnamaldehyde（0.21）
sweetness	ethyl_decanoate*（0.81）；iron*（0.52）；ethyl_octanoate（0.28）
t2n	germacrene.D*（0.58）；isopentyl.hexanoate（0.46）；X.Z..dihydro.apofarnesal（0.18）

150 条记录中，满足“入选频率 ≥ 0.5、至少 5 个训练折入选、普通与风格内置换贡献均为正”这三条描述性条件的有 36 条。完整记录（含系数方向、置换贡献、单位与文献核查状态）见支撑材料 top3_with_evidence.csv。

B 敏感性分析所用 32 字段清单

该集合为全部可比记录尺度一致、在浓度单位注册表中、且属于 226 个非综合输入的字段交集；与含 80 处不一致的 32 个字段没有重合。单位为来源表的参考单位；部分建模输入仍为对数变换值，diethyl.succinate 为恒等对应，单位登记本身不证明化学定值准确。

字段	单位	字段	单位
acetaldehyde	mg/L	MeSH	μg/L
CS2	μg/L	octyl_acetate	mg/L
DES	μg/L	phenethyl_acetate	mg/L
DMS	μg/L	phenethyl_alcohol	mg/L
ethyl_2.methyl_butyrate	mg/L	X4VG	mg/L
ethyl_acetate	mg/L	SO2.mg.L.	mg/L
ethyl_decanoate	mg/L	acetic_acid.g.L.	g/L
ethyl_hexanoate	mg/L	ammonia.mg.L.	mg/L
ethyl_isovalerate	mg/L	betaglucan.mg.L.	mg/L
ethyl_octanoate	mg/L	lactic_acid.mg.L.	mg/L
ethyl_propionate	mg/L	glycerol	g/L
H2S	μg/L	iron	mg/L
isoamyl_alcohol	mg/L	protein.g.L.	g/L
isobutanol	mg/L	GFS	g/L
isobutyl_acetate	mg/L	ethanol..v.v.	v/v
isopentyl_acetate	mg/L	diethyl.succinate	mg/L

C 源程序清单

全部源程序随支撑材料提交，运行环境为 Python 3.12、依赖版本见 requirements-lock.txt。下表给出模块与章节的对应关系；此后的完整清单按 src/、scripts/、tests/、入口脚本与配置文件顺序排列。

以下列出建模、审计、实验与验证程序。论文排版与图形生成脚本（prepare_*.py、build_*.py、render_*.py、verify_*.py 等）不属于建模程序，随支撑材料提交，此处不重复列出。

001___init__.py

```
"""Research support code. See docs/AI_USAGE.md before competition use."""
```

表 19　核心模块与对应章节

文件	功能	章节
src/audit.py	附件结构、字段尺度、模板公式核查	3
src/data.py	数据读入、按酒对齐、缺失与截尾	3–4
src/common.py	划分、预处理管线、随机种子	4–6
src/q1.py	面板均分与七级分布的基线模型	4
src/q2.py	共表达图构建与岭回归候选	5
src/q3.py	风格生成与迁移搜索原型	6
src/stage2.py	保形区间、分级敏感性、置换贡献	4–5
src/stage3_algorithms.py	图正则求解、对偶证书、阈值风险函数	5–6
src/stage3_generation.py	四种生成器与 MMD/KS/复制率评价	6
src/stage3_graph.py	交叉拟合的图消融与因素稳定性	5
src/stage3_prediction.py	嵌套预测与保形区间	4
src/stage4_audit.py	原始评分重建、化学尺度与 OAV 复核	3
src/stage4_models.py	有序分布模型与评分规则	4
src/deployment.py	无标签预测入口与输入校验	7
run.py / run_stage2.py / run_stage3.py / run_stage4.py	四个阶段的命令行入口	全文
run_final_model.py	最终模型训练与预测	7

002_audit.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24."""
from pathlib import Path

import numpy as np
import pandas as pd
from openpyxl import load_workbook

from .common import progress, save_csv, sha256, write_json
from .data import META


def run_audit(data, out, cfg):
    dest = Path(out) / "audit"
    dest.mkdir(parents=True, exist_ok=True)
    progress("Audit: checking source hashes, sample keys, scales and sensory templates")
    hashes, shapes = [], []
    for p in sorted(data.root.iterdir()):
        if p.is_file():
            hashes.append({"file": p.name, "bytes": p.stat().st_size, "sha256": sha256(p)})
            if p.suffix == ".csv":
                frame = pd.read_csv(p)
                shapes.append({"file": p.name, "rows": len(frame), "columns_including_metadata": len(frame.columns), "missing_cells": int(frame.isna().sum().sum())})
    save_csv(pd.DataFrame(hashes), dest / "source_manifest.csv")
    save_csv(pd.DataFrame(shapes), dest / "file_shapes.csv")
    save_csv(data.meta.groupby("tasting_category_fine").size().rename("n").reset_index(), dest / "style_counts.csv")
    save_csv(data.chemical_meta, dest / "chemical_dictionary.csv")
    names = data.sensory_names[data.sensory_names.dataset == "Trained panel"].drop_duplicates("R_name").set_index("R_name")
    targets = []
    for c in data.Y:
        alias = "X4vg" if c == "X4vg" else c
        targets.append({"field": c, "label": names.at[alias, "CleanName_PanelFeedback"] if alias in names.index else c,
                        "group": "Aroma" if c.startswith("A_") else "Flavour" if c.startswith("F_") else "Other sensory",
                        "observed_scale": "provided centered/standardized continuous values",
                        "absolute_0_6_calibration": "unresolved"})
    save_csv(pd.DataFrame(targets), dest / "sensory_dictionary.csv")
    records = []
    for col in data.X:
        a = data.X[col]
        raw = data.calibrated[col] if col in data.calibrated else None
        row = {"field": col, "n_missing": int(a.isna().sum()), "minimum": float(a.min()), "maximum": float(a.max()),
               "q01": float(a.quantile(.01)), "q99": float(a.quantile(.99)), "transform_evidence": "not_available",
               "raw_identity_fraction": np.nan, "raw_log10_fraction": np.nan, "n_raw_positive_pairs": 0}
        if raw is not None:
            mask = a.notna() & raw.notna() & (raw > 0)
            if mask.sum():
                identity = np.isclose(a[mask], raw[mask], atol=1e-7, rtol=1e-6)
                logarithm = np.isclose(a[mask], np.log10(raw[mask]), atol=1e-7, rtol=1e-6)
                fi, fl = float(identity.mean()), float(logarithm.mean())
                row.update(raw_identity_fraction=fi, raw_log10_fraction=fl, n_raw_positive_pairs=int(mask.sum()))
                row["transform_evidence"] = "verified_log10_positive_pairs" if fl >= .999 else "verified_identity_positive_pairs" if fi >= .999 else "mostly_identity_with_mismatches" if fi >= .9 else "mixed_or_unresolved"
        records.append(row)
    scales = pd.DataFrame(records)
    save_csv(scales, dest / "field_scales.csv")
    # Inspect actual input cells, not formula caches that evaluate blanks to zero.
    w = load_workbook(data.root / "3.xlsx", data_only=False)
    input_rows = [7, 20, 33, 45, 50, 54, 58, 62, 68, 72, 76, 82, 86, 90, 94]
    input_cols = [3, 5, 7, 9, 11, 13, 15, 19, 21, 23, 25, 27, 29, 31]
    template = []
    for sh in w:
        if not sh.title.startswith("beer_"):
```

```
            continue
        marks = [sh.cell(r, c).coordinate for r in input_rows for c in input_cols if sh.cell(r, c).value is not None]
        template.append({"sheet": sh.title, "nonempty_main_scoring_inputs": len(marks), "input_cells": ",".join(marks)})
    save_csv(pd.DataFrame(template), dest / "sensory_template_audit.csv")
    key_stats = []
    for n in [1, 4, 7]:
        d = pd.read_csv(data.root / f"{n}.csv")
        key_stats.append({"file": f"{n}.csv", "unique_beer": int(d.beer.nunique()), "unique_beer_id": int(d.beer_id.nunique())})
    save_csv(pd.DataFrame(key_stats), dest / "join_key_audit.csv")
    values = data.given_corr.to_numpy()
    evidence = {
        "pearson_matrix_mean_absolute_difference": float(np.nanmean(np.abs(data.X.corr().to_numpy() - values))),
        "spearman_matrix_mean_absolute_difference": float(np.nanmean(np.abs(data.X.corr(method="spearman").to_numpy() - values))),
        "n_samples": len(data.X), "n_features": data.X.shape[1], "n_targets": data.Y.shape[1],
        "n_styles": int(data.meta.tasting_category_fine.nunique()),
        "grade_calibration": "UNRESOLVED: no observed paired absolute 0-6 anchors confirmed",
        "template_summary_barnyard_formula": str(w["summary"]["AX2"].value),
        "template_barnyard_expected_cell": "beer_1!AG90 based on the form's Barnyard label; team review required",
        "scope": "source evidence verified by computation; model conclusions remain exploratory",
    }
    write_json(dest / "verified_facts.json", evidence)
```

003_common.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Team verification and modification are required; see docs/AI_USAGE.md.
"""
from __future__ import annotations

import hashlib
import json
import platform
from datetime import datetime
from importlib.metadata import version
from pathlib import Path
from zoneinfo import ZoneInfo

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler


def write_json(path, value):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")


def save_csv(frame, path):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(path, index=False, encoding="utf-8-sig", float_format="%.9g")


def sha256(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def environment():
    return {
        "timestamp": datetime.now(ZoneInfo("Asia/Shanghai")).isoformat(),
        "python": platform.python_version(),
        "packages": {p: version(p) for p in ["numpy", "pandas", "scipy", "scikit-learn", "networkx", "matplotlib", "openpyxl"]},
    }


class TrainQuantileClipper(TransformerMixin, BaseEstimator):
    """Learn bounds on training samples only. Missing values remain missing."""

    def __init__(self, lower=0.01, upper=0.99):
        self.lower = lower
        self.upper = upper

    def fit(self, X, y=None):
        a = np.asarray(X, dtype=float)
        self.lower_ = np.nanquantile(a, self.lower, axis=0)
        self.upper_ = np.nanquantile(a, self.upper, axis=0)
        self.n_features_in_ = a.shape[1]
        return self

    def transform(self, X):
        a = np.asarray(X, dtype=float)
        if a.shape[1] != self.n_features_in_:
            raise ValueError("Feature count differs from training")
        return np.clip(a, self.lower_, self.upper_)


def preprocessing():
    return Pipeline([
        ("clip", TrainQuantileClipper()),
        ("impute", SimpleImputer(strategy="median", keep_empty_features=True)),
        ("scale", StandardScaler()),
    ])

```

```

def ecdf_grades(reference, values):
    """ASSUMPTION: seven empirical bins, NOT calibrated sensory intensity."""
    bounds = np.quantile(np.asarray(reference), np.arange(1, 7) / 7, axis=0)
    values = np.asarray(values)
    return (values[:, None, :] > bounds[None, :, :]).sum(axis=1).astype(int)


def progress(text):
    print(text, flush=True)
```

004_data.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
All source files are read-only. Never join on precision-damaged beer_id.
"""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

META = ["beer", "beer_id", "tasting_category_fine"]
AGGREGATES = {"kcalperc", "esters_sum", "aroma_hops_sum", "acids_sum", "sulfur_sum"}
NON_MOLECULES = {"color.EBC.", "betaglucan.mg.L.", "pH", "protein.g.L.", "bitterness", "GFS", "CO2.PSI.", "unfermentables"} | AGGREGATES
CHEMICAL_DEFECTS = ["DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal"]


@dataclass
class BeerData:
    root: Path
    meta: pd.DataFrame
    X: pd.DataFrame
    Y: pd.DataFrame
    given_corr: pd.DataFrame
    thresholds: pd.DataFrame
    sensory_names: pd.DataFrame
    calibrated: pd.DataFrame
    chemical_meta: pd.DataFrame


def find_data(argument=None):
    if argument:
        root = Path(argument).resolve()
        if not (root / "1.csv").exists():
            raise FileNotFoundError(f"Expected 1.csv in {root}")
        return root
    repo = Path(__file__).resolve().parents[1]
    for root in [repo / "data/raw", repo.parent / "materials/A/附件"]:
        if (root / "1.csv").exists():
            return root
    raise FileNotFoundError("Supply --data-dir pointing to the original A attachments")


def align_by_beer(chemical, sensory):
    for label, frame in [("chemical", chemical), ("sensory", sensory)]:
        if frame["beer"].isna().any() or frame["beer"].duplicated().any():
            raise ValueError(f"{label}: beer must be non-null and unique")
    if set(chemical.beer) != set(sensory.beer):
        raise ValueError("Chemical and sensory sample sets differ")
    sensory = sensory.set_index("beer").loc[chemical.beer].reset_index()
    if not chemical.tasting_category_fine.reset_index(drop=True).equals(sensory.tasting_category_fine):
        raise ValueError("Style labels do not agree after beer alignment")
    return sensory


def load_data(root):
    root = Path(root)
    chemical = pd.read_csv(root / "1.csv")
    sensory = align_by_beer(chemical, pd.read_csv(root / "4.csv"))
    X = chemical.drop(columns=META).astype(float)
    Y = sensory.drop(columns=META).astype(float)
    if Y.isna().any().any():
        raise ValueError("Targets contain missing values; target handling needs explicit review")
    corr = pd.read_csv(root / "2.csv", index_col=0)
    if corr.shape != (X.shape[1], X.shape[1]):
        raise ValueError("Correlation matrix dimensions do not match chemical fields")
    if list(corr.index) != list(corr.columns):
        raise ValueError("Correlation row/column names differ")
    # The positional alias map is empirically checked over the full matrix.
    # Thus it is not inferred from just matching the number of columns.
    mismatch = np.nanmean(np.abs(X.corr(method="spearman").to_numpy() - corr.to_numpy()))
    if mismatch > 0.002:
        raise ValueError("Cannot verify ordered chemical aliases against sample correlations")
    book = root / "8.xlsx"
    thresholds = pd.read_excel(book, sheet_name="Sensory thresholds", na_values="NA")
    names = pd.read_excel(book, sheet_name="Sensory names", na_values="NA")
    calibrated = pd.read_excel(book, sheet_name="Calibrated chemical dataset", na_values="NA")
    if calibrated.beer.duplicated().any():
        raise ValueError("Calibrated reference has duplicate beer keys")
    calibrated = calibrated.set_index("beer").loc[chemical.beer].reset_index()
    cm = thresholds.dropna(subset=["field_name_R"]).drop_duplicates("field_name_R").set_index("field_name_R")
    rows = []
```

```
    for col, alias in zip(X.columns, corr.index):
        row = {"field": col, "matrix_alias": alias, "kind": "aggregate" if col in AGGREGATES else "physical_or_mixture" if col in NON_MOLECULES else "compound_or_element"}
        for name in ["chemical_name", "origin_paper", "unit", "off_flavors", "threshold_median", "threshold_mean"]:
            row[name] = cm.at[col, name] if col in cm.index else np.nan
        rows.append(row)
    return BeerData(root, chemical[META], X, Y, corr, thresholds, names, calibrated, pd.DataFrame(rows))
```

005_q1.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Nested preprocessing/tuning and paired out-of-fold prediction. No raw-score claim.
"""
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.base import clone
from sklearn.cross_decomposition import PLSRegression
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, KFold, RepeatedStratifiedKFold
from sklearn.pipeline import Pipeline

from .common import ecdf_grades, preprocessing, progress, save_csv, write_json


def model_specs(cfg):
    def pipe(model):
        return Pipeline([("prep", preprocessing()), ("model", model)])
    return {
        "mean": (pipe(DummyRegressor(strategy="mean")), {}),
        "ridge": (pipe(Ridge()), {"model__alpha": cfg["ridge_alphas"]}),
        "pls": (pipe(PLSRegression(scale=False)), {"model__n_components": cfg["pls_components"]}),
        "extra_trees": (pipe(ExtraTreesRegressor(n_estimators=cfg["extra_trees"], min_samples_leaf=3, max_features=.8, random_state=cfg["seed"], n_jobs=2)), {}),
    }


def fitted_model(name, X, y, cfg):
    estimator, params = model_specs(cfg)[name]
    if params:
        estimator = GridSearchCV(estimator, params, scoring="neg_mean_absolute_error", cv=KFold(cfg["inner_folds"], shuffle=True, random_state=cfg["seed"]), n_jobs=1, refit=True)
    estimator.fit(X, y)
    return estimator.best_estimator_ if params else estimator, estimator.best_params_ if params else {}


def metric_rows(y, pred, targets, model):
    return [{"model": model, "target": c,
             "mae": float(mean_absolute_error(y[:, j], pred[:, j])),
             "rmse": float(np.sqrt(mean_squared_error(y[:, j], pred[:, j]))),
             "r2": float(r2_score(y[:, j], pred[:, j]))}
            for j, c in enumerate(targets)]


def run_q1(data, out, cfg):
    dest = Path(out) / "q1"
    dest.mkdir(parents=True, exist_ok=True)
    (dest / "private").mkdir(exist_ok=True)
    X, Y = data.X.to_numpy(), data.Y.to_numpy()
    targets = data.Y.columns.to_list()
    split = RepeatedStratifiedKFold(n_splits=cfg["outer_folds"], n_repeats=cfg["outer_repeats"], random_state=cfg["seed"])
    sums = {m: np.zeros_like(Y) for m in model_specs(cfg)}
    count = np.zeros(len(X), dtype=int)
    assignments, selected, fold_metrics, robustness = [], [], [], []
    grades = np.zeros_like(Y)
    for fold, (train, test) in enumerate(split.split(X, data.meta.tasting_category_fine)):
        progress(f"Q1: outer fold {fold + 1}/{cfg['outer_folds'] * cfg['outer_repeats']}")
        count[test] += 1
        assignments.extend({"repeat": fold // cfg["outer_folds"], "fold": fold % cfg["outer_folds"], "beer": int(data.meta.iloc[i].beer), "role": "validation"} for i in test)
        for name in sums:
            est, params = fitted_model(name, X[train], Y[train], cfg)
            pred = est.predict(X[test])
            sums[name][test] += pred
            selected.append({"outer_fold": fold, "model": name, "parameters": str(params)})
            fold_metrics.append({"outer_fold": fold, "model": name, "mae": float(mean_absolute_error(Y[test], pred)), "macro_r2": float(r2_score(Y[test], pred)), "n_validation": len(test)})
            if name == "ridge":
                grades[test] += ecdf_grades(Y[train], pred)
                for rate in cfg["missing_rates"]:
                    corrupted = X[test].copy()
                    rng = np.random.default_rng(cfg["seed"] + fold)
                    artificial_missing = rng.random(corrupted.shape) < rate
                    corrupted[artificial_missing] = np.nan
                    pp = est.predict(corrupted)
                    robustness.append({"outer_fold": fold, "additional_missing_fraction": rate, "mae": float(mean_absolute_error(Y[test], pp)), "mask_seed": cfg["seed"] + fold})
    if not np.all(count == cfg["outer_repeats"]):
        raise AssertionError("Every sample must have exactly one held-out prediction per repeat")
    predictions = {name: value / count[:, None] for name, value in sums.items()}
```

```
    rows = []
    for name, pred in predictions.items():
        rows += metric_rows(Y, pred, targets, name)
        frame = pd.concat([data.meta[["beer", "tasting_category_fine"]], pd.DataFrame(pred, columns=targets)], axis=1)
        save_csv(frame, dest / f"oof_{name}_continuous.csv")
    metrics = pd.DataFrame(rows)
    save_csv(metrics, dest / "metrics_by_target.csv")
    save_csv(metrics.groupby("model")[["mae", "rmse", "r2"]].mean().reset_index(), dest / "metrics_summary.csv")
    save_csv(pd.DataFrame(fold_metrics), dest / "metrics_by_fold.csv")
    save_csv(pd.DataFrame(assignments), dest / "split_assignments.csv")
    save_csv(pd.DataFrame(selected), dest / "inner_selected_parameters.csv")
    save_csv(pd.DataFrame(robustness), dest / "missingness_robustness.csv")
    styles = []
    for style in sorted(data.meta.tasting_category_fine.unique()):
        ix = np.flatnonzero(data.meta.tasting_category_fine.to_numpy() == style)
        styles.append({"style": style, "n": len(ix), "ridge_mae": float(mean_absolute_error(Y[ix], predictions["ridge"][ix]))})
    save_csv(pd.DataFrame(styles), dest / "metrics_by_style.csv")
    # Predeclared ridge is used downstream; do not choose the winner using OOF scores.
    model, params = fitted_model("ridge", X, Y, cfg)
    joblib.dump({"model": model, "features": data.X.columns.tolist(), "targets": targets}, dest / "private/model.joblib")
    save_csv(pd.DataFrame(model.named_steps["model"].coef_, index=targets, columns=data.X.columns).reset_index(names="target"), dest / "final_
ridge_coefficients.csv")
    if cfg["export_assumption_grades"]:
        gg = np.rint(grades / count[:, None]).astype(int)
        frame = pd.concat([data.meta[["beer"]], pd.DataFrame(gg, columns=targets)], axis=1)
        frame["status"] = "ASSUMPTION_ONLY_training_ECDF_bins_not_absolute_sensory_scores"
        save_csv(frame, dest / "ASSUMPTION_percentile_grades_0_6.csv")
    write_json(dest / "status.json", {
        "continuous_prediction": "computed_repeated_out_of_fold_exploratory_baselines",
        "absolute_0_6_prediction": "UNRESOLVED_missing_raw_score_calibration",
        "proxy_grades": "empirical equal-frequency bins, not a recovery of the original 0-6 intensity",
        "primary_model": "ridge selected before evaluation; inner tuning only",
        "final_inner_parameters": params,
        "external_generalization": "not established; supplied targets already preprocessed by source authors",
        "intervals": "not calibrated; no nominal coverage claim is made",
    })
```

006_q2.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Descriptive graph-regularized attribution, not causal identification.
"""
from pathlib import Path

import networkx as nx
import numpy as np
import pandas as pd
from scipy.linalg import solve
from scipy.stats import rankdata

from .common import preprocessing, progress, save_csv, write_json
from .data import NON_MOLECULES


def signed_laplacian(corr, threshold):
    W = np.asarray(corr, float).copy()
    W[~np.isfinite(W)] = 0
    W[np.abs(W) < threshold] = 0
    np.fill_diagonal(W, 0)
    W = (W + W.T) / 2
    return np.diag(np.abs(W).sum(axis=1)) - W


def graph_coefficients(Z, Y, corr, cfg, graph=True):
    L = signed_laplacian(corr, cfg["graph_threshold"]) if graph else np.zeros((Z.shape[1], Z.shape[1]))
    yc = Y - Y.mean(axis=0)
    return solve(Z.T @ Z + cfg["graph_ridge_alpha"] * np.eye(Z.shape[1]) + cfg["graph_ridge_lambda"] * L, Z.T @ yc, assume_a="pos")


def cross_spearman(X, Y):
    a, b = rankdata(X, axis=0), rankdata(Y, axis=0)
    a -= a.mean(axis=0)
    b -= b.mean(axis=0)
    denom = np.sqrt((a * a).sum(axis=0)[:, None] * (b * b).sum(axis=0)[None, :])
    return np.divide(a.T @ b, denom, out=np.zeros((a.shape[1], b.shape[1])), where=denom > 0)


def top_indices(values, eligible):
    return eligible[np.argsort(-np.abs(values[eligible]), axis=0, kind="stable")[:3]]


def run_q2(data, out, cfg):
    dest = Path(out) / "q2"
    dest.mkdir(parents=True, exist_ok=True)
    X, Y = data.X.to_numpy(), data.Y.to_numpy()
    features, targets = data.X.columns.tolist(), data.Y.columns.tolist()
    eligible = np.array([j for j, c in enumerate(features) if c not in NON_MOLECULES])
    given = data.given_corr.to_numpy()
    graph = nx.Graph()
    graph.add_nodes_from(features)
    edge_rows = []
    for i in range(len(features)):
        for j in range(i + 1, len(features)):
            r = float(given[i, j])
            if abs(r) >= cfg["graph_threshold"]:
```

```
                graph.add_edge(features[i], features[j], weight=abs(r), correlation=r)
                edge_rows.append({"source": features[i], "target": features[j], "spearman_r": r, "absolute_weight": abs(r)})
    save_csv(pd.DataFrame(edge_rows), dest / "coexpression_edges.csv")
    communities = list(nx.community.greedy_modularity_communities(graph, weight="weight"))
    modules = {node: i for i, group in enumerate(communities) for node in group}
    save_csv(pd.DataFrame([{"field": c, "module": modules[c], "degree": graph.degree(c), "weighted_degree": graph.degree(c, weight="weight")} for c in features]), dest / "network_nodes.csv")
    graph_stats = []
    for threshold in [.6, .7, .8, .9]:
        gg = nx.Graph()
        gg.add_nodes_from(features)
        gg.add_edges_from((features[i], features[j]) for i in range(len(features)) for j in range(i + 1, len(features)) if abs(given[i, j]) >= threshold)
        graph_stats.append({"threshold": threshold, "edges": gg.number_of_edges(), "components": nx.number_connected_components(gg), "isolated_nodes": len(list(nx.isolates(gg)))})
    save_csv(pd.DataFrame(graph_stats), dest / "network_threshold_sensitivity.csv")
    prep = preprocessing()
    Z = prep.fit_transform(X)
    beta = graph_coefficients(Z, Y, given, cfg)
    ridge = graph_coefficients(Z, Y, given, cfg, graph=False)
    marginal = cross_spearman(Z, Y)
    frequencies = np.zeros_like(beta)
    rng = np.random.default_rng(cfg["seed"])
    for b in range(cfg["bootstrap_repeats"]):
        if b % 20 == 0:
            progress(f"Q2: bootstrap {b}/{cfg['bootstrap_repeats']}")
        ix = rng.integers(0, len(X), len(X))
        zb = preprocessing().fit_transform(X[ix])
        # Refit the network too. Do not merely resample previously computed ranks.
        corrb = np.corrcoef(rankdata(zb, axis=0), rowvar=False)
        bb = graph_coefficients(zb, Y[ix], corrb, cfg)
        for k in range(Y.shape[1]):
            frequencies[top_indices(bb[:, k], eligible), k] += 1
    frequencies /= cfg["bootstrap_repeats"]
    meta = data.chemical_meta.set_index("field")
    top_rows, agreement = [], []
    for k, target in enumerate(targets):
        chosen = top_indices(beta[:, k], eligible)
        other = top_indices(ridge[:, k], eligible)
        univariate = top_indices(marginal[:, k], eligible)
        def jac(a, b):
            return len(set(a) & set(b)) / len(set(a) | set(b))
        agreement.append({"target": target, "graph_vs_ridge_jaccard": jac(chosen, other), "graph_vs_spearman_jaccard": jac(chosen, univariate), "ridge_vs_spearman_jaccard": jac(other, univariate)})
        for rank, j in enumerate(chosen, 1):
            c = features[j]
            origin = meta.at[c, "origin_paper"]
            top_rows.append({"target": target, "rank": rank, "field": c, "chemical_name": meta.at[c, "matrix_alias"],
                             "graph_coefficient": beta[j, k], "association_spearman": marginal[j, k],
                             "bootstrap_top3_frequency": frequencies[j, k], "module": modules[c],
                             "origin": origin, "unit": meta.at[c, "unit"], "known_off_flavors": meta.at[c, "off_flavors"],
                             "stability_flag": "low_selection_frequency" if frequencies[j, k] < .5 else "frequently_selected_in_this_resampling",
                             "chemical_argument": f"Reference origin: {origin}. Verify target-specific mechanism and units; coefficient is association, not causation.",
                             "team_mechanism_review": "required"})
    save_csv(pd.DataFrame(top_rows), dest / "top3_by_sensory_dimension.csv")
    save_csv(pd.DataFrame(agreement), dest / "method_jaccard.csv")
    save_csv(pd.DataFrame(frequencies, index=features, columns=targets).reset_index(names="field"), dest / "bootstrap_selection_frequencies.csv")
    write_json(dest / "status.json", {
        "scope": "descriptive_exploratory_full_dataset_association",
        "nodes": graph.number_of_nodes(), "edges": graph.number_of_edges(), "modules": len(communities),
        "bootstrap_repeats": cfg["bootstrap_repeats"], "bootstrap_rebuilds_preprocessing_and_graph": True,
        "causal_status": "not_identified",
        "chemical_mechanism_status": "reference metadata attached; target-specific human literature review required",
        "regularization_parameters": "fixed research defaults, not optimized for causal recovery",
        "eligible_compounds": len(eligible),
    })
```

007_q3.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24.
Small-sample conditional Gaussian reference and explicitly model-based transfers.
"""
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import ks_2samp
from sklearn.covariance import LedoitWolf
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline

from .common import ecdf_grades, preprocessing, progress, save_csv, write_json
from .data import CHEMICAL_DEFECTS


class ConditionalPCAGaussian:
    def __init__(self, components=10, seed=20261024):
        self.components, self.seed = components, seed

```

```
    def fit(self, X, labels):
        self.prep = preprocessing()
        z = self.prep.fit_transform(X)
        self.pca = PCA(n_components=min(self.components, len(z) - 1, z.shape[1]), random_state=self.seed)
        latent = self.pca.fit_transform(z)
        self.low = np.nanquantile(X, .01, axis=0)
        self.high = np.nanquantile(X, .99, axis=0)
        self.median = np.nanmedian(X, axis=0)
        pooled = LedoitWolf().fit(latent).covariance_
        self.means, self.covariances = {}, {}
        labels = np.asarray(labels)
        for style in np.unique(labels):
            a = latent[labels == style]
            self.means[style] = a.mean(axis=0)
            local = np.atleast_2d(np.cov(a, rowvar=False)) if len(a) > 1 else pooled
            weight = len(a) / (len(a) + 10)
            self.covariances[style] = weight * local + (1 - weight) * pooled + 1e-8 * np.eye(pooled.shape[0])
        return self

    def encode(self, X):
        return self.pca.transform(self.prep.transform(X))

    def decode(self, latent):
        z = self.pca.inverse_transform(latent)
        values = self.prep.named_steps["scale"].inverse_transform(z)
        return np.clip(values, self.low, self.high)

    def sample(self, style, n, rng):
        latent = rng.multivariate_normal(self.means[style], self.covariances[style], size=n)
        return self.decode(latent)


def chemical_nonincrease(source, candidate, positions, tolerance=1e-9):
    return np.all(np.asarray(candidate)[..., positions] <= np.asarray(source)[..., positions] + tolerance, axis=-1)


def biased_mmd(a, b):
    """RBF MMD squared; bandwidth from real reference only."""
    if len(a) < 2 or len(b) < 2:
        return np.nan
    dist = cdist(b, b, metric="sqeuclidean")
    positive = dist[dist > 0]
    bandwidth = np.median(positive) if len(positive) else 1.
    return float(np.exp(-cdist(a, a, "sqeuclidean") / (2 * bandwidth)).mean() + np.exp(-dist / (2 * bandwidth)).mean() - 2 * np.exp(-cdist(a, b, "sqeuclidean") / (2 * bandwidth)).mean())


def run_q3(data, out, cfg):
    dest = Path(out) / "q3"
    dest.mkdir(parents=True, exist_ok=True)
    path = Path(out) / "q1/private/model.joblib"
    if not path.exists():
        raise FileNotFoundError("Q3 needs the Q1 reference predictor; run q1 first")
    bundle = joblib.load(path)
    predictor = bundle["model"]
    features, targets = data.X.columns.tolist(), data.Y.columns.tolist()
    if bundle["features"] != features or bundle["targets"] != targets:
        raise ValueError("Q1 predictor schema mismatch")
    X = data.X.to_numpy()
    labels = data.meta.tasting_category_fine.to_numpy()
    train, test = train_test_split(np.arange(len(X)), test_size=cfg["generation_holdout_fraction"], stratify=labels, random_state=cfg["seed"])
    save_csv(pd.DataFrame({"beer": data.meta.beer, "role": ["train" if i in set(train) else "holdout" for i in range(len(X))]}), dest / "generator_split.csv")
    progress("Q3: fitting conditional PCA Gaussian on generator training samples only")
    generator = ConditionalPCAGaussian(cfg["generation_components"], cfg["seed"]).fit(X[train], labels[train])
    classifier = Pipeline([("prep", preprocessing()), ("model", LogisticRegression(C=1, max_iter=2000, class_weight="balanced", random_state=cfg["seed"]))])
    classifier.fit(X[train], labels[train])
    class_index = {s: j for j, s in enumerate(classifier.classes_)}
    holdout_balanced = float(balanced_accuracy_score(labels[test], classifier.predict(X[test])))
    rng = np.random.default_rng(cfg["seed"])
    generated, metrics, portraits = [], [], []
    for style in cfg["generation_styles"]:
        values = generator.sample(style, cfg["generation_samples_per_style"], rng)
        ids = [f"{style}_{i + 1:03d}" for i in range(len(values))]
        frame = pd.DataFrame(values, columns=features)
        frame.insert(0, "style", style)
        frame.insert(0, "generated_id", ids)
        generated.append(frame)
        pred = predictor.predict(values)
        portrait = pd.DataFrame(pred, columns=targets)
        portrait.insert(0, "style", style)
        portrait.insert(0, "generated_id", ids)
        portraits.append(portrait)
        real = X[test[labels[test] == style]]
        zs = generator.prep.transform(values)
        zr = generator.prep.transform(real)
        ztrain = generator.prep.transform(X[train])
        distances = cdist(zs, ztrain)
        train_style = X[train[labels[train] == style]]
        bootstrap = train_style[rng.integers(0, len(train_style), len(values))]
        # Natural missingness in held-out measurements must not erase all KS diagnostics.
        ks_values = []
        for j in range(X.shape[1]):
            observed = real[:, j][np.isfinite(real[:, j])]
            if len(observed):
                ks_values.append(ks_2samp(values[:, j], observed, method="auto").statistic)
```

```
        # Both metrics use independent real holdout samples; small counts are explicit.
        metrics.append({"style": style, "n_generated": len(values), "n_training_style": len(train_style), "n_holdout_style": len(real),
                        "mean_marginal_KS_holdout": float(np.mean(ks_values)), "n_KS_fields_observed": len(ks_values),
                        "RBF_MMD2_holdout": biased_mmd(zs, zr),
                        "bootstrap_baseline_MMD2_holdout": biased_mmd(generator.prep.transform(bootstrap), zr),
                        "mean_target_style_probability": float(classifier.predict_proba(values)[:, class_index[style]].mean()),
                        "nearest_train_distance_mean": float(distances.min(axis=1).mean()),
                        "exact_training_copy_fraction": float((distances.min(axis=1) < 1e-8).mean()),
                        "heldout_reference_warning": "few independent style examples; descriptive distances only, no quality guarantee"})
    all_generated = pd.concat(generated, ignore_index=True)
    save_csv(all_generated, dest / "generated_compound_profiles.csv")
    save_csv(pd.concat(portraits, ignore_index=True), dest / "generated_sensory_continuous.csv")
    save_csv(pd.DataFrame(metrics), dest / "generation_diagnostics.csv")
    if cfg["export_assumption_grades"]:
        a = pd.concat(portraits, ignore_index=True)
        grade = ecdf_grades(data.Y.to_numpy()[train], a[targets].to_numpy())
        proxy = pd.concat([a[["generated_id", "style"]], pd.DataFrame(grade, columns=targets)], axis=1)
        proxy["status"] = "ASSUMPTION_ONLY_ECDF_not_absolute_sensory_scores"
        save_csv(proxy, dest / "ASSUMPTION_generated_percentile_grades.csv")
    positions = [features.index(c) for c in CHEMICAL_DEFECTS]
    stale = targets.index("stale_hops")
    target = cfg["transfer_target"]
    source = cfg["transfer_source"]
    transfers, diagnostics, checks, transfer_sensory = [], [], [], []
    for i in np.flatnonzero(labels == source):
        original = X[i]
        x = np.where(np.isfinite(original), original, generator.median)
        base_pred = predictor.predict(x[None])[0]
        base_prob = classifier.predict_proba(x[None])[0, class_index[target]]
        cleaning_only = np.clip(x, generator.low, generator.high)
        cleaning_only[positions] = np.minimum(cleaning_only[positions], x[positions])
        cleaning_prob = classifier.predict_proba(cleaning_only[None])[0, class_index[target]]
        latent = generator.encode(x[None])[0]
        desired = generator.decode((latent + generator.means[target] - generator.means[source])[None])[0]
        candidates = []
        for t in np.linspace(.025, 1.25, 50):
            candidate = np.clip(x + t * (desired - x), generator.low, generator.high)
            candidate[positions] = np.minimum(candidate[positions], x[positions])
            candidates.append(candidate)
        candidates = np.asarray(candidates)
        candidate_pred = predictor.predict(candidates)
        chemical_ok = chemical_nonincrease(x, candidates, positions)
        stale_ok = candidate_pred[:, stale] <= base_pred[stale] + 1e-9
        target_prob = classifier.predict_proba(candidates)[:, class_index[target]]
        # Separate genuine style movement from the effects of clipping source outliers.
        feasible = chemical_ok & stale_ok & (target_prob > max(base_prob, cleaning_prob) + .01)
        if feasible.any():
            eligible = np.flatnonzero(feasible)
            chosen = eligible[np.argmax(target_prob[eligible])]
            value, pp, probability = candidates[chosen], candidate_pred[chosen], target_prob[chosen]
            status = "nontrivial_model_feasible_candidate"
        else:
            value, pp, probability = x.copy(), base_pred.copy(), base_prob
            status = "no_nontrivial_candidate_found_source_retained"
        beer = int(data.meta.iloc[i].beer)
        missing_defects = [c for c, j in zip(CHEMICAL_DEFECTS, positions) if not np.isfinite(original[j])]
        for c, j in zip(CHEMICAL_DEFECTS, positions):
            checks.append({"beer": beer, "defect": c, "source_value": x[j], "candidate_value": value[j], "nonincrease": bool(value[j] <= x[j] + 1e
-9), "scale": "supplied_feature_scale", "source_observed": bool(np.isfinite(original[j])), "absolute_threshold_certified": False})
        checks.append({"beer": beer, "defect": "stale_hops", "source_value": base_pred[stale], "candidate_value": pp[stale], "nonincrease": bool(
pp[stale] <= base_pred[stale] + 1e-9), "scale": "Q1_model_predicted_continuous_sensory", "source_observed": False, "absolute_threshold_certified":
 False})
        transfers.append({"beer": beer, "source_style": source, "target_style": target, **dict(zip(features, value))})
        transfer_sensory.append({"beer": beer, **dict(zip(targets, pp))})
        diagnostics.append({"beer": beer, "generator_role": "holdout" if i in set(test) else "train", "status": status,
                            "source_target_probability": base_prob, "candidate_target_probability": probability,
                            "cleaning_only_target_probability": cleaning_prob,
                            "supplied_scale_change_l2": float(np.linalg.norm(value - x)),
                            "chemical_nonincrease": bool(chemical_nonincrease(x, value, positions)),
                            "predicted_stale_hops_nonincrease": bool(pp[stale] <= base_pred[stale] + 1e-9),
                            "missing_source_defects": ",".join(missing_defects),
                            "out_of_training_envelope_fields": int(((value < generator.low - 1e-9) | (value > generator.high + 1e-9)).sum()),
                            "guarantee_scope": "measurement nonincrease plus learned stale proxy; physical and sensory experiment not performed"})
    save_csv(pd.DataFrame(transfers), dest / "pils_to_tripel_profiles.csv")
    save_csv(pd.DataFrame(transfer_sensory), dest / "pils_to_tripel_sensory_continuous.csv")
    save_csv(pd.DataFrame(diagnostics), dest / "transfer_diagnostics.csv")
    save_csv(pd.DataFrame(checks), dest / "defect_nonincrease_checks.csv")
    write_json(dest / "status.json", {
        "model": "conditional_PCA_Gaussian_with_partial_covariance_pooling",
        "generator_train_n": len(train), "generator_holdout_n": len(test),
        "style_classifier_holdout_balanced_accuracy": holdout_balanced,
        "primary_feature_scale": "supplied_mixed_transforms; see audit/field_scales.csv",
        "marginal_bounds": "training 1%-99% descriptor range; not a thermodynamic or brewing-feasibility guarantee",
        "aggregate_consistency": "aggregate descriptors generated jointly; exact chemical sum identities not established",
        "sensory_portraits": "Q1 full-data fitted predictor, model-based only; not an independent tasting validation",
        "defect_thresholds": "absolute thresholds incomplete and some field scales unresolved; no complete OAV certification",
        "stale_hops_constraint": "prediction proxy only; actual sensory non-worsening unverified",
        "zero_change_transfers": "explicitly reported as no nontrivial feasible candidate, never counted as successful style transfer",
    })
```

008_reporting.py

```
"""AI-assisted implementation: OpenAI Codex / GPT-6, 2026-10-24."""
import json
from pathlib import Path
```

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from .common import progress, sha256, write_json


def make_report(data, out, cfg):
    out = Path(out)
    figures = out / "figures"
    figures.mkdir(exist_ok=True)
    progress("Report: preparing experiment summaries and scientific figures")
    metrics = pd.read_csv(out / "q1/metrics_summary.csv").set_index("model")
    per_target = pd.read_csv(out / "q1/metrics_by_target.csv")
    robustness = pd.read_csv(out / "q1/missingness_robustness.csv")
    top = pd.read_csv(out / "q2/top3_by_sensory_dimension.csv")
    jaccard = pd.read_csv(out / "q2/method_jaccard.csv")
    generation = pd.read_csv(out / "q3/generation_diagnostics.csv")
    transfers = pd.read_csv(out / "q3/transfer_diagnostics.csv")
    checks = pd.read_csv(out / "q3/defect_nonincrease_checks.csv")
    plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False, "figure.dpi": 140, "savefig.facecolor": "white"})
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
    order = ["mean", "ridge", "pls", "extra_trees"]
    ax[0].bar(order, metrics.loc[order, "mae"], color=["#9b9b9b", "#286eaa", "#549778", "#bd833a"])
    ax[0].set(ylabel="Macro MAE (provided continuous scale)", title="Repeated out-of-fold prediction")
    byrate = robustness.groupby("additional_missing_fraction").mae.agg(["mean", "std"])
    ax[1].errorbar(byrate.index * 100, byrate["mean"], yerr=byrate["std"], marker="o", capsize=4, color="#286eaa")
    ax[1].set(xlabel="Additional missing cells (%)", ylabel="Fold MAE", title="Ridge missingness sensitivity")
    ax[1].text(.02, .98, "Bars: fold SD, not a confidence interval", transform=ax[1].transAxes, va="top", fontsize=8)
    fig.savefig(figures / "q1_validation.png", dpi=170)
    plt.close(fig)
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
    ax[0].hist(top.bootstrap_top3_frequency, bins=np.linspace(0, 1, 11), color="#286eaa", edgecolor="white")
    ax[0].set(xlabel="Bootstrap Top-3 inclusion frequency", ylabel="Number of selected factors", title="Stability of 150 selected factors")
    cols = ["graph_vs_ridge_jaccard", "graph_vs_spearman_jaccard", "ridge_vs_spearman_jaccard"]
    ax[1].boxplot([jaccard[c] for c in cols], tick_labels=["Graph / ridge", "Graph / rank corr.", "Ridge / rank corr."])
    ax[1].set(ylabel="Top-3 Jaccard", ylim=(-.05, 1.05), title="Agreement across methods")
    fig.savefig(figures / "q2_stability.png", dpi=170)
    plt.close(fig)
    portraits = pd.read_csv(out / "q3/generated_sensory_continuous.csv")
    selected = ["A_malt_all", "A_hops_all", "A_esters_all", "F_malt_all", "F_hops_all", "F_esters_all", "acidity", "bitternes", "sweetness", "body
", "diacetyl", "dms", "stale_hops"]
    values = portraits.groupby("style")[selected].mean().reindex(cfg["generation_styles"])
    fig, ax = plt.subplots(figsize=(12, 3.4), layout="constrained")
    cap = max(.5, float(np.max(np.abs(values.to_numpy()))))
    im = ax.imshow(values, aspect="auto", cmap="RdBu_r", vmin=-cap, vmax=cap)
    ax.set_xticks(range(len(selected)), selected, rotation=45, ha="right")
    ax.set_yticks(range(len(values)), values.index)
    ax.set_title("Generated sensory portraits: model predictions on supplied continuous scale")
    fig.colorbar(im, ax=ax, label="Predicted continuous intensity", shrink=.85)
    fig.savefig(figures / "q3_sensory_portraits.png", dpi=170)
    plt.close(fig)
    fig, ax = plt.subplots(1, 2, figsize=(11, 4.5), layout="constrained")
    x = np.arange(len(transfers))
    ax[0].plot(x, transfers.source_target_probability, "o--", color="#999999", label="Source")
    ax[0].plot(x, transfers.candidate_target_probability, "o-", color="#286eaa", label="Candidate")
    ax[0].set_xticks(x, transfers.beer.astype(str), rotation=45)
    ax[0].set(xlabel="Source beer", ylabel="Tripel probability (model only)", title="Candidate style movement")
    ax[0].legend()
    groups = checks.groupby("defect").nonincrease.agg(["sum", "count"])
    ax[1].barh(groups.index, groups["sum"] / groups["count"], color="#549778")
    ax[1].set(xlim=(0, 1.05), xlabel="Nonincrease fraction in checked quantity", title="Chemical measurements and stale-hops proxy")
    ax[1].text(.02, -.2, "Unchanged failures are included; actual tasting unverified", transform=ax[1].transAxes, fontsize=8)
    fig.savefig(figures / "q3_transfer_checks.png", dpi=170)
    plt.close(fig)
    facts = json.loads((out / "audit/verified_facts.json").read_text(encoding="utf-8"))
    q3status = json.loads((out / "q3/status.json").read_text(encoding="utf-8"))
    successful = int((transfers.status == "nontrivial_model_feasible_candidate").sum())
    heldout = transfers[transfers.generator_role == "holdout"]
    heldout_success = int((heldout.status == "nontrivial_model_feasible_candidate").sum())
    baseline = float(metrics.loc["mean", "mae"])
    ridge_mae = float(metrics.loc["ridge", "mae"])
    improvement = (baseline - ridge_mae) / baseline
    low = int((top.bootstrap_top3_frequency < .5).sum())
    report = f"""# A 题阶段成果报告

本阶段为 AI 辅助研究基线，三问均有可运行实现和阶段产物。数值只对应当前配置与当前附件。原始 0–6 分标定、真实感官改善及化学因果仍未完成验证，不能将整份报告直接作为最终参赛结论。

## 已核实的数据事实

实际有 {facts['n_samples']} 个样本、{facts['n_features']} 个化学或理化数值字段、{facts['n_targets']} 个连续感官目标和 {facts['n_styles']} 种风格。以 beer 唯一键一对一关联，不使用受损 beer_id。附件相关矩阵与重算 Spearman 的平均绝对差为 {facts['spearman_matrix_mean_absolute_difference']:.6g}，与 Pearson 为 {facts['pearson_matrix_mean_absolute_difference']:.6g}。逐字段变换证据、模板检查、哈希和字典保存在 audit/。这些是文件证据，关于合理性的解释仍需队内复核。

## 第一问 处理 方法与结果

输入为附件 1 的数值字段，目标为附件 4 的 50 个连续感官字段。样本编号、条码和风格标签均不作为化合物预测特征。训练折内拟合 1%—99% 截尾、缺失中位数与标准化。采用按风格分层的 {cfg['outer_folds']} 折、{cfg['outer_repeats']} 次重复外层验证，岭回归和 PLS 的超参数在外层训练数据内再做 {cfg['inner_folds']} 折选择。每个样本的重复留出预测取平均。预先指定岭回归作为下游参考，未根据外层成绩选择冠军再声称独立测试性能。

| 方法 | 宏平均 MAE | 宏平均 RMSE | 宏平均 R² |
|---|---|---|---|
```

```
"""
    for name in order:
        v = metrics.loc[name]
        report += f"| {name} | {v.mae:.6f} | {v.rmse:.6f} | {v.r2:.6f} |\n"
    report += f"""
岭回归相对于训练均值参照的宏平均 MAE 变化为 {improvement:.2%} 的下降。该结果衡量给定连续标签的插值预测，不能当作真实 0–6 分精度，也不证明对全新风格的迁移能力。逐维误差、各风格误差、折分配和附加缺失压力实验均已保存。图中的折间标准差不是置信区间。

`ASSUMPTION_percentile_grades_0_6.csv` 采用训练折目标的七等频经验分位边界，只是有明确假设的参考等级。它不能恢复“未感知”等绝对评分语义，集中于同一值的目标也会出现重复分位边界。绝对标定仍为 UNRESOLVED。最终模型的全数据拟合系数仅供复现与下游画像，不用于报告样本外精度。

![第一问验证](figures/q1_validation.png)

## 第二问 处理 方法与结果

按全矩阵秩相关一致性核对附件 1 与 2 的变量别名。网络边保留绝对相关大于等于 {cfg['graph_threshold']} 的关系，保存符号与绝对权重。用符号拉普拉斯约束岭回归，求解 (ZᵀZ + αI + λL)B = Zᵀ(Y − 均值)。L 的非对角元素为负的有符号相关权，度数取绝对权和，因而为半正定。固定 α={cfg['graph_ridge_alpha']}、λ={cfg['graph_ridge_lambda']}；这是探索参数，尚未证明最优。

Top-3 仅从明确的化合物或元素字段中选择，排除 pH、色度、综合总量及混合指标。每个目标给出 3 项，共 {len(top)} 条，附方向、秩相关、网络模块、来源元数据及人工机理核查字段。{cfg['bootstrap_repeats']} 次样本 bootstrap 中重新拟合预处理和网络，再计算入选频率。与无图岭回归、边际秩相关 Top-3 比较 Jaccard。

当前选定因素中，入选频率低于 0.5 的有 {low}/{len(top)} 条，提示单个因素排序存在明显不稳定性。Graph/ridge 的平均 Jaccard 为 {jaccard.graph_vs_ridge_jaccard.mean():.4f}，Graph/Spearman 为 {jaccard.graph_vs_spearman_jaccard.mean():.4f}。这些差异应保留，不宜只展示一致的维度。来源与缺陷描述来自附件元数据，每个目标的具体化学机理论证仍需队员查阅原始文献并复核。全数据关联分析不构成独立预测验证或因果识别。

较高频候选包括 4VG 感官目标与 X4VG、金属味与 iron、烘烤麦芽感官与 methylpyrazine；它们仍只是当前重采样设计下的候选。某些目标排名出现反直觉化合物，说明模型和化学论证需要继续改进，不能把完整 Top-3 表当作已经确定的机理清单。

![第二问稳定性](figures/q2_stability.png)

## 第三问 处理 方法与结果

分层留出 20% 样本作为生成模型诊断参考。训练数据上拟合预处理和 {cfg['generation_components']} 维 PCA；各风格采用独立均值，协方差按 n/(n+10) 与全体训练协方差部分汇聚。采样后还原到附件特征尺度并限制在训练 1%—99% 范围，生成每种指定风格 {cfg['generation_samples_per_style']} 组化合物或理化描述符。全数据拟合的第一问参考模型输出感官画像，属于模型推断。

风格分类器在生成模型留出集上的平衡准确率为 {q3status['style_classifier_holdout_balanced_accuracy']:.4f}。该分类器与生成模型共享训练资料，并不是外部独立鉴定；其保留样本表现与生成物风格概率仅作模型诊断。Kriek、Lambic 等风格的真实留出样本很少，KS/MMD 指标无法证明完整分布已被可靠学会。KS 仅比较各字段实际非缺失的留出观测，字段数量另列；MMD 使用训练预处理后的表示。详见 generation_diagnostics.csv，并保留真实训练样本重采样的参照 MMD。

跨风格迁移先在潜空间施加风格均值差，再搜索连续插值候选。DMS、双乙酰、4VG 和 trans-2-nonenal 的附件字段逐项不得增加；Stale hops 没有直接化合物观测，以第一问的预测值不增加作为单独代理约束。候选还需使固定分类器的 Tripel 概率相对于原始样本以及仅做截尾的对照均增加超过 0.01，避免把异常清理效果混为风格迁移。未找到满足条件的非零改变候选时，保留原样并标为未找到，不计成功。

当前共有 {len(transfers)} 个 Pils/Lager 来源样本，找到非平凡模型内可行候选 {successful} 个；其中生成模型留出来源样本为 {len(heldout)} 个，找到候选 {heldout_success} 个。任何缺陷源值缺失、越出训练经验边界或仅能依赖预测代理的情况均在 transfer_diagnostics.csv 标注。所有字段检查保存在 defect_nonincrease_checks.csv。即使这些检查全部通过，也只能说明被检查的测量量或模型代理满足条件，不能声称实际品评无恶化。

![第三问感官画像](figures/q3_sensory_portraits.png)

![第三问迁移检查](figures/q3_transfer_checks.png)

## 保留结果与下一阶段

可保留：原件哈希和样本键审计、逐字段尺度证据、既定验证下的误差表、Top-3 与重采样频率、生成样本与诊断、逐项迁移约束核查。它们均绑定 config.json 与运行环境。

需要继续研究：真实评分标定锚点；未确定尺度与异常值的来源；目标相关的化学解释；网络与生成参数敏感性；新的独立验证；聚合指标的物理一致性；完整阈值和真实感官实验。原始研究或其他项目中的已发表成绩不属于本次实测结果。所有采纳的 AI 辅助成果须依赛事要求披露并由队内独立验证。

方法详见 docs/，来源详见 docs/SOURCES.md，运行日志详见 docs/RESEARCH_LOG.md。
"""
    (out / "STAGE_REPORT.md").write_text(report, encoding="utf-8")
    checksums = [{"file": str(p.relative_to(out)).replace("\\", "/"), "sha256": sha256(p)} for p in sorted(out.rglob("*")) if p.is_file() and "private" not in p.parts and p.suffix in {".csv", ".json", ".md", ".png"} and p.name not in ["result_checksums.json", "run_manifest.json"]]
    write_json(out / "result_checksums.json", checksums)
```

009_stage2.py

```
"""AI-assisted extension: OpenAI Codex / GPT-6, 2026-10-24.
Scientific limitations and all failed experiments must remain visible.
"""
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import ks_2samp, norm
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import balanced_accuracy_score, mean_absolute_error
from sklearn.model_selection import RepeatedStratifiedKFold, train_test_split
from sklearn.multioutput import MultiOutputRegressor
from sklearn.pipeline import Pipeline

from .audit import run_audit
from .common import ecdf_grades, preprocessing, progress, save_csv, sha256, write_json
from .data import CHEMICAL_DEFECTS
from .q1 import fitted_model, metric_rows
from .q3 import ConditionalPCAGaussian, biased_mmd, chemical_nonincrease


```

```
def conformal_quantile(errors, alpha):
    n = len(errors)
    rank = int(np.ceil((n + 1) * (1 - alpha)))
    if rank > n:
        raise ValueError("Calibration set too small for requested coverage")
    return np.sort(np.asarray(errors), axis=0)[rank - 1]


def assumption_grades(reference, pred):
    reference, pred = np.asarray(reference), np.asarray(pred)
    empirical = ecdf_grades(reference, pred)
    mu, sd = reference.mean(0), np.maximum(reference.std(0), 1e-8)
    gaussian = np.minimum(6, np.floor(7 * norm.cdf((pred - mu) / sd))).astype(int)
    low, high = np.quantile(reference, [.05, .95], axis=0)
    linear = np.rint(6 * np.clip((pred - low) / np.maximum(high - low, 1e-8), 0, 1)).astype(int)
    return np.stack([empirical, gaussian, linear])


def local_convex_samples(real, n, rng):
    """Conditional empirical local mixing; not a physical composition law."""
    real = np.asarray(real)
    scale = np.maximum(np.std(real, axis=0), 1e-8)
    dist = cdist(real / scale, real / scale)
    np.fill_diagonal(dist, np.inf)
    neighbors = np.argsort(dist, axis=1)[:, :min(3, len(real) - 1)]
    out = []
    for _ in range(n):
        anchor = rng.integers(len(real))
        partner = rng.choice(neighbors[anchor]) if len(real) > 1 else anchor
        weight = rng.uniform(.1, .9)
        out.append(weight * real[anchor] + (1 - weight) * real[partner])
    return np.asarray(out)


def run_extension(data, base, out, cfg, ext):
    manifest = pd.read_csv(base / "audit/source_manifest.csv")
    for row in manifest.itertuples():
        if sha256(data.root / row.file) != row.sha256:
            raise ValueError("Source version differs from stage one")
    run_audit(data, out, cfg)
    X, Y = data.X.to_numpy(), data.Y.to_numpy()
    labels = data.meta.tasting_category_fine.to_numpy()
    targets, features = data.Y.columns.tolist(), data.X.columns.tolist()
    for sub in ["q1", "q2", "q3"]:
        (out / sub).mkdir(exist_ok=True)
    splits = list(RepeatedStratifiedKFold(n_splits=3, n_repeats=2, random_state=cfg["seed"]).split(X, labels))
    sums = np.zeros_like(Y)
    counts = np.zeros(len(Y))
    permutation_sums = np.zeros((X.shape[1], Y.shape[1]))
    positive_folds = np.zeros_like(permutation_sums)
    grade_min, grade_max = np.full_like(Y, 6), np.zeros_like(Y)
    grade_disagreement, assignments = [], []
    for fold, (train, test) in enumerate(splits):
        progress(f"Stage 2 Q1/Q2: held-out fold {fold + 1}/6")
        model = Pipeline([("prep", preprocessing()), ("model", MultiOutputRegressor(GradientBoostingRegressor(n_estimators=ext["gbr_estimators"],
learning_rate=ext["gbr_learning_rate"], max_depth=ext["gbr_depth"], min_samples_leaf=5, max_features=.7, random_state=cfg["seed"]), n_jobs=2))])
        # Threads avoid process resource-tracker failures on Unicode Windows temp paths.
        with joblib.parallel_backend("threading"):
            model.fit(X[train], Y[train])
            prediction = model.predict(X[test])
        sums[test] += prediction
        counts[test] += 1
        assignments.extend({"repeat": fold // 3, "fold": fold % 3, "beer": int(data.meta.iloc[i].beer)} for i in test)
        ridge, _ = fitted_model("ridge", X[train], Y[train], cfg)
        ridge_pred = ridge.predict(X[test])
        grades = assumption_grades(Y[train], ridge_pred)
        grade_min[test] = np.minimum(grade_min[test], grades.min(axis=0))
        grade_max[test] = np.maximum(grade_max[test], grades.max(axis=0))
        grade_disagreement.append({"fold": fold, "any_rule_disagreement_fraction": float((grades.max(0) != grades.min(0)).mean()), "mean_grade_
spread": float((grades.max(0) - grades.min(0)).mean())})
        # Exact linear predictor permutation. The trained preprocessing is frozen.
        Z = ridge.named_steps["prep"].transform(X[test])
        coefficients = ridge.named_steps["model"].coef_
        baseline_error = np.abs(Y[test] - ridge_pred).mean(axis=0)
        rng = np.random.default_rng(cfg["seed"] + fold)
        deltas = np.zeros_like(permutation_sums)
        for j in range(X.shape[1]):
            for _ in range(ext["permutation_repeats"]):
                permuted = Z[rng.permutation(len(Z)), j] - Z[:, j]
                pp = ridge_pred + permuted[:, None] * coefficients[:, j]
                deltas[j] += np.abs(Y[test] - pp).mean(axis=0) - baseline_error
        deltas /= ext["permutation_repeats"]
        permutation_sums += deltas
        positive_folds += deltas > 0
    gbr = sums / counts[:, None]
    existing = pd.read_csv(base / "q1/metrics_by_target.csv")
    additional = pd.DataFrame(metric_rows(Y, gbr, targets, "gbr_fixed"))
    ridge_oof = pd.read_csv(base / "q1/oof_ridge_continuous.csv")[targets].to_numpy()
    tree_oof = pd.read_csv(base / "q1/oof_extra_trees_continuous.csv")[targets].to_numpy()
    blend = (ridge_oof + tree_oof) / 2
    additional = pd.concat([additional, pd.DataFrame(metric_rows(Y, blend, targets, "fixed_ridge_tree_blend"))], ignore_index=True)
    all_metrics = pd.concat([existing, additional], ignore_index=True)
    save_csv(all_metrics, out / "q1/metrics_by_target.csv")
    save_csv(all_metrics.groupby("model")[["mae", "rmse", "r2"]].mean().reset_index(), out / "q1/metrics_summary.csv")
    save_csv(pd.concat([data.meta[["beer"]], pd.DataFrame(gbr, columns=targets)], axis=1), out / "q1/oof_gbr_continuous.csv")
    save_csv(pd.DataFrame(assignments), out / "q1/split_assignments.csv")
    save_csv(pd.DataFrame(grade_disagreement), out / "q1/grade_rule_sensitivity.csv")
```

```
    grade_ranges = pd.concat([data.meta[["beer"]], pd.DataFrame(grade_min.astype(int), columns=[c + "__min" for c in targets]), pd.DataFrame(grade_max.astype(int), columns=[c + "__max" for c in targets])], axis=1)
    grade_ranges["status"] = "range_over_assumed_rules_and_CV_fits_not_a_confidence_interval"
    save_csv(grade_ranges, out / "q1/ASSUMPTION_grade_ranges.csv")

    # Proper train / calibration / test split for one demonstrative conformal experiment.
    train, rest = train_test_split(np.arange(len(X)), test_size=.4, random_state=cfg["seed"] + 71)
    calibration, test = train_test_split(rest, test_size=.5, random_state=cfg["seed"] + 72)
    ridge, _ = fitted_model("ridge", X[train], Y[train], cfg)
    q = conformal_quantile(np.abs(Y[calibration] - ridge.predict(X[calibration])), ext["conformal_alpha"])
    pp = ridge.predict(X[test])
    coverage = (np.abs(Y[test] - pp) <= q).mean(axis=0)
    save_csv(pd.DataFrame({"target": targets, "nominal_marginal_coverage": .9, "empirical_test_coverage": coverage, "interval_width": 2 * q, "calibration_n": len(calibration), "test_n": len(test)}), out / "q1/conformal_coverage.csv")
    save_csv(pd.DataFrame({"beer": data.meta.beer, "role": ["train" if i in set(train) else "calibration" if i in set(calibration) else "test" for i in range(len(X))]}), out / "q1/conformal_split.csv")

    progress("Stage 2 Q2: assembling held-out attribution evidence")
    importances = permutation_sums / len(splits)
    rows = [{"target": target, "field": field, "heldout_permutation_MAE_increase": importances[j, k], "positive_fold_fraction": positive_folds[j, k] / len(splits)} for k, target in enumerate(targets) for j, field in enumerate(features)]
    importance_frame = pd.DataFrame(rows)
    save_csv(importance_frame, out / "q2/heldout_permutation_importance.csv")
    top = pd.read_csv(base / "q2/top3_by_sensory_dimension.csv").merge(importance_frame, on=["target", "field"], validate="one_to_one")
    top["evidence_status"] = np.where((top.bootstrap_top3_frequency >= .5) & (top.heldout_permutation_MAE_increase > 0) & (top.positive_fold_fraction >= 2 / 3), "comparatively_supported_association", "weak_or_unstable_association")
    top["causality"] = "not_established"
    save_csv(top, out / "q2/top3_evidence_review.csv")

    progress("Stage 2 Q3: comparing conditional generators across held-out splits")
    rng = np.random.default_rng(cfg["seed"])
    results = []
    final_generator, final_classifier, generated_final = None, None, []
    first_train = None
    for repeat in range(ext["generation_repeats"]):
        train, test = train_test_split(np.arange(len(X)), test_size=.2, stratify=labels, random_state=cfg["seed"] + repeat)
        generator = ConditionalPCAGaussian(10, cfg["seed"]).fit(X[train], labels[train])
        clf = Pipeline([("prep", preprocessing()), ("model", LogisticRegression(C=1, max_iter=2000, class_weight="balanced", random_state=cfg["seed"]))]).fit(X[train], labels[train])
        acc = balanced_accuracy_score(labels[test], clf.predict(X[test]))
        for style in ext["generation_styles"]:
            real = X[test[labels[test] == style]]
            real_train = generator.prep.named_steps["scale"].inverse_transform(generator.prep.transform(X[train[labels[train] == style]]))
            samples = {"bootstrap": real_train[rng.integers(0, len(real_train), ext["generation_n"])], "local_convex": local_convex_samples(real_train, ext["generation_n"], rng), "PCA_Gaussian": generator.sample(style, ext["generation_n"], rng)}
            for method, values in samples.items():
                Z = generator.prep.transform(values)
                reference = generator.prep.transform(real)
                distances = cdist(Z, generator.prep.transform(X[train]))
                ks = [ks_2samp(values[:, j], real[:, j][np.isfinite(real[:, j])]).statistic for j in range(X.shape[1]) if np.isfinite(real[:, j]).any()]
                results.append({"repeat": repeat, "style": style, "method": method, "n_real_holdout": len(real), "KS_mean": np.mean(ks), "MMD2": biased_mmd(Z, reference), "exact_training_copy_fraction": float((distances.min(1) < 1e-8).mean()), "target_style_probability": clf.predict_proba(values)[:, list(clf.classes_).index(style)].mean(), "style_classifier_holdout_balanced_accuracy": acc})
                if repeat == 0 and method == "local_convex":
                    frame = pd.DataFrame(values, columns=features)
                    frame.insert(0, "style", style)
                    frame.insert(0, "generated_id", [f"LC_{style}_{j+1:03d}" for j in range(len(values))])
                    generated_final.append(frame)
        if repeat == 0:
            final_generator, final_classifier, first_train = generator, clf, train
    diagnostics = pd.DataFrame(results)
    save_csv(diagnostics, out / "q3/generator_comparison_by_split.csv")
    save_csv(diagnostics.groupby(["method", "style"])[["KS_mean", "MMD2", "exact_training_copy_fraction", "target_style_probability"]].mean().reset_index(), out / "q3/generator_comparison_mean.csv")
    generated = pd.concat(generated_final, ignore_index=True)
    save_csv(generated, out / "q3/local_convex_generated_profiles.csv")
    bundle = joblib.load(base / "q1/private/model.joblib")
    predictor = bundle["model"]
    portraits = pd.concat([generated[["generated_id", "style"]], pd.DataFrame(predictor.predict(generated[features].to_numpy()), columns=targets)], axis=1)
    save_csv(portraits, out / "q3/local_convex_sensory_continuous.csv")
    transfer_results = expanded_transfer(data, final_generator, final_classifier, first_train, predictor, ext)
    save_csv(transfer_results[0], out / "q3/expanded_transfer_diagnostics.csv")
    save_csv(transfer_results[1], out / "q3/expanded_transfer_profiles.csv")
    save_csv(transfer_results[2], out / "q3/expanded_transfer_defect_checks.csv")
    write_json(out / "status.json", {
        "absolute_score_calibration": "not_identifiable_from_provided_aggregated_per_taster_standardized_scores_without_additional_anchors",
        "grade_sensitivity_disagreement": float(pd.DataFrame(grade_disagreement).any_rule_disagreement_fraction.mean()),
        "conformal_nominal_marginal_coverage": .9, "conformal_empirical_average_coverage": float(coverage.mean()),
        "conformal_caveat": "single random split; marginal coverage on provided continuous labels, not simultaneous or original-scale coverage",
        "comparatively_supported_top3_associations": int((top.evidence_status == "comparatively_supported_association").sum()),
        "top3_total": len(top), "generation_repeated_splits": ext["generation_repeats"],
        "expanded_transfer_nontrivial": int((transfer_results[0].status == "nontrivial_model_feasible_candidate").sum()),
        "physical_or_tasting_validation": "not_performed", "research_stage": "exploratory_extension_after_stage_one_not_an_untouched_external_test",
    })


def expanded_transfer(data, generator, classifier, train, predictor, ext):
    X = data.X.to_numpy()
    features, targets = data.X.columns.tolist(), data.Y.columns.tolist()
    labels = data.meta.tasting_category_fine.to_numpy()
    positions = [features.index(c) for c in CHEMICAL_DEFECTS]
    stale = targets.index("stale_hops")
    target_train = X[train[labels[train] == "Tripel"]]
    target_train = generator.prep.named_steps["scale"].inverse_transform(generator.prep.transform(target_train))
```

```
    target_class = list(classifier.classes_).index("Tripel")
    diagnostics, profiles, checks = [], [], []
    for i in np.flatnonzero(labels == "Pils/Lager"):
        x = np.where(np.isfinite(X[i]), X[i], generator.median)
        source_pred = predictor.predict(x[None])[0]
        base_prob = classifier.predict_proba(x[None])[0, target_class]
        cleaned = np.clip(x, generator.low, generator.high)
        cleaned[positions] = np.minimum(cleaned[positions], x[positions])
        clean_prob = classifier.predict_proba(cleaned[None])[0, target_class]
        candidates = []
        for target in target_train:
            for weight in np.linspace(.025, 1, ext["transfer_weights"]):
                value = np.clip(x + weight * (target - x), generator.low, generator.high)
                value[positions] = np.minimum(value[positions], x[positions])
                candidates.append(value)
        candidates = np.asarray(candidates)
        pp = predictor.predict(candidates)
        probabilities = classifier.predict_proba(candidates)[:, target_class]
        valid = chemical_nonincrease(x, candidates, positions) & (pp[:, stale] <= source_pred[stale] + 1e-9) & (probabilities > max(base_prob, clean_prob) + .01)
        if valid.any():
            ids = np.flatnonzero(valid)
            j = ids[np.argmax(probabilities[ids])]
            value, portrait, prob, status = candidates[j], pp[j], probabilities[j], "nontrivial_model_feasible_candidate"
        else:
            value, portrait, prob, status = x, source_pred, base_prob, "no_nontrivial_candidate_found_source_retained"
        beer = int(data.meta.iloc[i].beer)
        missing = [c for c, j in zip(CHEMICAL_DEFECTS, positions) if not np.isfinite(X[i, j])]
        diagnostics.append({"beer": beer, "status": status, "generator_role": "train" if i in set(train) else "holdout", "source_trip_probability": base_prob, "candidate_trip_probability": prob, "cleaning_only_probability": clean_prob, "change_l2": np.linalg.norm(value - x), "missing_source_defects": ",".join(missing), "all_source_defects_observed": not missing, "stale_constraint": "predicted_proxy_only"})
        profiles.append({"beer": beer, **dict(zip(features, value))})
        for name, j in zip(CHEMICAL_DEFECTS, positions):
            checks.append({"beer": beer, "defect": name, "source": x[j], "candidate": value[j], "nonincrease": bool(value[j] <= x[j] + 1e-9), "source_observed": bool(np.isfinite(X[i, j])), "scale": "provided_measurement"})
        checks.append({"beer": beer, "defect": "stale_hops", "source": source_pred[stale], "candidate": portrait[stale], "nonincrease": bool(portrait[stale] <= source_pred[stale] + 1e-9), "source_observed": False, "scale": "learned_sensory_proxy"})
    return pd.DataFrame(diagnostics), pd.DataFrame(profiles), pd.DataFrame(checks)
```

010_stage3_algorithms.py

```
"""Stage-three algorithm prototypes; AI-assisted, OpenAI Codex, 2026-10-25.

These helpers validate mathematical properties. They are not evidence that the
proposed full study improves real prediction or chemical feasibility.
"""
import numpy as np


def fixed_grade_boundaries(training_targets):
    """Fit once on training labels; never refit with calibration/test labels."""
    y = np.asarray(training_targets, float)
    if y.ndim != 2 or not np.isfinite(y).all() or len(y) < 2:
        raise ValueError("Expected at least two finite training rows")
    return np.quantile(y, np.arange(1, 7) / 7, axis=0)


def grade_with_fixed_boundaries(values, boundaries):
    values, boundaries = np.asarray(values, float), np.asarray(boundaries, float)
    if values.ndim != 2 or boundaries.shape != (6, values.shape[1]):
        raise ValueError("Grade dimensions do not agree")
    if not np.isfinite(values).all() or not np.isfinite(boundaries).all() or np.any(np.diff(boundaries, axis=0) < 0):
        raise ValueError("Need finite values and ordered boundaries")
    return (values[:, None, :] > boundaries[None, :, :]).sum(axis=1).astype(int)


def ordinal_interval_image(lower, upper, boundaries):
    """Conservative image of a continuous interval under a fixed monotone map.

    Coverage, if applicable, is for this operationally defined grade only.
    Tied boundaries can make the interval hull include unreachable categories.
    """
    lower, upper = np.asarray(lower, float), np.asarray(upper, float)
    if lower.shape != upper.shape or np.any(lower > upper):
        raise ValueError("Invalid continuous intervals")
    return grade_with_fixed_boundaries(lower, boundaries), grade_with_fixed_boundaries(upper, boundaries)


def shrunk_blend_weights(y_inner_oof, et_inner_oof, gbr_inner_oof, penalty):
    """Fit using inner OOF predictions only; choose penalty within inner training.

    w[j] multiplies GBR and (1-w[j]) multiplies ExtraTrees. The closed-form
    objective is mean squared error plus penalty*(w[j]-global_weight)**2.
    """
    y, et, gbr = [np.asarray(a, float) for a in (y_inner_oof, et_inner_oof, gbr_inner_oof)]
    if y.ndim != 2 or et.shape != y.shape or gbr.shape != y.shape or penalty < 0:
        raise ValueError("Invalid blend inputs")
    if not all(np.isfinite(a).all() for a in (y, et, gbr)):
        raise ValueError("Blend arrays must be finite")
    diff, residual = gbr - et, y - et
    denom = np.mean(diff * diff)
    global_weight = float(np.clip(np.mean(diff * residual) / denom, 0, 1)) if denom > 1e-14 else 0.5
    local_denom = np.mean(diff * diff, axis=0) + penalty
    numerator = np.mean(diff * residual, axis=0) + penalty * global_weight
    weights = np.divide(numerator, local_denom, out=np.full(y.shape[1], global_weight), where=local_denom > 1e-14)
```

```
    return np.clip(weights, 0, 1), global_weight


def paired_oof_bootstrap(y, pred_a, pred_b, strata, repeats=2000, seed=20261025):
    """Stratified beer-level resampling of fixed predictions; NOT a retraining CI.

    Inputs must have one row per unique beer, with repeats already averaged.
    Negative differences favour A. Macro MAE is average over rows and targets.
    """
    y, a, b = [np.asarray(x, float) for x in (y, pred_a, pred_b)]
    labels = np.asarray(strata)
    if y.ndim != 2 or a.shape != y.shape or b.shape != y.shape or len(labels) != len(y):
        raise ValueError("Paired dimensions differ")
    if repeats < 1 or not all(np.isfinite(x).all() for x in (y, a, b)):
        raise ValueError("Need finite paired predictions and positive repeats")
    per_beer = (np.abs(y-a)-np.abs(y-b)).mean(axis=1)
    groups = [np.flatnonzero(labels == label) for label in np.unique(labels)]
    rng = np.random.default_rng(seed)
    draws = np.array([np.mean(per_beer[np.concatenate([rng.choice(g, len(g), replace=True) for g in groups])]) for _ in range(repeats)])
    return {"difference_A_minus_B": float(per_beer.mean()), "conditional_percentile_95": np.quantile(draws, [.025, .975]).tolist(),
            "scope": "conditional_on_fixed_OOF_predictions_does_not_include_model_retraining_or_selection_uncertainty",
            "n_unique_beers": len(y), "resamples": repeats}


def pareto_minimize(objectives):
    """Keep non-dominated finite candidates; no weighting can hide a violation."""
    values = np.asarray(objectives, float)
    if values.ndim != 2:
        raise ValueError("Expected candidates by objectives")
    keep = np.isfinite(values).all(axis=1)
    for i in np.flatnonzero(keep):
        dominated = np.all(values <= values[i], axis=1) & np.any(values < values[i], axis=1) & np.isfinite(values).all(axis=1)
        if dominated.any():
            keep[i] = False
    return keep


def threshold_excess(concentration, threshold):
    """Use only after unit and scale verification; invalid inputs stay unknown."""
    concentration, threshold = np.broadcast_arrays(np.asarray(concentration, float), np.asarray(threshold, float))
    valid = np.isfinite(concentration) & np.isfinite(threshold) & (concentration >= 0) & (threshold > 0)
    out = np.full(concentration.shape, np.nan)
    out[valid] = np.maximum(concentration[valid] / threshold[valid] - 1, 0)
    return out
```

011_stage3_generation.py

```
"""Disjoint-search/evaluation conditional generation and Pareto transfer.
AI-assisted reference implementation; no actual brewing or tasting validation.
"""
from pathlib import Path
import hashlib
import time
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist
from scipy.stats import ks_2samp
from sklearn.ensemble import ExtraTreesClassifier, ExtraTreesRegressor
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import balanced_accuracy_score
from sklearn.pipeline import Pipeline
from .common import preprocessing, save_csv, write_json, progress
from .data import AGGREGATES, CHEMICAL_DEFECTS
from .q3 import ConditionalPCAGaussian
from .stage2 import local_convex_samples
from .stage3_algorithms import pareto_minimize, threshold_excess


def mmd_fixed_bandwidth(a,b,bandwidth):
    if len(a)<2 or len(b)<2: return np.nan
    return float(np.exp(-cdist(a,a,'sqeuclidean')/(2*bandwidth)).mean()+np.exp(-cdist(b,b,'sqeuclidean')/(2*bandwidth)).mean()-2*np.exp(-cdist(a,b
,'sqeuclidean')/(2*bandwidth)).mean())


def loo_support_radius(z):
    if len(z)<2: return np.nan
    distance=cdist(z,z); np.fill_diagonal(distance,np.inf)
    return float(np.quantile(distance.min(1),.95))


def constrained_convex(real,generator,style_positions,n,rng):
    Z=generator.prep.transform(real); distance=cdist(Z,Z); np.fill_diagonal(distance,np.inf)
    neighbors=np.argsort(distance,axis=1)[:,:min(3,len(real)-1)]; radius=loo_support_radius(Z); result=[]; attempts=0
    if len(real)<2: return real[np.zeros(n,dtype=int)].copy(),{'attempts':n,'singleton_fallback':True,'support_radius':None}
    while len(result)<n and attempts<n*250:
        attempts+=1; a=rng.integers(len(real)); b=rng.choice(neighbors[a]); weight=rng.uniform(.1,.9)
        value=weight*real[a]+(1-weight)*real[b]
        for j in style_positions:
            value[j]=np.log10(weight*10**real[a,j]+(1-weight)*10**real[b,j])
        value=np.clip(value,generator.low,generator.high)
        if cdist(generator.prep.transform(value[None]),Z).min()<=radius+1e-9: result.append(value)
    return np.asarray(result).reshape((-1,real.shape[1])),{'attempts':attempts,'singleton_fallback':False,'support_radius':radius}
```

```
def locked_transfer_candidates(X,labels,beers,G,generator,searcher,sensory_a,consensus_a,features,targets,cfg):
    positions=[features.index(t) for t in CHEMICAL_DEFECTS]; logpos=[features.index(t) for t in ['DMS','X4VG']]
    target_class=list(searcher.classes_).index('Tripel'); stale=targets.index('stale_hops')
    target_data=generator.prep.named_steps['scale'].inverse_transform(generator.prep.transform(X[G[labels[G]=='Tripel']]))
    target_Z=generator.prep.transform(target_data); support=loo_support_radius(target_Z)
    result=[]
    for i in np.flatnonzero(labels=='Pils/Lager'):
        source=np.where(np.isfinite(X[i]),X[i],generator.median); sourceZ=generator.prep.transform(source[None])
        source_p=searcher.predict_proba(source[None])[0,target_class]; stale_a=sensory_a.predict(source[None])[0,stale]; stale_c=consensus_a.predict(source[None])[0,stale]
        cleaned=np.clip(source,generator.low,generator.high); cleaned[positions]=np.minimum(cleaned[positions],source[positions]); clean_p=searcher.predict_proba(cleaned[None])[0,target_class]
        values=[]
        for anchor in target_data:
            for weight in np.linspace(.025,1,cfg['transfer_weights']):
                v=np.clip(source+weight*(anchor-source),generator.low,generator.high); v[positions]=np.minimum(v[positions],source[positions]); values.append(v)
        values=np.asarray(values); Z=generator.prep.transform(values); changes=np.linalg.norm(Z-sourceZ,axis=1)/np.sqrt(len(features)); support_distance=cdist(Z,target_Z).min(1)
        probabilities=searcher.predict_proba(values)[:,target_class]; pa=sensory_a.predict(values); pc=consensus_a.predict(values)
        bounds=((values>=generator.low-1e-9)&(values<=generator.high+1e-9)).all(1)
        nonincrease=(values[:,positions]<=source[positions]+1e-9).all(1)
        known_proxy_risk=np.maximum(threshold_excess(10**values[:,logpos[0]],19.),threshold_excess(10**values[:,logpos[1]],.2))
        valid=bounds&nonincrease&(pa[:,stale]<=stale_a+1e-9)&(pc[:,stale]<=stale_c+1e-9)&(probabilities>=max(cfg['transfer_target_probability_floor'],source_p+.01,clean_p+.01))&(support_distance<=support+1e-9)&(changes>1e-8)
        for radius in cfg['trust_radii']:
            ids=np.flatnonzero(valid&(changes<=radius)); pareto_ids=[]
            if len(ids):
                nd=pareto_minimize(np.c_[changes[ids],support_distance[ids],known_proxy_risk[ids]]); pareto_ids=ids[nd]; j=int(pareto_ids[np.argmin(changes[pareto_ids])]); value=values[j]; status='model_feasible_nontrivial' if np.isfinite(X[i,positions]).all() else 'conditional_on_imputed_source_defect'
                p=float(probabilities[j]); change=float(changes[j]); target_distance=float(support_distance[j])
            else:
                value=source.copy(); status='no_feasible_nontrivial_candidate_source_retained'; p=float(source_p); change=0.; target_distance=float(cdist(sourceZ,target_Z).min())
            result.append({'row':int(i),'beer':int(beers[i]),'radius':radius,'source':source.copy(),'candidate':value.copy(),'status':status,'n_candidates':len(values),'n_valid':len(ids),'n_pareto':len(pareto_ids),
                'search_source_probability':float(source_p),'search_candidate_probability':p,'standardized_change':change,'target_support_distance':target_distance,
                'all_provided_source_defects_observed':bool(np.isfinite(X[i,positions]).all()),'missing_defects':','.join(features[j] for j in positions if not np.isfinite(X[i,j]))})
    return result


def run_generation(data,out,cfg):
    start=time.perf_counter(); dest=Path(out)/'q3'; dest.mkdir(parents=True,exist_ok=True)
    features=[f for f in data.X if f not in AGGREGATES]; targets=list(data.Y); X=data.X[features].to_numpy(); Y=data.Y.to_numpy(); labels=data.meta.tasting_category_fine.to_numpy(); beers=data.meta.beer.to_numpy()
    core=[j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]; log_positions=[features.index(f) for f in ['DMS','X4VG']]; stale=targets.index('stale_hops'); positions=[features.index(f) for f in CHEMICAL_DEFECTS]
    metrics=[]; transfers=[]; checks=[]; roles=[]; evaluator_rows=[]; locked=[]; profiles=[]; portraits=[]; generated_profiles=[]
    for repeat in range(cfg['generation_repeats']):
        seed=cfg['seed']+repeat; rng=np.random.default_rng(seed); progress(f'S3 Q3 disjoint partition {repeat+1}/5')
        dev,T=train_test_split(np.arange(len(X)),test_size=50,stratify=labels,random_state=seed)
        G,J=train_test_split(dev,test_size=100,stratify=labels[dev],random_state=seed+999)
        assert len(G)==len(J)==100 and len(T)==50 and not(set(G)&set(J) or set(G)&set(T) or set(J)&set(T))
        roles.extend({'repeat':repeat,'beer':int(beers[i]),'role':'G_search_training' if i in set(G) else 'J_evaluator_training' if i in set(J) else 'T_real_holdout'} for i in range(len(X)))
        generator=ConditionalPCAGaussian(10,seed).fit(X[G],labels[G])
        searcher=Pipeline([('prep',preprocessing()),('model',LogisticRegression(C=1,max_iter=2000,class_weight='balanced',random_state=seed))]).fit(X[G],labels[G])
        sensory_a=Pipeline([('prep',preprocessing()),('model',Ridge(alpha=100))]).fit(X[G],Y[G])
        consensus_a=Pipeline([('prep',preprocessing()),('model',ExtraTreesRegressor(n_estimators=200,min_samples_leaf=3,max_features=.8,random_state=seed,n_jobs=2))]).fit(X[G],Y[G])
        generated=[]
        for style in cfg['generation_styles']:
            real=generator.prep.named_steps['scale'].inverse_transform(generator.prep.transform(X[G[labels[G]==style]]))
            constrained,info=constrained_convex(real,generator,log_positions,cfg['generation_n'],rng)
            samples={'bootstrap':real[rng.integers(0,len(real),cfg['generation_n'])],'local_convex':local_convex_samples(real,cfg['generation_n'],rng),
                'PCA_Gaussian':generator.sample(style,cfg['generation_n'],rng),'constrained_local_convex':constrained}
            for method,values in samples.items(): generated.append({'style':style,'method':method,'values':values,'training_n':len(real),'extra':info if method=='constrained_local_convex' else {}})
        selected=locked_transfer_candidates(X,labels,beers,G,generator,searcher,sensory_a,consensus_a,features,targets,cfg)
        digest=hashlib.sha256()
        for group in generated: digest.update(group['values'].tobytes())
        for row in selected: digest.update(row['candidate'].tobytes())
        lock={'repeat':repeat,'candidate_sha256':digest.hexdigest(),'selection_used_J_or_T':False,'n_generated':sum(len(a['values']) for a in generated),'n_transfer_records':len(selected)}
        locked.append(lock); write_json(dest/f'locked_candidates_{repeat}.json',lock)
        # J models are fitted only after candidates and their order are fixed.
        evaluator=Pipeline([('prep',preprocessing()),('model',ExtraTreesClassifier(n_estimators=200,min_samples_leaf=2,max_features=.7,class_weight='balanced',random_state=seed+3000,n_jobs=2))]).fit(X[J],labels[J])
        sensory_b=Pipeline([('prep',preprocessing()),('model',ExtraTreesRegressor(n_estimators=200,min_samples_leaf=3,max_features=.8,random_state=seed+4000,n_jobs=2))]).fit(X[J],Y[J])
        acc_a=balanced_accuracy_score(labels[T],searcher.predict(X[T])); acc_b=balanced_accuracy_score(labels[T],evaluator.predict(X[T])); y_a=sensory_a.predict(X[T]); y_b=sensory_b.predict(X[T])
        evaluator_rows.append({'repeat':repeat,'searcher_holdout_balanced_accuracy':acc_a,'evaluator_holdout_balanced_accuracy':acc_b,'search_sensory_core30_mae':np.abs(Y[T][:,core]-y_a[:,core]).mean(),'evaluator_sensory_core30_mae':np.abs(Y[T][:,core]-y_b[:,core]).mean()})
        allG=generator.prep.transform(X[G]); distance=cdist(allG,allG,'sqeuclidean'); bandwidth=float(np.median(distance[distance>0]))
        for group in generated:
            style,method,values=group['style'],group['method'],group['values']; test=X[T[labels[T]==style]]; Z=generator.prep.transform(values) if len(values) else np.empty((0,len(features)))
            if not len(values):
                metrics.append({'repeat':repeat,'style':style,'method':method,'n_generated':0,'n_G_style':group['training_n'],'n_T_style':len(test),'status':'generation_budget_exhausted_no_sample'}); continue
            testZ=generator.prep.transform(test); nn=cdist(Z,allG).min(1)
```

```
            prob_a=searcher.predict_proba(values)[:,list(searcher.classes_).index(style)]; prob_b=evaluator.predict_proba(values)[:,list(evaluator.classes_).index(style)]
            jstyle=X[J[labels[J]==style]]; jz=evaluator.named_steps['prep'].transform(jstyle); threshold=loo_support_radius(jz); distances_b=cdist(evaluator.named_steps['prep'].transform(values),jz).min(1)
            ks=[ks_2samp(values[:,j],test[:,j][np.isfinite(test[:,j])]).statistic for j in range(X.shape[1]) if np.isfinite(test[:,j]).any()]
            metrics.append({'repeat':repeat,'style':style,'method':method,'n_generated':len(values),'n_G_style':group['training_n'],'n_J_style':len(jstyle),'n_T_style':len(test),
                'MMD2_fixed_training_bandwidth':mmd_fixed_bandwidth(Z,testZ,bandwidth),'KS_mean':np.mean(ks),'exact_copy_fraction':(nn<1e-8).mean(),
                'search_probability':prob_a.mean(),'independent_evaluator_probability':prob_b.mean(),'independent_evaluator_target_prediction_fraction':np.mean(evaluator.predict(values)==style),
                'J_support_fraction':np.mean(distances_b<=threshold) if np.isfinite(threshold) else np.nan,
                'provided_bounds_violation_fraction':np.mean(~((values>=generator.low-1e-9)&(values<=generator.high+1e-9)).all(1)),
                'sensory_proxy_disagreement':np.abs(sensory_a.predict(values)-sensory_b.predict(values)).mean(),
                'bandwidth_from_G':bandwidth,'status':'statistical_profiles_not_actual_tasting','singleton_fallback':group['extra'].get('singleton_fallback',False)})
            if repeat==0:
                ids=[f'S3_{method}_{style}_{i+1:03d}' for i in range(len(values))]
                frame=pd.DataFrame(values,columns=features); frame.insert(0,'style',style); frame.insert(0,'method',method); frame.insert(0,'generated_id',ids); generated_profiles.append(frame)
                if method=='constrained_local_convex':
                    portrait=pd.DataFrame(sensory_a.predict(values),columns=targets); portrait.insert(0,'style',style); portrait.insert(0,'generated_id',ids); portraits.append(portrait)
        for row in selected:
            i=row['row']; source,candidate=row['source'],row['candidate']; pb0=evaluator.predict_proba(source[None])[0,list(evaluator.classes_).index('Tripel')]; pb1=evaluator.predict_proba(candidate[None])[0,list(evaluator.classes_).index('Tripel')]
            sb0=sensory_b.predict(source[None])[0,stale]; sb1=sensory_b.predict(candidate[None])[0,stale]
            nontrivial=row['status']!='no_feasible_nontrivial_candidate_source_retained'
            b_accept=nontrivial and pb1>pb0+.01 and evaluator.predict(candidate[None])[0]=='Tripel' and sb1<=sb0+1e-9
            record={k:v for k,v in row.items() if k not in ['row','source','candidate']}
            record.update(repeat=repeat,source_role='G_search_training' if i in set(G) else 'J_evaluator_training' if i in set(J) else 'T_real_holdout',
                evaluator_source_probability=pb0,evaluator_candidate_probability=pb1,evaluator_predicts_Tripel=bool(evaluator.predict(candidate[None])[0]=='Tripel'),
                evaluator_stale_source=sb0,evaluator_stale_candidate=sb1,evaluator_proxy_acceptance=bool(b_accept),absolute_chemical_certification=False,real_tasting_validated=False)
            transfers.append(record)
            profiles.append({'repeat':repeat,'beer':int(beers[i]),'radius':row['radius'],'status':row['status'],**dict(zip(features,candidate))})
            for field,j in zip(CHEMICAL_DEFECTS,positions):
                raw=float(data.calibrated.iloc[i][field]) if field in data.calibrated else np.nan
                supported=bool(np.isfinite(raw) and raw>0 and np.isclose(X[i,j],np.log10(raw) if field in ['DMS','X4VG'] else raw,rtol=1e-6,atol=1e-7))
                checks.append({'repeat':repeat,'beer':int(beers[i]),'radius':row['radius'],'field':field,'source_provided':X[i,j],'source_for_search':source[j],'candidate_provided':candidate[j],
                    'provided_nonincrease':bool(candidate[j]<=source[j]+1e-9),'source_provided_observed':bool(np.isfinite(X[i,j])),
                    'source_positive_calibrated_pair_matches':supported,'absolute_threshold_available_in_attachment':field!='X.E..2.nonenal',
                    'complete_absolute_certification':False,'caveat':'scale_and_detection_limit_conditions_not_fully_verified; candidate_has_no_new_measurement'})
        save_csv(pd.DataFrame(metrics),dest/'generation_comparison.csv'); save_csv(pd.DataFrame(transfers),dest/'transfer_independent_evaluation.csv')
    save_csv(pd.DataFrame(roles),dest/'disjoint_roles.csv'); save_csv(pd.DataFrame(evaluator_rows),dest/'real_holdout_diagnostics.csv'); save_csv(pd.DataFrame(checks),dest/'transfer_defect_checks.csv')
    save_csv(pd.DataFrame(profiles),dest/'transfer_profiles.csv'); save_csv(pd.concat(generated_profiles,ignore_index=True),dest/'generated_profiles_first_split.csv'); save_csv(pd.concat(portraits,ignore_index=True),dest/'generated_sensory_proxy_first_split.csv')
    summary=pd.DataFrame(metrics).groupby(['method','style'])[['MMD2_fixed_training_bandwidth','KS_mean','exact_copy_fraction','search_probability','independent_evaluator_probability','independent_evaluator_target_prediction_fraction','provided_bounds_violation_fraction']].mean().reset_index(); save_csv(summary,dest/'generation_summary.csv')
    write_json(dest/'status.json',{'complete':True,'repeats':5,'G_n':100,'J_n':100,'T_n':50,'basic_fields':len(features),'excluded_unverified_aggregates':sorted(AGGREGATES),
        'candidate_locks':locked,'independent_evaluation_scope':'J is disjoint within the current run; all original beers were seen in earlier research stages, not new external validation',
        'absolute_chemical_certification':False,'real_brewing_or_tasting':False,'elapsed_seconds':time.perf_counter()-start})
    progress('S3 Q3 complete')
```

012_stage3_graph.py

```
"""Train-only graph ablations and cross-fitted exploratory explanations.
AI-assisted, OpenAI Codex, 2026-10-25. Graph regularization is an existing method.
"""
from pathlib import Path
import time
import numpy as np
import pandas as pd
import networkx as nx
from scipy.linalg import solve
from scipy.stats import rankdata
from sklearn.model_selection import KFold, RepeatedStratifiedKFold
from .common import preprocessing, progress, save_csv, write_json
from .q2 import signed_laplacian, top_indices
from .data import NON_MOLECULES
from .stage3_algorithms import paired_oof_bootstrap


def train_correlation(Z):
    with np.errstate(invalid='ignore',divide='ignore'):
        corr=np.corrcoef(rankdata(Z,axis=0),rowvar=False)
    return np.nan_to_num(corr)


def solve_coeff(Z,Y,L,alpha,lam):
    return solve(Z.T@Z+alpha*np.eye(Z.shape[1])+lam*L,Z.T@(Y-Y.mean(0)),assume_a='pos',check_finite=False)


def candidates(cfg,kind):
```

```
    return [(alpha,lam,tau) for alpha in cfg['ridge_alphas'] for lam in ([0] if kind=='none' else cfg['graph_lambdas']) for tau in ([.7] if lam==0 else cfg['graph_thresholds'])]


def fit_tuned_graph(X,Y,cfg,kind,core,seed):
    choices=candidates(cfg,kind); scores=np.zeros(len(choices))
    for fold,(train,test) in enumerate(KFold(3,shuffle=True,random_state=seed).split(X)):
        prep=preprocessing(); Z=prep.fit_transform(X[train]); V=prep.transform(X[test]); corr=train_correlation(Z)
        lhs=Z.T@Z; rhs=Z.T@(Y[train]-Y[train].mean(0)); identity=np.eye(Z.shape[1])
        layers={tau:signed_laplacian(corr,tau) for tau in cfg['graph_thresholds']}
        perms=[np.random.default_rng(seed+1009*k+fold).permutation(Z.shape[1]) for k in range(cfg['graph_null_repeats'])] if kind=='null' else [None]
        for c,(alpha,lam,tau) in enumerate(choices):
            L=layers[tau] if lam else np.zeros_like(lhs)
            for perm in perms:
                LL=L[np.ix_(perm,perm)] if perm is not None else L
                beta=solve(lhs+alpha*identity+lam*LL,rhs,assume_a='pos',check_finite=False)
                pp=V@beta+Y[train].mean(0)
                scores[c]+=np.abs(Y[test][:,core]-pp[:,core]).sum()/(len(Y)*len(core)*len(perms))
    best=min(range(len(choices)),key=lambda i:(scores[i],choices[i][1],-choices[i][0]))
    alpha,lam,tau=choices[best]; prep=preprocessing(); Z=prep.fit_transform(X); corr=train_correlation(Z); L=signed_laplacian(corr,tau)
    if kind=='none': L=np.zeros_like(L)
    betas=[]
    for k in range(cfg['graph_null_repeats'] if kind=='null' else 1):
        perm=np.random.default_rng(seed+20000+k).permutation(Z.shape[1]) if kind=='null' else None
        LL=L[np.ix_(perm,perm)] if perm is not None else L
        betas.append(solve_coeff(Z,Y,LL,alpha,lam))
    return {'prep':prep,'Z':Z,'corr':corr,'beta':betas,'mean':Y.mean(0),'params':{'alpha':alpha,'lambda':lam,'tau':tau},'inner_core_mae':float(scores[best])}


def within_style_indices(labels,rng):
    labels=np.asarray(labels); indices=np.arange(len(labels))
    for label in np.unique(labels):
        ids=np.flatnonzero(labels==label); indices[ids]=rng.permutation(ids)
    return indices


def run_graph(data,out,cfg):
    started=time.perf_counter(); dest=Path(out)/'q2'; dest.mkdir(parents=True,exist_ok=True)
    X,Y=data.X.to_numpy(),data.Y.to_numpy(); features,targets=list(data.X),list(data.Y); labels=data.meta.tasting_category_fine.to_numpy()
    core=np.array([j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]); eligible=np.array([j for j,t in enumerate(features) if t not in NON_MOLECULES])
    counts=np.zeros(len(Y)); totals={kind:np.zeros_like(Y) for kind in ['none','real']}; rows=[]; chosen=[]; effects=[]; modules=[]; style_effects=[]; selected_sets=[]
    for fold,(train,test) in enumerate(RepeatedStratifiedKFold(n_splits=3,n_repeats=cfg['outer_repeats'],random_state=cfg['seed']).split(X,labels)):
        progress(f'S3 Q2 graph fold {fold+1}/15, elapsed {(time.perf_counter()-started)/60:.1f} min')
        fits={kind:fit_tuned_graph(X[train],Y[train],cfg,kind,core,cfg['seed']+fold) for kind in ['none','real','null']}
        for kind,fitted in fits.items():
            V=fitted['prep'].transform(X[test])
            for k,beta in enumerate(fitted['beta']):
                pred=V@beta+fitted['mean']
                for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
                    rows.append({'fold':fold,'repeat':fold//3,'graph':'shuffled' if kind=='null' else kind,'null_rep':k if kind=='null' else -1,'scope':scope,'mae':np.abs(Y[test][:,ix]-pred[:,ix]).mean()})
                if kind in totals: totals[kind][test]+=pred
            chosen.append({'fold':fold,'graph':'shuffled' if kind=='null' else kind,**fitted['params'],'inner_core_mae':fitted['inner_core_mae']})
        counts[test]+=1
        fit=fits['real']; Z=fit['Z']; V=fit['prep'].transform(X[test]); B=fit['beta'][0]; original=V@B+fit['mean']; base=np.abs(Y[test]-original).mean(0)
        rng=np.random.default_rng(cfg['seed']+fold)
        fold_chosen=np.stack([top_indices(B[:,k],eligible) for k in range(Y.shape[1])]); selected_sets.append(fold_chosen)
        for k,target in enumerate(targets):
            for rank,j in enumerate(fold_chosen[k],1):
                all_delta=[]; within_delta=[]
                for repeat in range(cfg['permutation_repeats']):
                    ix=rng.permutation(len(V)); ixw=within_style_indices(labels[test],rng)
                    all_delta.append(np.abs(Y[test,k]-(original[:,k]+(V[ix,j]-V[:,j])*B[j,k])).mean()-base[k])
                    within_delta.append(np.abs(Y[test,k]-(original[:,k]+(V[ixw,j]-V[:,j])*B[j,k])).mean()-base[k])
                effects.append({'fold':fold,'target':target,'rank':rank,'field':features[j],'coefficient':B[j,k],
                    'heldout_delta_mae':np.mean(all_delta),'within_style_delta_mae':np.mean(within_delta),
                    'singleton_test_style_fraction':np.mean([np.sum(labels[test]==label)==1 for label in labels[test]])})
        # Joint module perturbations preserve internal feature relationships.
        graph=nx.Graph(); graph.add_nodes_from(range(X.shape[1])); W=np.abs(fit['corr'])>=fit['params']['tau']; np.fill_diagonal(W,False)
        graph.add_edges_from(zip(*np.where(np.triu(W,1))))
        interesting={frozenset(nx.node_connected_component(graph,int(j))) for j in np.unique(fold_chosen)}
        for group in interesting:
            ix=np.array(sorted(group)); keep=np.setdiff1d(np.arange(X.shape[1]),ix)
            L=signed_laplacian(fit['corr'][np.ix_(keep,keep)],fit['params']['tau'])
            reduced=solve_coeff(Z[:,keep],Y[train],L,fit['params']['alpha'],fit['params']['lambda'])
            drop=V[:,keep]@reduced+fit['mean']; joint=np.zeros(Y.shape[1])
            for repeat in range(cfg['permutation_repeats']):
                order=within_style_indices(labels[test],rng)
                pp=original+(V[np.ix_(order,ix)]-V[:,ix])@B[ix]
                joint+=np.abs(Y[test]-pp).mean(0)-base
            for k,t in enumerate(targets):
                if set(fold_chosen[k])&group:
                    modules.append({'fold':fold,'target':t,'module_fields':';'.join(features[j] for j in ix),'module_n':len(ix),'heldout_module_drop_mae':np.abs(Y[test,k]-drop[:,k]).mean()-base[k],'within_style_joint_permutation_mae':joint[k]/cfg['permutation_repeats']})
        # Style-centering is an explanatory diagnostic, not a Q1 model input.
        zr=Z.copy(); yr=Y[train].copy()
        for label in np.unique(labels[train]):
            ids=np.flatnonzero(labels[train]==label); zr[ids]-=Z[ids].mean(0); yr[ids]-=Y[train][ids].mean(0)
        adjusted=solve_coeff(zr,yr,signed_laplacian(train_correlation(zr),fit['params']['tau']),fit['params']['alpha'],fit['params']['lambda'])
        for k,t in enumerate(targets):
            adjusted_set=set(top_indices(adjusted[:,k],eligible))
```

```
            for j in fold_chosen[k]: style_effects.append({'fold':fold,'target':t,'field':features[j],'still_top3_after_style_centering':j in adjusted_set,'style_adjusted_coefficient':adjusted[j,k]})
        save_csv(pd.DataFrame(rows),dest/'graph_fold_metrics.csv'); save_csv(pd.DataFrame(chosen),dest/'selected_parameters.csv')
    for kind,pred in totals.items(): save_csv(pd.concat([data.meta[['beer']],pd.DataFrame(pred/counts[:,None],columns=targets)],axis=1),dest/f'oof_graph_{kind}.csv')
    summary=pd.DataFrame(rows).groupby(['graph','scope']).mae.agg(['mean','std']).reset_index(); save_csv(summary,dest/'graph_comparison.csv')
    save_csv(pd.DataFrame(effects),dest/'crossfit_top3_effects.csv'); save_csv(pd.DataFrame(modules),dest/'crossfit_module_effects.csv'); save_csv(pd.DataFrame(style_effects),dest/'style_centering_diagnostics.csv')
    paired=[]
    for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
        r=paired_oof_bootstrap(Y[:,ix],totals['real'][:,ix]/counts[:,None],totals['none'][:,ix]/counts[:,None],labels); r['target_scope']=scope; paired.append(r)
    write_json(dest/'paired_graph_vs_none.json',paired)
    progress('S3 Q2: full descriptive graph and 200 conditional bootstrap replicates')
    full=fit_tuned_graph(X,Y,cfg,'real',core,cfg['seed']+500); beta=full['beta'][0]; frequency=np.zeros_like(beta); rng=np.random.default_rng(cfg['seed']); p=full['params']
    for b in range(cfg['bootstrap_repeats']):
        ix=rng.integers(0,len(X),len(X)); z=preprocessing().fit_transform(X[ix]); L=signed_laplacian(train_correlation(z),p['tau']); bb=solve_coeff(z,Y[ix],L,p['alpha'],p['lambda'])
        for k in range(Y.shape[1]): frequency[top_indices(bb[:,k],eligible),k]+=1
    frequency/=cfg['bootstrap_repeats']; effect=pd.DataFrame(effects); adjusted=pd.DataFrame(style_effects); meta=data.chemical_meta.set_index('field'); top=[]
    for k,t in enumerate(targets):
        for rank,j in enumerate(top_indices(beta[:,k],eligible),1):
            selected=effect[(effect.target==t)&(effect.field==features[j])]; centered=adjusted[(adjusted.target==t)&(adjusted.field==features[j])]
            within=float(selected.within_style_delta_mae.mean()) if len(selected) else None
            delta=float(selected.heldout_delta_mae.mean()) if len(selected) else None
            supported=frequency[j,k]>=.5 and len(selected)>=5 and delta is not None and delta>0 and within is not None and within>0
            top.append({'target':t,'rank':rank,'field':features[j],'coefficient':beta[j,k],'bootstrap_frequency_conditional_on_full_selected_params':frequency[j,k],
                'crossfit_selection_count':len(selected),'heldout_delta_on_selected_folds':delta,'within_style_delta_on_selected_folds':within,
                'style_centering_retention_fraction':float(centered.still_top3_after_style_centering.mean()) if len(centered) else None,
                'origin':meta.at[features[j],'origin_paper'],'unit':meta.at[features[j],'unit'],'attachment_off_flavors':meta.at[features[j],'off_flavors'],
                'evidence_status':'comparatively_supported_exploratory_association' if supported else 'weak_or_incomplete_evidence',
                'chemical_review':'target_specific_literature_not_yet_complete','causal_status':'not_identified'})
    save_csv(pd.DataFrame(top),dest/'top3_with_evidence.csv'); save_csv(pd.DataFrame(frequency,index=features,columns=targets).reset_index(names='field'),dest/'bootstrap_frequencies.csv')
    write_json(dest/'status.json',{'complete':True,'outer_folds':15,'null_graphs_per_fit':cfg['graph_null_repeats'],'full_graph_params':p,'bootstrap_repeats':cfg['bootstrap_repeats'],'bootstrap_params_reselected':False,
        'bootstrap_caveat':'conditional_on_hyperparameters_selected_on_full_data; descriptive_stability_not_independent_inference','supported_top3':sum(r['evidence_status'].startswith('comparatively') for r in top),'scope':'exploratory_association_not_causal','elapsed_seconds':time.perf_counter()-started})
    progress('S3 Q2 complete')
```

013_stage3_prediction.py

```
"""Nested shrinkage blending, ordinal sets and robustness; no novelty claim."""
from pathlib import Path
import time
import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import ExtraTreesRegressor, GradientBoostingRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import KFold, RepeatedStratifiedKFold, train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from .common import preprocessing, TrainQuantileClipper, progress, save_csv, write_json
from .q1 import metric_rows
from .stage2 import conformal_quantile
from .stage3_algorithms import fixed_grade_boundaries, grade_with_fixed_boundaries, ordinal_interval_image, shrunk_blend_weights, paired_oof_bootstrap


def tree_model(cfg,kind='et',clip=.01):
    prep=preprocessing(); prep.set_params(clip=TrainQuantileClipper(clip,1-clip))
    if kind=='et':
        model=ExtraTreesRegressor(n_estimators=cfg['extra_trees'],min_samples_leaf=3,max_features=.8,random_state=cfg['seed'],n_jobs=2)
    else:
        model=MultiOutputRegressor(GradientBoostingRegressor(n_estimators=cfg['gbr_estimators'],learning_rate=.04,max_depth=2,min_samples_leaf=5,max_features=.7,random_state=cfg['seed']),n_jobs=2)
    return Pipeline([('prep',prep),('model',model)])


def fit_pair(X,Y,cfg):
    with joblib.parallel_backend('threading'):
        et=tree_model(cfg).fit(X,Y)
        gb=tree_model(cfg,'gbr').fit(X,Y)
    return et,gb


def predict_pair(pair,X):
    with joblib.parallel_backend('threading'):
        return pair[0].predict(X),pair[1].predict(X)


def learn_blend_nested(X,Y,cfg,core,fold_id):
    """No outer-test values enter this routine. V labels never enter S fits."""
    inner=list(KFold(3,shuffle=True,random_state=cfg['seed']+fold_id).split(X))
    pred_et,pred_gb=np.zeros_like(Y),np.zeros_like(Y)
    validation_predictions={eta:np.zeros_like(Y) for eta in cfg['blend_penalties']}
    for vfold,(s,v) in enumerate(inner):
        pe,pg=np.zeros_like(Y[s]),np.zeros_like(Y[s])
        for a,b in KFold(3,shuffle=True,random_state=cfg['seed']+vfold+99).split(s):
```

```
            fitted=fit_pair(X[s[a]],Y[s[a]],cfg)
            pe[b],pg[b]=predict_pair(fitted,X[s[b]])
        fitted=fit_pair(X[s],Y[s],cfg)
        ve,vg=predict_pair(fitted,X[v]); pred_et[v],pred_gb[v]=ve,vg
        for eta in cfg['blend_penalties']:
            w,_=shrunk_blend_weights(Y[s],pe,pg,eta)
            validation_predictions[eta][v]=ve*(1-w)+vg*w
    scores={eta:float(np.abs(Y[:,core]-pred[:,core]).mean()) for eta,pred in validation_predictions.items()}
    best=min(cfg['blend_penalties'],key=lambda eta:(scores[eta],-eta))
    w,glob=shrunk_blend_weights(Y,pred_et,pred_gb,best)
    unshrunk,_=shrunk_blend_weights(Y,pred_et,pred_gb,0)
    return w,unshrunk,glob,best,scores


def run_prediction(data,out,cfg):
    start=time.perf_counter(); dest=Path(out)/'q1'; dest.mkdir(parents=True,exist_ok=True)
    X,Y=data.X.to_numpy(),data.Y.to_numpy(); targets=list(data.Y)
    core=np.array([j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]); labels=data.meta.tasting_category_fine.to_numpy()
    names=['mean','ridge','extra_trees','gbr','fixed_half_blend','global_oof_blend','unshrunk_target_blend','shrunk_target_blend']
    sums={m:np.zeros_like(Y) for m in names}; count=np.zeros(len(Y)); assignments=[]; fold_rows=[]; weight_rows=[]; stress=[]; clipping=[]
    grade_votes=np.zeros((len(Y),Y.shape[1],7),int)
    for fold,(train,test) in enumerate(RepeatedStratifiedKFold(n_splits=cfg['outer_folds'],n_repeats=cfg['outer_repeats'],random_state=cfg['seed
']).split(X,labels)):
        progress(f'S3 Q1 fold {fold+1}/15: nested inner weights; elapsed {(time.perf_counter()-start)/60:.1f} min')
        w,wu,glob,eta,eta_scores=learn_blend_nested(X[train],Y[train],cfg,core,fold)
        pair=fit_pair(X[train],Y[train],cfg); etp,gbp=predict_pair(pair,X[test])
        ridge_search=GridSearchCV(Pipeline([('prep',preprocessing()),('model',Ridge())]),{'model__alpha':cfg['ridge_alphas']},
            scoring=lambda est,xx,yy:-np.abs(yy[:,core]-est.predict(xx)[:,core]).mean(),cv=KFold(3,shuffle=True,random_state=cfg['seed']),n_jobs
=1)
        ridge=ridge_search.fit(X[train],Y[train]).best_estimator_
        predictions={'mean':np.broadcast_to(Y[train].mean(0),Y[test].shape),'ridge':ridge.predict(X[test]),'extra_trees':etp,'gbr':gbp,
                     'fixed_half_blend':(etp+gbp)/2,'global_oof_blend':etp*(1-glob)+gbp*glob,'unshrunk_target_blend':etp*(1-wu)+gbp*wu,'shrunk_
target_blend':etp*(1-w)+gbp*w}
        for name,pred in predictions.items():
            sums[name][test]+=pred
            for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
                fold_rows.append({'fold':fold,'repeat':fold//3,'model':name,'scope':scope,'mae':np.abs(Y[test][:,ix]-pred[:,ix]).mean(),'n_test':
len(test)})
        count[test]+=1
        assignments.extend({'fold':fold,'repeat':fold//3,'beer':int(data.meta.iloc[i].beer),'role':'outer_validation'} for i in test)
        weight_rows.extend({'fold':fold,'target':target,'weight_GBR':w[j],'global_weight_GBR':glob,'eta':eta} for j,target in enumerate(targets))
        boundaries=fixed_grade_boundaries(Y[train]); grades=grade_with_fixed_boundaries(etp,boundaries)
        for j in range(Y.shape[1]): np.add.at(grade_votes[:,j],(test,grades[:,j]),1)
        rng=np.random.default_rng(cfg['seed']+fold); low=np.nanquantile(X[train],.2,axis=0)
        for scenario in ['MCAR','low_value_preferential']:
            for rate in [.0,.05,.1,.2]:
                eligible=np.flatnonzero(np.isfinite(X[test]).ravel()); n=int(rate*len(eligible)); a=X[test].copy()
                weights=np.where((a<=low).ravel()[eligible],4.,1.) if scenario!='MCAR' else np.ones(len(eligible))
                if n: a.ravel()[rng.choice(eligible,n,replace=False,p=weights/weights.sum())]=np.nan
                pe,pg=predict_pair(pair,a)
                for name,pred in [('extra_trees',pe),('shrunk_target_blend',pe*(1-w)+pg*w)]:
                    stress.append({'fold':fold,'scenario':scenario,'extra_missing_rate':rate,'model':name,'core30_mae':np.abs(Y[test][:,core]-pred
[:,core]).mean(),'all50_mae':np.abs(Y[test]-pred).mean()})
        for lower in [0,.01,.025]:
            alternative=pair[0] if lower==.01 else tree_model(cfg,clip=lower).fit(X[train],Y[train])
            pred=alternative.predict(X[test]); clipping.append({'fold':fold,'lower_clip':lower,'core30_mae':np.abs(Y[test][:,core]-pred[:,core]).
mean(),'all50_mae':np.abs(Y[test]-pred).mean()})
        # Save progress atomically per finished outer fold.
        save_csv(pd.DataFrame(fold_rows),dest/'fold_metrics.csv'); save_csv(pd.DataFrame(weight_rows),dest/'blend_weights.csv')
        write_json(dest/'progress.json',{'completed_outer_folds':fold+1,'total_outer_folds':15,'elapsed_seconds':time.perf_counter()-start})
    metric=[]
    for name in names:
        pred=sums[name]/count[:,None]
        save_csv(pd.concat([data.meta[['beer']],pd.DataFrame(pred,columns=targets)],axis=1),dest/f'oof_{name}.csv')
        metric+=metric_rows(Y,pred,targets,name)
    bytarget=pd.DataFrame(metric); save_csv(bytarget,dest/'metrics_by_target.csv')
    scopes=[]
    for scope,fields in [('core30',[targets[j] for j in core]),('all50',targets)]:
        a=bytarget[bytarget.target.isin(fields)].groupby('model')[['mae','rmse','r2']].mean().reset_index(); a.insert(0,'scope',scope); scopes.
append(a)
    summary=pd.concat(scopes,ignore_index=True); save_csv(summary,dest/'metrics_by_scope.csv')
    save_csv(pd.DataFrame(assignments),dest/'outer_roles.csv'); save_csv(pd.DataFrame(stress),dest/'missingness_stress.csv'); save_csv(pd.
DataFrame(clipping),dest/'clipping_ablation.csv')
    save_csv(pd.concat([data.meta[['beer']],pd.DataFrame(grade_votes.argmax(2),columns=targets)],axis=1),dest/'ASSUMPTION_oof_relative_grades.csv
')
    paired=[]
    for name in ['gbr','global_oof_blend','unshrunk_target_blend','shrunk_target_blend']:
        for scope,ix in [('core30',core),('all50',np.arange(Y.shape[1]))]:
            r=paired_oof_bootstrap(Y[:,ix],sums[name][:,ix]/count[:,None],sums['extra_trees'][:,ix]/count[:,None],labels)
            r.update(model_A=name,model_B='extra_trees',target_scope=scope); paired.append(r)
    write_json(dest/'paired_conditional_comparison.json',paired)
    intervals=[]; coverage_rows=[]
    for repeat in range(cfg['conformal_repeats']):
        train,rest=train_test_split(np.arange(len(X)),test_size=.4,random_state=cfg['seed']+71+repeat)
        calibration,test=train_test_split(rest,test_size=.5,random_state=cfg['seed']+72+repeat)
        model=tree_model(cfg).fit(X[train],Y[train]); pp=model.predict(X[test]); q=conformal_quantile(np.abs(Y[calibration]-model.predict(X[
calibration])),cfg['conformal_alpha'])
        boundaries=fixed_grade_boundaries(Y[train]); lo,hi=ordinal_interval_image(pp-q,pp+q,boundaries); true=grade_with_fixed_boundaries(Y[test],
boundaries); point=grade_with_fixed_boundaries(pp,boundaries)
        for j,t in enumerate(targets):
            coverage_rows.append({'repeat':repeat,'target':t,'continuous_coverage':((Y[test,j]>=pp[:,j]-q[j])&(Y[test,j]<=pp[:,j]+q[j])).mean(),'
interval_width':2*q[j],
                'relative_grade_set_coverage':((true[:,j]>=lo[:,j])&(true[:,j]<=hi[:,j])).mean(),'mean_grade_set_size':(hi[:,j]-lo[:,j]+1).mean()
})
        for i,row in enumerate(test):
            for j,t in enumerate(targets): intervals.append({'repeat':repeat,'beer':int(data.meta.iloc[row].beer),'target':t,'continuous_
prediction':pp[i,j],'lower':pp[i,j]-q[j],'upper':pp[i,j]+q[j],'relative_grade':point[i,j],'relative_grade_min':lo[i,j],'relative_grade_max':hi[i,j
```

```
],'status':'fixed_training_relative_bins_not_absolute_raw_scores'})
        save_csv(pd.DataFrame({'repeat':repeat,'beer':data.meta.beer,'role':['train' if i in set(train) else 'calibration' if i in set(calibration) else 'test' for i in range(len(X))]}),dest/f'conformal_roles_{repeat}.csv')
    save_csv(pd.DataFrame(intervals),dest/'ASSUMPTION_ordinal_sets.csv'); save_csv(pd.DataFrame(coverage_rows),dest/'conformal_coverage.csv')
    write_json(dest/'status.json',{'complete':True,'outer_folds':15,'unique_beers':len(X),'grade_semantics':'relative_training_quantile_not_original_0_6_intensity','absolute_anchors_observed':False,'elapsed_seconds':time.perf_counter()-start,'model_originality':'standard_estimators_and_regularized_stacking_variant_no_first_invention_claim'})
    progress('S3 Q1 complete')
```

014_stage4_audit.py

```
"""Source reconciliation and raw sensory endpoints. AI-assisted, 2026-10-25.
Identity matching is provenance analysis, never chemical-model fitting.
"""
from pathlib import Path
import hashlib
import numpy as np
import pandas as pd
from scipy.optimize import linear_sum_assignment
from scipy.spatial.distance import cdist
from scipy.special import logsumexp
from .common import save_csv, write_json, sha256
from .data import META, CHEMICAL_DEFECTS

ALIASES = {'A_ester_fruity': 'A_esters_fruity', 'F_ester_fruity': 'F_esters_fruity'}
RAW_NAME = 'Raw sensory data anonymized.csv'
BOOK_NAME = 'Supplemental Files and Figure source files.xlsx'


def aggregate_panel(raw, targets):
    """Repeated ratings from one person do not give that person extra weight."""
    person = raw.groupby(['beer', 'taster'])[targets].mean()
    return person.groupby('beer').mean(), raw.groupby('beer')[targets].mean(), person


def standardize_panel(raw, targets):
    group = raw.groupby('taster')[targets]
    sd = group.transform('std')
    return (raw[targets] - group.transform('mean')) / sd.replace(0, np.nan)


def panel_rating_distribution(raw, targets, beers):
    """Equal people, equal repeats per person; preserve actual integer categories."""
    encoded = np.eye(7)[raw[targets].to_numpy(int)].reshape(len(raw), -1)
    table = pd.DataFrame(encoded)
    per_person = table.groupby([raw.beer, raw.taster]).mean()
    return per_person.groupby(level=0).mean().reindex(beers).to_numpy().reshape(len(beers), len(targets), 7)


def infer_identity(standardized, raw_ids, published, discovery):
    means = standardized.groupby(raw_ids).mean().sort_index()
    distance = cdist(means[discovery], published[discovery]) / np.sqrt(len(discovery))
    i, j = linear_sum_assignment(distance)
    ordered = np.sort(distance, axis=1)
    return means, pd.DataFrame({'raw_beer': means.index.to_numpy()[i],
                                'beer': published.index.to_numpy()[j],
                                'discovery_rmse': distance[i, j],
                                'nearest_gap': ordered[:, 1] - ordered[:, 0],
                                'is_row_nearest': j == distance.argmin(1)})


def transform_candidates(raw):
    finite_positive = raw[np.isfinite(raw) & (raw > 0)]
    half_min = finite_positive.min() / 2 if len(finite_positive) else np.nan
    replacement = np.where(raw <= 0, half_min, raw)
    with np.errstate(invalid='ignore', divide='ignore'):
        return {'identity': raw, 'nonpositive_half_min': replacement,
                'log10_nonpositive_half_min': np.log10(replacement)}, half_min


def concentration_fields(calibrated, thresholds, units):
    metadata = thresholds.dropna(subset=['field_name_R']).drop_duplicates('field_name_R').set_index('field_name_R')
    fields = []
    for f in calibrated.columns:
        if f in META or f not in metadata.index or f == 'kcalperc':
            continue
        row = metadata.loc[f]
        if str(row.calibrated).startswith('yes') and row.unit in units:
            fields.append(f)
    return fields


def audit_aggregate_hypotheses(data, calibrated, out):
    """Reject unsupported aggregate identities on a disjoint row half.
    Fits are diagnostic hypotheses, not recovered author formulas.
    """
    x = data.X
    groups = {
        'esters_sum': [f for f in x if any(s in f for s in ['acetate', 'butyrate', 'hexanoate', 'octanoate', 'decanoate', 'propionate', 'isovalerate'])],
        'aroma_hops_sum': data.chemical_meta.loc[data.chemical_meta.origin_paper.astype(str).str.contains('Hops'), 'field'].tolist(),
        'acids_sum': ['acetic_acid.g.L.', 'lactic_acid.mg.L.'],
        'sulfur_sum': ['CS2', 'DES', 'DMS', 'H2S', 'MeSH', 'SO2.mg.L.']}
    rows = []
    for target, fields in groups.items():
```

```
        a = x[fields].to_numpy(float)
        probes = {'mean_processed': np.nanmean(a, axis=1), 'sum_processed': np.nansum(a, axis=1),
                  'log10_sum_exponentiated_processed': logsumexp(np.where(np.isfinite(a), a, -np.inf) * np.log(10), axis=1) / np.log(10)}
        y = x[target].to_numpy()
        for name, value in probes.items():
            valid = np.isfinite(value) & np.isfinite(y); train = valid & (np.arange(len(y)) % 2 == 0); test = valid & ~train
            slope, intercept = np.linalg.lstsq(np.c_[value[train], np.ones(train.sum())], y[train], rcond=None)[0]
            error = np.abs(value[test] * slope + intercept - y[test])
            rows.append({'aggregate': target, 'hypothesis': name, 'component_count': len(fields), 'fitted_slope_on_discovery': slope,
                         'fitted_intercept_on_discovery': intercept, 'validation_mae': error.mean(), 'validation_max_error': error.max(),
                         'identity_verified': bool(error.max() < 1e-7), 'scope': 'exploratory_formula_hypothesis_not_author_documentation'})
    features = ['ethanol..v.v.', 'GFS', 'unfermentables', 'protein.g.L.', 'glycerol']
    a = calibrated[features].to_numpy(float); y = calibrated.kcalperc.to_numpy(float)
    valid = np.isfinite(a).all(1) & np.isfinite(y); train = valid & (np.arange(len(y)) % 2 == 0); test = valid & ~train
    coef = np.linalg.lstsq(np.c_[a[train], np.ones(train.sum())], y[train], rcond=None)[0]
    error = np.abs(np.c_[a[test], np.ones(test.sum())] @ coef - y[test])
    rows.append({'aggregate': 'kcalperc', 'hypothesis': 'five_raw_nutrient_affine_fit', 'component_count': len(features),
                 'validation_mae': error.mean(), 'validation_max_error': error.max(), 'identity_verified': bool(error.max() < 1e-7),
                 'scope': 'exploratory_formula_hypothesis_not_author_documentation'})
    save_csv(pd.DataFrame(rows), Path(out) / 'aggregate_formula_hypotheses.csv')


def reconcile(data, external, out, cfg):
    external, out = Path(external), Path(out)
    out.mkdir(parents=True, exist_ok=True)
    for name, key in [(RAW_NAME, 'raw_md5'), (BOOK_NAME, 'workbook_md5')]:
        if hashlib.md5((external / name).read_bytes()).hexdigest() != cfg[key]:
            raise ValueError('External source checksum differs: ' + name)
    raw = pd.read_csv(external / RAW_NAME, encoding='cp1252').rename(columns=ALIASES)
    published = pd.read_excel(external / BOOK_NAME, sheet_name='Supplementary File S4')
    targets = list(data.Y)
    if set(raw.beer_name) != set(range(1, 251)) or len(published) != 250:
        raise ValueError('Unexpected source numbering; no positional fallback permitted')
    if set(raw.columns[5:-1]) != set(targets) or not raw[targets].isin(range(7)).all().all():
        raise ValueError('Raw target domain mismatch')
    reference = published.set_index('beer')[targets]
    if not np.allclose(reference.reindex(data.meta.beer), data.Y, atol=1e-7, rtol=0):
        raise ValueError('Published S4 differs from contest labels')
    standardized = standardize_panel(raw, targets)
    discovery = sorted(targets)[:25]
    validation = sorted(targets)[25:]
    means, mapping = infer_identity(standardized, raw.beer_name, reference, discovery)
    positional = dict(zip(range(1, 251), published.beer))
    if not (mapping.beer == mapping.raw_beer.map(positional)).all() or not mapping.is_row_nearest.all():
        raise ValueError('Source order and independent profile matching do not agree')
    # A second, disjoint attribute group must recover the same identity permutation.
    _, reverse_mapping = infer_identity(standardized, raw.beer_name, reference, validation)
    if not mapping.set_index('raw_beer').beer.equals(reverse_mapping.set_index('raw_beer').beer):
        raise ValueError('Disjoint-attribute identity validation failed')
    comparison = reference.reindex(mapping.beer).to_numpy()
    reconstructed = means.reindex(mapping.raw_beer).to_numpy()
    no_constant = ~standardized.isna().any().to_numpy()
    exact_error = np.abs(reconstructed[:, no_constant] - comparison[:, no_constant])
    if exact_error.max() > cfg['mapping_tolerance']:
        raise ValueError('Nonconstant attributes do not reproduce published scores')
    mapping['published_position'] = mapping.raw_beer
    mapping['identity_status'] = 'source_order_reconstructed_and_disjoint_attribute_verified_not_author_mapping_table'
    save_csv(mapping, out / 'identity_mapping.csv')
    # Reverse engineering only: fit unknown constant replacement on odd source IDs,
    # validate on even IDs. Do not call this the documented original preprocessing.
    counts = raw.groupby('beer_name').size().reindex(mapping.raw_beer).to_numpy()
    source_ids = mapping.raw_beer.to_numpy()
    replacement_rows, error_rows = [], []
    for j, target in enumerate(targets):
        sums = standardized[target].fillna(0).groupby(raw.beer_name).sum().reindex(source_ids).to_numpy()
        missing = standardized[target].isna().groupby(raw.beer_name).sum().reindex(source_ids).to_numpy()
        if missing.sum():
            fit = (source_ids % 2 == 1) & (missing > 0)
            constant = float(np.dot(missing[fit], comparison[fit, j] * counts[fit] - sums[fit]) / np.dot(missing[fit], missing[fit]))
            rebuilt = (sums + missing * constant) / counts
            held = (source_ids % 2 == 0) & (missing > 0)
            replacement_rows.append({'target': target, 'replacement': constant, 'fit_beers': int(fit.sum()),
                                     'validation_beers': int(held.sum()),
                                     'validation_max_error': float(np.abs(rebuilt[held] - comparison[held, j]).max()),
                                     'status': 'empirically_reconstructed_not_source_documented'})
        else:
            rebuilt = reconstructed[:, j]
        error_rows.append({'target': target, 'constant_taster_attribute': bool(missing.sum()),
                           'max_error_without_fitted_replacement': float(np.abs(reconstructed[:, j] - comparison[:, j]).max()),
                           'max_error_after_conditional_reconstruction': float(np.abs(rebuilt - comparison[:, j]).max())})
    save_csv(pd.DataFrame(replacement_rows), out / 'constant_attribute_reconstruction.csv')
    save_csv(pd.DataFrame(error_rows), out / 'sensory_reconstruction_errors.csv')
    raw['beer'] = raw.beer_name.map(mapping.set_index('raw_beer').beer)
    balanced, records, person = aggregate_panel(raw, targets)
    balanced, records = balanced.reindex(data.meta.beer), records.reindex(data.meta.beer)
    save_csv(balanced.reset_index(), out / 'observed_panel_means_0_6.csv')
    save_csv(records.reset_index(), out / 'record_weighted_means_sensitivity.csv')
    n = person.groupby('beer').size().reindex(data.meta.beer)
    save_csv(pd.DataFrame({'beer': data.meta.beer.to_numpy(), 'distinct_panelists': n.to_numpy(),
                           'rating_records': raw.groupby('beer').size().reindex(data.meta.beer).to_numpy()}), out / 'panel_sample_counts.csv')
    rng = np.random.default_rng(cfg['seed'])
    precision, repeats = [], []
    for beer, group in person.groupby(level='beer'):
        a = group.to_numpy()
        draws = a[rng.integers(len(a), size=(cfg['panel_bootstrap_repeats'], len(a)))].mean(1)
        low, high = np.quantile(draws, [.025, .975], axis=0)
        for j, target in enumerate(targets):
```

```
            precision.append({'beer': int(beer), 'target': target, 'observed_mean': a[:, j].mean(),
                              'panel_bootstrap_low': low[j], 'panel_bootstrap_high': high[j], 'panelists': len(a)})
    for (beer, taster), group in raw.groupby(['beer', 'taster']):
        if group.date.nunique() < 2:
            continue
        first, last = group.sort_values('date').iloc[[0, -1]][targets].to_numpy(float)
        repeats.extend({'beer': int(beer), 'target': target, 'absolute_first_last_difference': abs(last[j] - first[j])} for j, target in enumerate(targets))
    save_csv(pd.DataFrame(precision), out / 'panel_sampling_precision.csv')
    save_csv(pd.DataFrame(repeats).groupby('target', as_index=False).agg(mean_absolute_first_last_difference=('absolute_first_last_difference', 'mean'), repeated_person_beer_pairs=('beer', 'size')), out / 'within_person_repeatability.csv')
    calibrated = pd.read_excel(external / BOOK_NAME, sheet_name='Calibrated chemical dataset').set_index('beer').reindex(data.meta.beer)
    fields = list(calibrated.columns[2:])
    chemistry, mismatches = [], []
    for field in fields:
        a = calibrated[field].to_numpy(float)
        supplied = data.X[field].to_numpy(float)
        candidates, half = transform_candidates(a)
        valid = np.isfinite(a) & np.isfinite(supplied)
        rates = {name: float(np.isclose(pred[valid], supplied[valid], atol=1e-7, rtol=1e-7).mean()) for name, pred in candidates.items()}
        best = max(rates, key=rates.get)
        good = np.isclose(candidates[best], supplied, atol=1e-7, rtol=1e-7)
        chemistry.append({'field': field, 'best_transform': best, 'match_fraction': rates[best],
                          'comparable_pairs': int(valid.sum()), 'nonpositive_raw': int((a <= 0).sum()),
                          'missing_raw': int(np.isnan(a).sum()), 'half_min_positive': half,
                          'status': 'all_comparable_pairs_reconciled' if rates[best] == 1 else 'exceptions_retained'})
        mismatches.extend({'beer': int(data.meta.iloc[i].beer), 'field': field,
                           'absolute_difference': abs(candidates[best][i] - supplied[i]),
                           'status': 'unexplained_source_difference_not_repaired'} for i in np.flatnonzero(valid & ~good))
    save_csv(pd.DataFrame(chemistry), out / 'chemical_scale_reconciliation.csv')
    save_csv(pd.DataFrame(mismatches), out / 'chemical_source_exceptions.csv')
    selected = concentration_fields(calibrated, data.thresholds, cfg['concentration_units'])
    meta = data.thresholds.dropna(subset=['field_name_R']).drop_duplicates('field_name_R').set_index('field_name_R')
    save_csv(pd.DataFrame([{'field': f, 'unit': meta.at[f, 'unit'], 'calibrated_metadata': meta.at[f, 'calibrated'],
                            'negative_raw': int((calibrated[f] < 0).sum()), 'zero_raw': int((calibrated[f] == 0).sum()),
                            'missing_raw': int(calibrated[f].isna().sum())} for f in selected]), out / 'concentration_feature_registry.csv')
    thresholds = {'DMS': (19., 'ug/L', 'attachment_median'), 'X2.3.butanedione': (.09, 'mg/L', 'attachment_median'),
                  'X4VG': (.2, 'mg/L', 'attachment_median'),
                  'X.E..2.nonenal': (cfg['literature_nonenal_threshold_mg_L'], 'mg/L', 'Meilgaard_1993_Santos_2003_external_reference_not_batch_calibration')}
    oav = []
    for field, (threshold, unit, source) in thresholds.items():
        for factor in cfg['threshold_sensitivity_multipliers']:
            for i, value in enumerate(calibrated[field]):
                oav.append({'beer': int(data.meta.iloc[i].beer), 'field': field, 'unit': unit,
                            'threshold_source': source, 'reference_threshold': threshold, 'threshold_multiplier': factor,
                            'reference_oav': value / (factor * threshold) if pd.notna(value) and value >= 0 else np.nan,
                            'measurement_status': 'positive_reported_concentration_LOD_unknown' if pd.notna(value) and value > 0 else 'zero_or_missing_not_certified_absent',
                            'sensory_target': {'DMS': 'dms', 'X2.3.butanedione': 'diacetyl', 'X4VG': 'X4vg', 'X.E..2.nonenal': 't2n'}[field]})
    save_csv(pd.DataFrame(oav), out / 'reference_oav_sensitivity.csv')
    audit_aggregate_hypotheses(data, calibrated, out)
    summary = {'raw_rows': len(raw), 'beers': len(balanced), 'panelists': int(raw.taster.nunique()), 'targets': len(targets),
               'duplicate_person_beer_records': int(raw.duplicated(['beer', 'taster']).sum()),
               'identity_matches_both_attribute_groups': len(mapping), 'nonconstant_targets_reproduced': int(no_constant.sum()),
               'nonconstant_max_error': float(exact_error.max()), 'replacement_rules_empirically_reconstructed': len(replacement_rows),
               'panelists_per_beer_min': int(n.min()), 'panelists_per_beer_max': int(n.max()),
               'mean_abs_record_vs_person_aggregation_difference': float(np.abs(balanced - records).to_numpy().mean()),
               'calibrated_reference_fields': len(fields), 'concentration_model_fields': len(selected),
               'fully_reconciled_fields': sum(row['match_fraction'] == 1 for row in chemistry),
               'unexplained_chemical_pairs': len(mismatches),
               'source_sha256': {RAW_NAME: sha256(external / RAW_NAME), BOOK_NAME: sha256(external / BOOK_NAME)},
               'mapping_scope': 'reconstructed_source_identity_not_independent_new_external_samples',
               'absolute_target_scope': cfg['primary_absolute_target'], 'physical_validation_performed': False}
    write_json(out / 'audit_summary.json', summary)
    return raw, balanced, records, calibrated[selected].where(calibrated[selected] >= 0), selected
```

015_stage4_models.py

```
"""Absolute panel-mean prediction, factor checks, and physical-scale diagnostics.
AI-assisted research implementation, 2026-10-25; no original algorithm claim.
"""
from pathlib import Path
import hashlib
import time
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.isotonic import IsotonicRegression
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import balanced_accuracy_score
from sklearn.model_selection import GridSearchCV, KFold, RepeatedStratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline
from .common import preprocessing, save_csv, write_json, progress
from .data import NON_MOLECULES, CHEMICAL_DEFECTS, AGGREGATES
from .q1 import metric_rows
from .stage2 import conformal_quantile
from .stage3_prediction import fit_pair, predict_pair, tree_model
from .stage4_audit import panel_rating_distribution
from .review_diagnostics import distribution_scores


def calibrated_from_training(z_train, raw_train, z_pred):
    return np.column_stack([IsotonicRegression(out_of_bounds='clip', y_min=0, y_max=6).fit(z_train[:, j], raw_train[:, j]).predict(z_pred[:, j])
```

```
                           for j in range(raw_train.shape[1])])


def ridge_search(X, Y, core, cfg):
    return GridSearchCV(Pipeline([('prep', preprocessing()), ('model', Ridge())]),
                        {'model__alpha': cfg['ridge_alphas']}, cv=KFold(3, shuffle=True, random_state=cfg['seed']),
                        scoring=lambda est, xx, yy: -np.abs(yy[:, core] - est.predict(xx)[:, core]).mean(), n_jobs=1).fit(X, Y).best_estimator_


def raw_top3(model, fields, targets):
    candidates = np.array([j for j, f in enumerate(fields) if f not in NON_MOLECULES])
    coef = model.named_steps['model'].coef_
    return [(t, int(j), float(coef[k, j]), rank + 1) for k, t in enumerate(targets)
            for rank, j in enumerate(candidates[np.argsort(-np.abs(coef[k, candidates]), kind='stable')[:3]])]


def run_absolute(data, panel, records, concentration, out, cfg):
    dest = Path(out) / 'prediction'; dest.mkdir(parents=True, exist_ok=True)
    X, Y, C = data.X.to_numpy(), panel.to_numpy(), concentration.to_numpy()
    targets, fields = list(data.Y), list(data.X)
    core = np.array([j for j, t in enumerate(targets) if t.startswith(('A_', 'F_'))])
    labels, beers = data.meta.tasting_category_fine.to_numpy(), data.meta.beer.to_numpy()
    names = ['mean', 'ridge', 'extra_trees', 'gbr', 'fixed_half_blend', 'concentration_ET', 'standardized_ET_train_isotonic']
    sums = {m: np.zeros_like(Y) for m in names}; count = np.zeros(len(Y)); folds, factors, sensitivity, roles = [], [], [], []
    start = time.perf_counter()
    n_splits, n_repeats = cfg['outer_folds'], cfg['outer_repeats']
    for fold, (train, test) in enumerate(RepeatedStratifiedKFold(n_splits=n_splits, n_repeats=n_repeats, random_state=cfg['seed']).split(X, labels)):
        progress(f'S4 absolute fold {fold+1}/{n_splits*n_repeats}; {(time.perf_counter()-start):.0f}s')
        et, gb = fit_pair(X[train], Y[train], cfg)
        pe, pg = predict_pair((et, gb), X[test])
        ridge = ridge_search(X[train], Y[train], core, cfg)
        cp = tree_model(cfg).fit(C[train], Y[train]).predict(C[test])
        old = tree_model(cfg).fit(X[train], data.Y.to_numpy()[train]).predict(X[test])
        iso = calibrated_from_training(data.Y.to_numpy()[train], Y[train], old)
        predictions = {'mean': np.broadcast_to(Y[train].mean(0), Y[test].shape), 'ridge': ridge.predict(X[test]),
                       'extra_trees': pe, 'gbr': pg, 'fixed_half_blend': (pe + pg) / 2, 'concentration_ET': cp,
                       'standardized_ET_train_isotonic': iso}
        for name, pred in predictions.items():
            pred = np.clip(pred, 0, 6); sums[name][test] += pred
            for scope, indices in [('core30', core), ('all50', np.arange(50))]:
                folds.append({'fold': fold, 'repeat': fold // n_splits, 'model': name, 'scope': scope,
                              'mae_0_6': float(np.abs(Y[test][:, indices] - pred[:, indices]).mean()), 'n_test': len(test)})
        count[test] += 1
        record_model = tree_model(cfg).fit(X[train], records.to_numpy()[train])
        record_pred = record_model.predict(X[test])
        sensitivity.append({'fold': fold, 'person_balanced_ET_mae_own_endpoint': float(np.abs(Y[test] - pe).mean()),
                            'record_weighted_ET_mae_own_endpoint': float(np.abs(records.to_numpy()[test] - record_pred).mean()),
                            'prediction_aggregation_difference': float(np.abs(record_pred - pe).mean())})
        roles.extend({'fold': fold, 'repeat': fold // n_splits, 'beer': int(beers[i]), 'role': 'outer_test'} for i in test)
        # Linear model permits exact input-coordinate permutation changes without refits.
        zp = ridge.named_steps['prep'].transform(X[test]); rp = ridge.predict(X[test]); rng = np.random.default_rng(cfg['seed'] + fold)
        for target, j, coefficient, rank in raw_top3(ridge, fields, targets):
            k = targets.index(target); baseline = np.abs(Y[test, k] - rp[:, k]).mean(); ordinary, within = [], []
            for _ in range(20):
                permutation = rng.permutation(len(test))
                ordinary.append(np.abs(Y[test, k] - (rp[:, k] + coefficient * (zp[permutation, j] - zp[:, j]))).mean() - baseline)
                permutation = np.arange(len(test))
                for style in np.unique(labels[test]):
                    ids = np.flatnonzero(labels[test] == style); permutation[ids] = rng.permutation(ids)
                within.append(np.abs(Y[test, k] - (rp[:, k] + coefficient * (zp[permutation, j] - zp[:, j]))).mean() - baseline)
            factors.append({'fold': fold, 'target': target, 'field': fields[j], 'rank': rank, 'coefficient': coefficient,
                            'ordinary_delta_mae': np.mean(ordinary), 'within_style_delta_mae': np.mean(within)})
        save_csv(pd.DataFrame(folds), dest / 'fold_metrics.csv')
    metric = []
    for name in names:
        pred = sums[name] / count[:, None]
        save_csv(pd.concat([data.meta[['beer']], pd.DataFrame(pred, columns=targets)], axis=1), dest / f'oof_{name}_0_6.csv')
        metric += metric_rows(Y, pred, targets, name)
    detail = pd.DataFrame(metric); save_csv(detail, dest / 'metrics_by_target.csv')
    summaries = []
    for scope, selected in [('core30', [targets[j] for j in core]), ('all50', targets)]:
        f = detail[detail.target.isin(selected)].groupby('model')[['mae', 'rmse', 'r2']].mean().reset_index(); f.insert(0, 'scope', scope); summaries.append(f)
    save_csv(pd.concat(summaries), dest / 'metrics_by_scope.csv')
    save_csv(pd.DataFrame(roles), dest / 'outer_roles.csv')
    save_csv(pd.DataFrame(sensitivity), dest / 'aggregation_sensitivity.csv')
    effect = pd.DataFrame(factors); save_csv(effect, dest / 'raw_factor_crossfit.csv')
    # Prespecified ET is used for uncertainty; evaluation does not select the model.
    intervals, coverage, interval_roles = [], [], []
    for repeat in range(cfg['conformal_repeats']):
        train, rest = train_test_split(np.arange(len(X)), test_size=.4, random_state=cfg['seed'] + 71 + repeat)
        calibration, test = train_test_split(rest, test_size=.5, random_state=cfg['seed'] + 72 + repeat)
        model = tree_model(cfg).fit(X[train], Y[train]); pred = model.predict(X[test])
        q = conformal_quantile(np.abs(Y[calibration] - model.predict(X[calibration])), cfg['conformal_alpha'])
        lo, hi = np.maximum(0, pred - q), np.minimum(6, pred + q)
        for j, target in enumerate(targets):
            coverage.append({'repeat': repeat, 'target': target, 'coverage': float(((Y[test, j] >= lo[:, j]) & (Y[test, j] <= hi[:, j])).mean()),
                             'mean_width_0_6': float((hi[:, j] - lo[:, j]).mean())})
        intervals.extend({'repeat': repeat, 'beer': int(beers[i]), 'target': target, 'prediction_0_6': pred[k, j],
                          'lower_0_6': lo[k, j], 'upper_0_6': hi[k, j], 'observed_panel_mean': Y[i, j]}
                         for k, i in enumerate(test) for j, target in enumerate(targets))
        interval_roles.extend({'repeat': repeat, 'beer': int(beers[i]), 'role': 'fit' if i in train else 'calibration' if i in calibration else 'test'} for i in range(len(X)))
    save_csv(pd.DataFrame(intervals), dest / 'absolute_panel_mean_intervals.csv')
    save_csv(pd.DataFrame(coverage), dest / 'conformal_coverage.csv')
    save_csv(pd.DataFrame(interval_roles), dest / 'conformal_roles.csv')
```

```
    final_ridge = ridge_search(X, Y, core, cfg); alpha = float(final_ridge.named_steps['model'].alpha)
    final = raw_top3(final_ridge, fields, targets); frequencies = {}
    rng = np.random.default_rng(cfg['seed'] + 333)
    for _ in range(200):
        ii = rng.integers(len(X), size=len(X))
        m = Pipeline([('prep', preprocessing()), ('model', Ridge(alpha=alpha))]).fit(X[ii], Y[ii])
        for target, j, _, _ in raw_top3(m, fields, targets): frequencies[(target, fields[j])] = frequencies.get((target, fields[j]), 0) + 1
    grouped = effect.groupby(['target', 'field']).agg(selected_folds=('fold', 'size'), ordinary_delta_mae=('ordinary_delta_mae', 'mean'), within_style_delta_mae=('within_style_delta_mae', 'mean'))
    metadata = data.chemical_meta.set_index('field'); top_rows = []
    for target, j, coefficient, rank in final:
        field = fields[j]; key = (target, field); f = frequencies.get(key, 0) / 200
        evidence = grouped.loc[key] if key in grouped.index else pd.Series({'selected_folds': 0, 'ordinary_delta_mae': np.nan, 'within_style_delta_mae': np.nan})
        top_rows.append({'target': target, 'rank': rank, 'field': field, 'coefficient_raw_score_per_standardized_input': coefficient,
                         'bootstrap_frequency_fixed_alpha': f, **evidence.to_dict(),
                         'descriptive_support': bool(f >= .5 and evidence.selected_folds >= 5 and evidence.ordinary_delta_mae > 0 and evidence.within_style_delta_mae > 0),
                         'unit': metadata.at[field, 'unit'], 'origin': metadata.at[field, 'origin_paper'], 'causal_validated': False})
    save_csv(pd.DataFrame(top_rows), dest / 'absolute_raw_top3.csv')
    write_json(dest / 'status.json', {'complete': True, 'elapsed_seconds': time.perf_counter() - start, 'outer_fits': n_splits * n_repeats,
                                      'outer_folds': n_splits, 'outer_repeats': n_repeats, 'conformal_alpha': cfg['conformal_alpha'],
                                      'selected_full_ridge_alpha': alpha, 'absolute_endpoint': cfg['primary_absolute_target'],
                                      'source_identity_reconstructed': True, 'new_external_beers': 0})


def mix_concentrations(source, anchor, weights):
    return source[None, :] + np.asarray(weights)[:, None] * (anchor[None, :] - source[None, :])


def exact_nonincrease(values, source, positions):
    if not np.isfinite(source[positions]).all():
        return np.zeros(len(values), dtype=bool)
    return np.isfinite(values[:, positions]).all(1) & (values[:, positions] <= source[positions] + 1e-12).all(1)


def ordinal_summary(observed, probabilities, grades):
    axis = np.arange(7)
    individual_mae = (observed * np.abs(axis - grades[..., None])).sum(-1)
    individual_accuracy = np.take_along_axis(observed, grades[..., None], axis=-1)[..., 0]
    modal_accuracy = grades == observed.argmax(-1)
    result = {'expected_individual_grade_MAE': float(individual_mae.mean()),
              'expected_individual_exact_accuracy': float(individual_accuracy.mean()),
              'observed_panel_modal_grade_accuracy': float(modal_accuracy.mean()),
              'predicted_zero_fraction': float((grades == 0).mean()),
              'observed_panel_zero_mode_fraction': float((observed.argmax(-1) == 0).mean())}
    nonzero = observed.argmax(-1) > 0
    result['nonzero_panel_mode_accuracy'] = float(modal_accuracy[nonzero].mean()) if nonzero.any() else np.nan
    if probabilities is not None:
        result.update(**{name: float(value.mean()) for name, value in distribution_scores(observed, probabilities).items()},
                      expected_panel_mean_MAE=float(np.abs((probabilities * axis).sum(-1) - (observed * axis).sum(-1)).mean()))
    return result


def run_ordinal(data, raw, panel, out, cfg, previous):
    dest = Path(out) / 'ordinal'; dest.mkdir(parents=True, exist_ok=True)
    targets = list(data.Y); X = data.X.to_numpy(); Y = panel.to_numpy(); beers = data.meta.beer.to_numpy()
    labels = data.meta.tasting_category_fine.to_numpy(); observed = panel_rating_distribution(raw, targets, beers)
    core = np.array([j for j, t in enumerate(targets) if t.startswith(('A_', 'F_'))]); scopes = [('core30', core), ('all50', np.arange(50))]
    sums = np.zeros_like(observed); base_sum = np.zeros_like(observed); point_sum = np.zeros_like(Y); counts = np.zeros(len(X)); folds, stress = [], []
    n_splits, n_repeats = cfg['outer_folds'], cfg['outer_repeats']
    for fold, (train, test) in enumerate(RepeatedStratifiedKFold(n_splits=n_splits, n_repeats=n_repeats, random_state=cfg['seed']).split(X, labels)):
        progress(f'S4 ordinal distribution fold {fold+1}/{n_splits*n_repeats}')
        model = tree_model(cfg).fit(X[train], observed[train].reshape(len(train), -1))
        p = model.predict(X[test]).reshape(len(test), 50, 7); p /= p.sum(-1, keepdims=True)
        base = np.broadcast_to(observed[train].mean(0), p.shape)
        point = tree_model(cfg).fit(X[train], Y[train]).predict(X[test])
        sums[test] += p; base_sum[test] += base; point_sum[test] += point; counts[test] += 1
        for name, prob, grades in [('ET_rating_distribution', p, p.argmax(-1)),
                                   ('ET_rating_distribution_median', p, (p.cumsum(-1) >= .5).argmax(-1)),
                                   ('training_population_distribution', base, base.argmax(-1)),
                                   ('ET_mean_round_half_up', None, np.floor(point + .5).clip(0, 6).astype(int))]:
            for scope, selected in scopes:
                folds.append({'fold': fold, 'scope': scope, 'method': name,
                              **ordinal_summary(observed[test][:, selected], None if prob is None else prob[:, selected], grades[:, selected])})
        rng = np.random.default_rng(cfg['seed'] + fold)
        for rate in [0, .05, .1, .2]:
            masked = X[test].copy(); eligible = np.flatnonzero(np.isfinite(masked).ravel()); n = int(rate * len(eligible))
            if n: masked.ravel()[rng.choice(eligible, n, replace=False)] = np.nan
            changed = model.predict(masked).reshape(len(test), 50, 7)
            for scope, selected in scopes:
                stress.append({'fold': fold, 'scope': scope, 'additional_MCAR_rate': rate,
                               'modal_grade_change_fraction': float((changed.argmax(-1)[:, selected] != p.argmax(-1)[:, selected]).mean()),
                               **ordinal_summary(observed[test][:, selected], changed[:, selected], changed.argmax(-1)[:, selected])})
    probability = sums / counts[:, None, None]; baseline = base_sum / counts[:, None, None]; point = point_sum / counts[:, None]
    results = []
    for method, prob, grades in [('ET_rating_distribution', probability, probability.argmax(-1)),
                                 ('ET_rating_distribution_median', probability, (probability.cumsum(-1) >= .5).argmax(-1)),
                                 ('training_population_distribution', baseline, baseline.argmax(-1)),
                                 ('ET_mean_round_half_up', None, np.floor(point + .5).clip(0, 6).astype(int))]:
        for scope, selected in scopes:
            results.append({'scope': scope, 'method': method, **ordinal_summary(observed[:, selected], None if prob is None else prob[:, selected], grades[:, selected])})
    save_csv(pd.DataFrame(results), dest / 'metrics_by_scope.csv')
    save_csv(pd.DataFrame(folds), dest / 'fold_metrics.csv'); save_csv(pd.DataFrame(stress), dest / 'missingness_stress.csv')
```

```
    save_csv(pd.concat([data.meta[['beer']], pd.DataFrame(probability.argmax(-1), columns=targets)], axis=1), dest / 'oof_modal_grades_0_6.csv')
    save_csv(pd.concat([data.meta[['beer']], pd.DataFrame((probability.cumsum(-1) >= .5).argmax(-1), columns=targets)], axis=1), dest / 'oof_median_grades_0_6.csv')
    save_csv(pd.concat([data.meta[['beer']], pd.DataFrame(np.floor(point + .5).clip(0, 6).astype(int), columns=targets)], axis=1), dest / 'oof_rounded_panel_mean_grades_0_6.csv')
    rows = []
    for i, beer in enumerate(beers):
        for j, target in enumerate(targets):
            rows.append({'beer': int(beer), 'target': target, 'predicted_modal_grade': int(probability[i, j].argmax()),
                         'observed_panel_modal_grade': int(observed[i, j].argmax()),
                         **{f'predicted_p{k}': probability[i, j, k] for k in range(7)},
                         **{f'observed_panel_p{k}': observed[i, j, k] for k in range(7)}})
    save_csv(pd.DataFrame(rows), dest / 'oof_rating_distributions.csv')
    roles = pd.read_csv(Path(previous) / 'q3/disjoint_roles.csv'); generated = pd.read_csv(Path(previous) / 'q3/generated_profiles_first_split.csv')
    features = [f for f in data.X if f not in AGGREGATES]; repeat0 = roles[roles.repeat == 0].set_index('beer').role.reindex(beers)
    for name, role in [('G', 'G_search_training'), ('J', 'J_evaluator_training')]:
        subset = np.flatnonzero(repeat0.to_numpy() == role)
        model = tree_model(cfg).fit(data.X[features].to_numpy()[subset], observed[subset].reshape(len(subset), -1))
        p = model.predict(generated[features].to_numpy()).reshape(len(generated), 50, 7)
        save_csv(pd.concat([generated[['generated_id', 'method', 'style']], pd.DataFrame(p.argmax(-1), columns=targets)], axis=1),
                 dest / f'generated_modal_grade_proxy_{name}_first_split.csv')
    write_json(dest / 'status.json', {'complete': True, 'beer_grouped_outer_fits': n_splits * n_repeats, 'outer_folds': n_splits, 'outer_repeats': n_repeats, 'grade_definition': 'modal_prediction_of_actual_0_6_panelist_rating_distribution',
                                      'zero_definition': 'most_probable_panelist_rating_zero_not_chemical_absence',
                                      'equal_panelist_and_within_person_repeat_weights': True, 'probabilities_claimed_calibrated': False,
                                      'new_physical_measurements': False})


def run_transfer_diagnostics(data, panel, concentration, out, cfg, previous):
    dest = Path(out) / 'transfer'; dest.mkdir(parents=True, exist_ok=True)
    previous = Path(previous); roles = pd.read_csv(previous / 'q3/disjoint_roles.csv')
    profiles = pd.read_csv(previous / 'q3/transfer_profiles.csv'); old_eval = pd.read_csv(previous / 'q3/transfer_independent_evaluation.csv')
    targets = list(data.Y); stale = targets.index('stale_hops'); beers = data.meta.beer.to_numpy(); labels = data.meta.tasting_category_fine.to_numpy()
    nonaggregate = [f for f in data.X if f not in AGGREGATES]; X = data.X[nonaggregate].to_numpy(); Y = panel.to_numpy(); C = concentration.to_numpy(); features = list(concentration)
    positions = [features.index(f) for f in CHEMICAL_DEFECTS]
    frozen_rows, mixture_rows, mixture_profiles, locks, diagnostics, lp_rows = [], [], [], [], [], []
    for repeat in range(5):
        progress(f'S4 frozen-candidate and concentration mixture evaluation {repeat+1}/5')
        role = roles[roles.repeat == repeat].set_index('beer').role.reindex(beers).to_numpy()
        G = np.flatnonzero(role == 'G_search_training'); J = np.flatnonzero(role == 'J_evaluator_training'); T = np.flatnonzero(role == 'T_real_holdout')
        assert len(G) == len(J) == 100 and len(T) == 50 and not set(G) & set(J)
        # Old candidates are already frozen; no raw-scale J feedback changes them.
        graw = tree_model(cfg).fit(X[G], Y[G]); jraw = tree_model(cfg).fit(X[J], Y[J])
        if repeat == 0:
            generated = pd.read_csv(previous / 'q3/generated_profiles_first_split.csv')
            for name, predictor in [('G', graw), ('J', jraw)]:
                portrait = pd.DataFrame(predictor.predict(generated[nonaggregate].to_numpy()), columns=targets)
                save_csv(pd.concat([generated[['generated_id', 'method', 'style']].reset_index(drop=True), portrait], axis=1),
                         dest / f'generated_raw_score_proxy_{name}_first_split.csv')
        old = profiles[profiles.repeat == repeat].copy(); candidate = old[nonaggregate].to_numpy(); ids = pd.Index(beers).get_indexer(old.beer)
        sp, cp = jraw.predict(X[ids])[:, stale], jraw.predict(candidate)[:, stale]
        ga, gc = graw.predict(X[ids])[:, stale], graw.predict(candidate)[:, stale]
        prior = old.merge(old_eval[['repeat', 'beer', 'radius', 'evaluator_proxy_acceptance']], on=['repeat', 'beer', 'radius'], validate='one_to_one')
        for k, row in old.reset_index(drop=True).iterrows():
            feasible = not row.status.startswith('no_feasible')
            frozen_rows.append({'repeat': repeat, 'beer': int(row.beer), 'radius': row.radius, 'source_role': role[ids[k]],
                                'original_search_feasible': feasible, 'raw_G_stale_source': ga[k], 'raw_G_stale_candidate': gc[k],
                                'raw_J_stale_source': sp[k], 'raw_J_stale_candidate': cp[k],
                                'raw_G_nonincrease': bool(gc[k] <= ga[k] + 1e-9), 'raw_J_nonincrease': bool(cp[k] <= sp[k] + 1e-9),
                                'original_proxy_acceptance': bool(prior.iloc[k].evaluator_proxy_acceptance),
                                'original_and_raw_proxy_support': bool(feasible and prior.iloc[k].evaluator_proxy_acceptance and gc[k] <= ga[k] + 1e-9 and cp[k] <= sp[k] + 1e-9),
                                'physical_validation': False})
        # New mixtures use only documented concentration-unit fields, not pH/pressure/A.U.
        generator_prep = preprocessing().fit(C[G]); fill = generator_prep.named_steps['impute'].statistics_
        source_data = np.where(np.isfinite(C), C, fill)
        search = Pipeline([('prep', preprocessing()), ('model', LogisticRegression(C=1, max_iter=3000, random_state=cfg['seed']))]).fit(C[G], labels[G])
        sensory = tree_model(cfg).fit(C[G], Y[G])
        consensus = Pipeline([('prep', preprocessing()), ('model', Ridge(alpha=1000))]).fit(C[G], Y[G])
        target_position = list(search.classes_).index('Tripel'); anchors = G[labels[G] == 'Tripel']; locked = []
        for i in np.flatnonzero(labels == 'Pils/Lager'):
            source = source_data[i]; srcp = search.predict_proba(source[None])[0, target_position]
            candidates = np.concatenate([mix_concentrations(source, source_data[a], cfg['mixture_weights']) for a in anchors])
            candidate_recipes = [(int(beers[a]), weight, {int(beers[a]): weight}) for a in anchors for weight in cfg['mixture_weights']]
            # An exploratory LP extension checks combinations of multiple target beers.
            # Source/anchor missing defect values cannot support a chemical certificate.
            known_anchors = anchors[np.isfinite(C[anchors][:, positions]).all(1)]
            lp_fraction = None
            if np.isfinite(C[i, positions]).all() and len(known_anchors):
                difference = (C[known_anchors][:, positions] - C[i, positions]).T
                lp = linprog(-np.ones(len(known_anchors)), A_ub=np.vstack([difference, np.ones(len(known_anchors))]),
                             b_ub=np.r_[np.zeros(len(positions)), .75], bounds=(0, None), method='highs')
                if lp.success:
                    lp_fraction = float(lp.x.sum())
                    if lp_fraction > 1e-8:
                        for shrink in np.linspace(.05, 1, 20):
                            weights = shrink * lp.x
                            value = source * (1 - weights.sum()) + weights @ source_data[known_anchors]
                            candidates = np.vstack([candidates, value])
                            recipe = {int(beers[a]): float(w) for a, w in zip(known_anchors, weights) if w > 1e-10}
                            candidate_recipes.append((None, float(weights.sum()), recipe))
```

```
            lp_rows.append({'repeat': repeat, 'beer': int(beers[i]), 'source_role': role[i],
                            'max_target_mixture_fraction': lp_fraction, 'known_target_anchors': len(known_anchors),
                            'scope': 'reported_concentration_four_linear_constraints_only_no_LOD_or_reaction_certification'})
            known = np.isfinite(C[i, positions]).all()
            chem_ok = exact_nonincrease(candidates, C[i], positions)
            probabilities = search.predict_proba(candidates)[:, target_position]
            gs = sensory.predict(source[None])[0, stale]; rs = consensus.predict(source[None])[0, stale]
            gpred = sensory.predict(candidates)[:, stale]; rpred = consensus.predict(candidates)[:, stale]
            valid = chem_ok & (probabilities >= max(.5, srcp + .01)) & (gpred <= gs + 1e-9) & (rpred <= rs + 1e-9)
            change = np.linalg.norm(generator_prep.transform(candidates) - generator_prep.transform(source[None]), axis=1) / np.sqrt(C.shape[1])
            valid &= change > 1e-8
            good = np.flatnonzero(valid); k = int(good[np.argmin(change[good])]) if len(good) else None
            value = candidates[k].copy() if k is not None else source.copy()
            locked.append({'row': i, 'candidate': value, 'source': source, 'candidate_index': k,
                           'anchor_beer': candidate_recipes[k][0] if k is not None else None,
                           'weight': candidate_recipes[k][1] if k is not None else 0,
                           'recipe': candidate_recipes[k][2] if k is not None else {},
                           'source_prob': srcp, 'candidate_prob': probabilities[k] if k is not None else srcp,
                           'candidates': len(candidates), 'chemical_feasible': int(chem_ok.sum()), 'valid': len(good),
                           'known_defects': bool(known), 'positive_defects': bool(known and (C[i, positions] > 0).all()),
                           'conditional_missing_any_source_input': bool(np.isnan(C[i]).any()),
                           'source_role': role[i], 'status': 'concentration_mixture_proxy_feasible' if k is not None else 'no_feasible_mixture_
source_retained'})
        # Lock exact numeric candidates BEFORE fitting independent J classifiers/sensory.
        digest = hashlib.sha256(np.asarray([v['candidate'] for v in locked], dtype='<f8').tobytes()).hexdigest()
        write_json(dest / f'concentration_candidate_lock_{repeat}.json', {'repeat': repeat, 'sha256_float64_candidates': digest,
                                                                          'features': features, 'selection_completed_before_J_fitting': True})
        evaluator = Pipeline([('prep', preprocessing()), ('model', ExtraTreesClassifier(n_estimators=200, min_samples_leaf=3, max_features=.7,
random_state=cfg['seed'], n_jobs=2))]).fit(C[J], labels[J])
        judge = tree_model(cfg).fit(C[J], Y[J]); target_j = list(evaluator.classes_).index('Tripel')
        diagnostics.append({'repeat': repeat, 'concentration_search_balanced_accuracy_on_T': balanced_accuracy_score(labels[T], search.predict(C[T
])),
                            'concentration_J_balanced_accuracy_on_T': balanced_accuracy_score(labels[T], evaluator.predict(C[T])),
                            'raw_stale_J_mae_on_T': float(np.abs(judge.predict(C[T])[:, stale] - Y[T, stale]).mean())})
        for v in locked:
            i = v['row']; source, value = v['source'], v['candidate']; jp = evaluator.predict_proba(np.stack([source, value]))[:, target_j]
            js = judge.predict(np.stack([source, value]))[:, stale]; predicted = evaluator.predict(value[None])[0]
            feasible = v['candidate_index'] is not None
            accepted = feasible and predicted == 'Tripel' and jp[1] > jp[0] + .01 and js[1] <= js[0] + 1e-9
            mixture_rows.append({'repeat': repeat, 'beer': int(beers[i]), 'source_role': v['source_role'], 'status': v['status'],
                                 'anchor_beer': v['anchor_beer'], 'anchor_volume_fraction': v['weight'], 'n_candidates': v['candidates'],
                                 'target_component_fractions': str(v['recipe']),
                                 'n_chemical_feasible': v['chemical_feasible'], 'n_search_feasible': v['valid'],
                                 'four_source_defects_observed': v['known_defects'], 'four_source_defects_positive': v['positive_defects'],
                                 'conditional_on_other_source_input_imputation': v['conditional_missing_any_source_input'],
                                 'G_source_Tripel_probability': v['source_prob'], 'G_candidate_Tripel_probability': v['candidate_prob'],
                                 'J_source_Tripel_probability': jp[0], 'J_candidate_Tripel_probability': jp[1],
                                 'J_stale_source_raw_0_6': js[0], 'J_stale_candidate_raw_0_6': js[1],
                                 'J_predicts_Tripel': predicted == 'Tripel', 'independent_proxy_support': bool(accepted),
                                 'real_tasting_validated': False, 'LOD_certification': False})
            mixture_profiles.append({'repeat': repeat, 'beer': int(beers[i]), 'status': v['status'], **dict(zip(features, value))})
    save_csv(pd.DataFrame(frozen_rows), dest / 'frozen_stage3_candidates_raw_score_check.csv')
    save_csv(pd.DataFrame(mixture_rows), dest / 'concentration_mixture_evaluation.csv')
    save_csv(pd.DataFrame(mixture_profiles), dest / 'concentration_mixture_profiles.csv')
    save_csv(pd.DataFrame(diagnostics), dest / 'concentration_holdout_diagnostics.csv')
    save_csv(pd.DataFrame(lp_rows), dest / 'mixture_lp_feasibility.csv')
    write_json(dest / 'status.json', {'complete': True, 'concentration_features': features, 'frozen_candidates_evaluated': len(frozen_rows),
                                      'mixture_records': len(mixture_rows), 'physical_experiments_performed': False})
```

016_stage4_trial.py

```
"""Power planning and analysis for a future paired, blinded physical experiment.
No human observations are generated here. AI-assisted, 2026-10-25.
"""
import numpy as np
import pandas as pd
from scipy.stats import t, nct

DEFECT_ENDPOINTS = ['dms', 'diacetyl', 'X4vg', 't2n', 'stale_hops']


def planned_noninferiority_power(n, sigma, margin=.5, alpha=.01):
    critical = t.ppf(1 - alpha, n - 1)
    return float(nct.cdf(-critical, n - 1, -margin * np.sqrt(n) / sigma))


def analyze_paired_trial(frame, margin=.5):
    endpoints = DEFECT_ENDPOINTS + ['style_Tripel_fit']
    required = ['panelist', 'batch', 'condition'] + endpoints
    if not set(required).issubset(frame) or frame.empty:
        raise ValueError('No observed paired data; a blank template is not an experiment')
    if frame[required].isna().any().any() or frame.duplicated(['panelist', 'batch', 'condition']).any():
        raise ValueError('Missing values or duplicated participant-batch-condition records require review')
    if set(frame.condition) != {'control', 'candidate'} or not frame[endpoints].ge(0).all().all() or not frame[endpoints].le(6).all().all():
        raise ValueError('Two conditions and valid 0-6 ratings are required')
    control = frame[frame.condition == 'control'].set_index(['panelist', 'batch'])[endpoints]
    candidate = frame[frame.condition == 'candidate'].set_index(['panelist', 'batch'])[endpoints]
    if set(control.index) != set(candidate.index):
        raise ValueError('Unpaired batches cannot silently enter paired analysis')
    difference = (candidate - control).groupby(level='panelist').mean()
    if len(difference) < 3:
        raise ValueError('At least three independent panelists are required for the diagnostic')
    rows = []
    for endpoint in endpoints:
        d = difference[endpoint].to_numpy(); n = len(d); mean = d.mean(); se = d.std(ddof=1) / np.sqrt(n)
```

```
        alpha = .01 if endpoint in DEFECT_ENDPOINTS else .05
        upper = mean + t.ppf(1 - alpha, n - 1) * se
        lower = mean - t.ppf(1 - alpha, n - 1) * se
        rows.append({'endpoint': endpoint, 'independent_panelists': n, 'candidate_minus_control': mean,
                     'one_sided_alpha': alpha, 'lower_confidence_limit': lower, 'upper_confidence_limit': upper,
                     'criterion_passed': bool(upper < margin if endpoint in DEFECT_ENDPOINTS else lower > 0),
                     'criterion': 'upper_limit_below_0.5_noninferiority_margin' if endpoint in DEFECT_ENDPOINTS else 'lower_limit_above_zero_style
_improvement'})
    return pd.DataFrame(rows)
```

017_analyze_trial.py

```
"""Analyze real future blinded paired ratings; refuse empty/simulated templates."""
from pathlib import Path
import sys
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import argparse
import pandas as pd
from src.stage4_trial import analyze_paired_trial
from src.common import save_csv
parser = argparse.ArgumentParser()
parser.add_argument('--ratings', required=True)
parser.add_argument('--out', required=True)
args = parser.parse_args()
result = analyze_paired_trial(pd.read_csv(args.ratings))
save_csv(result, args.out)
print('Analysis uses independent participant means; repeated batches do not increase the participant count.')
```

020_fetch_stage4_sources.py

```
"""Fetch two public author sources with pinned MD5; keep raw files outside repo."""
from pathlib import Path
from urllib.request import urlopen
from urllib.parse import quote
import argparse
import hashlib
import json

ROOT = Path(__file__).resolve().parents[1]
parser = argparse.ArgumentParser()
parser.add_argument('--out', default=str(ROOT.parent / 'external_sources/zenodo_v4_23624915'))
args = parser.parse_args(); out = Path(args.out); out.mkdir(parents=True, exist_ok=True)
cfg = json.loads((ROOT / 'config_stage4.json').read_text(encoding='utf-8'))
for filename, key in [('Raw sensory data anonymized.csv', 'raw_md5'), ('Supplemental Files and Figure source files.xlsx', 'workbook_md5')]:
    path = out / filename
    if path.exists() and hashlib.md5(path.read_bytes()).hexdigest() == cfg[key]:
        print('Verified existing source: ' + filename); continue
    url = cfg['external_record'] + '/files/' + quote(filename) + '?download=1'
    raw = urlopen(url, timeout=60).read()
    if hashlib.md5(raw).hexdigest() != cfg[key]:
        raise ValueError('Publisher checksum mismatch: ' + filename)
    path.write_bytes(raw)
    print('Downloaded and verified: ' + filename)
```

030_stage3_preflight.py

```
"""Read-only source audit and conditional analysis of existing OOF predictions.

Run from the bundled document runtime (openpyxl + numpy). No workbook is edited.
This is a protocol preflight, not the full stage-three experiment runner.
"""
from pathlib import Path
import argparse
import csv
import hashlib
import json
import re
import sys
import xml.etree.ElementTree as ET
import numpy as np
from openpyxl import load_workbook

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.stage3_algorithms import paired_oof_bootstrap

parser = argparse.ArgumentParser()
parser.add_argument("--data-dir", type=Path, default=ROOT.parent / "materials/A/附件")
parser.add_argument("--out", type=Path, default=ROOT / "outputs/stage3_design")
args = parser.parse_args()
DATA, OUT = args.data_dir, args.out
OUT.mkdir(parents=True, exist_ok=True)


def read_csv(path):
    with path.open(encoding="utf-8-sig", newline="") as stream:
        return list(csv.DictReader(stream))


def write(name, value):
```

```
    (OUT / name).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")


original_hashes = {name: hashlib.sha256((DATA / name).read_bytes()).hexdigest() for name in ["1.csv", "3.xlsx", "4.csv", "8.xlsx"]}
previous_hashes = {r['file']:r['sha256'] for r in read_csv(ROOT/'outputs/stage2/audit/source_manifest.csv')}
if any(previous_hashes.get(name) != sha for name,sha in original_hashes.items()):
    raise ValueError("Source version differs from the existing stage-two predictions/evidence")
target_rows = read_csv(DATA / "4.csv")
targets = [c for c in target_rows[0] if c not in ["beer", "beer_id", "tasting_category_fine"]]
aliases = {"A_ester_fruity": "A_esters_fruity", "F_ester_fruity": "F_esters_fruity", "4vg": "X4vg"}
workbook = load_workbook(DATA / "3.xlsx", data_only=False, read_only=True)
summary = workbook["summary"]
mapping, anomalies = [], []
for col in range(2, summary.max_column+1):
    original = summary.cell(1,col).value
    target = aliases.get(original,original)
    if target not in targets:
        continue
    formula = str(summary.cell(2,col).value)
    match = re.fullmatch(r"=beer_1!\$([A-Z]+)\$(\d+)",formula)
    if not match:
        raise ValueError(f"Unexpected score formula for {target}: {formula}")
    column, row = match.group(1), int(match.group(2))
    expected_cell = "AG90" if target == "barnyard" else f"{column}{row}"
    section = ("Aroma_core" if target.startswith("A_") else "Flavour_core" if target.startswith("F_") else
               "Faults" if target in ["dms","metallic","X4vg","t2n","diacetyl","stale_hops"] else
               "Spiced_beers" if target in ["orange","coriander","clove"] else
               "Lambic_Geuze" if target in ["lactic","acetic","barnyard"] else
               "Overall" if target == "overall" else "Palate_or_basic_taste")
    mapping.append({"target":target,"summary_header":original,"summary_column":col,"original_formula":formula,
                    "expected_score_cell":expected_cell,"section":section,
                    "formula_status":"suspected_wrong_reference_derived_map_corrects_only" if target=='barnyard' else "consistent_with_form_
position",
                    "official_submission_scope":"requires_explicit_reporting_core_30_and_full_50_not_inferred_from_count"})
    if target == "barnyard":
        assert "Barnyard" in str(workbook["beer_1"]["R89"].value)
        assert str(workbook["beer_1"]["AG90"].value).startswith("=")
        for i in range(2,14):
            actual=str(summary.cell(i,col).value)
            expected=f"=beer_{i-1}!$AG$90"
            if actual != expected:
                anomalies.append({"summary_cell":summary.cell(i,col).coordinate,"target":target,"actual_formula":actual,"form_position_expected_
formula":expected})
assert len(mapping) == len(targets) == 50 and {r['target'] for r in mapping} == set(targets)
write("scoring_field_map.json",mapping)
write("scoring_scope_audit.json",{"mapped_targets":50,"core_Aroma":15,"core_Flavour":15,"additional_targets":20,
    "aliases":aliases,"suspected_formula_mismatches":anomalies,
    "card_items_without_separate_supervised_targets":[{"card_label":"Acetaldehyde under Aroma","score_cell":"Q45"},{"card_label":"Acetaldehyde
under Flavour","score_cell":"AG45"}],
    "original_files_edited":False,"absolute_score_anchors_available":False,
    "scope_caveat":"30 is the prefixed core comparison scope, not a ruling that only 30 answers are required."})

scales = {r['field']:r for r in read_csv(ROOT / "outputs/stage2/audit/field_scales.csv")}
chem = {r['field']:r for r in read_csv(ROOT / "outputs/stage2/audit/chemical_dictionary.csv")}
registry=[]
for field in ["DMS","X2.3.butanedione","X4VG","X.E..2.nonenal"]:
    scale,meta=scales[field],chem[field]
    try: threshold=float(meta['threshold_median'])
    except (TypeError,ValueError): threshold=None
    if threshold is not None and not np.isfinite(threshold): threshold=None
    supported=scale['transform_evidence']=='verified_log10_positive_pairs'
    registry.append({"field":field,"unit_in_attachment":meta['unit'],"threshold_median_in_attachment":threshold,
        "transformation_evidence":scale['transform_evidence'],"positive_pair_log10_supported":supported,
        "concentration_rule_if_eligible":"10**x" if supported else "unresolved_per_sample_check_needed",
        "zero_LOD_or_substitution_provenance":"not_fully_verified",
        "absolute_defect_certification":"not_complete_even_if_positive_pairs_match",
        "next_action":"audit_per_sample_calibrated_pairs_and_detection_limits" if supported else "resolve_mismatching_pairs_and_scale_before_
concentration_claim"})
write("defect_registry_preflight.json",registry)

beer_ids=[int(r['beer']) for r in target_rows]
truth=np.array([[float(r[t]) for t in targets] for r in target_rows])
def predictions(path):
    rows=read_csv(path)
    bybeer={int(r['beer']):r for r in rows}
    if len(bybeer)!=len(rows) or set(bybeer)!=set(beer_ids): raise ValueError("OOF keys do not align uniquely")
    return np.array([[float(bybeer[b][t]) for t in targets] for b in beer_ids])
gbr=predictions(ROOT/'outputs/stage2/q1/oof_gbr_continuous.csv')
et=predictions(ROOT/'outputs/stage1/q1/oof_extra_trees_continuous.csv')
labels=[r['tasting_category_fine'] for r in target_rows]
reports=[]
for scope,indices in [('all_50',list(range(50))),('prefixed_core_30',[j for j,t in enumerate(targets) if t.startswith(('A_','F_'))])]:
    result=paired_oof_bootstrap(truth[:,indices],gbr[:,indices],et[:,indices],labels)
    result.update(scope=scope,comparison='fixed_stage2_GBR_minus_stage1_ExtraTrees',metric='macro_MAE',
                  uncertainty_scope='beer-resampling_conditional_on_existing_predictions_not_fresh_external_validation')
    reports.append(result)
write('existing_oof_paired_pilot.json',reports)

test_report=ROOT/'tmp/stage3_tests.xml'
suite=ET.parse(test_report).getroot().find('testsuite')
assert all(int(suite.attrib[k])==0 for k in ['failures','errors','skipped'])
for name,old in original_hashes.items():
    assert hashlib.sha256((DATA/name).read_bytes()).hexdigest()==old
manifest={'date':'2026-10-25','stage':'design_and_preflight','complete_stage3_training_run':False,
    'source_sha256':original_hashes,'protocol_sha256':hashlib.sha256((ROOT/'config_stage3_design.json').read_bytes()).hexdigest(),
    'algorithm_tests':{k:int(suite.attrib[k]) for k in ['tests','failures','errors','skipped']},
    'no_human_validation_claimed':True,'no_original_workbook_edited':True,
```

```
    'full_experiments_status':'planned_not_executed; only source mapping and conditional existing-OOF pilot executed'}
write('preflight_manifest.json',manifest)
print(json.dumps({'mapped_targets':len(mapping),'formula_mismatches':len(anomalies),'paired_pilot':reports,'test_summary':manifest['algorithm_tests']},ensure_ascii=False))
```

032_stage4_graph_comparison.py

```
"""Repeat the full graph control experiment on the newly observed raw endpoint."""
from pathlib import Path
import os
for name in ['OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS']:
    os.environ.setdefault(name, '2')
import sys
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import json
import argparse
from dataclasses import replace
import pandas as pd
from src.common import environment, write_json, sha256
from src.data import load_data, find_data
from src.stage3_graph import run_graph
ROOT = Path(__file__).resolve().parents[1]
parser = argparse.ArgumentParser(); parser.add_argument('--data-dir'); args = parser.parse_args()
cfg = json.loads((ROOT / 'config_stage3.json').read_text(encoding='utf-8'))
cfg.update(json.loads((ROOT / 'config_stage4.json').read_text(encoding='utf-8')))
data = load_data(find_data(args.data_dir))
panel = pd.read_csv(ROOT / 'outputs/stage4/audit/observed_panel_means_0_6.csv').set_index('beer').reindex(data.meta.beer)
raw_data = replace(data, Y=pd.DataFrame(panel.to_numpy(), columns=data.Y.columns))
destination = ROOT / 'outputs/stage4/raw_graph'
run_graph(raw_data, destination, cfg)
write_json(destination / 'run_manifest.json', {**environment(), 'config': cfg, 'target': 'observed_raw_equal_panelist_mean_0_6',
                                               'code_sha256': {p.relative_to(ROOT).as_posix(): sha256(p) for p in [Path(__file__), ROOT / 'src/stage3_graph.py']}})
```

036_verify_stage3.py

```
"""Cross-check saved scientific artifacts and record exact executed scope."""
from pathlib import Path
import hashlib
import json
import xml.etree.ElementTree as ET
import numpy as np
import pandas as pd
ROOT=Path(__file__).resolve().parents[1]; OUT=ROOT/'outputs/stage3'
suite=ET.parse(ROOT/'tmp/stage3_full_tests.xml').getroot().find('testsuite')
tests={k:int(suite.attrib[k]) for k in ['tests','failures','errors','skipped']}
assert tests['tests']>=40 and tests['failures']==tests['errors']==tests['skipped']==0
status={part:json.loads((OUT/part/'status.json').read_text()) for part in ['q1','q2','q3']}
assert all(a['complete'] for a in status.values())
roles=pd.read_csv(OUT/'q1/outer_roles.csv')
assert len(roles)==1250 and roles['repeat'].nunique()==5
assert (roles.groupby(['repeat','beer']).size()==1).all()
grades=pd.read_csv(OUT/'q1/ASSUMPTION_oof_relative_grades.csv'); a=grades.drop(columns='beer').to_numpy()
assert a.shape==(250,50) and np.isfinite(a).all() and np.all((a>=0)&(a<=6)&(a==a.astype(int)))
weights=pd.read_csv(OUT/'q1/blend_weights.csv'); assert len(weights)==750 and weights.weight_GBR.between(0,1).all()
metadata=pd.read_csv(ROOT/'outputs/stage2/audit/source_manifest.csv')
from sys import path
path.insert(0,str(ROOT))
from src.data import find_data,load_data
original=find_data(); data=load_data(original)
for row in metadata.itertuples(): assert hashlib.sha256((original/row.file).read_bytes()).hexdigest()==row.sha256
scope=pd.read_csv(OUT/'q1/metrics_by_scope.csv'); core=[c for c in data.Y if c.startswith(('A_','F_'))]
for model in scope.model.unique():
    pred=pd.read_csv(OUT/f'q1/oof_{model}.csv').set_index('beer').loc[data.meta.beer]
    for name,fields in [('core30',core),('all50',list(data.Y))]:
        expected=float(np.abs(data.Y[fields].to_numpy()-pred[fields].to_numpy()).mean())
        actual=float(scope[(scope.model==model)&(scope.scope==name)].mae.iloc[0]); assert abs(expected-actual)<2e-8
graph=pd.read_csv(OUT/'q2/graph_fold_metrics.csv'); assert graph.graph.notna().all() and len(graph[graph.graph=='shuffled'])==300
gen=pd.read_csv(OUT/'q3/generation_comparison.csv'); assert len(gen)==60 and (gen.n_generated==40).all()
assert (gen.provided_bounds_violation_fraction==0).all()
transfer=pd.read_csv(OUT/'q3/transfer_independent_evaluation.csv'); assert len(transfer)==220
assert not transfer.absolute_chemical_certification.any() and not transfer.real_tasting_validated.any()
defects=pd.read_csv(OUT/'q3/transfer_defect_checks.csv'); assert defects.provided_nonincrease.all()
top=pd.read_csv(OUT/'q2/top3_with_evidence.csv'); assert len(top)==150 and (top.groupby('target').size()==3).all()
literature=[]
for r in top.itertuples():
    row={'target':r.target,'field':r.field,'statistical_status':r.evidence_status,'causal_status':'not_identified','independent_reference':'','chemical_evidence':'attachment_metadata_only_target_specific_mechanism_unverified','limitations':'direction_and_style_confounding_require_review'}
    if r.target=='X4vg' and r.field=='X4VG': row.update(independent_reference='https://pubmed.ncbi.nlm.nih.gov/19053838/',chemical_evidence='beer_research_supports_clove_related_aroma_not_current_dataset_causality',limitations='aging_and_matrix_conditions_not_a_new_spiking_test')
    elif r.target=='metallic' and r.field=='iron': row.update(independent_reference='https://pubmed.ncbi.nlm.nih.gov/22304665/',chemical_evidence='metal_solution_and_salivary_lipid_oxidation_mechanism',limitations='iron_speciation_and_beer_matrix_not_resolved_no_beer_threshold_inferred')
    elif r.target=='coriander' and r.field=='linalool': row.update(independent_reference='https://pubmed.ncbi.nlm.nih.gov/38398596/',chemical_evidence='coriander_seed_low_alcohol_beer_primary_research_context',limitations='not_specific_causality_for_this_panel_or_this_concentration')
    elif (r.target=='acidity' and r.field in ['lactic_acid.mg.L.','acetic_acid.g.L.']) or (r.target=='lactic' and r.field=='lactic_acid.mg.L.') or (r.target=='alcohol' and r.field=='ethanol..v.v.'):
        row.update(chemical_evidence='measurement_compatible_relation_not_independent_causal_validation',limitations='matrix_mixture_and_source_preprocessing_remain')
    literature.append(row)
pd.DataFrame(literature).to_csv(OUT/'q2/chemical_evidence_ledger.csv',index=False,encoding='utf-8-sig')
summary={'client_date':'2026-10-25','tests':tests,'original_sources_unchanged':True,'prediction_outer_folds':15,'unique_beers':250,
 'graph_null_fits_per_outer':10,'generation_disjoint_partitions':5,'generated_records_per_group':40,
```

```
 'core_targets':30,'all_targets':50,'relative_grade_cells':12500,
 'complete_absolute_score_calibration':False,'complete_chemical_certification':False,'real_tasting_validated':False,
 'scientific_scope':'exploratory_internal_validation_after_earlier_stages_not_new_external_validation',
 'executed':['nested_prediction_and_shrinkage','fixed_relative_grade_sets','missingness_and_clipping','train_only_graph_ablations','crossfit_within_style_module_effects','disjoint_generation_and_Pareto_transfer','failure_and_unknown_item_preservation'],
 'still_unresolved':['real_absolute_score_anchors','detection_limits_and_all_units','all_aggregate_formulas','full_target_specific_chemical_mechanisms','new_external_beers_and_tasting','human_independent_review'],
 'source_hashes':[{ 'file':r.file,'sha256':r.sha256} for r in metadata.itertuples()],
 'code_sha256':{p.relative_to(ROOT).as_posix():hashlib.sha256(p.read_bytes()).hexdigest() for p in sorted((ROOT/'src').glob('stage3*.py'))}}
(OUT/'validation_summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
checks=[{'file':p.relative_to(OUT).as_posix(),'sha256':hashlib.sha256(p.read_bytes()).hexdigest()} for p in sorted(OUT.rglob('*')) if p.is_file() and p.name!='result_checksums.json' and 'private' not in p.parts]
(OUT/'result_checksums.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps({'tests':tests,'files_hashed':len(checks),'relative_grade_cells':12500,'all_three_experiments_complete':True},ensure_ascii=False))
```

037_verify_stage4.py

```
"""Validate source identity, absolute outputs, grading and frozen-candidate roles."""
from pathlib import Path
import json
import hashlib
import xml.etree.ElementTree as ET
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]; OUT = ROOT / 'outputs/stage4'
audit = json.loads((OUT / 'audit/audit_summary.json').read_text(encoding='utf-8'))
assert audit['identity_matches_both_attribute_groups'] == 250 and audit['nonconstant_targets_reproduced'] == 38
assert audit['nonconstant_max_error'] < 1e-7 and audit['concentration_model_fields'] == 61
mapping = pd.read_csv(OUT / 'audit/identity_mapping.csv'); assert mapping.beer.nunique() == 250 and mapping.raw_beer.nunique() == 250 and mapping.is_row_nearest.all()
reconstruction = pd.read_csv(OUT / 'audit/constant_attribute_reconstruction.csv'); assert len(reconstruction) == 12 and reconstruction.validation_max_error.max() < 1e-7
panel = pd.read_csv(OUT / 'audit/observed_panel_means_0_6.csv').set_index('beer'); assert panel.shape == (250, 50) and panel.ge(0).all().all() and panel.le(6).all().all()
for part in ['prediction', 'ordinal', 'transfer']: assert json.loads((OUT / part / 'status.json').read_text())['complete']
graph_status = json.loads((OUT / 'raw_graph/q2/status.json').read_text()); assert graph_status['complete'] and graph_status['outer_folds'] == 15
graph_folds = pd.read_csv(OUT / 'raw_graph/q2/graph_fold_metrics.csv')
assert len(graph_folds[graph_folds.graph == 'shuffled']) == 300
roles = pd.read_csv(OUT / 'prediction/outer_roles.csv'); assert len(roles) == 1250 and (roles.groupby(['repeat', 'beer']).size() == 1).all()
metrics = pd.read_csv(OUT / 'prediction/metrics_by_scope.csv')
core = [f for f in panel if f.startswith(('A_', 'F_'))]
for model in metrics.model.unique():
    p = pd.read_csv(OUT / f'prediction/oof_{model}_0_6.csv').set_index('beer').reindex(panel.index)
    assert p.ge(0).all().all() and p.le(6).all().all()
    for scope, fields in [('core30', core), ('all150', list(panel))]:
        value = np.abs(panel[fields] - p[fields]).to_numpy().mean()
        assert abs(value - metrics[(metrics.model == model) & (metrics.scope == scope)].mae.iloc[0]) < 1e-7
for name in ['oof_modal_grades_0_6.csv', 'oof_median_grades_0_6.csv', 'oof_rounded_panel_mean_grades_0_6.csv']:
    grades = pd.read_csv(OUT / 'ordinal' / name).iloc[:, 1:].to_numpy(); assert grades.shape == (250, 50) and np.isin(grades, np.arange(7)).all()
distributions = pd.read_csv(OUT / 'ordinal/oof_rating_distributions.csv'); assert len(distributions) == 12500
prob = distributions[[f'predicted_p{k}' for k in range(7)]].to_numpy(); observed = distributions[[f'observed_panel_p{k}' for k in range(7)]].to_numpy()
assert (prob >= 0).all() and np.allclose(prob.sum(1), 1, atol=1e-8) and np.allclose(observed.sum(1), 1, atol=1e-8)
assert np.array_equal(prob.argmax(1), distributions.predicted_modal_grade)
assert abs(np.abs((prob * np.arange(7)).sum(1) - (observed * np.arange(7)).sum(1)).mean() - pd.read_csv(OUT / 'ordinal/metrics_by_scope.csv').query("scope == 'all50' and method == 'ET_rating_distribution'").expected_panel_mean_MAE.iloc[0]) < 1e-7
intervals = pd.read_csv(OUT / 'prediction/absolute_panel_mean_intervals.csv'); assert len(intervals) == 12500 and (intervals.lower_0_6 <= intervals.upper_0_6).all() and intervals.lower_0_6.ge(0).all() and intervals.upper_0_6.le(6).all()
top = pd.read_csv(OUT / 'prediction/absolute_raw_top3.csv'); assert len(top) == 150 and (top.groupby('target').size() == 3).all() and not top.causal_validated.any()
frozen = pd.read_csv(OUT / 'transfer/frozen_stage3_candidates_raw_score_check.csv'); assert len(frozen) == 220 and not frozen.physical_validation.any()
assert not (frozen.original_and_raw_proxy_support & ~frozen.original_proxy_acceptance).any()
mixture = pd.read_csv(OUT / 'transfer/concentration_mixture_evaluation.csv'); assert len(mixture) == 55 and not mixture.real_tasting_validated.any() and not mixture.LOD_certification.any()
assert all(json.loads((OUT / f'transfer/concentration_candidate_lock_{i}.json').read_text())['selection_completed_before_J_fitting'] for i in range(5))
assert pd.read_csv(OUT / 'protocol/blank_panel_rating_template.csv').empty
tests = ET.parse(ROOT / 'tmp/stage4_tests.xml').getroot(); suites = list(tests.iter('testsuite'))
test_summary = {key: sum(int(s.attrib.get(key, 0)) for s in suites) for key in ['tests', 'failures', 'errors', 'skipped']}
assert test_summary['tests'] == 48 and not any(test_summary[k] for k in ['failures', 'errors', 'skipped'])
summary = {'all_four_stage4_components_complete': True, 'tests': test_summary, 'identity_mapping_verified': True,
           'raw_endpoint_graph_controls_complete': True,
           'absolute_panel_mean_cells': 12500, 'integer_grade_cells_per_rule': 12500, 'ordinal_probability_rows': 12500,
           'frozen_transfer_records': len(frozen), 'concentration_mixture_records': len(mixture), 'physical_trial_observations': 0}
(OUT / 'validation_summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
hashes = {p.relative_to(OUT).as_posix(): hashlib.sha256(p.read_bytes()).hexdigest() for p in sorted(OUT.rglob('*')) if p.is_file() and p.name != 'result_checksums.json'}
(OUT / 'result_checksums.json').write_text(json.dumps(hashes, ensure_ascii=False, indent=2), encoding='utf-8')
print(json.dumps({**summary, 'hashed_files': len(hashes)}, ensure_ascii=False))
```

038_test_invariants.py

```
"""AI-assisted verification: OpenAI Codex / GPT-6, 2026-10-24."""
import json
from pathlib import Path

import numpy as np
import pandas as pd
import pytest
```

```
from sklearn.model_selection import RepeatedStratifiedKFold

from src.common import TrainQuantileClipper, ecdf_grades, sha256
from src.data import CHEMICAL_DEFECTS, NON_MOLECULES, align_by_beer, find_data, load_data
from src.q2 import signed_laplacian
from src.q3 import ConditionalPCAGaussian, chemical_nonincrease


ROOT = Path(__file__).resolve().parents[1]
OUTPUT = ROOT / "outputs/stage1"


def test_join_uses_unique_beer_and_recovers_permuted_rows():
    x = pd.DataFrame({"beer": [1, 2], "beer_id": [100, 101], "tasting_category_fine": ["a", "b"]})
    y = pd.DataFrame({"beer": [2, 1], "beer_id": [100, 100], "tasting_category_fine": ["b", "a"], "score": [9, 4]})
    assert align_by_beer(x, y).score.tolist() == [4, 9]


def test_duplicate_key_is_rejected_before_join():
    a = pd.DataFrame({"beer": [1, 1], "tasting_category_fine": ["a", "a"]})
    with pytest.raises(ValueError, match="unique"):
        align_by_beer(a, a)


def test_preprocessing_bounds_do_not_learn_from_validation_extreme():
    train = np.array([[0.], [1.], [2.], [3.]])
    clip = TrainQuantileClipper(lower=0, upper=1).fit(train)
    assert clip.transform([[1e12]])[0, 0] == 3
    np.testing.assert_array_equal(clip.upper_, [3.])


def test_signed_laplacian_is_psd_with_negative_edges():
    L = signed_laplacian([[1, -.8, .7], [-.8, 1, .1], [.7, .1, 1]], .6)
    assert np.linalg.eigvalsh(L).min() > -1e-10


def test_ecdf_grades_are_bounded_and_monotone():
    ref = np.arange(100.)[:, None]
    g = ecdf_grades(ref, np.array([[-1.], [50.], [1000.]]))[:, 0]
    assert g[0] == 0 and g[-1] == 6 and np.all(np.diff(g) >= 0)


def test_each_defect_is_checked_separately():
    # A falling total must not hide a rise in one defect.
    source, candidate = np.array([2., 2.]), np.array([0., 3.])
    assert candidate.sum() < source.sum()
    assert not chemical_nonincrease(source, candidate, [0, 1])


def test_conditional_generator_is_finite_bounded_and_reproducible():
    rng = np.random.default_rng(4)
    x = rng.normal(size=(30, 7))
    x[0, 0] = np.nan
    gen = ConditionalPCAGaussian(3).fit(x, np.array(["a"] * 15 + ["b"] * 15))
    a = gen.sample("a", 10, np.random.default_rng(1))
    b = gen.sample("a", 10, np.random.default_rng(1))
    np.testing.assert_allclose(a, b)
    assert np.isfinite(a).all()
    assert np.all(a >= gen.low - 1e-10) and np.all(a <= gen.high + 1e-10)


@pytest.fixture(scope="module")
def real_data():
    try:
        root = find_data()
    except FileNotFoundError:
        pytest.skip("Original competition data not available")
    return load_data(root)


def need_results():
    if not (OUTPUT / "q3/status.json").exists():
        pytest.skip("Run all stages before artifact validation")


def test_original_sources_unchanged(real_data):
    need_results()
    manifest = pd.read_csv(OUTPUT / "audit/source_manifest.csv")
    for row in manifest.itertuples():
        assert sha256(real_data.root / row.file) == row.sha256


def test_oof_mean_matches_actual_training_only_mean(real_data):
    need_results()
    cfg = json.loads((ROOT / "config.json").read_text())
    Y = real_data.Y.to_numpy()
    expected = np.zeros_like(Y)
    counts = np.zeros(len(Y))
    splitter = RepeatedStratifiedKFold(n_splits=cfg["outer_folds"], n_repeats=cfg["outer_repeats"], random_state=cfg["seed"])
    for train, test in splitter.split(real_data.X, real_data.meta.tasting_category_fine):
        expected[test] += Y[train].mean(axis=0)
        counts[test] += 1
    saved = pd.read_csv(OUTPUT / "q1/oof_mean_continuous.csv")[real_data.Y.columns].to_numpy()
    np.testing.assert_allclose(saved, expected / counts[:, None], atol=1e-7)


def test_each_sample_held_out_once_per_repeat(real_data):
```

```
    need_results()
    s = pd.read_csv(OUTPUT / "q1/split_assignments.csv")
    assert not s.duplicated(["repeat", "beer"]).any()
    for _, part in s.groupby("repeat"):
        assert set(part.beer) == set(real_data.meta.beer)


def test_top3_has_unique_eligible_factors_per_target(real_data):
    need_results()
    top = pd.read_csv(OUTPUT / "q2/top3_by_sensory_dimension.csv")
    assert len(top) == 3 * real_data.Y.shape[1]
    assert set(top.target) == set(real_data.Y.columns)
    assert not top.duplicated(["target", "field"]).any()
    assert not set(top.field) & NON_MOLECULES
    assert top.bootstrap_top3_frequency.between(0, 1).all()


def test_transfer_failure_is_not_hidden_as_success():
    need_results()
    t = pd.read_csv(OUTPUT / "q3/transfer_diagnostics.csv")
    failed = t[t.status == "no_nontrivial_candidate_found_source_retained"]
    assert np.allclose(failed.supplied_scale_change_l2, 0)
    success = t[t.status == "nontrivial_model_feasible_candidate"]
    assert (success.candidate_target_probability > success.source_target_probability + .01).all()
    assert (success.candidate_target_probability > success.cleaning_only_target_probability + .01).all()
    assert t.chemical_nonincrease.all() and t.predicted_stale_hops_nonincrease.all()
    checks = pd.read_csv(OUTPUT / "q3/defect_nonincrease_checks.csv")
    assert checks.nonincrease.all()
    assert set(checks.defect) == set(CHEMICAL_DEFECTS) | {"stale_hops"}


def test_calibration_remains_explicitly_unresolved():
    need_results()
    status = json.loads((OUTPUT / "q1/status.json").read_text())
    assert status["absolute_0_6_prediction"].startswith("UNRESOLVED")
    grades = pd.read_csv(OUTPUT / "q1/ASSUMPTION_percentile_grades_0_6.csv")
    assert grades.status.str.startswith("ASSUMPTION_ONLY").all()


def test_generator_holdout_does_not_overlap_training(real_data):
    need_results()
    split = pd.read_csv(OUTPUT / "q3/generator_split.csv")
    train = set(split[split.role == "train"].beer)
    heldout = set(split[split.role == "holdout"].beer)
    assert train.isdisjoint(heldout)
    assert train | heldout == set(real_data.meta.beer)


def test_generation_diagnostics_do_not_hide_missing_reference_values():
    need_results()
    diagnostics = pd.read_csv(OUTPUT / "q3/generation_diagnostics.csv")
    assert np.isfinite(diagnostics.mean_marginal_KS_holdout).all()
    assert diagnostics.mean_marginal_KS_holdout.between(0, 1).all()
    assert (diagnostics.n_KS_fields_observed > 0).all()
```

039_test_stage2.py

```
"""AI-assisted validation: OpenAI Codex / GPT-6, 2026-10-24."""
import json
from pathlib import Path

import numpy as np
import pandas as pd
import pytest

from src.stage2 import assumption_grades, conformal_quantile, local_convex_samples

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "outputs/stage2"


def test_conformal_quantile_uses_finite_sample_rank():
    errors = np.arange(1, 51)[:, None]
    assert conformal_quantile(errors, .1)[0] == 46
    with pytest.raises(ValueError):
        conformal_quantile(errors[:2], .01)


def test_three_grade_assumptions_remain_bounded():
    rng = np.random.default_rng(3)
    ref = rng.normal(size=(50, 4))
    grades = assumption_grades(ref, np.array([[-100.] * 4, [100.] * 4]))
    assert grades.shape == (3, 2, 4)
    assert np.all((grades >= 0) & (grades <= 6))
    assert np.all(grades[:, 0] <= grades[:, 1])


def test_local_convex_samples_stay_inside_training_hull_box():
    real = np.array([[0., 1.], [1., 2.], [2., 3.]])
    generated = local_convex_samples(real, 50, np.random.default_rng(4))
    assert np.all(generated >= real.min(0)) and np.all(generated <= real.max(0))
    assert np.allclose(generated[:, 1] - generated[:, 0], 1.)
    assert not np.any(np.all(np.isclose(generated[:, None], real[None]), axis=2))


```

```
def need_stage2():
    if not (OUT / "status.json").exists():
        pytest.skip("Stage two has not been run")


def test_conformal_roles_are_disjoint():
    need_stage2()
    frame = pd.read_csv(OUT / "q1/conformal_split.csv")
    assert len(frame) == frame.beer.nunique() == 250
    assert frame.role.value_counts().to_dict() == {"train": 150, "calibration": 50, "test": 50}


def test_attribution_flags_do_not_claim_causality():
    need_stage2()
    top = pd.read_csv(OUT / "q2/top3_evidence_review.csv")
    assert len(top) == 150
    selected = top[top.evidence_status == "comparatively_supported_association"]
    assert (selected.bootstrap_top3_frequency >= .5).all()
    assert (selected.heldout_permutation_MAE_increase > 0).all()
    assert (selected.positive_fold_fraction >= 2 / 3).all()
    assert (top.causality == "not_established").all()


def test_repeated_generator_comparison_has_all_methods_and_splits():
    need_stage2()
    d = pd.read_csv(OUT / "q3/generator_comparison_by_split.csv")
    assert len(d) == 5 * 3 * 3
    assert set(d.method) == {"bootstrap", "local_convex", "PCA_Gaussian"}
    assert np.isfinite(d[["KS_mean", "MMD2", "target_style_probability"]]).all().all()


def test_extended_transfer_preserves_failure_and_each_defect():
    need_stage2()
    d = pd.read_csv(OUT / "q3/expanded_transfer_diagnostics.csv")
    good = d[d.status == "nontrivial_model_feasible_candidate"]
    assert (good.candidate_trip_probability > good.source_trip_probability + .01).all()
    assert (good.candidate_trip_probability > good.cleaning_only_probability + .01).all()
    bad = d[d.status != "nontrivial_model_feasible_candidate"]
    assert np.allclose(bad.change_l2, 0)
    assert pd.read_csv(OUT / "q3/expanded_transfer_defect_checks.csv").nonincrease.all()


def test_stage2_keeps_absolute_calibration_unidentified():
    need_stage2()
    assert json.loads((OUT / "status.json").read_text())["absolute_score_calibration"].startswith("not_identifiable")
```

040_test_stage3_algorithms.py

```
"""Meaningful property tests for the new mathematical helpers."""
import numpy as np
import pytest
from src.stage3_algorithms import (fixed_grade_boundaries, grade_with_fixed_boundaries,
    ordinal_interval_image, shrunk_blend_weights, paired_oof_bootstrap, pareto_minimize, threshold_excess)


def test_fixed_ordinal_map_preserves_contained_outcome_with_tied_boundaries():
    boundaries = fixed_grade_boundaries(np.array([[0,0],[0,1],[0,2],[1,3],[1,4],[1,5],[2,6]], float))
    lower, upper = np.array([[-1,1.5],[0,2]]), np.array([[2,4],[0,2]])
    lo, hi = ordinal_interval_image(lower, upper, boundaries)
    for weight in np.linspace(0,1,21):
        inside = grade_with_fixed_boundaries(lower+weight*(upper-lower),boundaries)
        assert np.all((lo <= inside) & (inside <= hi))


def test_invalid_interval_is_rejected():
    with pytest.raises(ValueError):
        ordinal_interval_image(np.ones((1,2)), np.zeros((1,2)), np.zeros((6,2)))


def test_blend_recovers_opposite_best_models_and_shrinks_to_global():
    y = np.zeros((20,2)); et = np.tile([0,1.],(20,1)); gbr=np.tile([1.,0],(20,1))
    w,_ = shrunk_blend_weights(y,et,gbr,0)
    np.testing.assert_allclose(w,[0,1])
    w,global_w = shrunk_blend_weights(y,et,gbr,1e8)
    np.testing.assert_allclose(w,global_w,atol=1e-7)


def test_fixed_prediction_resampling_uses_paired_errors_and_declares_scope():
    y=np.zeros((12,2)); a=np.zeros_like(y); b=np.ones_like(y)
    r=paired_oof_bootstrap(y,a,b,np.repeat(['A','B'],6),100)
    assert r['difference_A_minus_B'] == -1
    assert r['conditional_percentile_95'] == [-1,-1]
    assert 'does_not_include_model_retraining' in r['scope']


def test_pareto_retains_tradeoffs_and_rejects_nonfinite_candidate():
    keep=pareto_minimize([[0,2],[1,1],[2,0],[2,2],[np.nan,0],[1,1]])
    np.testing.assert_array_equal(keep,[True,True,True,False,False,True])


def test_unknown_threshold_and_source_values_never_become_safe_zero():
    result=threshold_excess([.5,2,2,np.nan,-1],[1,1,np.nan,1,1])
    np.testing.assert_allclose(result[:2],[0,1])
    assert np.isnan(result[2:]).all()
```

041_test_stage3_research.py

```python
"""Check actual isolation and mathematical invariants of the executed study."""
from pathlib import Path
import json
import numpy as np
import pandas as pd
import pytest
from src.stage3_prediction import learn_blend_nested
from src.stage3_graph import fit_tuned_graph, within_style_indices
from src.stage3_generation import mmd_fixed_bandwidth, constrained_convex
from src.q3 import ConditionalPCAGaussian

ROOT=Path(__file__).resolve().parents[1]


def test_nested_blend_base_predictions_never_predict_their_own_training_rows(monkeypatch):
    import src.stage3_prediction as module
    calls=[]
    def fit(X,Y,cfg): return set(X[:,0].astype(int))
    def predict(seen,X):
        assert not seen.intersection(X[:,0].astype(int)); calls.append((seen,set(X[:,0].astype(int))))
        return np.zeros((len(X),2)),np.ones((len(X),2))
    monkeypatch.setattr(module,'fit_pair',fit); monkeypatch.setattr(module,'predict_pair',predict)
    X=np.c_[np.arange(12),np.arange(12)**2]; Y=np.c_[np.arange(12)/12,np.arange(12)/24]
    w,_,_,eta,_=learn_blend_nested(X,Y,{'seed':1,'blend_penalties':[0,.1]},np.array([0]),0)
    assert len(calls)==12 and np.all((w>=0)&(w<=1)) and eta in [0,.1]


def test_zero_penalty_graph_and_permuted_graph_reduce_to_same_ridge():
    rng=np.random.default_rng(5); X=rng.normal(size=(18,4)); Y=rng.normal(size=(18,2))
    cfg={'ridge_alphas':[10],'graph_lambdas':[0],'graph_thresholds':[.7],'graph_null_repeats':2}
    fits=[fit_tuned_graph(X,Y,cfg,k,np.array([0]),17) for k in ['none','real','null']]
    for f in fits:
        for b in f['beta']: np.testing.assert_allclose(b,fits[0]['beta'][0],atol=1e-12)


def test_style_permutation_preserves_each_style_and_singleton():
    labels=np.array(['A','A','B','B','B','C']); order=within_style_indices(labels,np.random.default_rng(9))
    np.testing.assert_array_equal(labels[order],labels); assert sorted(order)==list(range(6)) and order[-1]==5


def test_fixed_bandwidth_mmd_is_symmetric_and_zero_for_identical_samples():
    rng=np.random.default_rng(8); a=rng.normal(size=(12,4)); b=rng.normal(size=(7,4))
    assert abs(mmd_fixed_bandwidth(a,a,2))<1e-12
    assert np.isclose(mmd_fixed_bandwidth(a,b,2),mmd_fixed_bandwidth(b,a,2))


def test_constrained_convex_respects_fitted_bounds_and_budget():
    rng=np.random.default_rng(3); X=rng.normal(size=(12,4)); g=ConditionalPCAGaussian(3,2).fit(X,np.repeat('A',12))
    real=g.prep.named_steps['scale'].inverse_transform(g.prep.transform(X))
    a,info=constrained_convex(real,g,[0],20,rng)
    assert len(a)==20 and info['attempts']<=5000
    assert np.all(a>=g.low-1e-9) and np.all(a<=g.high+1e-9)


def test_saved_graph_comparison_contains_all_15_folds_and_ten_null_replicates():
    path=ROOT/'outputs/stage3/q2/graph_fold_metrics.csv'
    if not path.exists(): pytest.skip('Third-stage graph study not present')
    d=pd.read_csv(path); assert d.fold.nunique()==15
    shuffled=d[d.graph=='shuffled']; assert len(shuffled)==300
    assert (shuffled.groupby(['fold','scope']).size()==10).all()


def test_saved_search_evaluator_and_test_roles_are_disjoint():
    path=ROOT/'outputs/stage3/q3/disjoint_roles.csv'
    if not path.exists(): pytest.skip('Third-stage generation study not present')
    d=pd.read_csv(path)
    for _,a in d.groupby('repeat'):
        assert len(a)==a.beer.nunique()==250
        assert sorted(a.groupby('role').size())==[50,100,100]


def test_saved_failed_transfers_are_not_counted_as_improvements():
    path=ROOT/'outputs/stage3/q3/transfer_independent_evaluation.csv'
    if not path.exists(): pytest.skip('Third-stage transfer study not present')
    d=pd.read_csv(path); failed=d.status=='no_feasible_nontrivial_candidate_source_retained'
    assert not d.loc[failed,'evaluator_proxy_acceptance'].any()
    assert (d.loc[failed,'standardized_change']==0).all()


def test_missing_sources_are_never_reported_as_unconditional_feasible():
    path=ROOT/'outputs/stage3/q3/transfer_independent_evaluation.csv'
    if not path.exists(): pytest.skip('Third-stage transfer study not present')
    d=pd.read_csv(path)
    assert not ((~d.all_provided_source_defects_observed)&(d.status=='model_feasible_nontrivial')).any()
    assert not d.absolute_chemical_certification.any() and not d.real_tasting_validated.any()


def test_saved_ordinal_calibration_roles_are_disjoint():
    paths=list((ROOT/'outputs/stage3/q1').glob('conformal_roles_*.csv'))
    if len(paths)!=5: pytest.skip('Third-stage conformal experiments not complete')
    for path in paths:
        d=pd.read_csv(path); assert len(d)==d.beer.nunique()==250
        assert d.groupby('role').size().to_dict()=={'calibration':50,'test':50,'train':150}

```

```

def test_saved_prediction_study_is_complete_and_absolute_scores_remain_unidentified():
    path=ROOT/'outputs/stage3/q1/status.json'
    if not path.exists(): pytest.skip('Third-stage prediction study not complete')
    d=json.loads(path.read_text()); assert d['complete'] and d['outer_folds']==15
    assert not d['absolute_anchors_observed']
```

042_test_stage4.py

```
"""Check identity, repeated-rater weighting and physical constraint boundaries."""
import numpy as np
import pandas as pd
from src.stage4_audit import aggregate_panel, infer_identity, transform_candidates, concentration_fields, panel_rating_distribution
from src.stage4_models import calibrated_from_training, mix_concentrations, exact_nonincrease
from src.stage4_trial import analyze_paired_trial, planned_noninferiority_power
import pytest


def test_repeat_ratings_do_not_overweight_one_panelist():
    raw = pd.DataFrame({'beer': [1, 1, 1], 'taster': [1, 1, 2], 'rating': [0, 2, 6]})
    balanced, records, _ = aggregate_panel(raw, ['rating'])
    assert balanced.loc[1, 'rating'] == 3.5
    assert records.loc[1, 'rating'] == 8 / 3
    distribution = panel_rating_distribution(raw, ['rating'], [1])
    assert np.allclose(distribution[0, 0, [0, 2, 6]], [.25, .25, .5])
    assert np.allclose(distribution.sum(-1), 1)


def test_identity_uses_attributes_despite_shuffled_published_rows():
    z = pd.DataFrame({'a': [-2., 0., 3.], 'b': [3., -1., 0.]})
    raw_id = pd.Series([1, 2, 3])
    published = pd.DataFrame({'a': [3., -2., 0.], 'b': [0., 3., -1.]}, index=[40, 90, 7])
    _, mapping = infer_identity(z, raw_id, published, ['a', 'b'])
    assert mapping.set_index('raw_beer').beer.to_dict() == {1: 90, 2: 7, 3: 40}


def test_negative_blank_signal_is_not_logged_as_negative_concentration():
    values, half = transform_candidates(np.array([-2., 0., 2.5, 10., np.nan]))
    assert half == 1.25
    assert np.allclose(values['log10_nonpositive_half_min'][:2], np.log10(1.25))
    assert np.isnan(values['log10_nonpositive_half_min'][-1])


def test_unit_and_calibration_registry_excludes_relative_area_and_pressure():
    calibrated = pd.DataFrame(columns=['beer_id', 'tasting_category_fine', 'a', 'b', 'c'])
    registry = pd.DataFrame({'field_name_R': ['a', 'b', 'c'], 'calibrated': ['yes', np.nan, 'yes'], 'unit': ['mg/L', 'A.U.', 'PSI']})
    assert concentration_fields(calibrated, registry, ['mg/L']) == ['a']


def test_isotonic_calibration_respects_only_supplied_training_anchor_range():
    result = calibrated_from_training(np.array([[0.], [1.], [2.]]), np.array([[1.], [2.], [3.]]), np.array([[-10.], [10.]]))
    assert np.array_equal(result[:, 0], [1., 3.])


def test_mixture_is_linear_in_concentration_and_missing_source_cannot_certify():
    source, anchor = np.array([1., 10.]), np.array([3., 2.])
    values = mix_concentrations(source, anchor, [.25, .5])
    assert np.allclose(values, [[1.5, 8.], [2., 6.]])
    assert not exact_nonincrease(values, source, [0, 1]).any()
    assert exact_nonincrease(values, source, [1]).all()
    assert not exact_nonincrease(values, np.array([1., np.nan]), [1]).any()


def test_trial_repeated_batches_do_not_inflate_independent_participant_count():
    rows = []
    for person in range(4):
        for batch in range(3):
            for condition in ['control', 'candidate']:
                values = dict.fromkeys(['dms', 'diacetyl', 'X4vg', 't2n', 'stale_hops'], 1.)
                rows.append({'panelist': person, 'batch': batch, 'condition': condition,
                             'style_Tripel_fit': 1. if condition == 'control' else 3., **values})
    result = analyze_paired_trial(pd.DataFrame(rows))
    assert (result.independent_panelists == 4).all() and result.criterion_passed.all()


def test_no_real_trial_claim_from_an_empty_or_unpaired_template():
    with pytest.raises(ValueError):
        analyze_paired_trial(pd.DataFrame())
    assert planned_noninferiority_power(30, .8) > planned_noninferiority_power(10, .8)
```

043_run.py

```
"""AI-assisted orchestration: OpenAI Codex / GPT-6, 2026-10-24."""
import argparse
import json
import os
from pathlib import Path

os.environ.setdefault("MPLBACKEND", "Agg")
os.environ.setdefault("OMP_NUM_THREADS", "2")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "2")

```

```
from src.common import environment, progress, write_json
from src.data import find_data, load_data


def main():
    parser = argparse.ArgumentParser(description="Reproducible A-track research baselines")
    parser.add_argument("stage", choices=["audit", "q1", "q2", "q3", "report", "all"])
    parser.add_argument("--data-dir")
    parser.add_argument("--out", default="outputs/stage1")
    parser.add_argument("--config", default="config.json")
    args = parser.parse_args()
    cfg = json.loads(Path(args.config).read_text(encoding="utf-8"))
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    data = load_data(find_data(args.data_dir))
    if args.stage in ["all", "audit"]:
        from src.audit import run_audit
        run_audit(data, out, cfg)
    if args.stage in ["all", "q1"]:
        from src.q1 import run_q1
        run_q1(data, out, cfg)
    if args.stage in ["all", "q2"]:
        from src.q2 import run_q2
        run_q2(data, out, cfg)
    if args.stage in ["all", "q3"]:
        from src.q3 import run_q3
        run_q3(data, out, cfg)
    if args.stage in ["all", "report"]:
        from src.reporting import make_report
        make_report(data, out, cfg)
    manifest = environment()
    manifest["stage"] = args.stage
    manifest["config"] = cfg
    manifest["files"] = sorted(str(p.relative_to(out)).replace("\\", "/") for p in out.rglob("*") if p.is_file() and "private" not in p.parts and
p.suffix in {".csv", ".json", ".md", ".png"})
    write_json(out / "run_manifest.json", manifest)
    progress(f"Completed {args.stage}; outputs: {out.resolve()}")


if __name__ == "__main__":
    main()
```

044_run_stage2.py

```
"""AI-assisted research extension, OpenAI Codex / GPT-6, 2026-10-24."""
import argparse
import json
import os
from pathlib import Path

os.environ.setdefault("MPLBACKEND", "Agg")
os.environ.setdefault("OMP_NUM_THREADS", "2")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "2")

from src.common import environment, write_json
from src.data import find_data, load_data
from src.stage2 import run_extension

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-dir")
    parser.add_argument("--out", default="outputs/stage2")
    parser.add_argument("--base", default="outputs/stage1")
    args = parser.parse_args()
    cfg = json.loads(Path("config.json").read_text())
    ext = json.loads(Path("config_stage2.json").read_text())
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    run_extension(load_data(find_data(args.data_dir)), Path(args.base), out, cfg, ext)
    write_json(out / "run_manifest.json", {**environment(), "base_config": cfg, "extension_config": ext})
```

045_run_stage3.py

```
"""Run saved, explicitly scoped third-stage experiments; AI-assisted, 2026-10-25."""
import os
os.environ.setdefault("OMP_NUM_THREADS", "2")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "2")
os.environ.setdefault("MKL_NUM_THREADS", "2")
os.environ.setdefault("MPLBACKEND", "Agg")
import argparse
import json
from pathlib import Path
from src.common import environment, write_json, sha256
from src.data import load_data, find_data

if __name__ == '__main__':
    parser=argparse.ArgumentParser()
    parser.add_argument('part',choices=['q1','q2','q3'])
    parser.add_argument('--data-dir')
    parser.add_argument('--out',default='outputs/stage3')
    args=parser.parse_args()
    cfg=json.loads(Path('config_stage3.json').read_text(encoding='utf-8'))
```

```
    data=load_data(find_data(args.data_dir)); out=Path(args.out); out.mkdir(parents=True,exist_ok=True)
    expected={r['file']:r['sha256'] for r in __import__('pandas').read_csv('outputs/stage2/audit/source_manifest.csv').to_dict('records')}
    for file in ['1.csv','2.csv','3.xlsx','4.csv','8.xlsx']:
        if sha256(data.root/file)!=expected[file]: raise ValueError('Original source version changed: '+file)
    if args.part=='q1':
        from src.stage3_prediction import run_prediction
        run_prediction(data,out,cfg)
    elif args.part=='q2':
        from src.stage3_graph import run_graph
        run_graph(data,out,cfg)
    else:
        from src.stage3_generation import run_generation
        run_generation(data,out,cfg)
    write_json(out/f'{args.part}_run_manifest.json',{**environment(),'config':cfg,'config_sha256':sha256('config_stage3.json'),'sources_verified':
expected})
```

046_run_stage4.py

```
"""Reconcile newly available raw sensory data, then evaluate absolute scores."""
from pathlib import Path
import argparse
import json
import os
for name in ['OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS']:
    os.environ.setdefault(name, '2')
from src.common import environment, write_json, sha256
from src.data import load_data, find_data
from src.stage4_audit import reconcile
from src.stage4_models import run_absolute, run_transfer_diagnostics, run_ordinal

ROOT = Path(__file__).resolve().parent
parser = argparse.ArgumentParser()
parser.add_argument('part', choices=['audit', 'prediction', 'ordinal', 'transfer', 'all'])
parser.add_argument('--external-dir', default=str(ROOT.parent / 'external_sources/zenodo_v4_23624915'))
parser.add_argument('--data-dir')
parser.add_argument('--config', default=str(ROOT / 'config_stage4.json'))
parser.add_argument('--out', default=str(ROOT / 'outputs/stage4'))
parser.add_argument('--stage3', default=str(ROOT / 'outputs/stage3'))
parser.add_argument('--allow-overwrite', action='store_true', help='Permit writing into an existing non-empty output directory')
args = parser.parse_args()
config_path = Path(args.config).resolve()
cfg = json.loads(config_path.read_text(encoding='utf-8'))
if cfg['outer_folds'] < 2 or cfg['outer_repeats'] < 1 or not 0 < cfg['conformal_alpha'] < 1:
    parser.error('outer_folds >= 2, outer_repeats >= 1, and 0 < conformal_alpha < 1 are required')
out = Path(args.out).resolve()
if out.exists() and any(out.iterdir()) and not args.allow_overwrite:
    parser.error('Output directory is not empty. Select a new --out or explicitly use --allow-overwrite.')
data = load_data(find_data(args.data_dir))
raw, panel, records, concentration, fields = reconcile(data, args.external_dir, out / 'audit', cfg)
if args.part in ['prediction', 'all']:
    run_absolute(data, panel, records, concentration, out, cfg)
if args.part in ['ordinal', 'all']:
    run_ordinal(data, raw, panel, out, cfg, Path(args.stage3))
if args.part in ['transfer', 'all']:
    run_transfer_diagnostics(data, panel, concentration, out, cfg, Path(args.stage3))
write_json(out / f'{args.part}_run_manifest.json', {**environment(), 'config': cfg, 'source_identity_audit': 'audit/audit_summary.json',
                                                    'concentration_features': fields, 'part': args.part,
                                                    'config_source': str(config_path), 'config_sha256': sha256(config_path),
                                                    'code_sha256': {p.relative_to(ROOT).as_posix(): sha256(p) for p in [ROOT / 'run_stage4.py'] +
sorted((ROOT / 'src').glob('*.py'))}})
print(json.dumps({'part': args.part, 'completed': True, 'concentration_fields': len(fields)}))
```

047_config.json

```
{
  "seed": 20261024,
  "outer_folds": 3,
  "outer_repeats": 2,
  "inner_folds": 3,
  "ridge_alphas": [1.0, 10.0, 100.0, 1000.0],
  "pls_components": [3, 8],
  "extra_trees": 200,
  "bootstrap_repeats": 100,
  "graph_threshold": 0.7,
  "graph_ridge_alpha": 20.0,
  "graph_ridge_lambda": 5.0,
  "generation_holdout_fraction": 0.2,
  "generation_components": 10,
  "generation_samples_per_style": 40,
  "generation_styles": ["Tripel", "Kriek", "Lambic"],
  "transfer_source": "Pils/Lager",
  "transfer_target": "Tripel",
  "missing_rates": [0.0, 0.05, 0.1, 0.2],
  "export_assumption_grades": true
}
```

048_config_stage2.json

```
{
  "seed": 20261024,
```

```
  "gbr_estimators": 100,
  "gbr_learning_rate": 0.04,
  "gbr_depth": 2,
  "permutation_repeats": 5,
  "conformal_alpha": 0.1,
  "generation_repeats": 5,
  "generation_n": 40,
  "generation_styles": ["Tripel", "Kriek", "Lambic"],
  "transfer_weights": 30
}
```


049_config_stage3.json


```
{
  "date": "2026-10-25",
  "seed": 20261025,
  "outer_folds": 3,
  "outer_repeats": 5,
  "inner_folds": 3,
  "primary_targets": "explicit_A_and_F_prefixed_30_not_official_scope_ruling",
  "blend_penalties": [0, 0.01, 0.1, 1, 10],
  "ridge_alphas": [1, 10, 100, 1000],
  "extra_trees": 200,
  "gbr_estimators": 100,
  "graph_thresholds": [0.6, 0.7, 0.8],
  "graph_lambdas": [0, 1, 5, 20],
  "graph_null_repeats": 10,
  "bootstrap_repeats": 200,
  "permutation_repeats": 20,
  "conformal_repeats": 5,
  "conformal_alpha": 0.1,
  "generation_repeats": 5,
  "generation_n": 40,
  "generation_styles": ["Tripel", "Kriek", "Lambic"],
  "generator_n": 100,
  "evaluator_n": 100,
  "real_holdout_n": 50,
  "transfer_weights": 30,
  "trust_radii": [0.25, 0.5, 1, 2],
  "transfer_target_probability_floor": 0.5,
  "transfer_min_probability_gain": 0.01,
  "claim_scope": "exploratory_internal_validation_after_stages_one_and_two_no_new_external_or_tasting_data",
  "execution_differences_from_design": [
    "Clipping and missingness ablations use fixed ExtraTrees, not all nested fusion variants; reported separately.",
    "Chemical literature evidence is graded and incomplete; no fabricated absolute-score anchors or physical validation."
  ]
}
```


050_config_stage3_design.json


```
{
  "stage": "stage3_design",
  "date": "2026-10-25",
  "seed": 20261025,
  "status": "protocol_and_preflight_only_not_completed_stage3_experiments",
  "validation": {
    "outer_folds": 3,
    "outer_repeats": 5,
    "inner_folds": 3,
    "stratification": "beer_style",
    "unit": "unique_beer",
    "primary_metric": "macro_MAE_on_explicitly_mapped_Aroma_Flavour_30_targets",
    "secondary_scope": "all_50_targets_and_card_sections_reported_separately",
    "paired_bootstrap_repeats": 2000,
    "bootstrap_scope": "conditional_on_saved_OOF_predictions_not_full_algorithm_uncertainty",
    "practical_relative_MAE_margin": 0.01,
    "new_external_data": "not_available_existing_data_are_exploratory"
  },
  "grading": {
    "primary_without_anchors": "training_only_ECDF_relative_rank_bins_0_to_6",
    "conformal_alpha": 0.1,
    "absolute_grade_accuracy": "not_estimable_without_observed_absolute_anchors",
    "warning_if_grade_set_size_at_least": 4,
    "anchor_analysis": "conditional_on_new_observed_same_protocol_pairs_only"
  },
  "models": {
    "baselines": ["mean", "ridge", "extra_trees", "gbr_fixed"],
    "proposed": "inner_OOF_per_target_GBR_ET_blend_shrunk_to_global_weight",
    "blend_shrinkage_grid": [0, 0.01, 0.1, 1, 10],
    "clip_quantiles": [0, 0.01, 0.025],
    "missing_rates": [0, 0.05, 0.1, 0.2],
    "mask_scenarios": ["MCAR", "preferentially_mask_low_training_quantile_values"]
  },
  "graph": {
    "thresholds": [0.6, 0.7, 0.8],
    "ridge_alphas": [1, 10, 100, 1000],
    "graph_lambdas": [0, 1, 5, 20],
    "null_graph": "permute_node_labels_preserving_unlabelled_weighted_signed_graph",
    "bootstrap_repeats": 200,
    "permutation_repeats": 20,
    "inference": "descriptive_training_selection_with_heldout_effects_no_causal_or_p_value_claim"
  },
```

```
  "generation": {
    "repeat_seeds": [20261025, 20261026, 20261027, 20261028, 20261029],
    "generator_search_training_n": 100,
    "external_evaluator_training_n": 100,
    "real_holdout_n": 50,
    "samples_per_style": 40,
    "styles": ["Tripel", "Kriek", "Lambic"],
    "primary_fields": "all_nonaggregate_fields_226",
    "unknown_aggregate_policy": "exclude_from_primary_models_until_formula_verified",
    "methods": ["bootstrap", "local_convex", "conditional_PCA_Gaussian", "constrained_local_convex"],
    "MMD_bandwidth": "fit_on_generator_training_data_once_per_split_shared_by_all_methods"
  },
  "transfer": {
    "source": "Pils/Lager",
    "target": "Tripel",
    "chemical_fields": ["DMS", "X2.3.butanedione", "X4VG", "X.E..2.nonenal"],
    "trust_region_radii": [0.25, 0.5, 1, 2],
    "objective": "Pareto_minimize_standardized_change_target_distance_and_defect_excess",
    "threshold_sensitivity_multipliers": [0.5, 1, 2],
    "missing_defect_source": "unknown_not_certified_against_median",
    "stale_hops": "multiple_model_proxy_consensus_not_real_tasting_certification",
    "failure": "preserve_source_and_record_no_feasible_nontrivial_candidate"
  }
}
```

051_config_stage4.json

```
{
  "date": "2026-10-25",
  "seed": 20261025,
  "outer_folds": 3,
  "outer_repeats": 5,
  "extra_trees": 200,
  "gbr_estimators": 100,
  "ridge_alphas": [1, 10, 100, 1000],
  "conformal_repeats": 5,
  "conformal_alpha": 0.1,
  "panel_bootstrap_repeats": 1000,
  "mapping_tolerance": 0.0000001,
  "external_record": "https://zenodo.org/records/23624915",
  "raw_md5": "a8f911d8c0061e096abf3b087678e164",
  "workbook_md5": "52645e38d90aafda272532e19e2fea1c",
  "primary_absolute_target": "equal_panelist_mean_after_within_beer_panelist_repeat_averaging",
  "absolute_models": ["mean", "ridge", "extra_trees", "gbr", "fixed_half_blend", "concentration_ET", "standardized_ET_train_isotonic"],
  "ordinal_target": "equal_panelist_equal_repeat_observed_rating_distribution_7_categories",
  "ordinal_decision_rules": ["mode_for_exact_match", "median_for_absolute_grade_loss", "rounded_mean_sensitivity"],
  "concentration_mixture_LP_extension": "exploratory_after_failed_single_anchor_pilot_not_pre_registered",
  "concentration_units": ["mg/L", "g/L", "ug/L", "v/v"],
  "literature_nonenal_threshold_mg_L": 0.0001,
  "threshold_sensitivity_multipliers": [0.5, 1, 2],
  "mixture_weights": [0.025, 0.05, 0.075, 0.1, 0.125, 0.15, 0.175, 0.2, 0.225, 0.25, 0.275, 0.3, 0.325, 0.35, 0.375, 0.4, 0.425, 0.45, 0.475, 0.5,
 0.525, 0.55, 0.575, 0.6, 0.625, 0.65, 0.675, 0.7, 0.725, 0.75],
  "claim_scope": "Internal prediction of observed panel means; reconstructed identity mapping is audited separately; no new physical beer or
tasting experiment"
}
```

052_review_experiments.py

```
"""Exploratory follow-up: fixed folds, ordinal objective, honest baselines, LP proof."""
from pathlib import Path
import argparse
import json
import os
import sys
for name in ['OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS']:
    os.environ.setdefault(name, '2')
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
import numpy as np
import pandas as pd
from sklearn.model_selection import RepeatedStratifiedKFold
from src.data import find_data, load_data, CHEMICAL_DEFECTS
from src.common import save_csv, write_json, sha256, environment
from src.stage3_prediction import tree_model
from src.stage4_models import ordinal_summary
from src.review_diagnostics import distribution_scores, cdf_to_probabilities, conditional_paired_interval, mixture_certificate


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--data-dir')
    args = parser.parse_args()
    out = ROOT / 'outputs/review_20261028'; out.mkdir(parents=True, exist_ok=True)
    old = ROOT / 'outputs/stage4'
    cfg = json.loads((ROOT / 'config_stage4.json').read_text(encoding='utf-8'))
    data = load_data(find_data(args.data_dir))
    beers = data.meta.beer.to_numpy(); styles = data.meta.tasting_category_fine.to_numpy()
    targets = list(data.Y); X = data.X.to_numpy()
    Y = pd.read_csv(old / 'audit/observed_panel_means_0_6.csv').set_index('beer').loc[beers, targets].to_numpy()
    index = pd.MultiIndex.from_product([beers, targets], names=['beer', 'target'])
    table = pd.read_csv(old / 'ordinal/oof_rating_distributions.csv').set_index(['beer', 'target']).loc[index]
    q = table[[f'observed_panel_p{k}' for k in range(7)]].to_numpy().reshape(250, 50, 7)
```

```
    q /= q.sum(-1, keepdims=True)
    probability = {'frequency_ET': table[[f'predicted_p{k}' for k in range(7)]].to_numpy().reshape(250, 50, 7)}
    probability['frequency_ET'] /= probability['frequency_ET'].sum(-1, keepdims=True)
    sums = {m: np.zeros_like(q) for m in ['CDF_ET', 'population', 'style_only']}
    points = {m: np.zeros_like(Y) for m in ['training_median', 'style_median']}
    counts = np.zeros(len(beers)); fold_rows = []; role_rows = []
    scopes = {'core30': np.array([j for j, t in enumerate(targets) if t.startswith(('A_', 'F_'))]), 'all50': np.arange(50)}
    previous_roles = pd.read_csv(old / 'prediction/outer_roles.csv')
    for fold, (train, test) in enumerate(RepeatedStratifiedKFold(n_splits=3, n_repeats=5, random_state=cfg['seed']).split(X, styles)):
        assert set(beers[test]) == set(previous_roles.loc[previous_roles.fold == fold, 'beer'])
        print(f'CDF and baseline comparison {fold + 1}/15', flush=True)
        model = tree_model(cfg).fit(X[train], q[train].cumsum(-1)[..., :-1].reshape(len(train), -1))
        cdf = model.predict(X[test]).reshape(len(test), 50, 6)
        predicted = {'CDF_ET': cdf_to_probabilities(cdf), 'population': np.broadcast_to(q[train].mean(0), (len(test), 50, 7)).copy(), 'style_only': np.empty((len(test), 50, 7))}
        median = np.broadcast_to(np.median(Y[train], axis=0), (len(test), 50)).copy()
        style_median = np.empty_like(median)
        for k, i in enumerate(test):
            group = train[styles[train] == styles[i]]
            if not len(group): group = train
            predicted['style_only'][k] = q[group].mean(0)
            style_median[k] = np.median(Y[group], axis=0)
        points['training_median'][test] += median; points['style_median'][test] += style_median
        for name, p in predicted.items():
            sums[name][test] += p
            for scope, selected in scopes.items():
                metrics = distribution_scores(q[test][:, selected], p[:, selected])
                fold_rows.append({'fold': fold, 'model': name, 'scope': scope, 'n_test': len(test), **{k: float(v.mean()) for k, v in metrics.items()}})
        counts[test] += 1
        role_rows += [{'fold': fold, 'beer': int(beers[i]), 'role': 'test'} for i in test]
    assert np.all(counts == 5)
    for name, p in sums.items(): probability[name] = p / counts[:, None, None]
    summary = []; by_target = []; paired = []; calibration = []
    for name, p in probability.items():
        metric = distribution_scores(q, p)
        grades = p.argmax(-1)
        for scope, selected in scopes.items():
            row = ordinal_summary(q[:, selected], p[:, selected], grades[:, selected])
            # Recompute from the common scoring function for each requested scope.
            for name in ('cdf_distance', 'panel_disagreement', 'expected_individual_RPS', 'brier_distance', 'expected_individual_Brier'):
                row.pop(name, None)
            summary.append({'model': name, 'scope': scope, **row, **{k: float(v[:, selected].mean()) for k, v in metric.items()}})
        for j, target in enumerate(targets):
            by_target.append({'model': name, 'target': target, **ordinal_summary(q[:, j:j+1], None, grades[:, j:j+1]), **{k: float(v[:, j].mean()) for k, v in metric.items()}})
        for scope, selected in scopes.items():
            x = p[:, selected, 0].ravel(); y = q[:, selected, 0].ravel()
            for b in range(10):
                mask = np.minimum((x * 10).astype(int), 9) == b
                if mask.any(): calibration.append({'model': name, 'scope': scope, 'zero_probability_bin': b, 'beer_attribute_pairs': int(mask.sum()), 'predicted_p0': float(x[mask].mean()), 'observed_p0': float(y[mask].mean())})
    for a, b in [('CDF_ET', 'frequency_ET'), ('frequency_ET', 'population'), ('CDF_ET', 'style_only')]:
        for scope, selected in scopes.items():
            d = (distribution_scores(q, probability[a])['cdf_distance'][:, selected] - distribution_scores(q, probability[b])['cdf_distance'][:, selected]).mean(1)
            paired.append({'endpoint': 'cdf_distance', 'scope': scope, 'A': a, 'B': b, **conditional_paired_interval(d, styles)})
    point_preds = {}
    for name in ['mean', 'ridge', 'extra_trees', 'gbr', 'fixed_half_blend', 'concentration_ET']:
        point_preds[name] = pd.read_csv(old / f'prediction/oof_{name}_0_6.csv').set_index('beer').loc[beers, targets].to_numpy()
    for name, v in points.items(): point_preds[name] = v / counts[:, None]
    point_rows = []; style_rows = []; graph_rows = []
    for name, p in point_preds.items():
        for scope, selected in scopes.items():
            error = np.abs(Y[:, selected] - p[:, selected]).mean(1)
            point_rows.append({'model': name, 'scope': scope, 'mae': float(error.mean())})
            for label in np.unique(styles):
                mask = styles == label
                style_rows.append({'model': name, 'scope': scope, 'style': label, 'unique_beers': int(mask.sum()), 'mae': float(error[mask].mean())})
    for name in ['mean', 'training_median', 'extra_trees', 'gbr', 'concentration_ET', 'style_median']:
        for scope, selected in scopes.items():
            d = (np.abs(Y - point_preds['fixed_half_blend'])[:, selected] - np.abs(Y - point_preds[name])[:, selected]).mean(1)
            paired.append({'endpoint': 'panel_mean_MAE', 'scope': scope, 'A': 'fixed_half_blend', 'B': name, **conditional_paired_interval(d, styles)})
    graph_preds = {kind: pd.read_csv(old / f'raw_graph/q2/oof_graph_{kind}.csv').set_index('beer').loc[beers, targets].to_numpy() for kind in ['real', 'none']}
    for scope, selected in scopes.items():
        for kind, p in graph_preds.items(): graph_rows.append({'graph': kind, 'scope': scope, 'aggregated_oof_mae': float(np.abs(Y[:, selected] - p[:, selected]).mean())})
        d = (np.abs(Y - graph_preds['real'])[:, selected] - np.abs(Y - graph_preds['none'])[:, selected]).mean(1)
        paired.append({'endpoint': 'graph_panel_mean_MAE', 'scope': scope, 'A': 'real', 'B': 'none', **conditional_paired_interval(d, styles)})
    roles = pd.read_csv(ROOT / 'outputs/stage3/q3/disjoint_roles.csv')
    concentration = data.calibrated.set_index('beer').loc[beers, CHEMICAL_DEFECTS].to_numpy(float)
    certificates = []; ablations = []
    from scipy.optimize import linprog
    for repeat in range(5):
        role = roles[roles.repeat == repeat].set_index('beer').loc[beers, 'role'].to_numpy()
        anchors = np.flatnonzero((role == 'G_search_training') & (styles == 'Tripel') & np.isfinite(concentration).all(1))
        for i in np.flatnonzero(styles == 'Pils/Lager'):
            d = (concentration[anchors] - concentration[i]).T
            cert = mixture_certificate(d)
            certificates.append({'repeat': repeat, 'beer': int(beers[i]), 'n_anchors': len(anchors), 'certified_zero_only': cert['certified_zero_only'], 'normalized_margin': cert['margin'], 'single_row_blockers': ';'.join(CHEMICAL_DEFECTS[k] for k in cert['single_row_blockers']), **{f'weight_{f}': cert['weights'][k] for k, f in enumerate(CHEMICAL_DEFECTS)}})
            for dropped in [None] + CHEMICAL_DEFECTS:
                keep = [k for k, f in enumerate(CHEMICAL_DEFECTS) if f != dropped]
```

```
                a = d[keep]; scales = np.maximum(np.abs(a).max(1), 1e-12)
                solution = linprog(-np.ones(len(anchors)), A_ub=np.vstack([a / scales[:, None], np.ones(len(anchors))]), b_ub=np.r_[np.zeros(len(keep)), .75], bounds=(0, None), method='highs')
                if not solution.success: raise RuntimeError(solution.message)
                ablations.append({'repeat': repeat, 'beer': int(beers[i]), 'dropped_constraint': dropped or 'none', 'max_target_fraction': float(solution.x.sum())})
    for name, frame in [('ordinal_metrics', summary), ('ordinal_by_target', by_target), ('ordinal_fold_metrics', fold_rows), ('zero_calibration', calibration), ('paired_comparisons', paired), ('point_metrics', point_rows), ('point_by_style', style_rows), ('graph_aggregation', graph_rows), ('mixture_certificates', certificates), ('mixture_constraint_ablation', ablations), ('outer_roles', role_rows)]:
        save_csv(pd.DataFrame(frame), out / f'{name}.csv')
    rows = table.reset_index()[['beer', 'target']].copy()
    for k in range(7): rows[f'predicted_p{k}'] = probability['CDF_ET'][..., k].ravel()
    rows['modal_grade'] = probability['CDF_ET'].argmax(-1).ravel()
    rows['median_grade'] = (probability['CDF_ET'].cumsum(-1) >= .5).argmax(-1).ravel()
    save_csv(rows, out / 'cdf_et_oof.csv')
    for name in ['training_median', 'style_median']:
        save_csv(pd.concat([data.meta[['beer']], pd.DataFrame(point_preds[name], columns=targets)], axis=1), out / f'oof_{name}.csv')
    inputs = [old / 'ordinal/oof_rating_distributions.csv', old / 'audit/observed_panel_means_0_6.csv', old / 'prediction/outer_roles.csv', ROOT / 'outputs/stage3/q3/disjoint_roles.csv', ROOT / 'config_stage4.json']
    inputs += list((old / 'prediction').glob('oof_*_0_6.csv')) + list((old / 'raw_graph/q2').glob('oof_graph_*.csv'))
    code = [Path(__file__), ROOT / 'src/review_diagnostics.py', ROOT / 'src/stage3_prediction.py', ROOT / 'src/common.py', ROOT / 'src/stage4_models.py']
    write_json(out / 'run_manifest.json', {**environment(), 'design': 'exploratory_after_stage4_review_fixed_hyperparameters_no_test_tuning', 'config': cfg, 'resamples': 5000, 'unique_beers': 250, 'new_external_samples': 0, 'conditional_intervals': 'fixed_OOF_predictions_stratified_by_style_no_refit_or_selection_uncertainty', 'input_sha256': {p.relative_to(ROOT).as_posix(): sha256(p) for p in inputs}, 'code_sha256': {p.relative_to(ROOT).as_posix(): sha256(p) for p in code}, 'official_inputs_sha256': {p.name: sha256(p) for p in data.root.iterdir() if p.is_file() and p.suffix in ['.csv', '.xlsx']}, 'result_sha256': {p.name: sha256(p) for p in sorted(out.glob('*.csv'))}})
    print(pd.DataFrame(summary)[['model', 'scope', 'cdf_distance', 'expected_individual_RPS', 'nonzero_panel_mode_accuracy']].to_string(index=False))


if __name__ == '__main__':
    main()
```

053_review_diagnostics.py

```
"""Scoring identities and certificates used in the 2026-10-28 revision."""
import numpy as np
from scipy.optimize import linprog


def distribution_scores(observed, predicted):
    """Return per-beer/per-attribute distances and expected proper scores."""
    q, p = np.asarray(observed), np.asarray(predicted)
    if q.shape != p.shape or q.shape[-1] != 7:
        raise ValueError('Expected matching seven-category distributions')
    for a in (q, p):
        if not np.isfinite(a).all() or a.min() < -1e-8 or not np.allclose(a.sum(-1), 1, atol=1e-7):
            raise ValueError('Invalid probability distribution')
    Q, P = q.cumsum(-1)[..., :-1], p.cumsum(-1)[..., :-1]
    distance = ((P - Q) ** 2).mean(-1)
    uncertainty = (Q * (1 - Q)).mean(-1)
    brier_distance = ((p - q) ** 2).sum(-1)
    return {'cdf_distance': distance, 'panel_disagreement': uncertainty,
            'expected_individual_RPS': distance + uncertainty,
            'brier_distance': brier_distance,
            'expected_individual_Brier': brier_distance + 1 - (q ** 2).sum(-1)}


def cdf_to_probabilities(cdf):
    a = np.asarray(cdf)
    if a.shape[-1] != 6 or not np.isfinite(a).all() or (np.diff(a, axis=-1) < -1e-10).any():
        raise ValueError('Six finite, monotone cumulative probabilities required')
    if a.min() < -1e-10 or a.max() > 1 + 1e-10:
        raise ValueError('CDF outside [0,1]')
    p = np.diff(np.concatenate([np.zeros_like(a[..., :1]), a, np.ones_like(a[..., :1])], axis=-1), axis=-1)
    p = np.maximum(p, 0)
    return p / p.sum(-1, keepdims=True)


def conditional_paired_interval(difference, strata, seed=20261028, resamples=5000):
    """Stratified beer bootstrap conditional on fixed OOF predictions."""
    d, strata = np.asarray(difference), np.asarray(strata)
    if d.ndim != 1 or len(d) != len(strata) or not np.isfinite(d).all():
        raise ValueError('One finite paired difference per beer required')
    rng = np.random.default_rng(seed)
    values = np.zeros(resamples)
    for label in np.unique(strata):
        v = d[strata == label]
        values += v[rng.integers(len(v), size=(resamples, len(v)))].sum(1)
    lo, hi = np.quantile(values / len(d), [.025, .975])
    return {'difference_A_minus_B': float(d.mean()), 'conditional_low95': float(lo),
            'conditional_high95': float(hi), 'unique_beers': len(d), 'resamples': resamples}


def mixture_certificate(difference):
    """D w <= 0, w>=0: exhibit v>=0 with D.T v >0 to exclude w!=0.

    Row normalization preserves inequalities and avoids mixing concentration units.
    Maximize the minimum normalized anchor excess over v on the simplex.
    """
    d = np.asarray(difference, dtype=float)
    if d.ndim != 2 or not d.size or not np.isfinite(d).all():
        raise ValueError('Finite defect-by-anchor differences required')
    scales = np.max(np.abs(d), axis=1)
```

```
    a = d / np.where(scales > 0, scales, 1)[:, None]
    m, n = a.shape
    result = linprog(np.r_[np.zeros(m), -1.],
                     A_ub=np.column_stack([-a.T, np.ones(n)]), b_ub=np.zeros(n),
                     A_eq=[np.r_[np.ones(m), 0]], b_eq=[1],
                     bounds=[(0, None)] * m + [(None, None)], method='highs')
    if not result.success:
        raise RuntimeError(result.message)
    v = result.x[:-1]
    margin = float((a.T @ v).min())
    return {'certified_zero_only': margin > 1e-8, 'margin': margin,
            'weights': v.tolist(), 'row_scales': scales.tolist(),
            'single_row_blockers': np.flatnonzero(d.min(1) > 1e-10).tolist()}
```

055_run_final_model.py

```
"""Train final 0-6 models or predict from a label-free chemical CSV."""
import argparse
import json
from pathlib import Path
import joblib
from src.common import save_csv, write_json
from src.deployment import fit_final, load_trusted_model, predict

ROOT = Path(__file__).resolve().parent
parser = argparse.ArgumentParser()
sub = parser.add_subparsers(dest='command', required=True)
train = sub.add_parser('train')
train.add_argument('--chemical', required=True, help='Official attachment 1.csv')
train.add_argument('--means', default=str(ROOT / 'outputs/stage4/audit/observed_panel_means_0_6.csv'))
train.add_argument('--distributions', default=str(ROOT / 'outputs/stage4/ordinal/oof_rating_distributions.csv'))
train.add_argument('--config', default=str(ROOT / 'config_stage4.json'))
train.add_argument('--unit-map', default=str(ROOT / 'outputs/stage4/audit/concentration_feature_registry.csv'))
train.add_argument('--scale-map', default=str(ROOT / 'outputs/stage4/audit/chemical_scale_reconciliation.csv'))
train.add_argument('--model', required=True, help='New output .joblib path')
train.add_argument('--overwrite', action='store_true')
infer = sub.add_parser('predict')
infer.add_argument('--model', required=True, help='Trusted joblib model from this project')
infer.add_argument('--input', required=True, help='Chemical CSV, no labels needed')
infer.add_argument('--out', required=True, help='New output directory')
args = parser.parse_args()
if args.command == 'train':
    path = Path(args.model)
    if (path.exists() or path.with_suffix('.manifest.json').exists()) and not args.overwrite:
        parser.error('Model exists; choose another path or use --overwrite')
    cfg = json.loads(Path(args.config).read_text(encoding='utf-8'))
    bundle = fit_final(args.chemical, args.means, args.distributions, cfg, args.unit_map, args.scale_map)
    path.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(bundle, path, compress=3)
    write_json(path.with_suffix('.manifest.json'), bundle['manifest'])
    print(json.dumps({'model': str(path), 'training_beers': bundle['manifest']['training_beers'],
                      'features': len(bundle['manifest']['features']), 'targets': len(bundle['manifest']['targets'])}))
else:
    out = Path(args.out)
    if out.exists() and any(out.iterdir()):
        parser.error('Output directory is not empty; choose a new --out')
    tables = predict(load_trusted_model(args.model), args.input)
    for name, table in tables.items(): save_csv(table, out / f'{name}.csv')
    print(json.dumps({'output': str(out), 'samples': len(tables['panel_mean']), 'targets': len(tables['panel_mean'].columns)-1}))
```

056_deployment.py

```
"""Final full-data models and label-free inference for the historical panel.

Only load joblib files from a trusted source: deserialization executes code.
"""
from pathlib import Path
import csv
import json
import platform
from datetime import datetime, timezone
from importlib.metadata import version

import joblib
import numpy as np
import pandas as pd

from .common import sha256
from .data import META
from .stage3_prediction import fit_pair, predict_pair, tree_model


def read_input(path, features):
    """Require the complete training schema; reorder columns by recorded names."""
    with Path(path).open(encoding='utf-8-sig', newline='') as stream:
        header = next(csv.reader(stream), [])
    if len(header) != len(set(header)):
        raise ValueError('Input CSV contains duplicate column names')
    data = pd.read_csv(path)
    missing = [name for name in features if name not in data.columns]
    if missing:
        raise ValueError(f'Missing {len(missing)} required feature(s): {missing[:12]}')
```

```
    if data.empty:
        raise ValueError('Input CSV contains no samples')
    numeric = data.loc[:, features].apply(pd.to_numeric, errors='raise').to_numpy(dtype=float)
    if np.isinf(numeric).any():
        raise ValueError('Infinite feature values are not supported')
    if 'beer' in data and (data.beer.isna().any() or data.beer.duplicated().any()):
        raise ValueError('beer identifiers must be non-null and unique')
    return data, numeric


def fit_final(chemical_path, means_path, distribution_path, cfg, unit_map=None, scale_map=None):
    """Fit the same half ET/GBR mean and frequency ET models on all available beers."""
    chemical = pd.read_csv(chemical_path)
    if 'beer' not in chemical or chemical.beer.isna().any() or chemical.beer.duplicated().any():
        raise ValueError('Training chemical data need unique beer identifiers')
    features = [c for c in chemical.columns if c not in META]
    _, X = read_input(chemical_path, features)
    means = pd.read_csv(means_path)
    if means.beer.isna().any() or means.beer.duplicated().any() or set(means.beer) != set(chemical.beer):
        raise ValueError('Training means must match all chemical beer identifiers exactly')
    targets = [c for c in means if c != 'beer']
    if not targets:
        raise ValueError('No sensory targets found')
    Y = means.set_index('beer').loc[chemical.beer, targets].to_numpy(dtype=float)
    if not np.isfinite(Y).all() or (Y < 0).any() or (Y > 6).any():
        raise ValueError('Panel means must be finite and in [0,6]')
    q_frame = pd.read_csv(distribution_path)
    q_cols = [f'observed_panel_p{k}' for k in range(7)]
    if q_frame.duplicated(['beer', 'target']).any():
        raise ValueError('Observed distribution has duplicate beer/target records')
    idx = pd.MultiIndex.from_product([chemical.beer, targets], names=['beer', 'target'])
    q_frame = q_frame.set_index(['beer', 'target'])
    if len(q_frame) != len(idx) or not idx.isin(q_frame.index).all():
        raise ValueError('Observed distributions must match all beers and targets')
    q = q_frame.loc[idx, q_cols].to_numpy(dtype=float).reshape(len(X), len(targets), 7)
    if not np.isfinite(q).all() or (q < -1e-8).any() or not np.allclose(q.sum(-1), 1, atol=1e-6):
        raise ValueError('Observed ratings must form valid probability distributions')
    q = np.maximum(q, 0); q /= q.sum(-1, keepdims=True)
    et, gbr = fit_pair(X, Y, cfg)
    ordinal = tree_model(cfg).fit(X, q.reshape(len(X), -1))
    descriptions = {name: {'field': name, 'reference_concentration_unit': None,
                           'empirical_transform': None, 'transform_match_fraction': None}
                    for name in features}
    for path, columns in [(unit_map, {'unit': 'reference_concentration_unit'}),
                          (scale_map, {'best_transform': 'empirical_transform', 'match_fraction': 'transform_match_fraction'})]:
        if path is not None:
            table = pd.read_csv(path).drop_duplicates('field').set_index('field')
            for field in set(features) & set(table.index):
                for original, dest in columns.items():
                    value = table.at[field, original]
                    descriptions[field][dest] = None if pd.isna(value) else value.item() if isinstance(value, np.generic) else value
    manifest = {
        'format_version': 1, 'created_utc': datetime.now(timezone.utc).isoformat(),
        'features': features, 'targets': targets, 'training_beers': len(X),
        'feature_descriptions': [descriptions[name] for name in features],
        'feature_scale': 'same field names, units and transformations as the official attachment 1.csv; do not substitute raw concentration for log-scale input',
        'mean_endpoint': 'equal-person panel mean on 0-6 scale; ET and GBR fixed equal-weight blend',
        'grade_endpoint': 'mode and median of predicted seven-category individual-rating distribution',
        'probability_calibration': 'not independently established',
        'model_assessment': 'full-data fit, not an independent accuracy evaluation',
        'training_hashes_sha256': {Path(p).name: sha256(p) for p in (chemical_path, means_path, distribution_path, unit_map, scale_map) if p is not None},
        'versions': {'python': platform.python_version(), **{p: version(p) for p in ('numpy', 'pandas', 'scikit-learn', 'joblib')}},
        'config': cfg,
    }
    return {'manifest': manifest, 'mean_et': et, 'mean_gbr': gbr, 'ordinal_et': ordinal}


def predict(bundle, input_path):
    metadata = bundle['manifest']; data, X = read_input(input_path, metadata['features'])
    pe, pg = predict_pair((bundle['mean_et'], bundle['mean_gbr']), X)
    pe = np.asarray(pe).reshape(len(X), len(metadata['targets']))
    pg = np.asarray(pg).reshape(len(X), len(metadata['targets']))
    means = np.clip((pe + pg) / 2, 0, 6)
    p = bundle['ordinal_et'].predict(X).reshape(len(X), len(metadata['targets']), 7)
    if not np.isfinite(p).all() or (p < -1e-8).any() or not np.allclose(p.sum(-1), 1, atol=1e-6):
        raise ValueError('Model produced invalid rating probabilities')
    p = np.maximum(p, 0); p /= p.sum(-1, keepdims=True)
    ids = data['beer'].to_numpy() if 'beer' in data else np.arange(1, len(data) + 1)
    result = {'beer': ids}
    targets = metadata['targets']
    tables = {}
    for name, values in [('panel_mean', means), ('modal_grade', p.argmax(-1)), ('median_grade', (p.cumsum(-1) >= .5).argmax(-1))]:
        tables[name] = pd.concat([pd.DataFrame(result), pd.DataFrame(values, columns=targets)], axis=1)
    records = pd.DataFrame({'beer': np.repeat(ids, len(targets)), 'target': np.tile(targets, len(ids))})
    for k in range(7): records[f'p{k}'] = p[:, :, k].ravel()
    tables['rating_distribution'] = records
    return tables


def load_trusted_model(path):
    bundle = joblib.load(path)
    if bundle.get('manifest', {}).get('format_version') != 1:
        raise ValueError('Unsupported model bundle')
    installed = version('scikit-learn')
    if installed != bundle['manifest']['versions']['scikit-learn']:
```

```
        raise ValueError(f'scikit-learn version mismatch: trained {bundle["manifest"]["versions"]["scikit-learn"]}, installed {installed}')
    return bundle
```

057_diagnose_stage4.py

```
"""Describe saved out-of-fold grades and split-conformal coverage without refitting."""
from pathlib import Path
import argparse
import json
import sys
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.common import save_csv, write_json, sha256

parser = argparse.ArgumentParser()
parser.add_argument('--chemical', required=True)
parser.add_argument('--out', required=True)
args = parser.parse_args()
out = Path(args.out)
if out.exists() and any(out.iterdir()):
    parser.error('Choose an empty output directory')
out.mkdir(parents=True, exist_ok=True)
base = ROOT / 'outputs/stage4'
ordinal_path = base / 'ordinal/oof_rating_distributions.csv'
interval_path = base / 'prediction/absolute_panel_mean_intervals.csv'
population_path = base / 'audit/panel_sample_counts.csv'
meta = pd.read_csv(args.chemical, usecols=['beer', 'tasting_category_fine'])
counts = pd.read_csv(population_path)
if meta.beer.duplicated().any() or counts.beer.duplicated().any():
    raise ValueError('Duplicate sample metadata')
ratings = pd.read_csv(ordinal_path).merge(meta, on='beer', validate='many_to_one')
ratings['core'] = ratings.target.str.startswith(('A_', 'F_'))
ratings['absolute_grade_error'] = (ratings.predicted_modal_grade - ratings.observed_panel_modal_grade).abs()
confusion, target_metrics, style_metrics = [], [], []
for scope, group in [('core30', ratings[ratings.core]), ('all50', ratings)]:
    for observed in range(7):
        for predicted in range(7):
            confusion.append({'scope': scope, 'observed_grade': observed, 'predicted_grade': predicted,
                              'beer_target_count': int(((group.observed_panel_modal_grade == observed) &
                                                        (group.predicted_modal_grade == predicted)).sum())})
    for field, destination in [('target', target_metrics), ('tasting_category_fine', style_metrics)]:
        for value, sample in group.groupby(field, sort=True):
            nonzero = sample[sample.observed_panel_modal_grade > 0]
            destination.append({'scope': scope, field: value, 'unique_beers': int(sample.beer.nunique()),
                                'beer_target_count': len(sample), 'nonzero_observed_count': len(nonzero),
                                'overall_modal_accuracy': float((sample.predicted_modal_grade == sample.observed_panel_modal_grade).mean()),
                                'nonzero_modal_accuracy': float((nonzero.predicted_modal_grade == nonzero.observed_panel_modal_grade).mean()) if
len(nonzero) else np.nan,
                                'nonzero_predicted_zero_fraction': float((nonzero.predicted_modal_grade == 0).mean()) if len(nonzero) else np.nan,
                                'modal_absolute_grade_error': float(sample.absolute_grade_error.mean())})
intervals = pd.read_csv(interval_path).merge(meta, on='beer', validate='many_to_one').merge(counts, on='beer', validate='many_to_one')
intervals['covered'] = (intervals.observed_panel_mean >= intervals.lower_0_6) & (intervals.observed_panel_mean <= intervals.upper_0_6)
intervals['width'] = intervals.upper_0_6 - intervals.lower_0_6
intervals['panel_group'] = pd.cut(intervals.distinct_panelists, bins=[0, 4, 7, 100], labels=['3-4', '5-7', '8+'])
coverage = []
for grouping in ['target', 'tasting_category_fine', 'panel_group']:
    for value, sample in intervals.groupby(grouping, observed=True, sort=True):
        coverage.append({'grouping': grouping, 'group': str(value), 'unique_beers': int(sample.beer.nunique()),
                         'test_beer_attribute_records': len(sample), 'coverage': float(sample.covered.mean()),
                         'mean_width': float(sample.width.mean())})
save_csv(pd.DataFrame(confusion), out / 'modal_confusion.csv')
save_csv(pd.DataFrame(target_metrics), out / 'modal_by_target.csv')
save_csv(pd.DataFrame(style_metrics), out / 'modal_by_style.csv')
save_csv(pd.DataFrame(coverage), out / 'coverage_by_group.csv')
core = ratings[ratings.core]; nz = core[core.observed_panel_modal_grade > 0]
summary = {'core_beer_target_records': len(core), 'core_nonzero_records': len(nz),
           'core_nonzero_predicted_zero_fraction': float((nz.predicted_modal_grade == 0).mean()),
           'core_nonzero_modal_accuracy': float((nz.predicted_modal_grade == nz.observed_panel_modal_grade).mean()),
           'coverage_repeat_warning': 'Repeated test records share beers; group rates are descriptive, not independent-binomial guarantees.',
           'input_sha256': {p.name: sha256(p) for p in [ordinal_path, interval_path, population_path]}}
write_json(out / 'summary.json', summary)
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
matrix = pd.DataFrame(confusion).query("scope == 'core30'").pivot(index='observed_grade', columns='predicted_grade', values='beer_target_count').
to_numpy()
fig, ax = plt.subplots(figsize=(7, 5.2), constrained_layout=True)
im = ax.imshow(matrix, cmap='Blues')
for i in range(7):
    for j in range(7): ax.text(j, i, str(matrix[i, j]), ha='center', va='center', color='white' if matrix[i, j] > matrix.max() * .55 else 'black',
 fontsize=8)
ax.set(xlabel='预测面板众数等级', ylabel='观测面板众数等级', xticks=range(7), yticks=range(7), title='核心30项：按啤酒留出预测的等级混淆')
fig.colorbar(im, ax=ax, label='啤酒-属性记录数'); fig.savefig(out / 'ordinal_confusion.png', dpi=220); plt.close(fig)
print(json.dumps(summary, ensure_ascii=False))
```

058_test_deployment.py

```python
"""Exercise model packaging and prediction on unseen files without sensory inputs."""
import numpy as np
import pandas as pd
import pytest
from src.deployment import fit_final, predict, read_input


def test_final_model_predicts_from_unlabelled_reordered_chemical_input(tmp_path):
    n = 14
    chem = pd.DataFrame({'beer': np.arange(n), 'tasting_category_fine': ['a'] * n,
                         'beer_id': np.arange(n), 'x1': np.linspace(-1, 1, n), 'x2': np.arange(n) % 3})
    means = pd.DataFrame({'beer': np.arange(n), 'taste': np.arange(n) % 6})
    prob = pd.DataFrame({'beer': np.arange(n), 'target': ['taste'] * n})
    for k in range(7): prob[f'observed_panel_p{k}'] = (means.taste == k).astype(float)
    for path, frame in [('chemical.csv', chem), ('means.csv', means), ('q.csv', prob)]:
        frame.to_csv(tmp_path / path, index=False)
    config = {'seed': 11, 'extra_trees': 5, 'gbr_estimators': 5}
    model = fit_final(tmp_path / 'chemical.csv', tmp_path / 'means.csv', tmp_path / 'q.csv', config)
    unlabelled = chem[['beer', 'x2', 'x1']].iloc[:3].copy()
    unlabelled.loc[1, 'x1'] = np.nan
    unlabelled.to_csv(tmp_path / 'new.csv', index=False)
    result = predict(model, tmp_path / 'new.csv')
    assert list(result['panel_mean'].columns) == ['beer', 'taste']
    assert result['panel_mean'].taste.between(0, 6).all()
    assert result['modal_grade'].taste.between(0, 6).all()
    assert np.allclose(result['rating_distribution'].filter(regex=r'^p\d$').sum(axis=1), 1)
    assert 'taste' not in unlabelled.columns
    with pytest.raises(ValueError, match='Missing'):
        read_input(tmp_path / 'new.csv', ['x1', 'missing'])


def test_duplicate_and_infinite_input_is_rejected(tmp_path):
    (tmp_path / 'duplicate.csv').write_text('beer,x,x\n1,1,2\n', encoding='utf-8')
    with pytest.raises(ValueError, match='duplicate'):
        read_input(tmp_path / 'duplicate.csv', ['x'])
    (tmp_path / 'infinite.csv').write_text('beer,x\n1,inf\n', encoding='utf-8')
    with pytest.raises(ValueError, match='Infinite'):
        read_input(tmp_path / 'infinite.csv', ['x'])
```

059_ordinal_two_stage_experiment.py

```python
"""Zero/nonzero and conditional-grade ET pilot on fixed, beer-held-out folds."""
from pathlib import Path
import argparse
import json
import sys
import time
import numpy as np
import pandas as pd
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.model_selection import RepeatedStratifiedKFold

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.common import preprocessing, save_csv, sha256, write_json
from src.data import META
from src.review_diagnostics import distribution_scores, conditional_paired_interval
from src.stage4_models import ordinal_summary

parser = argparse.ArgumentParser()
parser.add_argument('--chemical', required=True)
parser.add_argument('--out', required=True)
parser.add_argument('--trees', type=int, default=32)
args = parser.parse_args()
out = Path(args.out)
if out.exists() and any(out.iterdir()): parser.error('Use a new output directory')
if args.trees < 1: parser.error('--trees must be positive')
out.mkdir(parents=True, exist_ok=True)
cfg = json.loads((ROOT / 'config_stage4.json').read_text(encoding='utf8'))
chemical = pd.read_csv(args.chemical)
beers = chemical.beer.to_numpy(); styles = chemical.tasting_category_fine.to_numpy()
X = chemical.drop(columns=META).to_numpy(dtype=float)
base_path = ROOT / 'outputs/stage4/ordinal/oof_rating_distributions.csv'
source = pd.read_csv(base_path)
targets = list(dict.fromkeys(source.target))
idx = pd.MultiIndex.from_product([beers, targets], names=['beer', 'target'])
source = source.set_index(['beer', 'target']).loc[idx]
q = source[[f'observed_panel_p{k}' for k in range(7)]].to_numpy().reshape(len(X), len(targets), 7)
q /= q.sum(-1, keepdims=True)
baseline = source[[f'predicted_p{k}' for k in range(7)]].to_numpy().reshape(q.shape)
baseline /= baseline.sum(-1, keepdims=True)
sum_p = np.zeros_like(q); matched_sum = np.zeros_like(q); counts = np.zeros(len(X)); roles = pd.read_csv(ROOT / 'outputs/stage4/prediction/outer_roles.csv')
start = time.perf_counter()
split = RepeatedStratifiedKFold(n_splits=cfg['outer_folds'], n_repeats=cfg['outer_repeats'], random_state=cfg['seed'])
for fold, (train, test) in enumerate(split.split(X, styles)):
    if set(beers[test]) != set(roles.loc[roles.fold == fold, 'beer']): raise ValueError('Outer roles do not match saved baseline')
    prep = preprocessing().fit(X[train]); a = prep.transform(X[train]); b = prep.transform(X[test])
    matched = ExtraTreesRegressor(n_estimators=args.trees, min_samples_leaf=3, max_features=.8,
                                  random_state=cfg['seed'], n_jobs=2).fit(a, q[train].reshape(len(train), -1)).predict(b)
    matched_sum[test] += matched.reshape(len(test), len(targets), 7)
    zero = ExtraTreesRegressor(n_estimators=args.trees, min_samples_leaf=3, max_features=.8,
                               random_state=cfg['seed'] + fold, n_jobs=2).fit(a, q[train, :, 0]).predict(b)
    zero = np.asarray(zero).reshape(len(test), len(targets)).clip(0, 1)
    prediction = np.zeros((len(test), len(targets), 7)); prediction[:, :, 0] = zero
```

```
    for j in range(len(targets)):
        mass = 1 - q[train, j, 0]
        use = mass > 1e-10
        if use.sum() < 3:
            conditional = q[train, j, 1:].sum(0)
            conditional = conditional / conditional.sum() if conditional.sum() > 0 else np.full(6, 1 / 6)
            r = np.broadcast_to(conditional, (len(test), 6))
        else:
            y = q[train[use], j, 1:] / mass[use, None]
            model = ExtraTreesRegressor(n_estimators=args.trees, min_samples_leaf=3, max_features=.8,
                                        random_state=cfg['seed'] + 100 * fold + j, n_jobs=2)
            r = model.fit(a[use], y, sample_weight=mass[use]).predict(b)
            r = np.maximum(r, 0); r /= r.sum(-1, keepdims=True)
        prediction[:, j, 1:] = (1 - zero[:, j, None]) * r
    sum_p[test] += prediction; counts[test] += 1
    print(f'fold {fold+1}/{cfg["outer_folds"]*cfg["outer_repeats"]}; elapsed {time.perf_counter()-start:.1f}s', flush=True)
if not np.all(counts == cfg['outer_repeats']): raise ValueError('Unexpected OOF test counts')
p = sum_p / counts[:, None, None]
matched = matched_sum / counts[:, None, None]
matched /= matched.sum(-1, keepdims=True)
scopes = {'core30': [j for j, t in enumerate(targets) if t.startswith(('A_', 'F_'))], 'all50': list(range(len(targets)))}
summaries = []; comparisons = []
for scope, columns in scopes.items():
    for name, prob in [(f'frequency_ET_{cfg["extra_trees"]}_trees', baseline), (f'frequency_ET_{args.trees}_trees', matched), (f'two_stage_ET_{
args.trees}_trees', p)]:
        summaries.append({'scope': scope, 'model': name, **ordinal_summary(q[:, columns], prob[:, columns], prob[:, columns].argmax(-1))})
    for name, comparator in [('frequency_200', baseline), ('frequency_matched', matched)]:
        d = (distribution_scores(q, p)['cdf_distance'][:, columns] - distribution_scores(q, comparator)['cdf_distance'][:, columns]).mean(1)
        comparisons.append({'scope': scope, 'comparison': f'two_stage_minus_{name}', **conditional_paired_interval(d, styles)})
rows = pd.DataFrame({'beer': np.repeat(beers, len(targets)), 'target': np.tile(targets, len(beers))})
for k in range(7): rows[f'predicted_p{k}'] = p[:, :, k].ravel()
rows['predicted_modal_grade'] = p.argmax(-1).ravel()
save_csv(rows, out / 'oof_distribution.csv'); save_csv(pd.DataFrame(summaries), out / 'metrics.csv'); save_csv(pd.DataFrame(comparisons), out / '
paired_comparison.csv')
matched_rows = pd.DataFrame({'beer': np.repeat(beers, len(targets)), 'target': np.tile(targets, len(beers)), 'predicted_modal_grade': matched.
argmax(-1).ravel()})
for k in range(7): matched_rows[f'predicted_p{k}'] = matched[:, :, k].ravel()
save_csv(matched_rows, out / 'oof_frequency_matched.csv')
write_json(out / 'manifest.json', {'trees_per_two_stage_model': args.trees, 'baseline_trees': cfg['extra_trees'],
    'folds': cfg['outer_folds'], 'repeats': cfg['outer_repeats'], 'seed': cfg['seed'],
    'design': 'exploratory; conditional nonzero distributions weighted by training panel nonzero mass; fixed saved folds; no calibration',
    'conditional_interval': 'beer stratified bootstrap of fixed OOF predictions; no refitting uncertainty',
    'source_sha256': {'baseline_ratings': sha256(base_path), 'chemical_file': sha256(args.chemical)}})
print(pd.DataFrame(summaries)[['scope','model','cdf_distance','expected_individual_RPS','nonzero_panel_mode_accuracy']].to_string(index=False))
```

060_summarize_two_stage.py

```
"""Paired beer-level bootstrap of nonzero-grade recognition, conditional on saved OOF predictions."""
from pathlib import Path
import argparse
import numpy as np
import pandas as pd
import sys
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from src.common import save_csv

parser = argparse.ArgumentParser()
parser.add_argument('--chemical', required=True)
parser.add_argument('--result-dir', default=str(ROOT / 'outputs/stage5_two_stage_20261028'))
args = parser.parse_args()
out = Path(args.result_dir)
base = pd.read_csv(ROOT / 'outputs/stage4/ordinal/oof_rating_distributions.csv')
candidate = pd.read_csv(out / 'oof_distribution.csv')
styles = pd.read_csv(args.chemical, usecols=['beer', 'tasting_category_fine'])
joined = base[['beer', 'target', 'observed_panel_modal_grade', 'predicted_modal_grade']].merge(
    candidate[['beer', 'target', 'predicted_modal_grade']], on=['beer', 'target'], suffixes=('_base', '_new'), validate='one_to_one')
joined = joined.merge(styles, on='beer', validate='many_to_one')
rows = []
matched_path = out / 'oof_frequency_matched.csv'
if matched_path.exists():
    matched = pd.read_csv(matched_path, usecols=['beer', 'target', 'predicted_modal_grade']).rename(columns={'predicted_modal_grade': 'predicted_
modal_grade_matched'})
    joined = joined.merge(matched, on=['beer', 'target'], validate='one_to_one')
comparators = ['base'] + (['matched'] if matched_path.exists() else [])
for scope, frame in [('core30', joined[joined.target.str.startswith(('A_', 'F_'))]), ('all50', joined)]:
  for comparator in comparators:
    frame = frame[frame.observed_panel_modal_grade > 0].copy()
    frame['difference'] = (frame.predicted_modal_grade_new == frame.observed_panel_modal_grade).astype(int) - (frame[f'predicted_modal_grade_{
comparator}'] == frame.observed_panel_modal_grade).astype(int)
    per = frame.groupby(['beer', 'tasting_category_fine']).agg(difference=('difference', 'sum'), denominator=('difference', 'size')).reset_index()
    rng = np.random.default_rng(20261028); numerator = np.zeros(5000); denominator = np.zeros(5000)
    # Include beers with no nonzero records in their style as zero-contribution sampling units.
    per = styles.merge(per[['beer', 'difference', 'denominator']], on='beer', how='left').fillna({'difference': 0, 'denominator': 0})
    for _, g in per.groupby('tasting_category_fine'):
        index = rng.integers(len(g), size=(5000, len(g)))
        numerator += g.difference.to_numpy()[index].sum(axis=1)
        denominator += g.denominator.to_numpy()[index].sum(axis=1)
    draws = numerator[denominator > 0] / denominator[denominator > 0]
    low, high = np.quantile(draws, [.025, .975])
    rows.append({'scope': scope, 'comparator': comparator, 'nonzero_beer_target_records': len(frame), 'difference_two_stage_minus_frequency':
float(frame.difference.mean()),
                 'conditional_low95': float(low), 'conditional_high95': float(high), 'resamples': 5000,
                 'unit': 'beer stratified by style; fixed OOF predictions'})
save_csv(pd.DataFrame(rows), out / 'paired_nonzero_accuracy.csv')
```

```
print(pd.DataFrame(rows).to_string(index=False))
```

061_joint_review_experiments.py

```
from pathlib import Path
import os, sys, json, hashlib, argparse
for n in ['OMP_NUM_THREADS','OPENBLAS_NUM_THREADS','MKL_NUM_THREADS']: os.environ.setdefault(n,'2')
ROOT=Path(__file__).resolve().parents[1]; sys.path.insert(0,str(ROOT))
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge, LogisticRegression
from sklearn.ensemble import ExtraTreesClassifier
from src.data import find_data, load_data, CHEMICAL_DEFECTS, AGGREGATES, NON_MOLECULES
from src.common import preprocessing, save_csv, write_json, sha256
from src.stage3_prediction import tree_model


def risk_mix(source, anchors, threshold):
    d=(anchors-source).T
    b=np.maximum(source,threshold)-source
    scale=np.maximum(np.abs(d).max(axis=1),1e-12)
    result=linprog(-np.ones(len(anchors)),A_ub=np.vstack([d/scale[:,None],np.ones(len(anchors))]),b_ub=np.r_[b/scale,.75],bounds=(0,None),method='highs')
    if not result.success: raise RuntimeError(result.message)
    return result.x


def main():
    cfg=json.loads((ROOT/'config_stage4.json').read_text(encoding='utf-8'))
    parser=argparse.ArgumentParser(); parser.add_argument('--data-dir'); args=parser.parse_args()
    data=load_data(find_data(args.data_dir)); beers=data.meta.beer.to_numpy(); styles=data.meta.tasting_category_fine.to_numpy(); targets=list(data.Y)
    out=ROOT/'outputs/joint_20261028'; out.mkdir(parents=True,exist_ok=True)
    Y=pd.read_csv(ROOT/'outputs/stage4/audit/observed_panel_means_0_6.csv').set_index('beer').loc[beers,targets].to_numpy()
    features=[f for f in data.X if f not in AGGREGATES]; X=data.X[features].copy(); exceptions=pd.read_csv(ROOT/'outputs/stage4/audit/chemical_source_exceptions.csv')
    assert not exceptions.duplicated(['beer','field']).any()
    index={int(b):i for i,b in enumerate(beers)}; masked=X.copy(); applied=[]
    for row in exceptions.itertuples():
        if row.field in masked.columns:
            i=index[int(row.beer)]; masked.iloc[i,masked.columns.get_loc(row.field)]=np.nan; applied.append((row.beer,row.field))
    registry=pd.read_csv(ROOT/'outputs/stage4/audit/concentration_feature_registry.csv')
    reconciled=pd.read_csv(ROOT/'outputs/stage4/audit/chemical_scale_reconciliation.csv')
    verified=[f for f in features if f in set(registry.field) and f in set(reconciled.loc[reconciled.status=='all_comparable_pairs_reconciled','field'])]
    variants={'original':X,'masked_exceptions':masked,'reconciled_unit_fields':X[verified]}
    preds={k:np.zeros_like(Y) for k in variants}; counts=np.zeros(len(beers)); top={k:[] for k in variants}; rows=[]
    previous=pd.read_csv(ROOT/'outputs/stage4/prediction/outer_roles.csv')
    split=RepeatedStratifiedKFold(n_splits=cfg['outer_folds'],n_repeats=cfg['outer_repeats'],random_state=cfg['seed'])
    for fold,(train,test) in enumerate(split.split(X,styles)):
        assert set(beers[test])==set(previous.loc[previous.fold==fold,'beer'])
        print(f'Chemical sensitivity fold {fold+1}/15',flush=True)
        for name,frame in variants.items():
            a=frame.to_numpy(); model=tree_model(cfg).fit(a[train],Y[train]); pred=model.predict(a[test]); preds[name][test]+=pred
            prep=preprocessing().fit(a[train]); ridge=Ridge(alpha=1000).fit(prep.transform(a[train]),Y[train]); coef=np.abs(ridge.coef_); cols=list(frame)
            valid=np.array([i for i,f in enumerate(cols) if f not in NON_MOLECULES]); ranks=[]
            for j in range(len(targets)): ranks.append(set(cols[i] for i in valid[np.argsort(coef[j,valid])[-3:]]))
            top[name].append(ranks)
        counts[test]+=1
    assert np.all(counts==cfg['outer_repeats'])
    for k in preds: preds[k]/=counts[:,None]
    core=[j for j,t in enumerate(targets) if t.startswith(('A_','F_'))]
    targetrows=[]
    for name,p in preds.items():
        for scope,idx in [('core30',core),('all50',list(range(len(targets))))]:
            err=np.abs(Y[:,idx]-p[:,idx]).mean(axis=1); ref=np.abs(Y[:,idx]-preds['original'][:,idx]).mean(axis=1)
            rows.append({'variant':name,'scope':scope,'fields':variants[name].shape[1],'mae':err.mean(),'paired_mae_change':(err-ref).mean(),'mean_absolute_prediction_change':np.abs(p[:,idx]-preds['original'][:,idx]).mean()})
        for j,t in enumerate(targets):
            overlaps=[len(top[name][f][j]&top['original'][f][j])/3 for f in range(len(top[name]))]
            save={'variant':name,'target':t,'mae':np.abs(Y[:,j]-p[:,j]).mean(),'original_mae':np.abs(Y[:,j]-preds['original'][:,j]).mean(),'mean_top3_retention_fixed_ridge':np.mean(overlaps)}
            targetrows.append(save)
    save_csv(pd.DataFrame(rows),out/'chemical_sensitivity_summary.csv'); save_csv(pd.DataFrame(targetrows),out/'chemical_sensitivity_by_target.csv')
    # Candidate search uses G alone; candidate array is frozen before J is fitted.
    fields=registry.field.tolist(); C=data.calibrated.set_index('beer').loc[beers,fields].where(lambda x:x>=0).to_numpy(); pos=[fields.index(f) for f in CHEMICAL_DEFECTS]; stale=targets.index('stale_hops')
    roles=pd.read_csv(ROOT/'outputs/stage3/q3/disjoint_roles.csv'); allrows=[]; allprofiles=[]; locks=[]
    for repeat in range(5):
        print(f'Joint risk and proxy evaluation repeat {repeat+1}/5',flush=True)
        role=roles[roles.repeat==repeat].set_index('beer').loc[beers,'role'].to_numpy(); G=np.flatnonzero(role=='G_search_training'); J=np.flatnonzero(role=='J_evaluator_training'); T=np.flatnonzero(role=='T_real_holdout')
        fill=preprocessing().fit(C[G]); values=np.where(np.isfinite(C),C,fill.named_steps['impute'].statistics_)
        search=Pipeline([('prep',preprocessing()),('model',LogisticRegression(C=1,max_iter=3000,random_state=cfg['seed']))]).fit(C[G],styles[G]); sensory=tree_model(cfg).fit(C[G],Y[G]); consensus=Pipeline([('prep',preprocessing()),('model',Ridge(alpha=1000))]).fit(C[G],Y[G]); pt=list(search.classes_).index('Tripel')
        anchors=G[(styles[G]=='Tripel') & np.isfinite(C[G][:,pos]).all(axis=1)]; locked=[]
        for i in np.flatnonzero(styles=='Pils/Lager'):
            s=values[i]; base=search.predict_proba(s[None])[0,pt]; gs=sensory.predict(s[None])[0,stale]; rs=consensus.predict(s[None])[0,stale]
            for multiplier in [.5,1.,2.]:
```

```
                threshold=multiplier*np.array([19.,.09,.2,.0001]); known=np.isfinite(C[i,pos]).all(); candidates=[]; recipes=[]; maxfraction=0.
                if known and len(anchors):
                    w=risk_mix(C[i,pos],C[anchors][:,pos],threshold); maxfraction=w.sum()
                    for shrink in np.linspace(.05,1,20):
                        ww=w*shrink; candidates.append(s*(1-ww.sum())+ww@values[anchors]); recipes.append({int(beers[a]):float(z) for a,z in zip(anchors,ww) if z>1e-10})
                    for a in anchors:
                        for z in cfg['mixture_weights']:
                            candidates.append(s*(1-z)+values[a]*z); recipes.append({int(beers[a]):float(z)})
                found=None; valid_count=0
                if candidates:
                    V=np.array(candidates); risk=(V[:,pos] <= np.maximum(C[i,pos],threshold)+1e-12).all(axis=1); p=search.predict_proba(V)[:,pt]; sv=sensory.predict(V)[:,stale]; rv=consensus.predict(V)[:,stale]
                    distance=np.linalg.norm(fill.transform(V)-fill.transform(s[None]),axis=1)/np.sqrt(len(fields)); valid=risk & (p>=max(.5,base+.01)) & (sv<=gs+1e-9) & (rv<=rs+1e-9) & (distance>1e-8); good=np.flatnonzero(valid); valid_count=len(good)
                    if len(good): found=int(good[np.argmin(distance[good])])
                value=V[found] if found is not None else s.copy()
                locked.append({'i':i,'multiplier':multiplier,'source':s,'value':value,'feasible':found is not None,'recipe':recipes[found] if found is not None else {},'maxfraction':maxfraction,'valid_count':valid_count,'base':base,'gprob':p[found] if found is not None else base,'gs':gs,'gscore':sv[found] if found is not None else gs,'rs':rs,'rscore':rv[found] if found is not None else rs})
        digest=hashlib.sha256(np.array([v['value'] for v in locked],dtype='<f8').tobytes()).hexdigest(); locks.append({'repeat':repeat,'sha256_float64_candidates':digest,'selection_completed_before_J_fitting':True,'features':fields})
        judge=Pipeline([('prep',preprocessing()),('model',ExtraTreesClassifier(n_estimators=200,min_samples_leaf=3,max_features=.7,random_state=cfg['seed'],n_jobs=2))]).fit(C[J],styles[J]); sensoryj=tree_model(cfg).fit(C[J],Y[J]); pj=list(judge.classes_).index('Tripel')
        for v in locked:
            i=v['i']; z=np.stack([v['source'],v['value']]); prob=judge.predict_proba(z)[:,pj]; score=sensoryj.predict(z)[:,stale]; predicted=judge.predict(z[1:])[0]; accepted=v['feasible'] and predicted=='Tripel' and prob[1]>prob[0]+.01 and score[1]<=score[0]+1e-9
            row={'repeat':repeat,'beer':int(beers[i]),'source_role':role[i],'threshold_multiplier':v['multiplier'],'max_chemical_fraction':v['maxfraction'],'search_feasible':v['feasible'],'search_candidate_count':v['valid_count'],'target_fractions':json.dumps(v['recipe'],sort_keys=True),'G_source_Tripel_probability':v['base'],'G_candidate_Tripel_probability':v['gprob'],'G_source_stale':v['gs'],'G_candidate_stale':v['gscore'],'G_ridge_source_stale':v['rs'],'G_ridge_candidate_stale':v['rscore'],'J_source_Tripel_probability':prob[0],'J_candidate_Tripel_probability':prob[1],'J_predicts_Tripel':predicted=='Tripel','J_source_stale':score[0],'J_candidate_stale':score[1],'J_proxy_support':bool(accepted),'source_has_imputed_concentration':bool(np.isnan(C[i]).any())}
            for k,f in enumerate(CHEMICAL_DEFECTS): row['source_'+f]=C[i,pos[k]]; row['candidate_'+f]=v['value'][pos[k]]
            allrows.append(row); allprofiles.append({'repeat':repeat,'beer':int(beers[i]),'threshold_multiplier':v['multiplier'],'search_feasible':v['feasible'],**dict(zip(fields,v['value']))})
    save_csv(pd.DataFrame(allrows),out/'joint_candidate_evaluation.csv'); save_csv(pd.DataFrame(allprofiles),out/'joint_candidate_profiles.csv'); write_json(out/'candidate_locks.json',locks)
    summary=pd.DataFrame(allrows).groupby(['threshold_multiplier','source_role'],as_index=False).agg(records=('beer','size'),unique_beers=('beer','nunique'),chemical_possible=('max_chemical_fraction',lambda x:int((x>1e-8).sum())),search_feasible=('search_feasible','sum'),J_proxy_support=('J_proxy_support','sum'))
    save_csv(summary,out/'joint_summary_by_role.csv')
    write_json(out/'manifest.json',{'design':'exploratory after review; G candidate selection frozen before fitting J; source role J is in J training; T is internally held out, not external','exception_cells_in_nonaggregate_features':len(applied),'verified_unit_field_count':len(verified),'verified_fields':verified,'ridge_alpha_fixed':1000,'folds':cfg['outer_folds'],'repeats':cfg['outer_repeats'],'physical_experiment':False,'code_sha256':sha256(Path(__file__))})
    print(pd.DataFrame(rows).to_string(index=False)); print(summary.to_string(index=False))

if __name__=='__main__': main()
```

062_test_joint_review.py

```
import numpy as np
from scripts.joint_review_experiments import risk_mix


def test_risk_mix_finds_mixture_within_all_four_bounds():
    source=np.array([10., .08, .1, .00005])
    anchors=np.array([[20., .1, .3, .00015], [12., .06, .08, .00004]])
    threshold=np.array([19., .09, .2, .0001])
    w=risk_mix(source, anchors, threshold)
    candidate=source+(anchors-source).T@w
    assert 0 < w.sum() <= .75+1e-9
    assert np.all(candidate <= np.maximum(source,threshold)+1e-10)


def test_risk_mix_returns_zero_when_each_anchor_increases_above_threshold():
    source=np.array([20., .1, .3, .0002])
    anchors=np.array([[21., .11, .4, .0003], [22., .12, .5, .0004]])
    w=risk_mix(source, anchors, np.array([19., .09, .2, .0001]))
    assert w.sum() < 1e-8
```

补充核查程序

```
"""Auditable field-set, classification-metric and candidate evidence checks."""
from pathlib import Path
import argparse,json,sys,hashlib
import numpy as np,pandas as pd
from sklearn.metrics import f1_score,balanced_accuracy_score,confusion_matrix
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT))
from src.data import load_data,find_data,AGGREGATES
from src.common import save_csv,write_json,sha256

def main():
 ap=argparse.ArgumentParser();ap.add_argument('--data-dir');ap.add_argument('--out',default='outputs/submission_review_20261029');a=ap.parse_args();out=ROOT/a.out
 if out.exists() and any(out.iterdir()):raise SystemExit('choose an empty output directory')
 out.mkdir(parents=True,exist_ok=True);D=load_data(find_data(a.data_dir));beers=D.meta.beer.to_numpy();styles=D.meta.tasting_category_fine.to_numpy();targets=list(D.Y);a4=ROOT/'outputs/stage4/audit';rec=pd.read_csv(a4/'chemical_scale_reconciliation.csv');reg=pd.read_csv(a4/'concentration_feature_registry.csv');exc=pd.read_csv(a4/'chemical_source_exceptions.csv');manifest=json.loads((ROOT/'outputs/joint_20261028/manifest.json').read_text(encoding='utf8'));selected=set(manifest['verified_fields']);equal=set(rec.loc[rec.status=='all_comparable_pairs_reconciled','field']);diff=set(rec.loc[rec.status=='exceptions_retained','field']);unit=set(reg.field);nonagg=set(D.X)-AGGREGATES;computed=equal & unit & nonagg
```

```
 assert len(equal)==38 and len(diff)==32 and len(exc)==80 and len(computed)==32 and selected==computed and not selected&diff
 rows=[]
 for r in rec.itertuples():rows.append({'field':r.field,'audit_status':r.status,'best_transform':r.best_transform,'comparable_pairs':r.comparable_pairs,'exception_pairs':int((exc.field==r.field).sum()),'documented_unit_in_registry':r.field in unit,'nonaggregate_model_field':r.field in nonagg,'selected_for_32_subset':r.field in selected,'unit':reg.set_index('field').unit.get(r.field,'')})
 save_csv(pd.DataFrame(rows),out/'field_sets.csv')
 x=D.X['diethyl.succinate'].to_numpy(); raw=D.calibrated['diethyl.succinate'].to_numpy();mask=np.isfinite(x)&np.isfinite(raw)&(raw>0);r=10**x[mask]/raw[mask]
 identity={'field':'diethyl.succinate','comparable_positive_pairs':int(mask.sum()),'identity_max_absolute_difference':float(np.max(abs(x[mask]-raw[mask]))),'counterfactual_exponentiation_ratio_min_median_max':[float(np.min(r)),float(np.median(r)),float(np.max(r))],'counterfactual_ratio_note':'10^X/reference is a forced log hypothesis applied to identity-aligned values; variation is not evidence of a constant unit conversion or actual quantification bias','independent_chemical_calibration':False}
 write_json(out/'identity_field_check.json',identity)
 ratings=pd.read_csv(ROOT/'outputs/stage4/ordinal/oof_rating_distributions.csv').set_index(['beer','target']).loc[pd.MultiIndex.from_product([beers,targets])];q=ratings[[f'observed_panel_p{k}' for k in range(7)]].to_numpy().reshape(len(beers),50,7);q/=q.sum(-1,keepdims=True);prob=ratings[[f'predicted_p{k}' for k in range(7)]].to_numpy().reshape(q.shape);prob/=prob.sum(-1,keepdims=True);roles=pd.read_csv(ROOT/'outputs/stage4/prediction/outer_roles.csv');preds={'frequency_ET':prob,'population':np.zeros_like(q),'style_only':np.zeros_like(q)};count=np.zeros(len(beers))
 for fold in sorted(roles.fold.unique()):
  test=np.flatnonzero(np.isin(beers,roles.loc[roles.fold==fold,'beer']));train=np.setdiff1d(np.arange(len(beers)),test);preds['population'][test]+=q[train].mean(0)
  for i in test:
   ids=train[styles[train]==styles[i]]; preds['style_only'][i]+=q[ids].mean(0) if len(ids) else q[train].mean(0)
  count[test]+=1
 assert np.all(count==5)
 for name in ['population','style_only']:preds[name]/=count[:,None,None]
 observed=q.argmax(-1);scopes={'core30':[i for i,t in enumerate(targets) if t.startswith(('A_','F_'))],'all150':list(range(50))};metrics=[];targetrows=[];confusions=[]
 for name,p in preds.items():
  grade=p.argmax(-1)
  for scope,cols in scopes.items():
   y=observed[:,cols].ravel();z=grade[:,cols].ravel();counts=np.bincount(y,minlength=7);active=np.flatnonzero(counts>0); nz=y>0;cm=confusion_matrix(y,z,labels=np.arange(7));
   metrics.append({'model':name,'scope':scope,'pooled_records':len(y),'zero_fraction':float((y==0).mean()),'pooled_macro_f1_7_observed_grades':float(f1_score(y,z,labels=active,average='macro',zero_division=0)),'pooled_balanced_accuracy':float(balanced_accuracy_score(y,z)),'modal_grade_mae':float(np.abs(y-z).mean()),'exact_accuracy':float((y==z).mean()),'nonzero_exact_accuracy':float((z[nz]==y[nz]).mean()),'nonzero_detection_recall':float((z[nz]>0).mean())})
   for i in range(7):
    for j in range(7):confusions.append({'model':name,'scope':scope,'observed_grade':i,'predicted_grade':j,'count':int(cm[i,j])})
  for j,t in enumerate(targets):
   y=observed[:,j];z=grade[:,j];active=np.unique(y);targetrows.append({'model':name,'target':t,'n':len(y),'observed_nonzero':int((y>0).sum()),'observed_grades':';'.join(str(k) for k in active),'macro_f1_over_observed_grades':float(f1_score(y,z,labels=active,average='macro',zero_division=0)),'balanced_accuracy':float(balanced_accuracy_score(y,z)),'modal_grade_mae':float(np.abs(y-z).mean())})
 save_csv(pd.DataFrame(metrics),out/'ordinal_metrics.csv');save_csv(pd.DataFrame(targetrows),out/'ordinal_metrics_by_target.csv');save_csv(pd.DataFrame(confusions),out/'ordinal_confusion.csv')
 evidence=pd.read_csv(ROOT/'outputs/stage4/raw_graph/q2/top3_with_evidence.csv');assert len(evidence)==150
 support=(evidence.bootstrap_frequency_conditional_on_full_selected_params>=.5)&(evidence.crossfit_selection_count>=5)&(evidence.heldout_delta_on_selected_folds>0)&(evidence.within_style_delta_on_selected_folds>0);assert support.sum()==36;evidence['support_class']=np.where(support,'supported_descriptive','exploratory');save_csv(evidence,out/'top3_classified.csv');group=evidence.groupby('target').support_class.apply(lambda z:(z=='supported_descriptive').sum());save_csv(group.reset_index(name='supported_count'),out/'top3_support_by_target.csv')
 write_json(out/'manifest.json',{'source_files':{str(p.relative_to(ROOT)):sha256(p) for p in [a4/'chemical_scale_reconciliation.csv',a4/'concentration_feature_registry.csv',a4/'chemical_source_exceptions.csv',ROOT/'outputs/joint_20261028/manifest.json',ROOT/'outputs/stage4/ordinal/oof_rating_distributions.csv',ROOT/'outputs/stage4/prediction/outer_roles.csv',ROOT/'outputs/stage4/raw_graph/q2/top3_with_evidence.csv']},'script_sha256':sha256(Path(__file__)),'32_unit_subset_and_32_exception_overlap':len(selected&diff),'metric_scope':'pooled beer-attribute modal grades, macro across observed grades; repeated OOF probabilities averaged per beer before decisions; 7 grades are ordered; no independence claim','classification_thresholds':'descriptive exploratory evidence thresholds; selection not independent confirmatory test','no_physical_validation':True})
 print('32 verified subset vs exceptions overlap',len(selected&diff));print('identity',identity);print(pd.DataFrame(metrics).to_string(index=False));print('supported by target',group.value_counts().sort_index().to_string())
if __name__=='__main__':main()
```

```
"""Recover full-data graph tuning scores, using the original inner split and grid."""
from pathlib import Path
import os,sys,json,argparse
for n in ['OMP_NUM_THREADS','OPENBLAS_NUM_THREADS','MKL_NUM_THREADS']:os.environ[n]='2'
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT))
import numpy as np,pandas as pd
from scipy.linalg import solve
from sklearn.model_selection import KFold
from src.data import load_data,find_data
from src.stage3_graph import candidates,train_correlation
from src.q2 import signed_laplacian
from src.common import preprocessing,save_csv,write_json,sha256

def main():
 p=argparse.ArgumentParser();p.add_argument('--data-dir');a=p.parse_args();d=load_data(find_data(a.data_dir));cfg=json.loads((ROOT/'config_stage3.json').read_text(encoding='utf8'));cfg.update(json.loads((ROOT/'config_stage4.json').read_text(encoding='utf8')));X=d.X.to_numpy();Y=pd.read_csv(ROOT/'outputs/stage4/audit/observed_panel_means_0_6.csv').set_index('beer').loc[d.meta.beer,list(d.Y)].to_numpy();core=[i for i,t in enumerate(d.Y) if t.startswith(('A_','F_'))];choices=candidates(cfg,'real');scores=np.zeros(len(choices));seed=cfg['seed']+500
 for train,test in KFold(3,shuffle=True,random_state=seed).split(X):
  prep=preprocessing().fit(X[train]);z=prep.transform(X[train]);v=prep.transform(X[test]);corr=train_correlation(z);layers={t:signed_laplacian(corr,t) for t in cfg['graph_thresholds']};lhs=z.T@z;rhs=z.T@(Y[train]-Y[train].mean(0));eye=np.eye(z.shape[1])
  for k,(alpha,lam,tau) in enumerate(choices):
   beta=solve(lhs+alpha*eye+(lam*layers[tau] if lam else 0),rhs,assume_a='pos',check_finite=False);pred=v@beta+Y[train].mean(0);scores[k]+=np.abs(Y[test][:,core]-pred[:,core]).sum()/(len(Y)*len(core))
 best=min(range(len(choices)),key=lambda i:(scores[i],choices[i][1],-choices[i][0]));rows=[{'alpha':aa,'lambda':ll,'tau':tt,'inner_mae':scores[i],'selected':i==best} for i,(aa,ll,tt) in enumerate(choices)];out=ROOT/'outputs/submission_review_20261029';save_csv(pd.DataFrame(rows),out/'full_graph_tuning_grid.csv');write_json(out/'full_graph_tuning_manifest.json',{'split':'original full-data 3-fold KFold seed stage4_seed+500; scoring core30 MAE','target':'audited panel means exported CSV','code_sha256':sha256(Path(__file__))});print(pd.DataFrame(rows).sort_values('inner_mae').head(8).to_string(index=False));print('selected',choices[best])
if __name__=='__main__':main()
```

```
"""Rebuild the scale figure using commensurate, auditable quantities."""
from pathlib import Path
import argparse,sys
```

```
import numpy as np,pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT))
from src.data import load_data,find_data
from src.common import save_csv,write_json,sha256

def main():
 ap=argparse.ArgumentParser();ap.add_argument('--data-dir');a=ap.parse_args();D=load_data(find_data(a.data_dir));out=ROOT/'outputs/submission_review_20261029';rec=pd.read_csv(out/'field_sets.csv');exc=pd.read_csv(ROOT/'outputs/stage4/audit/chemical_source_exceptions.csv');fs=rec[(rec.audit_status=='all_comparable_pairs_reconciled')&(rec.best_transform=='log10_nonpositive_half_min')].field.tolist();assert len(fs)==37
 vals=[]
 for f in fs:
  x=D.X[f].to_numpy();c=D.calibrated[f].to_numpy();m=np.isfinite(x)&np.isfinite(c)&(c>0);vals.extend(zip(c[m],10**x[m]))
 xy=np.array(vals);counts=exc.field.value_counts();top=counts.head(8);other=int(counts.iloc[8:].sum());fig,(ax,bx)=plt.subplots(1,2,figsize=(10,4.3),constrained_layout=True);ax.scatter(xy[:,0],xy[:,1],s=3,alpha=.16,c='#287c91',rasterized=True);lo=max(min(xy.min(),1e-5),1e-8);hi=max(xy.max(),1e2);ax.plot([lo,hi],[lo,hi],color='#ac4d45',linestyle='--',linewidth=1);ax.set(xscale='log',yscale='log',xlabel='Reference concentration (reported units)',ylabel='Reconstructed concentration: $10^x$',title='(a) 37 log-scale fields, positive pairs');labels=list(top.index)+['Other 24 fields'];v=list(top.values)+[other];bp=bx.barh(range(len(v)),v,color=['#477f91']*8+['#999999']);bx.set_yticks(range(len(v)),labels,fontsize=7);bx.invert_yaxis();bx.set_xlabel('Number of mismatched cells');bx.set_title('(b) 80 exceptions in 32 fields');bx.bar_label(bp,padding=2,fontsize=8);bx.set_xlim(0,max(v)*1.18);fig.savefig(ROOT/'paper/submission_revision_20261029/figures/fig02_scale.png',dpi=220);plt.close(fig);save_csv(counts.rename_axis('field').reset_index(name='exception_count'),out/'exception_counts_by_field.csv');print('positive pairs',len(xy),'bar total',sum(v))
if __name__=='__main__':main()
```

```
"""Exploratory matched ET comparisons: known style alone versus style plus chemistry."""
from pathlib import Path
import argparse,json,sys
import numpy as np,pandas as pd
from sklearn.model_selection import RepeatedStratifiedKFold
ROOT=Path(__file__).resolve().parents[1];sys.path.insert(0,str(ROOT))
from src.data import load_data,find_data
from src.common import save_csv,write_json,sha256
from src.stage3_prediction import tree_model
from src.review_diagnostics import conditional_paired_interval

def main():
 p=argparse.ArgumentParser();p.add_argument('--data-dir');p.add_argument('--out',default='outputs/submission_style_increment_20261029');a=p.parse_args();out=ROOT/a.out
 if out.exists() and any(out.iterdir()):raise SystemExit('choose an empty output directory')
 out.mkdir(parents=True,exist_ok=True);D=load_data(find_data(a.data_dir));cfg=json.loads((ROOT/'config_stage4.json').read_text(encoding='utf8'));beers=D.meta.beer.to_numpy();styles=D.meta.tasting_category_fine.to_numpy();targets=list(D.Y);X=D.X.to_numpy();Y=pd.read_csv(ROOT/'outputs/stage4/audit/observed_panel_means_0_6.csv').set_index('beer').loc[beers,targets].to_numpy();onehot=pd.get_dummies(styles,dtype=float).to_numpy();inputs={'style_ET':onehot,'style_chemistry_ET':np.column_stack([onehot,X])};oof={k:np.zeros_like(Y) for k in inputs};count=np.zeros(len(X));roles=pd.read_csv(ROOT/'outputs/stage4/prediction/outer_roles.csv')
 for fold,(train,test) in enumerate(RepeatedStratifiedKFold(n_splits=cfg['outer_folds'],n_repeats=cfg['outer_repeats'],random_state=cfg['seed']).split(X,styles)):
  assert set(beers[test])==set(roles.loc[roles.fold==fold,'beer'])
  for name,v in inputs.items():oof[name][test]+=tree_model(cfg).fit(v[train],Y[train]).predict(v[test])
  count[test]+=1;print('matched style/chemistry fold',fold+1,flush=True)
 assert np.all(count==cfg['outer_repeats']);oof={k:v/count[:,None] for k,v in oof.items()};oof['style_median']=pd.read_csv(ROOT/'outputs/review_20261028/oof_style_median.csv').set_index('beer').loc[beers,targets].to_numpy();rows=[];paired=[];bytarget=[]
 for name,pred in oof.items():
  for scope,cols in [('core30',[i for i,t in enumerate(targets) if t.startswith(('A_','F_'))]),('all50',list(range(len(targets))))]:rows.append({'model':name,'scope':scope,'mae':float(abs(pred[:,cols]-Y[:,cols]).mean())})
  for j,t in enumerate(targets):bytarget.append({'model':name,'target':t,'mae':float(abs(pred[:,j]-Y[:,j]).mean())})
 for scope,cols in [('core30',[i for i,t in enumerate(targets) if t.startswith(('A_','F_'))]),('all50',list(range(len(targets))))]:
  for comparator in ['style_ET','style_median']:
   diff=(abs(oof['style_chemistry_ET'][:,cols]-Y[:,cols])-abs(oof[comparator][:,cols]-Y[:,cols])).mean(1);paired.append({'scope':scope,'comparison':'style_chemistry_ET_minus_'+comparator,**conditional_paired_interval(diff,styles)})
 save_csv(pd.DataFrame(rows),out/'metrics.csv');save_csv(pd.DataFrame(paired),out/'paired.csv');save_csv(pd.DataFrame(bytarget),out/'by_target.csv')
 for name,arr in oof.items():save_csv(pd.concat([D.meta[['beer']],pd.DataFrame(arr,columns=targets)],axis=1),out/f'oof_{name}.csv')
 write_json(out/'manifest.json',{'design':'exploratory after review; same ET hyperparameters, outer folds and known one-hot style; compare style only and style plus chemical, each train-fold preprocessing; style-median retained; fixed OOF beer bootstrap, no retraining uncertainty','data_sha256':sha256(ROOT/'outputs/stage4/audit/observed_panel_means_0_6.csv'),'code_sha256':sha256(Path(__file__)),'note':'max_features=.8 operates on different dimensionalities; not a fully matched optimization budget'});print(pd.DataFrame(rows).to_string(index=False));print(pd.DataFrame(paired).to_string(index=False))
if __name__=='__main__':main()
```